Technical documentation — HYDRA

Processing methods, uncertainty propagation, image correlation and dispersion corrections

Version 1.0

Document generated from a full reading of the source code

Edition v1.0 — November 2026

Contents

Foreword **11**

1 Monte-Carlo Uncertainty Propagation and Covariance Calculations **14**

1.1 General principle: Monte-Carlo sampling, no linearized analytical propagation 14

1.1.1 Step 1 — definition and sampling of input parameters 14

1.1.2 Step 2 — replaying the full pipeline for each draw 15

1.1.3 Step 3 — aggregation: point-by-point empirical mean and standard deviation . . 16

1.2 Supported draw distributions 17

1.2.1 Normal law (Distribution.NORMAL) 17

1.2.2 Uniform law (Distribution.UNIFORM) 17

1.2.3 Degenerate case (fixed value) 18

1.3 Monte-Carlo covariance/correlation between points of a single curve 18

1.3.1 Need for a common grid, and interpolation 18

1.3.2 Covariance matrix formula 19

1.3.3 Correlation: no normalization implemented in the code 19

1.3.4 Origin of the raw draws used 20

1.3.5 Calibration library (covariance_library) — bridge to Bayesian Calibration . . . 20

1.4 Uncertainty band of a custom sensor (custom_sensor.py) 22

1.4.1 Calibrated data: multiplicative factor, low-pass filter, thermal correction 23

1.4.2 Uncertainty band (uncertainty_band) 23

1.4.3 Time trimming (cut_from) 24

1.4.4 Time-varying thermal output correction (optional) 24

1.5 User-facing parameterization of the number of draws (N) 25

1.6 Summary of traceability points (file :: function) 25

1.7 Another uncertainty paradigm: asymptotic least-squares covariance 27

1.8 Limitations and assumptions explicitly flagged 27

2 Quasi-Static (QS) Reduction and Specimen Geometries **29**

2.1 Overview of the QS computation chain 29

2.2 Signal preprocessing 30

2.2.1 Cutting the beginning of the curve (core/curve_cutting.py: cut_curve_start) . . 30

2.2.2 Machine compliance correction 30

2.2.3 Monte-Carlo measurement uncertainties (coeff_load, coeff_dd) 31

2.3 The four reduction kinematics (type_check) 31

2.3.1 Compression and Tension (type_check == 1 or 2) 31

2.3.2 Notched specimen — Shear-Compression/Tension (type_check == 3) 33

2.3.3 Shear (type_check == 4) . . . 33
2.3.4 Adiabatic temperature model (Taylor-Quinney), common to all 4 modes . . . 33
2.4 Specimen geometries (core/models.py) . . . 34
2.4.1 Cylindrical (Compression) . . . 34
2.4.2 Square/Rectangular (Compression) . . . 34
2.4.3 Axisymmetric (Tension) . . . 34
2.4.4 Flat (Tension, "Flat") . . . 34
2.4.5 Notched specimen (Shear-Compression/Tension, Notched) . . . 35
2.4.6 Hat-shaped (Shear) . . . 35
2.4.7 Double shear . . . 35
2.4.8 Single shear . . . 35
2.5 k / sk coefficients for the notched specimen . . . 36
2.5.1 File format (core/k_value_io.py) . . . 36
2.5.2 Kinematic role of k_value (no split, sk_check false) . . . 36
2.5.3 "Split fit" mechanism (sk_check true) . . . 36
2.5.4 Consistency with the "notched" geometry (§4.5) . . . 37
2.6 Limitations, assumptions and explicitly flagged ambiguities . . . 37
2.6.1 On the QS tab, the detailed geometry (cylindrical vs. square/rectangular, axisymmetric vs. flat) is not taken into account . . . 37
2.6.2 On the QS tab, the shear sub-type is not selectable . . . 38
2.6.3 Notch angle: unit "deg" declared, but used directly inside cos() . . . 38
2.6.4 Geometry fields declared but unused in QS . . . 38
2.6.5 No cross-section recomputation at large strain . . . 38
2.6.6 Temperature model: adiabatic assumption not justified in the code . . . 39
2.7 File / function summary . . . 39

3 SHPB Analysis (Classic Two-Bar Hopkinson Bars) . . . 40
3.1 Physical principle and general computation chain . . . 40
3.1.1 Classic assumptions . . . 40
3.1.2 Exact convention used in the code — verified line by line . . . 41
3.1.3 Time windowing of the three waves ("Cutting waves") . . . 41
3.1.4 Strain rate, nominal strain, true strain (compression/tension case) . . . 42
3.1.5 Nominal and true stress (compression/tension case) . . . 42
3.1.6 Force-balance quality indicators ("balance quality") . . . 43
3.1.7 Numerical safeguards (not physical corrections) . . . 43
3.2 Kinematics variants of the same computation engine . . . 43
3.2.1 Compression / tension (type_check = 1 or 2) . . . 43

3.2.2 Notched shear-compression/tension — ”Notched” (type_check == 3, lines 383-497) 44
3.2.3 Pure shear — ”Shear” (type_check == 4, lines 609-723) 44
3.3 Choice of the force used for stress (force_check) 44
3.4 Available physical corrections 45
3.4.1 Bar thermal-gradient correction (”heat_check”, heated specimen) 45
3.4.2 Platen correction (plat_check) 46
3.4.3 Punch/indentation correction (punch_check) 47
3.4.4 Inertia correction (inertia_check) 48
3.4.5 Friction coefficient (friction) 48
3.5 ”Toe” correction of the curve (core/toe_correction.py) 49
3.5.1 Automatic detection of the linear region — detect_linear_region() (lines 51-89) . 49
3.5.2 Manual adjustment — fit_linear_region() (lines 40-48) 49
3.5.3 Origin realignment — apply_toe_correction() (lines 92-102) 50
3.5.4 Recomputing true strain after realignment — infer_true_strain_relation() / true_strain_from_nominal() (lines 105-148) 50
3.5.5 Consistency with raw Monte-Carlo draws and confinement quantities 50
3.6 Wave-alignment verification tool (core/wave_alignment.py) 52
3.6.1 Mechanical impedance and transmission coefficient 52
3.6.2 Theoretical transmitted wave — predicted_transmitted_wave() (lines 45-73) . . 52
3.6.3 Cross-correlation auto-alignment — best_alignment_shift() (lines 88-129) . . . 52
3.7 Time alignment of a sensor mounted directly on the specimen 53
3.8 Summary of limitations, assumptions and points of ambiguity found in the code 53

4 ”Direct Impact” (DI) Analysis 55
4.1 Physical principle and differences from classic SHPB 55
4.2 Pipeline overview (process_di()) 55
4.3 Cutting the signals and the incident velocity 56
4.3.1 Time windowing 56
4.3.2 Dispersion-corrected signal (disp_ela / disp_visco) 57
4.3.3 ”Platen” correction 57
4.3.4 Bar thermal-gradient correction (”heat”) 57
4.3.5 Caution — plat_check / heat_check interaction on E_mat_OB / C0_mat_OB 58
4.3.6 Diameter/properties used for F_output vs. the punch correction 59
4.3.7 Viscoelastic modulus and wave speed (_viscoelastic_modulus_and_speed, disp_visco) 59
4.4 Strain / strain-rate / force / stress chain 60
4.4.1 Compression / Tension (type_check 1 or 2) 60

4.4.2 Shear (type_check == 4: hat-shaped / double / single) . . . 61
4.4.3 Notched specimen — Shear-Compression/Tension (type_check == 3) . . . 62
4.5 Additional physical corrections . . . 63
4.5.1 Punch/indentation correction (punch_check) . . . 63
4.5.2 Friction (friction[d] != 0) . . . 64
4.5.3 Radial/axial inertia (inertia_check) . . . 64
4.5.4 Adiabatic heating (temperature) . . . 64
4.6 Realignment of the incident-velocity profile (”Cut”/”shift”) and calibration factor . . . 65
4.6.1 Cutting/shifting (”Cut velocity profile at selected start” / ”shift by (s)”) . . . 65
4.6.2 Multiplicative calibration factor for the incident velocity . . . 65
4.6.3 Auxiliary helper for custom sensors mounted on the specimen . . . 66
4.7 Reuse of toe_correction.py and specimen_sensor_alignment.py . . . 66
4.8 Equation code correspondence table . . . 67
4.9 Limitations, warnings and points to verify . . . 68

5 Quasi-Oedometric (Confined) Test: Radial Stress, Hydrostatic Pressure and Deviator 70

5.1 Physical principle of the test . . . 70
5.2 Linear cell model . . . 71
5.2.1 Formula . . . 71
5.2.2 Interpretation — C1, C2 and the transition mechanism . . . 71
5.3 Nonlinear cell model . . . 72
5.3.1 Overview . . . 72
5.3.2 Step 1 — Normalization to hoop strain and polynomial fit . . . 72
5.3.3 Step 2 — Evaluating the polynomial without its constant term . . . 73
5.3.4 Step 3 — Interpolating between the two curves according to the specimen’s current height 73
5.4 Constants alpha00 and alpha0l/2 — radial-strain calibration . . . 74
5.4.1 Implicit relation . . . 74
5.4.2 Numerical resolution . . . 74
5.4.3 Sign convention — an explicit point of caution in the code . . . 75
5.5 Derived quantities: radial stress, pressure, deviator, strains . . . 75
5.5.1 Stress invariants: hydrostatic pressure and deviator . . . 75
5.5.2 Radial strain . . . 76
5.5.3 Volumetric strain . . . 76
5.5.4 Equivalent (von Mises) strain . . . 76
5.5.5 Summary — full computation chain . . . 76

5.5.6 Alternative axial stress via laser velocimetry . . . 77
5.5.7 Synthetic fallback envelope (“imitation”) — triaxial_synthetic_draws . . . 77
5.5.8 Real computation in the Monte-Carlo loop (SHPB / Direct Impact / Quasi-Static) 78
5.6 Summary of function quantity correspondences . . . 80
5.7 Limitations, assumptions and warnings found in the code . . . 81

6 Digital Image Correlation (DIC) . . . 83
6.1 Pipeline overview . . . 83
6.2 Preprocessing (core/dic_preprocessing.py) . . . 84
6.3 ROI and correlation mesh (core/dic_correlation.py) . . . 84
6.3.1 ROI shapes . . . 85
6.3.2 Building the grid — build_mesh(roi, subset_size, step) . . . 85
6.3.3 Non-rectangular ROIs . . . 85
6.3.4 MeshGrid structure . . . 85
6.4 Tracking/correlation algorithm (core/dic_processing.py) . . . 86
6.4.1 General principle . . . 86
6.4.2 Normalized cross-correlation and search window . . . 86
6.4.3 Sub-pixel refinement — _subpixel_peak . . . 86
6.4.4 “Fixed” vs. “Moving” tracking modes . . . 87
6.4.5 Correlation quality filter — correlation_threshold . . . 87
6.4.6 Neighbor consistency filter — _apply_neighbor_consistency_filter . . . 87
6.4.7 Main loop — run_dic . . . 88
6.4.8 Exporting and re-importing results (CSV) — core/dic_result_io.py . . . 88
6.5 Local strain field — _grid_strain (core/dic_processing.py, lines 188–202) . . . 89
6.5.1 Von Mises equivalent strain — core/dic_kinematics.von_mises_strain . . . 90
6.6 Derived kinematics — core/dic_kinematics.compute_kinematics . . . 91
6.6.1 Time axis . . . 91
6.6.2 Velocity and acceleration . . . 91
6.6.3 ROI dynamic force — constant-section assumption . . . 91
6.6.4 Smoothing — smooth_series . . . 92
6.7 Conversion to true strain — nominal_to_true_strain . . . 92
6.8 Time scale / Time offset — camera-gage synchronization . . . 93
6.9 DIC measurement-uncertainty model (core/dic_uncertainty.py) . . . 94
6.9.1 What this model IS and is NOT . . . 94
6.9.2 Mechanism 1 — Sub-pixel registration precision floor (photometric) . . . 94
6.9.3 Mechanism 2 — Correlation-quality degradation . . . 95
6.9.4 Mechanism 3 — Subset distortion (intra-subset averaging) . . . 96

6.9.5 Combining the three terms in quadrature . . . 96
6.9.6 Propagation to the strains . . . 96
6.9.7 Temporal propagation (rate, velocity, acceleration, force) . . . 97
6.9.8 Effective point count and ROI-mean uncertainty . . . 98
6.9.9 Orchestration — compute_uncertainty . . . 98
6.10 Visual rendering (core/dic_render.py) — brief summary . . . 99
6.11 Summary of file function correspondences . . . 99
6.12 Summary of limitations and warnings found in the code . . . 101

7 Wave Dispersion Correction — Elastic Regime . . . 103
7.1 Physical principle: why a finite bar disperses the wave . . . 103
7.2 Dispersion theory implemented: the Pochhammer-Chree equation . . . 104
7.2.1 Exact variables and notation from the code . . . 104
7.2.2 Characteristic equation solved . . . 105
7.2.3 Bar wave speed c0 and Poisson's ratio: _bar_elastic_wave_speed() . . . 105
7.3 Numerical solution method: frequency-by-frequency root search . . . 106
7.3.1 Algorithm (_solve_pochhammer_branch, lines 162–197) . . . 106
7.3.2 Parallelization across branches (_solve_pochhammer_branches, lines 209–239) . 107
7.4 Full signal correction algorithm (FFT per-bin correction IFFT) . . . 107
7.4.1 Time-domain window cutting (before the FFT) . . . 107
7.4.2 Moving to the frequency domain (FFT) . . . 108
7.4.3 Per-frequency-component correction: _corrected() (lines 373–385) . . . 108
7.4.4 Signed distance z and sign convention of the three waves . . . 109
7.4.5 "Output bar only" case (Direct Impact): elastic_correction_direct_impact() . . 109
7.5 Frequency windowing and bandwidth bounding of the correction . . . 110
7.5.1 Absence of time-domain windowing (apodization) before the FFT — always true 110
7.5.2 Bounding the correction's own bandwidth — _bandwidth_rolloff_weight() (lines 92–153) . . . 110
7.6 Switching between elastic and viscoelastic within the module . . . 112
7.7 Exposed results (DispersionResult, lines 99–110) . . . 112
7.8 Summary of code correspondences . . . 113
7.9 Limitations and warnings identified in the code . . . 113

8 Viscoelastic dispersion correction, Bayesian calibration and damping . . . 115
8.1 Why a viscoelastic model? . . . 115
8.2 The four rheological models (core/rheology_models.py) . . . 117
8.2.1 Kelvin–Voigt (_kelvin_voigt_modulus, line 120) . . . 117
8.2.2 Standard Linear Solid / Zener (_sls_modulus, line 126) – the only model wired into the

Dispersion Corrections page prior to this revision . . . 118
8.2.3 Generalized Maxwell / Wiechert, 2 branches (_generalized_maxwell_2_modulus, line 139) . . . 118
8.2.4 Burgers (_burgers_modulus, line 150) . . . 119
8.2.5 Summary table . . . 120
8.3 Pulse propagation through the viscoelastic bar model (core/visco_calibration.py) . . . 120
8.3.1 Complex modulus, speed and Lame moduli (_complex_modulus_and_speed, line 88) 120
8.3.2 Per-frequency-bin Pochhammer-Chree root search . . . 121
8.3.3 Phase, attenuation and damping correction, then time-domain reconstruction . . . 124
8.3.4 Comparison score: Pearson vs R^2(SCORE_FUNCS, lines 233-236) . . . 124
8.4 Bayesian calibration (core/bayes_visco_calibration.py) . . . 125
8.4.1 Why a Bayesian search rather than a grid . . . 125
8.4.2 Evaluating a candidate point, priors and hyperpriors . . . 125
8.4.3 Initial sampling (Latin Hypercube Sampling) . . . 127
8.4.4 Adaptive loop: Gaussian Process surrogate + acquisition strategy . . . 127
8.4.5 Posterior mean, standard deviation and parameter correlation . . . 128
8.4.6 Predicted pulse returned: reused, not recomputed . . . 129
8.5 The surrogate Gaussian Process (core/gaussian_process.py) . . . 130
8.5.1 Covariance kernel . . . 130
8.5.2 Hyperparameter fitting (marginal likelihood maximization) . . . 130
8.5.3 Prediction (posterior mean/variance) . . . 130
8.6 Sub-sample-accurate pulse auto-alignment (core/pulse_alignment.py) . . . 131
8.6.1 Why . . . 131
8.6.2 Algorithm (align_pulse2, line 50) . . . 131
8.7 The ”Free-Decay Damping” standalone tool (core/decay.py) . . . 132
8.7.1 Precise localization of the peak and the half-power points . . . 132
8.7.2 Damping factor . . . 132
8.7.3 Downstream use . . . 133
8.8 Summary of file/function cross-references . . . 133
8.9 Summary of limitations and warnings found in the code . . . 134

9 Standalone tools (SGBC, camera/video sync, batch analysis, Bayesian calibration) 136
9.1 Strain-gage bridge coefficient calculator (SGBC) . . . 136
9.1.1 Context and role . . . 136
9.1.2 Physical inputs . . . 137
9.1.3 Common thermal term . . . 137

9.1.4 CPG coefficient formula by bridge type . . . 137
9.1.5 Uncertainty propagation . . . 138
9.1.6 Interface / synchronization . . . 138
9.1.7 Transverse-sensitivity correction (optional) . . . 138
9.1.8 Thermal-output correction (optional, informative) . . . 139
9.2 Camera / video synchronization . . . 140
9.2.1 Context . . . 140
9.2.2 Reading the clock file . . . 140
9.2.3 Detecting the exposure pulse train . . . 141
9.2.4 Estimating the absolute time of each frame . . . 141
9.2.5 Detecting a rising edge (single-step trigger signal) . . . 142
9.2.6 Method 1 — Locating the trigger via a trigger signal . . . 142
9.2.7 Method 2 — Locating the trigger via a known frame number + frequency . . . 142
9.2.8 Returned result . . . 143
9.2.9 Limitations noted in the code . . . 143
9.3 Batch analysis . . . 143
9.3.1 Context . . . 144
9.3.2 Loading files . . . 144
9.3.3 Isothermal correction (adiabatic heating) . . . 144
9.3.4 Resampling onto a common grid . . . 145
9.3.5 Mean curve and variability bands . . . 146
9.3.6 Export . . . 147
9.3.7 Limitations and remarks noted in the code . . . 147
9.4 Downsampling (core/downsampling.py) . . . 147
9.5 Video / image-sequence abstraction (core/frame_source.py) . . . 147
9.6 Annotated DIC video export (core/dic_video_export.py) . . . 148
9.7 Bayesian calibration of constitutive laws — port of MAMBA . . . 148
9.7.1 Context: MAMBA and the porting decision . . . 148
9.7.2 Equation parser and evaluator (core/calib_equation.py) . . . 149
9.7.3 Three prior treatments per free parameter (core/calib_priors.py) . . . 151
9.7.4 From the covariance library to a test dataset (core/calib_library_adapter.py) . . . 151
9.7.5 Building and sampling the PyMC model (core/calib_pymc_model.py) . . . 153
9.7.6 Orchestrating a run (core/calib_run.py) . . . 155
9.7.7 Posterior diagnostics and genuine cross-validation (core/calib_diagnostics.py) . . . 157
9.7.8 Shared worker thread (core/qt_worker.py) . . . 158
9.7.9 Interface — the Bayesian Calibration page's cards . . . 158
9.7.10 Limitations and warnings noted in the code . . . 160

9.7.11 Summary of file/function references 162

10 Crack & wave analysis 164

10.1 Traced crack skeleton (core/crack_skeleton.py) 165

10.2 Elastic wave speed via front tracking (core/elastic_wave_speed.py) 165

10.2.1 Principle: time-of-flight along a front 165

10.2.2 Automatic detection of multiple branches (ridge + RANSAC) 166

10.2.3 Manual picking via ROI polygon (extract_ridge_in_polygon) 166

10.2.4 Absence of formal uncertainty 167

10.3 Crack-tip detection and tracking (core/crack_tip_tracking.py) 167

10.3.1 Overview of available methods 167

10.3.2 Method used: opening jump (crack_opening_jump_profile, track_tip_opening_jump) 168

10.3.3 Position uncertainty (opening_extent_position_uncertainty_px) 169

10.3.4 Local speed and its uncertainty 171

10.3.5 Fitted speed (regression, weighted when an uncertainty is available) 171

10.3.6 CSV export 172

10.4 Stress intensity factors K_I/K_II: X-FEM-DIC (core/xfem_dic.py) 172

10.4.1 Principle and rationale: why the raw images 172

10.4.2 Displacement model and deliberate simplifications 173

10.4.3 Tip-centered Q4 mesh (uniform or graded) 173

10.4.4 Enrichment classification (classify_mesh_enrichment) 174

10.4.5 Solving: damped Gauss-Newton, Jacobi scaling 174

10.4.6 Automatic tip-position detection 175

10.4.7 Formal uncertainty (asymptotic Gauss-Newton covariance) 176

10.4.8 “Critical K_I” convention and uncertainty envelope on the plot 177

10.4.9 CSV export (export_xfem_k_csv) 177

10.5 Additional methods available in the code 178

10.5.1 Standard ring-fit DIC (fit_k_single_frame, compute_k_time_series) 178

10.5.2 Optional extension: T-stress term (extended Williams series) 179

10.5.3 Interaction integral / J-integral (core/crack_interaction_integral.py) 180

10.6 Limitations and remarks noted in the code 181

Appendix — Summary of Caution Points Noted in the Code 184

Supplements to the v1.0 edition (chapters 11 to 14) 193

11 Synchronisation and pre-processing of N cameras 195

11.1 General architecture 195

11.2 Calibration . . . 196
11.3 Alignment and crop . . . 196
11.4 Pairwise photometric equalisation . . . 197
11.5 Time stamping . . . 198
11.6 “ImageJ Auto” contrast stretch . . . 198
11.7 Lossless video exports . . . 198
11.8 File / function summary table . . . 199
11.9 Limitations, assumptions and points to check . . . 199

12 Image correlation, supplements . . . 200
12.1 Sub-pixel refinement by affine IC-GN . . . 200
12.2 Strain-tensor invariants and spatial smoothing . . . 201
12.3 Continuous isocontours . . . 201
12.4 Enlarged window for frame-by-frame reading (`ui/frame_popup.py`) . . . 202
12.5 File / function summary . . . 202
12.6 Limitations, assumptions and points to check . . . 202

13 Crack & wave analysis, supplements . . . 203
13.1 Crack representation and automatic ridge detection . . . 203
13.2 Tip tracking: methods and rejection of elastic waves . . . 204
13.3 Local crack frame and local opening (`core/crack_local_frame.py`) . . . 206
13.4 Smoothing of the tip position . . . 206
13.5 Direct K_I and K_{II} from the measured field . . . 206
13.6 X-FEM-DIC: sub-pixel sampling and K from the fitted field . . . 207
13.7 K_I versus tip speed . . . 207
13.8 Video exports of the tip analyses . . . 208
13.9 File / function summary . . . 208
13.10 Limitations, assumptions and points to check . . . 208

14 Processing and exports, supplements . . . 210
14.1 Strain rates of the confined test . . . 210
14.2 Curves before toe correction (“pre-toe”) . . . 210
14.3 Bar signals in profiles and in the full-test export . . . 210
14.4 Axial stress from laser velocimetry: uncertainty propagation . . . 211
14.5 Setup profiles and session . . . 211
14.6 File / function summary . . . 211
14.7 Limitations, assumptions and points to check . . . 211

Foreword

Purpose of this document

This document describes, chapter by chapter, the technical workings of `shpb_app`, the application for processing dynamic material characterization tests (Hopkinson bars / Split-Hopkinson Pressure Bar — SHPB, Direct Impact, quasi-static tests, confined quasi-oedometric tests) and digital image correlation (DIC). It covers ten functional areas:

1. Monte-Carlo uncertainty propagation and the associated covariance computations;
2. processing of quasi-static (QS) tests, for every supported specimen geometry;
3. processing of classical (two-bar) SHPB tests;
4. processing of Direct Impact tests (a single instrumented bar);
5. processing of quasi-oedometric (confined) tests;
6. digital image correlation (DIC);
7. elastic-regime wave dispersion correction;
8. viscoelastic-regime wave dispersion correction, Bayesian calibration of rheological models, and the free-decay damping tool;
9. the standalone tools: strain-gauge bridge coefficient calculator, camera/video synchronization, batch analysis, free-text Bayesian calibration of constitutive laws (a port of MAMBA, on PyMC 5/PyTensor), and supporting technical modules;
10. crack & wave speed analysis ("Crack & Wave Analysis"): elastic wave speed by front tracking, crack-tip detection/tracking, and stress-intensity factors K_I/K_{II} by several independent, mutually verifiable methods (standard ring-fit DIC — with an optional T-stress extension —, interaction integral/J-integral, X-FEM-DIC).

Writing method and level of equation detail

This document was produced by reading the entire source code of the application's `core/` folder (roughly 9,000 lines spread across about thirty Python modules), then formalizing, for each module, the equations exactly as they are actually implemented in the code — using the same variable notation as the source code whenever that aids traceability — together with a brief physical justification. It is therefore not a theoretical manual that would re-derive each method from the fundamental equations of continuum mechanics or signal processing; it is verification documentation, whose displayed equation must be traceable, term for term, in the file and function cited right next to it.

Each technical section is accordingly systematically accompanied by an explicit reference to the source file and function or method (`core/file.py::function`), and many code excerpts are reproduced verbatim when they clarify a formula. A summary table of file/function references appears at the end of most chapters.

Why Monte Carlo, and not analytical propagation — except in one corner of the application

A cross-cutting methodological point to keep in mind before reading the following chapters: in the overwhelming majority of the application (chapters 2 to 9), no uncertainty is computed by linearized analytical derivation (the first-order GUM propagation law, $\sigma_y^2 = \sum_i (\partial f/\partial x_i)^2 \sigma_{x_i}^2$, applied to a complete physical pipeline). Every uncertainty shown in these chapters — on a stress-strain curve, on a strain-gauge bridge coefficient, on a confinement radial stress, on a DIC strain field — results from a Monte Carlo draw: each uncertain input quantity is sampled N times according to its declared distribution (Normal or Uniform), the entire computation pipeline is replayed once per draw, and the empirical mean/standard deviation of the N results give the mean curve and its uncertainty band. Chapter 1 details this mechanism, common to nearly the whole application.

Exception: the "Crack & Wave Analysis" tool (chapter 10) computes its uncertainties differently, via the asymptotic covariance of a least-squares fit (exact and linear for the standard ring-fit DIC, asymptotic Gauss-Newton at the converged point for X-FEM-DIC) — a single evaluation, not a re-sampling. See chapter 1, §7, for how the two paradigms relate to each other, and chapter 10 for the full detail of both methods.

On the points of caution flagged throughout this document

A careful reading of the source code surfaced, in several modules, points that need to be flagged explicitly rather than passed over in silence: apparent inconsistencies between two neighboring computation paths, implicit assumptions left uncommented in the code, ambiguous units, interface features that do not pass along all the information the underlying computation function expects, and so on. Every technical chapter accordingly ends with a "Limitations, assumptions and points to verify" section that lists these observations, phrased as they were observed in the code, without interpretation or correction on our part — it is left to the reader to confirm or refute them by comparison with the original application or with the code's authors. A consolidated summary of all these points also appears in an appendix at the end of the document, to serve as a single checklist.

Several code modules themselves carry, in their header, an explicit warning of no independent validation against real experimental data or against the output of the original application this code is ported from; these warnings are systematically reproduced in the relevant chapters.

Conventions

- Equations are given using the code's own notation (subscripts, `_pa` suffix for a size-N Monte Carlo array, IB/OB suffixes for input/output bar, etc.): the corresponding section defines this notation at its first occurrence.
- Code references are given as `file::function` or `file::Class.method`, paths relative to the `shpb_app/` project root.
- The § symbol refers to a section within the same chapter.
- Code excerpts are reproduced verbatim from the source code, for direct verification purposes.

Note on this edition

This edition updates the previous version of this document (September 2026) to reflect source-code changes made after its initial writing — chiefly the completion of the ”Bayesian Calibration” tab (MAMBA port, chapter 9 §7), a fix to the Pochhammer-Chree root search shared between the two viscoelastic computation paths (chapter 8, §8.1 through §8.4), a fix to the machine compliance correction (chapter 2, §2.2), and the addition of a save/load-to-file feature for the calibration library (chapter 1, §3.5). Only the sections affected by these changes were revised; the rest of the document carries over the previous edition’s text unchanged. This edition is also, for the first time, translated into English (a separate document, with identical technical content).

1. Monte-Carlo Uncertainty Propagation and Covariance Calculations

This chapter describes the methodological foundation used by every reduction module of `shpb_app` (Quasi-Static QS, SHPB, Direct Impact DI, auxiliary sensors, triaxial tests). All code references are given in the form `file::Class.method` or `file::function`, paths relative to `shpb_app/`.

1.1 General principle: Monte-Carlo sampling, no linearized analytical propagation

The application never computes the final uncertainty by analytical derivation/linearization (the first-order GUM "law of propagation of uncertainty", $\sigma_y^2 = \sum_i (\partial f/\partial x_i)^2 \sigma_{x_i}^2$). It exclusively uses a Monte-Carlo draw-based method: each uncertain physical input parameter (bar geometry, density, Young's modulus, gauge factor, etc.) is drawn randomly N times according to its declared distribution law, producing N complete parameter sets. The full computation pipeline (wave → stress/strain conversion for SHPB/DI, or load/displacement → stress/strain for QS) is then replayed in full N times, once per parameter set, producing N independent result curves. The point-by-point mean and standard deviation of these N curves give the displayed mean curve and uncertainty band.

This is explicitly documented at the head of `core/parallel.py`:

> *"Monte-Carlo uncertainty propagation (core/process_shpb.py, core/process_di.py, called from ui/pages/page_process.py): one independent call per Monte-Carlo draw."*

1.1.1 Step 1 — definition and sampling of input parameters

Each uncertain physical parameter is represented by an `MCValue` object (`core/mc.py`), which groups into a single structure the mean, dispersion and distribution of the parameter ("mean value / dispersion / distribution", header comment of `core/mc.py`):

```
# core/mc.py
@dataclass
class MCValue:
    name: str
    unit: str = ""
    mean: float = 0.0
    dispersion: float = 0.0
    distribution: Distribution = Distribution.NORMAL
```

Sampling the N realizations of a parameter is performed by `core/mc.py::MCValue.sample(n, rng)`. The parameters of a bar or a specimen are grouped in an `MCParameterSet` (`core/mc.py`), whose `sample_all(n, rng)` method draws each parameter in the group independently of the others — no correlation between input parameters is modeled: the sampling uses a distinct `numpy.random.Generator` per call (or a shared one if passed explicitly), and each `MCValue.sample()` is independent

of the others. This is an implicit assumption of the code, not explicitly commented as such in `mc.py`, but which follows directly from the implementation (no input correlation matrix, each parameter has its own `rng.normal`/`rng.uniform` call).

For `Bar` objects (`core/models.py::Bar`), the Monte-Carlo expansion is triggered by `Bar.expand(n, rng)`, which calls `_mc_array(params, key, n, rng)` (itself a simple relay to `MCValue.sample`) for each bar parameter: `rho_pa`, `C0_pa`, `D_pa`, `e_pa`, `L_pa`, `PG_pa`, `nu_pa`, `CPG_pa`. The `_pa` suffix denotes a numpy array of length N ("population array"), a convention inherited from the original code (`OB.D_pa`, `IB.rho_pa`, header comment of `core/models.py`). The number of draws N comes from `core/project.py::Project.n_montecarlo` (default value 500, "Number of Monte-Carlo Draws" field of the analysis identity sheet on the Test Setup page). The global expansion (input bar, output bar, specimen) is triggered by `core/project.py::Project.expand_montecarlo()`:

```
# core/project.py
def expand_montecarlo(self):
    self.input_bar.expand(self.n_montecarlo)
    self.output_bar.expand(self.n_montecarlo)
    self.specimen.expand(self.n_montecarlo)
```

A special case exists for the constants of the bars' viscoelastic model (Standard Linear Solid), drawn by `core/models.py::Bar.expand_viscoelastic(n, e1, e1_std, e2, e2_std, n2, n2_std, rng)` — always a normal law, falling back to a constant value (`np.full(n, e1)`) if the supplied standard deviation is zero.

1.1.2 Step 2 — replaying the full pipeline for each draw

For SHPB and DI, execution of the N draws is handed to `core/parallel.py::run_map(func, items, n_workers, context, progress_callback, use_threads)`, called from `ui/pages/page_process.py` with `items = list(range(n))` (one draw index per item) and `func = core/process_shpb.py::_process_shpb_task` (resp. `core/process_di.py::_process_di_task`). Each call runs the full physical pipeline for draw d: it picks `IB.D_pa[d]`, `IB.rho_pa[d]`, etc. from the pre-drawn `_pa` arrays, and produces one entire stress-strain curve (and all derived quantities: force, temperature, strain rate, …) for that single draw. `n_workers` (the "Performance" setting of the Test Setup page, `Project.n_workers`) determines whether these N calls run serially (`n_workers <= 1`) or in parallel across several processes (`ProcessPoolExecutor`), without changing the numerical result — only the execution speed changes. The context (bars, specimen, settings) is pickled once per worker process, not once per draw, for performance reasons.

For QS, the same scheme is used directly in `ui/pages/page_process.py::_QSTab._on_run` via `run_map(_process_qs_task, qs_items, n_workers=project.n_workers, context=qs_context, ...)`, with, in addition, two random multiplicative coefficients (correction factors, drawn independently per draw):

```
rng = np.random.default_rng()
coeff_load_arr = rng.normal(1.0, clu_pct / 100.0, n)
coeff_dd_arr = rng.normal(1.0, meu_pct / 100.0, n)
```

(`clu_pct`/`meu_pct`: load- and displacement-calibration uncertainty percentages — normal laws with mean 1.0 and standard deviation equal to percentage/100).

1.1.3 Step 3 — aggregation: point-by-point empirical mean and standard deviation

Important: no interpolation onto a common grid is performed at this stage. Since each draw produces a curve of potentially different length (the number of points of an SHPB/DI/QS stress-strain curve depends on the computation window, itself dependent on the drawn values), the aggregation simply truncates each curve to the length of the shortest draw, then computes mean and standard deviation by array index (not by identical strain level). This is done identically at the three locations of the QS, SHPB, DI tabs of `ui/pages/page_process.py` (lines ≈813–818, ≈1878–1883, ≈2969–2974):

```
min_len = min(len(s) for s in all_stress)
stress_arr = np.array([s[:min_len] for s in all_stress])                # (N_valides,
    min_len)
strain_arr = np.array([s[:min_len] for s in all_strain])                # (N_valides,
    min_len)
mean_strain = strain_arr.mean(axis=0)
mean_stress = stress_arr.mean(axis=0)
std_stress     = stress_arr.std(axis=0)
```

Exact formulas, for N' valid draws (those that did not raise an exception) truncated to L common points, with $x_{d,i}$ the strain and $y_{d,i}$ the stress of draw d at point i:

$$\bar{x}_i = \frac{1}{N'} \sum_{d=1}^{N'} x_{d,i}, \qquad \bar{y}_i = \frac{1}{N'} \sum_{d=1}^{N'} y_{d,i} \tag{1.1}$$

$$s_{y,i} = \sqrt{\frac{1}{N'} \sum_{d=1}^{N'} (y_{d,i} - \bar{y}_i)^2} \tag{1.2}$$

This is the population standard deviation (`numpy.ndarray.std` with `ddof=0` by default, not the sample standard deviation `ddof=1`) — worth noting explicitly, this is a small underestimate relative to the unbiased standard deviation, negligible once N' is large (default value $N = 500$).

The plotted mean curve uses the strain axis itself averaged ($\bar{x}_i$, not a fixed reference axis), and the displayed uncertainty band is:

$$[\bar{y}_i - k_\sigma \, s_{y,i}, \ \bar{y}_i + k_\sigma \, s_{y,i}] \tag{1.3}$$

where k_σ = `Project.uncertainty_sigma` $\in \{1, 2, 3\}$ — the "Uncertainty Band (σ)" setting of the analysis identity sheet (Test Setup). The corresponding code (identical on the 3 tabs):

```
sigma = self.project.uncertainty_sigma
ax.fill_between(mean_strain, mean_stress - sigma * std_stress, mean_stress + sigma *
    std_stress, ...
```

Limitation to flag: since there is no resampling onto a common strain grid at this stage (unlike the covariance module, see §3), the mean/standard-deviation by array index is rigorously valid only if all curves have substantially the same point-by-point strain axis — which is generally true when the number of points is set by the time-sampling step of the raw signal (common to all draws) rather than by the dynamics of the draw itself, but can introduce some blurring if the time windows (hence curve lengths) vary significantly from one draw to another — in that case truncation to `min_len` aligns curves by index, not by strain value.

All per-draw quantities are further generically aggregated (not just stress/strain) by `ui/pages/page_process.py::_aggregate_quantities(per_draw)`, which applies exactly the same principle (truncation to the minimal common length for the quantity, then mean/standard deviation by index) to every key present in the per-draw dictionaries (`strain`, `stress`, `strain_rate`, `temperature`, `load`, `displacement`, `nominal_strain`, `force_input`, `force_output`, `inertia_stress`, `friction_stress`, etc.), producing for each quantity `<key>` a pair `<key>` (mean) and `<key>_std` (standard deviation) stored in `project.last_results[...]`:

```
# ui/pages/page_process.py
def _aggregate_quantities(per_draw: list[dict]) -> dict:
    ...
    for key in keys:
        arrays = [np.asarray(d[key]) for d in per_draw if key in d and len(d[key]) > 0]
        min_len = min(len(a) for a in arrays)
        stacked = np.array([a[:min_len] for a in arrays])
        out[key] = stacked.mean(axis=0)
        out[f"{key}_std"] = stacked.std(axis=0)
    return out
```

In parallel, `ui/pages/page_process.py::_store_raw_draws(project, result_key, per_draw)` keeps every draw individually (not just mean/standard deviation) in `project.last_raw_draws[result_key]` as `{quantity: [draw_0_array, draw_1_array, ...]}`. It is this raw structure (not the aggregated means) that feeds the covariance module (§3).

1.2 Supported draw distributions

The code supports only two laws, enumerated in `core/mc.py::Distribution(str, Enum): NORMAL = "Normal"` and `UNIFORM = "Uniform"`. Sampling is performed by `core/mc.py::MCValue.sample(n, rng)`:

```
def sample(self, n, rng=None):
    rng = rng or np.random.default_rng()
    if n <= 1 or self.dispersion == 0:
        return np.full(max(n, 1), float(self.mean))
    if self.distribution == Distribution.NORMAL:
        return rng.normal(self.mean, self.dispersion, size=n)
    return rng.uniform(self.mean - self.dispersion, self.mean + self.dispersion, size=n)
```

1.2.1 Normal law (Distribution.NORMAL)

Function: `numpy.random.Generator.normal(mean, dispersion, size=n)`, mathematically equivalent to a transformed standard draw:

$$x_d = \mu + \sigma \cdot Z_d, \qquad Z_d \sim \mathcal{N}(0,1), \qquad d = 1, \ldots, N \tag{1.4}$$

where $\mu =$ `MCValue.mean`, $\sigma =$ `MCValue.dispersion` (interpreted here as a standard deviation).

1.2.2 Uniform law (Distribution.UNIFORM)

Function: `numpy.random.Generator.uniform(mean - dispersion, mean + dispersion, size=n)`. The `dispersion` parameter is here treated as a half-interval (half-range) around the mean, a con-

vention explicitly documented in the code ("dispersion is treated as a half-range around the mean, matching the 'Dispersion Value' convention used throughout the original GUI"):

$$x_d \sim \mathcal{U}(\mu - \Delta, \mu + \Delta), \qquad \Delta = \text{dispersion} \tag{1.5}$$

i.e. $x_d = \mu - \Delta + 2\Delta \cdot U_d$ with $U_d \sim \mathcal{U}(0, 1)$.

1.2.3 Degenerate case (fixed value)

If `n <= 1` or `dispersion == 0`, no draw is performed: `MCValue.sample` returns a constant array `np.full(max(n,1), mean)` — the parameter is treated as deterministic, regardless of the declared distribution. This is how the user "disables" the uncertainty on a given parameter (`dispersion = 0`).

Traceability note: this is the one and only entry point for random sampling of physical parameters in the application; no other distribution (log-normal, triangular, beta, etc.) is implemented in `core/mc.py`. Separately, `core/models.py::Bar.expand_viscoelastic` and `core/confinement.py` (see note at the end of this document) call `rng.normal(...)` directly without going through `MCValue`/`Distribution` — they are therefore always normal, with no uniform-law option, unlike the standard bar/specimen parameters.

1.3 Monte-Carlo covariance/correlation between points of a single curve

Module: `core/covariance.py`, ported from the original `export_res.py`. Objective stated at the head of the file: beyond the point-by-point standard deviation already displayed everywhere (which corresponds, in practice, to the diagonal of this matrix), compute the full covariance matrix between points of the same curve, to find out whether the uncertainty at one strain (or time) level is correlated with that at another level — for example, a particular draw of the bar diameter shifts the whole curve globally rather than perturbing each point independently.

1.3.1 Need for a common grid, and interpolation

Unlike the mean/standard-deviation aggregation of section 1.3 (which truncates by index), the covariance module explicitly interpolates each draw onto a common grid before any computation, because each draw has its own strain axis (strain itself being Monte-Carlo-dependent):

> *"each draw has its own strain axis (since strain is itself Monte-Carlo-dependent), so before stacking draws into one matrix, every draw's quantity is linearly interpolated onto one common strain grid"*

Function: `core/covariance.py::interpolate_draws_to_common_grid(per_draw_strain, per_draw_quantity, strain_start, strain_end, n_points)`.

1. Construction of the common grid: `strain_grid = np.linspace(strain_start, strain_end, num=n_points)` — *n_points* values evenly spaced between `strain_start` (called "Reference (start) strain" in the UI, corresponding to the `bay_cut_value` of the old code) and `strain_end`.
2. For each draw d, linear interpolation of its own curve (x_d, y_d) onto this grid via `scipy.interpolate.interp1d(strain, quantity, kind="linear")`.

3. A draw is entirely excluded if its own axis does not fully cover `[strain_start, strain_end]` (`if strain_grid.min() < strain.min() or strain_grid.max() > strain.max(): continue`) — a linear interpolator cannot be evaluated outside its domain; no extrapolation is attempted.
4. Result: `stacked`, an array `(n_valid_draws, n_points)`, one draw per row, each column corresponding to the same strain value (quantity X) across all draws.

On the UI side (`ui/pages/page_covariance.py`), `strain_start`/`strain_end`/`n_points` are the "Reference (start) strain" / "End strain" / "Number of points (N)" fields of the "Common grid" card, with default values 0.0, 0.05, 50.

1.3.2 Covariance matrix formula

Function: `core/covariance.py::compute_covariance_matrix(per_draw_arrays)`, called with `per_draw_arrays = stacked` (from the previous step). It requires at least 2 valid draws (`len(per_draw_arrays) < 2` → `None`, since a covariance requires at least 2 observations), transposes `stacked` from `(n_draws, n_points)` to `(n_points, n_draws)`, then applies `numpy.cov`:

```
def compute_covariance_matrix(per_draw_arrays):
    if per_draw_arrays is None or len(per_draw_arrays) < 2:
        return None
    stacked = np.asarray(per_draw_arrays)
    if stacked.ndim == 1:
        return None
    return np.cov(stacked.T)        # (n_points, n_points)
```

`numpy.cov` treats each row of its argument as a variable and each column as an observation; by transposing, each grid point i becomes a "variable" observed over the N' draws, exactly as the original does (`np.transpose(...)` before `np.cov(...)`, cited in the docstring). The exact mathematical formula, with $y_{d,i}$ the value of draw d at grid point i (after interpolation), N' the number of retained valid draws, and $\bar{y}_i = \frac{1}{N'}\sum_d y_{d,i}$:

$$C_{ij} = \frac{1}{N'-1}\sum_{d=1}^{N'}(y_{d,i}-\bar{y}_i)(y_{d,j}-\bar{y}_j) \tag{1.6}$$

This is the unbiased estimator (`ddof=1`, `numpy.cov`'s default behavior, to be distinguished from the `ddof=0` used by `ndarray.std` in §1.3 — a minor inconsistency between the two modules worth noting). The diagonal C_{ii} recovers the empirical variance of point i (in principle consistent with `<quantity>_std**2` of section 1.3, up to the common-grid interpolation, which makes the two quantities slightly different: §1.3 aligns by array index, §3 aligns by interpolated strain value).

1.3.3 Correlation: no normalization implemented in the code

The UI and docstring vocabulary uses "covariance" and "correlation" interchangeably ("how uncertainty at one strain/time point correlates with uncertainty at another"), but no function in the code normalizes the covariance matrix into a correlation matrix (no call to `numpy.corrcoef`, no division by $\sqrt{C_{ii}C_{jj}}$ found in `core/covariance.py` nor in `ui/pages/page_covariance.py`). What the application displays and exports is the raw covariance matrix C_{ij} (units = square of the quantity's unit), visualized as a color map (`ax.imshow(matrix, cmap="viridis", ...)`), with the diagonal = variance. A normalized correlation matrix $\rho_{ij} = C_{ij}/\sqrt{C_{ii}C_{jj}}$ could be derived from C_{ij} by the user but is not computed internally — if the final document must present a

"correlation matrix", it must be flagged as a post-hoc transformation, absent from the current source code.

1.3.4 Origin of the raw draws used

Depending on the quantity chosen on the Covariance Analysis page, `per_draw_strain`/`per_draw_quantity` come from three distinct sources, all converging onto the same pair of functions `interpolate_draws_to_common_grid` + `compute_covariance_matrix`:

- Already-computed QS/SHPB/DI result: `project.last_raw_draws[result_key]` (populated by `_store_raw_draws`, §1.3), X axis = `nominal_strain` if available, otherwise `strain`.
- Custom Sensor: synthetic draws generated on the fly by `ui/pages/page_covariance.py` (perturbation around the measured data, within the sensor's uncertainty band — see §4), with `N_SYNTHETIC_SENSOR_DRAWS = 200` draws.
- Triaxial (confinement) test: two possible sources, automatically selected by `ui/pages/page_covariance.py::CovariancePage._triax_draws()` depending on what is available for the chosen result (see chapter 5 §5.5/§5.7/§5.8 for the complete detail):
  - Real draws (preferred when available): the six confinement-cell-gauge-based quantities (radial stress, p, q, ε_r, ε_v, ε_{eq}) were computed once per draw, within the Monte-Carlo loop of the Processing tab itself (`core/confinement.py::per_draw_confinement_quantities`, chapter 5 §5.8), from the actual axial stress/strain and cell signal of THAT draw — a physical resampling already present in `project.last_raw_draws`, not an approximation added after the fact. `_triax_draws()` detects this availability (the `confinement_radial_stress` key in the raw draws) and reuses them directly.
  - Synthetic fallback draws (used when real draws are not available for that result, and systematically for the three laser-velocimetry-derived quantities): `core/confinement.py::triaxial_synthetic_draws(..., n_draws=200)`, which perturbs the axial stress and the confinement-cell deflection each by centered Gaussian noise (`rng.normal(0, axial_std)`, `rng.normal(0, defl_std)`) and the cell modulus by a relative normal law (`rng.normal(cell_modulus, cell_modulus_std_fraction * cell_modulus)`, default fraction 2%), to produce $N = 200$ synthetic realizations of the derived triaxial quantities (pressure, deviator, radial stress, etc.). This mechanism is an independent Monte-Carlo propagation, outside the `Bar.expand`/`MCValue` pipeline, which resamples an independent Gaussian around the already-averaged curves rather than reusing the actual draw-by-draw correlation — an imitation of the uncertainty, not a propagation in the strict sense (see chapter 5 §5.7 for the complete discussion of this limitation).

1.3.5 Calibration library (covariance_library) — bridge to Bayesian Calibration

Recent addition: the "Save to calibration library" card on the Covariance Analysis page (`ui/pages/page_covariance.py`) lets the displayed matrix — along with its common grid and mean curve, both already computed in §3.1 but until now discarded after display — be frozen into a named entry, kept on the project rather than written to disk. This is the data-feeding mechanism for the "Bayesian Calibration" tab (chapter 9, §7), which no longer loads folders of per-test `.txt` files (as the original software, MAMBA, did) but instead draws directly from this library.

Structure (`core/project.py::CovarianceLibraryEntry`, dataclass):

Field	Content
`name`	User-given name, unique within the library (pre-filled with the matrix's label; on a collision, a numeric suffix `_2`, `_3`, … is added automatically rather than overwriting the existing entry — `_on_save_to_library`).
`source_label`	Provenance (the displayed matrix's label, e.g. `SHPB_stress`).
`quantity_key`	Key into `QUANTITY_LABELS` (e.g. `stress`, `strain`, `pressure`, `volumetric_strain`) — advisory only, see below.
`grid`	Common grid (N points), list of floats.
`mean`	Mean curve on that grid (N points) — `stacked.mean(axis=0)`, computed in §3.1 but never kept before this addition.
`covariance`	$N \times N$ matrix (list of lists) — the one from §3.2.
`n_draws_used`, `grid_start`, `grid_end`, `created_at`	Traceability (number of draws retained, grid bounds used, ISO timestamp).

The choice to fully duplicate the data (rather than keep a mere reference to `project.last_raw_draws`/`last_results`) is deliberate: the latter are recomputed on demand from the currently loaded test and do not survive loading a different test, whereas a calibration library must precisely accumulate (grid, mean, covariance) pairs across several tests processed successively within the same session — process test 1, save the desired quantities, load test 2, repeat. An entry therefore remains usable even after the files of the test that produced it have been unloaded.

`quantity_key` deserves a remark: it is a purely advisory label, drawn from the dictionary `core/covariance.py::QUANTITY_LABELS` (27 entries: true/nominal strain, true/nominal stress, strain rate, temperature, displacement, load, input/output bar forces, the full set of triaxial quantities — radial stress, pressure, deviator, radial/volumetric/equivalent strains, with their "laser" and "in-loop" variants — and a custom sensor's calibrated signal). Nothing in `core/covariance.py` or `core/calib_library_adapter.py` enforces that an entry with `quantity_key="strain"` actually contains a strain: this field only pre-fills and filters the drop-down menus on the Bayesian Calibration page (chapter 9, §7.9, "Calibration tests" card), never to validate the data itself.

Persistence: `covariance_library` is saved/restored by `core/session.py` (`_covariance_library_to_dict`/`_covariance_library_from_dict`, registered in `to_dict`/`from_dict`), following the same scheme as `last_correction` (arrays → JSON-native lists, converted back to `np.ndarray` on read by the consuming code) — and not the "recomputed on demand, never persisted" scheme used for `last_results`/`last_raw_draws`, since this library is only ever populated by an explicit user action (never silently) and is specifically designed to survive across several test-loading cycles. It is not, however, tied to `core/setup_profile.py` (reusable configuration profiles concern the equipment/rig, not computed test data; applying a profile to an unrelated test must never silently drag along another test's library).

Saving/loading the library to a file (core/covariance_library_io.py) — feature added since the previous version of this document

Until now, `covariance_library` only survived through the session's automatic persistence (`core/session.py`, above) — there was no way to export it for backup, sharing between machines/colleagues, or reloading into a fresh project. A new module, `core/covariance_library_io.py`, fills this gap with the same file-pairing convention as `core/calib_project_io.py` (chapter 9, §7.6) and Results & Export's "Full test" export: *Save* asks for a `.csv` path and derives the paired `.json` path from it; *Load* asks directly for the `.json` path.

Unlike `calib_project_io.py`, the two files here are not complementary halves of one format: the **JSON** is the complete, authoritative, genuinely round-trippable data — every field of every entry, including the full grid/mean/covariance arrays, reusing the exact same shape `core/session.py` already persists a session with (via `_covariance_library_to_dict`/`_from_dict`, so a manually exported-then- reimported library is byte-for-byte the same shape a session restore produces) — while the **CSV** is a courtesy, human-readable summary table (one row per entry, metadata columns only: name/source/quantity/ size/provenance, no grid/mean/covariance numbers, which don't fit a flat table at all once entries have different grid lengths). Loading only ever reads the JSON; the CSV is written for review in a spreadsheet, it is neither required nor re-read by `import_covariance_library`.

Exposed functions:

- `export_covariance_library(csv_path, entries) -> json_path`: writes the full `.json` file (key `"entries"`, plus a `"format_version"` of 1) as well as the `.csv` summary, and returns the `.json` path actually written. Does not catch write errors (`OSError` propagates unchanged, as for every other export in the application).
- `import_covariance_library(json_path) -> list`: reads the `.json` file back into a list of `CovarianceLibraryEntry`. Tolerates both the wrapped shape this module writes (`{"entries": […]}`) and a bare list (e.g. a file obtained by directly serializing `core.session._covariance_library_to_dict(…)`). Raises `CovarianceLibraryIOError` (a `ValueError`) if the JSON is malformed, if the content is not a list, or if an entry is missing its `"name"` field.
- `unique_entry_name(existing_names, name) -> str`: appends a numeric suffix (`_2`, `_3`, …) until `name` no longer collides with `existing_names` — exactly the same collision policy already used by `ui/pages/page_covariance.py::_on_save_to_library` for a single manually saved entry, reused here so that an imported library never silently overwrites an entry already present in the current project.

Interface wiring (`ui/pages/page_covariance.py`): two new buttons on the library card, `_on_save_library_file` ("Save library to file..." button, disabled when the library is empty, opens a `QFileDialog.getSaveFileName` pre-filled with `calibration_library.csv`) and `_on_load_library_file` ("Load library from file..." button, `QFileDialog.getOpenFileName` filtered to `*.json`). On load, each imported entry that collides with one already present in `project.covariance_library` is renamed via `unique_entry_name` (never merged nor overwritten), and the card's status label summarizes the number of entries loaded along with any renamings.

A pure-stdlib module (`json`/`csv`) plus `core.project` — no PySide6 — so it is fully testable with real data, the same as `core/covariance.py` itself.

1.4 Uncertainty band of a custom sensor (custom_sensor.py)

Module: `core/custom_sensor.py::CustomSensor`, ported from `config_custom_sensors.py` (up to three auxiliary sensors: additional load cells, extensometers, thermocouples, etc., configurable in unlimited number on the "Custom Sensors" page). Unlike bar/specimen parameters, this band is not obtained by Monte-Carlo sampling but by a direct deterministic computation combining a relative (percentage) uncertainty and an absolute uncertainty declared by the user.

1.4.1 Calibrated data: multiplicative factor, low-pass filter, thermal correction

`core/custom_sensor.py::CustomSensor.calibrated_data()` chains, in this order, up to three optional/always-active transformations on the raw data `self.data`:

```
def calibrated_data(self) -> np.ndarray:
    factor = self.multiplicative_factor if self.multiplicative_factor else 1.0
    data = self.data * factor
    if self.lowpass_enabled and len(self.time) > 1:
        dt = float(np.median(np.diff(self.time)))
        data = apply_lowpass_filter(data, dt, self.lowpass_cutoff_hz)
    if self.thermal_correction_enabled:
        offset = self.thermal_offset_strain()
        n = min(len(data), len(offset))
        data = data.copy()
        data[:n] = data[:n] - offset[:n]
    return data
```

1. Multiplicative factor (always applied, 1.0 by default if null/empty): $y = f \cdot d$.
2. Low-pass filter (optional, `lowpass_enabled`): zero-phase Butterworth filter (`core/lowpass.py::apply_lowpass_filter`, cutoff frequency `lowpass_cutoff_hz`, 300 kHz by default), to remove high-frequency noise — an invalid time step or a cutoff beyond the sensor's Nyquist frequency simply leaves the data unfiltered.
3. Time-varying thermal output correction (optional, `thermal_correction_enabled`, §4.4 below): subtracts an apparent strain offset, point by point.

`calibrated_data()` remains pure/non-mutating: always recomputed from `self.data`, never written back into it — enabling/disabling either of the two optional corrections can therefore never accumulate a repeated effect.

1.4.2 Uncertainty band (uncertainty_band)

Via `core/custom_sensor.py::CustomSensor.uncertainty_band()` — the two uncertainty components (percentage and absolute) are added together to form the half-width of the band, exactly as in the original code (docstring: "combining the percentage and absolute uncertainty additively, exactly matching the original's three if sensor_unc_check == ... combinations"):

$$\Delta(t) = u_{abs} + y(t) \cdot \frac{u_{\%}}{100} \tag{1.7}$$

$$\text{lower bound}(t) = y(t) - \Delta(t), \qquad \text{upper bound}(t) = y(t) + \Delta(t) \tag{1.8}$$

```
def uncertainty_band(self) -> tuple[np.ndarray, np.ndarray]:
    data = self.calibrated_data()
    half_width = self.uncertainty_absolute + data * (self.uncertainty_percent / 100.0)
    return data - half_width, data + half_width
```

where `uncertainty_absolute` (u_{abs}, in the sensor's physical unit) and `uncertainty_percent` ($u_{\%}$, in %) are two independent sensor fields, which can be used alone or together (in the old code, three possible combinations depending on the checkbox: percentage only, absolute only, or both — here always summed, with zero values leaving the corresponding term neutral). Note that the relative part is computed on the calibrated `data` (after multiplicative factor, low-pass filter and

thermal correction if applicable), not on the raw data — the uncertainty band, and everything downstream of it (§4.3), therefore automatically reflects both optional corrections when they are active.

This half-width band $\Delta(t)$ is subsequently reused as a synthetic standard deviation ($\sigma \approx \Delta(t)$, `upper_bound - lower_bound over 2`, cf. `cell_defl_std = (cell_hi - cell_lo) / 2` in `ui/pages/page_covariance.py`) everywhere a custom sensor must be injected into a downstream Monte-Carlo computation (centered Gaussian perturbation of standard deviation $\Delta(t)$ used to generate synthetic draws, §3.4) — a treatment that implicitly assumes a normal law for this band, whereas `uncertainty_band()` itself makes no distributional assumption: it is a deterministic symmetric band, reinterpreted as a Gaussian 1σ only at the moment of its downstream use (`core/confinement.py`, `ui/pages/page_covariance.py::_sensor_synthetic_draws/_triax_draws`).

1.4.3 Time trimming (cut_from)

The module also provides `core/custom_sensor.py::CustomSensor.cut_from(cut_time)`, a time trimming tool (cuts everything before `cut_time` and re-zeros the time axis), unrelated to the uncertainty computation itself but used to prepare the sensor data before sampling.

1.4.4 Time-varying thermal output correction (optional)

Unlike the static thermal output correction of the CPG Calc tab (chapter 9 §1.8 — a single operating temperature, offset reported but not applied to the signal), this correction is applied directly inside `calibrated_data()` (§4.1): it is therefore automatically reflected in the Overview graph of the Custom Sensors page, in `uncertainty_band()` (§4.2), and in any downstream Monte-Carlo processing that consumes this sensor (sensor-specimen alignment, export, synthetic covariance draws).

Related `CustomSensor` fields: `thermal_correction_enabled`, `thermal_unit` ("C"/"F", purely indicative — does not enter the computation), `thermal_degree` (3 to 7), `thermal_coeffs` (coefficients a_i of the manufacturer polynomial, in microstrain/unit^i), `thermal_t_ref` (reference/bridge-balancing temperature), and an independently loaded temperature-vs-time profile (`thermal_time`, `thermal_temperature`).

`core/custom_sensor.py::CustomSensor.thermal_offset_strain()`:

```
def thermal_offset_strain(self) -> np.ndarray:
    if not self.thermal_correction_enabled or len(self.time) == 0:
        return np.zeros(len(self.time))
    if len(self.thermal_time) < 2 or len(self.thermal_temperature) < 2:
        return np.zeros(len(self.time))
    coeffs = self.thermal_coeffs[: self.thermal_degree + 1]
    t_profile = np.interp(
        self.time, self.thermal_time, self.thermal_temperature,
        left=self.thermal_temperature[0], right=self.thermal_temperature[-1],
    )
    offset_ue = np.array([
        thermal_output_strain(coeffs, self.thermal_t_ref, t) for t in t_profile
    ])
    return offset_ue * 1e-6         # microdéformation -> déformation
```

The supplied temperature-vs-time profile (potentially on its own time step/sampling — for example an adiabatic heating profile exported from a Processing tab, or an ambient thermocouple reading) is linearly interpolated onto `self.time`, clamped (not extrapolated) to the first/last known temperature outside its own covered range rather than set to NaN — an unknown

temperature just before/after the profile must fall back to the nearest known value, not blind the sensor signal at that instant. Each interpolated temperature is then passed through the same manufacturer polynomial as the static correction of the CPG Calc tab (`core/cpg_calc.py::thermal_output_strain`, Vishay Tech Note TN-504-1, eq. 3):

$$\varepsilon_{T/O}(t) = P(T(t)) - P(T_{ref}), \qquad P(T) = \sum_i a_i T^i \tag{1.9}$$

and the final corrected strain is $y_{corr}(t) = y(t) - \varepsilon_{T/O}(t) \times 10^{-6}$.

Explicit design decision — no thermal lag model: unlike a first proposal that was considered (a first-order 1D diffusion model to represent the time lag between the actual specimen temperature and the one actually seen by the gauge, whose order-of-magnitude analysis gave a time constant $\tau \approx L^2/\alpha$ on the order of 1 to 125 ms depending on the thickness of the gauge's bonding/backing — comparable to or greater than the duration of a typical SHPB test), the gauge is assumed to follow the supplied temperature profile directly, with no intermediate dynamics. This choice was made deliberately so that the same tool remains valid both for a fast dynamic test with adiabatic heating of the specimen (where a lag would be physically plausible but remains unmodeled here) and for a slow quasi-static test with ambient temperature drift (where a lag model would not make physical sense anyway, thermal equilibrium being reached continuously). The profile supplied as input must therefore already represent the temperature actually seen by the gauge.

As with the static correction of chapter 9 §1.8, this correction is physically meaningful only for a sensor whose calibrated signal is itself a strain-gauge signal (typically role `specimen_strain_gage` or `confinement_cell`) — the application does not check this and leaves the checkbox available for any sensor, without warning.

1.5 User-facing parameterization of the number of draws (N)

`core/project.py::Project.n_montecarlo` (default 500) is the number of draws N used by `Project.expand_montecarlo()` to size every `_pa` array, and is passed directly as the loop bound (`n = project.n_montecarlo`) to the three processing tabs (`ui/pages/page_process.py`). It is a single, shared field (not a per-module QS/SHPB/DI setting), appearing on the analysis identity sheet ("Test Setup"). The number of draws actually used in the means/standard deviations can be lower than N if some draws fail (exception raised in the physical pipeline, individually caught by `core/parallel.py::run_map` and simply ignored in the aggregation, with diagnostic message "Processed X/N draws" / "Draw d failed: …").

A complementary setting, `core/project.py::Project.uncertainty_sigma` (1, 2 or 3), multiplies the empirical standard deviation to set the width of the displayed/exported band ($\pm 1\sigma/\pm 2\sigma/\pm 3\sigma$), see §1.3 — a simple scale factor applied after the fact, not a parameter of the draw itself.

1.6 Summary of traceability points (file :: function)

Topic	File :: function/method
Supported distributions	`core/mc.py::Distribution` (NORMAL, UNIFORM)
Sampling a parameter	`core/mc.py::MCValue.sample(n, rng)`

Topic	File :: function/method
Sampling a group of parameters	`core/mc.py::MCParameterSet.sample_all(n, rng)`
Monte-Carlo expansion of a bar	`core/models.py::Bar.expand(n, rng)`
Viscoelastic (SLS) expansion of a bar	`core/models.py::Bar.expand_viscoelastic(...)`
Project-level trigger (IB/OB/ specimen)	`core/project.py::Project.expand_montecarlo()`
Number of draws N	`core/project.py::Project.n_montecarlo` (default 500)
Scale factor of the displayed band	`core/project.py::Project.uncertainty_sigma` (1/2/3)
Parallel/serial execution of the N draws	`core/parallel.py::run_map(func, items, n_workers, context, progress_callback, use_threads)`
Per-draw computation, SHPB	`core/process_shpb.py::_process_shpb_task`
Per-draw computation, DI	`core/process_di.py::_process_di_task`
Per-draw computation, QS (+ calibration coeffs.)	`ui/pages/page_process.py::_QSTab._on_run` (see `coeff_load_arr`/`coeff_dd_arr`)
Mean/standard-deviation aggregation by index	`ui/pages/page_process.py::_aggregate_quantities(per_draw)`
Storage of raw draws (for covariance)	`ui/pages/page_process.py::_store_raw_draws(project, result_key, per_draw)`
Interpolation onto common grid	`core/covariance.py::interpolate_draws_to_common_grid(...)`
Monte-Carlo covariance matrix	`core/covariance.py::compute_covariance_matrix(per_draw_arrays)`
Covariance UI page	`ui/pages/page_covariance.py::CovariancePage._on_compute`
Custom sensor uncertainty band	`core/custom_sensor.py::CustomSensor.uncertainty_band()`
Custom sensor calibrated data	`core/custom_sensor.py::CustomSensor.calibrated_data()`
Custom sensor time trimming	`core/custom_sensor.py::CustomSensor.cut_from(cut_time)`
Time-varying thermal correction (custom sensor)	`core/custom_sensor.py::CustomSensor.thermal_offset_strain()`
Triaxial synthetic draws (fallback)	`core/confinement.py::triaxial_synthetic_draws(...)`
Real triaxial draws (within the Monte-Carlo loop)	`core/confinement.py::per_draw_confinement_quantities(...)`, orchestrated by `ui/pages/page_process.py::_prepare_confinement(...)`
Real/synthetic selection per quantity (Covariance page)	`ui/pages/page_covariance.py::CovariancePage._triax_draws()`
Calibration library (named grid/mean/covariance)	`core/project.py::CovarianceLibraryEntry`, `Project.covariance_library`
Saving/removing a library entry	`ui/pages/page_covariance.py::CovariancePage._on_save_to_library`
Persistence of the calibration library	`core/session.py::_covariance_library_to_dict`/`_covariance_library_from_dict`

1.7 Another uncertainty paradigm: asymptotic least-squares covariance

Everything above in this chapter relies on RESAMPLING: replaying the physical pipeline N times with drawn inputs, then aggregating empirically (point-by-point mean/standard deviation, §1.3) or via an explicit covariance matrix (§3). This is the only uncertainty paradigm used by the SHPB/QS/Direct Impact/Confinement/Dispersion modules (chapters 2 to 8).

The "Crack & Wave Analysis" tool (chapter 10) introduces a second paradigm, unrelated to Monte-Carlo sampling: the asymptotic covariance of a least-squares fit, computed ONCE, at the fitted point, from the local curvature (linear or linearized) of the least-squares problem itself, rather than by repeated resampling. For an exact linear fit (`core/crack_k1c.py`'s `fit_k_single_frame`, chapter 10 §4.2, including its optional T-stress extension §4.5), this is the classical, exact OLS theory ($\sigma^2 \cdot (X^T X)^{-1}$); for a nonlinear fit (`core/xfem_dic.py`'s `fit_xfem_dic_frame`, chapter 10 §7.7), it is the same formula applied to the local linearization at the converged point (the Jacobian reassembled one last time) — the standard, textbook approach for reporting parameter uncertainty of a nonlinear least-squares fit, distinct from a Monte-Carlo or bootstrap confidence interval. A third method for computing K_I/K_{II} (the interaction integral, chapter 10 §5) falls under NEITHER of these two paradigms — it is not a least-squares fit at all, hence no covariance; its confidence is assessed via a sub-domain independence diagnostic rather than an error bar (chapter 10 §5.5/§5.7).

The two paradigms are not directly comparable term by term: least-squares covariance captures only the noise that directly affects the fit's own residual (grey-level noise for X-FEM-DIC, displacement-field noise for the ring fit), at a single instant, whereas the Monte-Carlo approach of this chapter propagates the uncertainty of physical INPUT PARAMETERS (dimensions, calibration, …) through an entire processing pipeline. See chapter 10 for the full detail of the least-squares methods (standard ring DIC fit, X-FEM-DIC), of the non-covariance method (interaction integral) and of their own limitations.

1.8 Limitations and assumptions explicitly flagged

1. Independence of input parameters: each `MCValue` is drawn independently of the others (no input correlation matrix) — a possible real physical correlation between parameters (e.g. diameter and density of the same machined bar) is not modeled.

2. Only two distributions: Normal and Uniform (`core/mc.py::Distribution`). No log-normal, triangular, etc. law.

3. The `dispersion` parameter has a different meaning depending on the law: standard deviation for Normal, half-range for Uniform — the same numeric `dispersion` value therefore does not represent the same uncertainty width depending on the chosen law.

4. Mean/standard-deviation aggregation by array index (§1.3), with no interpolation onto a common grid, unlike the covariance module (§3) which does interpolate explicitly. The two standard deviations (`<quantity>_std` of the general aggregation vs. the diagonal of the covariance matrix) can therefore differ slightly.

5. Biased vs. unbiased standard deviation: `ndarray.std()` (§1.3, general aggregation) uses `ddof=0` (biased/population estimator), whereas `numpy.cov()` (§3.2, covariance matrix) uses `ddof=1` (unbiased estimator) — a minor inconsistency between the two modules, negligible for large N.

6. Out-of-domain draws silently excluded: in `interpolate_draws_to_common_grid`, a draw whose curve does not cover the entire requested `[strain_start, strain_end]` range is simply dropped (no extrapolation) — the number of draws actually used in the covariance matrix (`n_used`) can therefore be significantly lower than the total number of successful draws (`n_total`), with no warning other than the `n_used`/`n_total` indication displayed in the page's status.

7. No normalized correlation matrix computed (§3.3): only the raw covariance is produced by the code; any correlation matrix presented in the documentation must be flagged as an external derivation ($\rho_{ij} = C_{ij}/\sqrt{C_{ii}C_{jj}}$), not a native output of the application.

8. Custom sensor uncertainty band = deterministic additive combination, not a Monte-Carlo draw in itself; it is treated as a Gaussian standard deviation only at the point where it is reused upstream of a downstream Monte-Carlo computation (triaxial test, Covariance page) — a normality assumption introduced only at that stage, not within `uncertainty_band()` itself.

9. `quantity_key` of a calibration library entry (§3.5) is a purely indicative label, never validated against the entry's actual content: nothing prevents saving a stress under `quantity_key="strain"`, and nothing in `core/calib_library_adapter.py` (chapter 9, §7.4) would detect it — only the filtering of the Bayesian Calibration page's dropdown menus relies on it.

10. Lost draws not reweighted/not replaced: a failing Monte-Carlo draw (exception in the physical pipeline) is simply removed from the mean/standard deviation (`n_used`/`n_total`) — no attempt to redraw, which can slightly bias the estimate if the failures are not independent of the region of the drawn parameter space (this point is not discussed in the code itself, it is an analysis observation).

11. Two levels of quality coexist for triaxial/confinement uncertainty, without one systematically replacing the other: the real per-draw computation (§3.4, chapter 5 §5.8) is only available if the confinement-cell sensor was cross-checked per draw (SHPB/Direct Impact, Monte-Carlo checkbox checked) or comes from the QS tab (always active, but with a fixed, unvaried cell signal — chapter 5 §7 limitation 10); failing that, the application silently falls back (with an explicit message) to the synthetic fallback envelope, which resamples an independent Gaussian rather than propagating the actual correlation. The quantities derived from laser velocimetry, for their part, are always computed via the synthetic route, regardless of the state of the other quantities for that same result.

2. Quasi-Static (QS) Reduction and Specimen Geometries

This chapter covers the "Load/Displacement Data" mode (slow test on a conventional testing machine, not Hopkinson bars) and the specimen geometries common to every loading mode of the application.

Source files analyzed:

- `core/qs_processing.py` — QS computation engine (`process_qs()`)
- `core/models.py` — specimen geometries (`Specimen`, `SPECIMEN_FIELD_ORDER`)
- `core/machine_compliance.py` — machine compliance correction
- `core/k_value_io.py` — reading/writing of the k/sk coefficients (notched specimen)
- `core/curve_cutting.py` — cutting the beginning of the curve
- `ui/pages/page_process.py`, class `_QSTab` (≈ lines 148–800) — for the user-facing vocabulary and to find exactly which arguments this page passes to `process_qs()` (see §6, limitations)

2.1 Overview of the QS computation chain

The `core/qs_processing.py` module explicitly states in its header that it is a direct port of the old code (`process_QS.py: process_QS()`), the physical formulas being "unchanged" ("The physics/formulas for each of the four test kinematics are unchanged from the original"). The central function is:

```
process_qs(*, type_check, shear_check=0, tens_check=0, comp_check=0,
              displacement, load, timet, specimen, d,
              coeff_load, coeff_dd, t0,
              k_value=None, sk_value=None, sk_check=False) -> QSResult
```

It processes a single Monte-Carlo draw d; the caller (the UI page, via `_process_qs_task` and `core/parallel.run_map`) loops over the n draws and aggregates the results (mean ± standard deviation, see `_aggregate_quantities()` in `ui/pages/page_process.py`).

`type_check` selects the reduction kinematics (independently of the detailed specimen geometry — see §6):

type_check	Mode
1	Compression (true strain)
2	Tension (true strain)
3	Notched specimen, Shear-Compression/Tension (fitted k coefficients)
4	Shear (hat-shaped, double or single)

The output is a `QSResult` object (dataclass) containing: `load`, `displacement` (signals corrected by the uncertainty factor, see §2.3), `stress`, `strain`, `strain_rate`, `temperature`, and — for `type_check` 1 and 2 only — `nominal_stress`, `nominal_strain`, `nominal_strain_rate`.

Upstream chain (before process_qs)

Before calling `process_qs()`, the `_QSTab` page (`ui/pages/page_process.py`, method `_on_run`, lines ≈720–790) performs, in this order:

1. Loading of the raw signals `timet`, `load`, `disp` (text files, one value per line);
2. Optional cutting of the beginning of the curve (§2.1, applied upstream via the "Cut curve beginning" button, hence before `_on_run`, on the arrays `self._timet_raw/_load_raw/_disp_raw`);
3. Optional machine compliance correction (§2.2), applied to `disp` just before the call;
4. Generation of the Monte-Carlo uncertainty factors `coeff_load_arr`, `coeff_dd_arr` (§2.3);
5. Call to `process_qs()` for each draw d.

2.2 Signal preprocessing

2.2.1 Cutting the beginning of the curve (core/curve_cutting.py: cut_curve_start)

Objective stated in the docstring: remove an initial "slack"/"toe" zone before the specimen is actually in contact, and re-zero the displacement.

Exact algorithm (`cut_curve_start(timet, load, displacement, cut_displacement)`):

1. The last index `idx` such that `displacement[idx] < cut_displacement` is found (if none, `idx = 0`);
2. `timet`, `load`, `displacement` are truncated starting from `idx`;
3. The displacement is re-zeroed:

$$d'_i = d_i - d_{idx} \tag{2.1}$$

The cut point is chosen interactively by the user on a load-displacement plot (`TimeWindowPicker`, "Cut curve beginning" card in `_QSTab`); only the start of the selected window matters.

2.2.2 Machine compliance correction (core/machine_compliance.py: apply_compliance_correction)

Principle (module docstring, ported from `process_t.py`'s `if corr_file.disp_comp_check == 1 and corr_file.load_comp_check == 1:` block): a rigid calibration specimen (or the load frame alone) is loaded over the same load range as the real test; its own displacement (the frame's own compliance) is recorded as a function of load. This load → machine-displacement curve is then used to subtract, at each load level of the real test, the portion of displacement due to the frame itself.

Exact algorithm (`apply_compliance_correction(load, displacement, calib_load, calib_displacement)`):

1. The calibration points `(calib_load, calib_displacement)` are sorted by increasing load (`np.argsort(calib_load)`, applied to both arrays in parallel to keep them synchronized).
2. **Recent fix, absent from the previous version of this document**: `scipy.interpolate.PchipInterpolator` requires a *strictly* increasing array of abscissas, whereas a set of calibration points may legitimately contain two equal loads (or, after the sort in the previous step, two consecutive non-strictly-increasing values). The code now explicitly discards, after sorting, any calibration point whose load does not strictly exceed that of the previously kept point:

```
keep_indices = [0]
for i in range(1, len(calib_load)):
    if calib_load[i] > calib_load[keep_indices[-1]]:
        keep_indices.append(i)
calib_load = calib_load[keep_indices]
calib_displacement = calib_displacement[keep_indices]
```

A code comment explicitly labels this filtering step *"Manual fix (preserved across future deliveries)"* — a point fix that future deliveries of the code must take care not to lose, rather than a behaviour documented as definitive in an upstream specification.

3. A monotone PCHIP interpolant (`scipy.interpolate.PchipInterpolator(calib_load, calib_displacement, extrapolate=True)`) is built on these now-deduplicated calibration points.
4. This interpolant is evaluated:
   - at the actually measured load signal: $\delta_{machine}(F) = \mathrm{PCHIP}(F)$;
   - at the calibration points themselves, to obtain a baseline $\delta_0 = \delta_{machine}(F_{calib,0})$ (first value after sorting/deduplication, i.e. the smallest retained calibration load).
5. The machine displacement is re-centred: $\delta'_{machine}(F) = \delta_{machine}(F) - \delta_0$.
6. The measured displacement is corrected and then re-zeroed to its own minimum:

$$d_{corr} = \left(d_{measured} - \delta'_{machine}(F)\right) - \min\left(d_{measured} - \delta'_{machine}(F)\right) \tag{2.2}$$

This function returns the corrected displacement, which replaces `disp` before the call to `process_qs()` (the load/displacement calibration files are stored in `project.qs.calib_load` / `project.qs.calib_displacement`).

2.2.3 Monte-Carlo measurement uncertainties (coeff_load, coeff_dd)

These factors are generated once per Monte-Carlo draw in `ui/pages/page_process.py` (`_on_run`, lines ≈766–768), not within `qs_processing.py` itself:

$$\mathrm{coeff_load}_d \sim \mathcal{N}\left(1, \left(\frac{\mathrm{clu}\%}{100}\right)^2\right), \qquad \mathrm{coeff_dd}_d \sim \mathcal{N}\left(1, \left(\frac{\mathrm{meu}\%}{100}\right)^2\right) \tag{2.3}$$

where `clu_pct` = "Cell Load Uncertainty (%)" and `meu_pct` = "Measurement Uncertainty (%) (displacement/extensometer)" are entered by the user (the "Measurement uncertainties" card of `_QSTab`). Each draw d therefore receives its own `coeff_load[d]` and `coeff_dd[d]`, applied multiplicatively to the raw load and displacement within `process_qs()` (see all the formulas below, where `coeff_load * load[...]` and `coeff_dd * displacement[...]` appear systematically). This mechanism is what produces the ±1 standard-deviation uncertainty band displayed on the "Result" plot.

2.3 The four reduction kinematics (type_check)

2.3.1 Compression and Tension (type_check == 1 or 2)

Code: `core/qs_processing.py`, block `if type_check in (1, 2):` (lines 56–78).

Initial resisting cross-section A_0 ("section"):

$$A_0 = \begin{cases} \pi \left(\dfrac{D_0}{2} \right)^2 & \text{if tens_check} = 1 \text{ or comp_check} = 1 \\ w_i \cdot th_i & \text{otherwise} \end{cases} \tag{2.4}$$

with `D_pa[d]` (diameter), `wi_pa[d]` (width), `thi_pa[d]` (thickness) — Monte-Carlo fields of `Specimen`. Note — see the limitation detailed in §6.1: on the QS tab, `comp_check`/`tens_check` are always 1 as soon as `type_check` is respectively 1 or 2, so the "rectangular" branch (`wi_pa * thi_pa`) is in practice never reached from that tab.

Nominal (engineering) strain:

$$\varepsilon_{n,i} = \text{clamp}\left(\frac{\text{coeff_dd} \cdot d_i}{1000\, L_0} \right), \qquad L_0 = \text{specimen.L_pa}[d] \tag{2.5}$$

The factor of 1000 in the denominator suggests that the displacement file is expected in mm while `L_pa` (initial length) is in m — this is a deduction from the code, not an explicit comment. The `clamp` function is `_clamp_nominal_strain()` (imported from `core/process_shpb.py`, shared with the SHPB/DI reduction): it bounds every value to the interval $[-0.99999, 0.99999]$ solely to avoid a `math domain error` in the logarithmic conversion that follows ($\log(1-v)$ as $v \to 1$, $\log(1+v)$ as $v \to -1$); per its docstring, this is a safety net that does not alter the result for values already within the valid domain.

Nominal (engineering) stress, in MPa (assuming load in N and lengths in m):

$$\sigma_{n,j} = \frac{\text{coeff_load} \cdot F_j}{A_0 \cdot 10^6} \tag{2.6}$$

True (logarithmic) strain:

$$\varepsilon_t = \begin{cases} -\ln(1-\varepsilon_n) & \text{compression (type_check} = 1) \\ \ln(1+\varepsilon_n) & \text{tension (type_check} = 2) \end{cases} \tag{2.7}$$

True stress — formula as implemented (not the "textbook" formula $\sigma_t = \sigma_n(1+\varepsilon_n)$, but a correction parameterized by the material's Poisson's ratio `nu_pa[d]`):

$$\sigma_t = \begin{cases} \sigma_n(1-2\nu\varepsilon_n) & \text{compression} \\ \sigma_n(1+2\nu\varepsilon_n) & \text{tension} \end{cases} \tag{2.8}$$

No comment in the code explains the physical origin of this factor ($1 \mp 2\nu\varepsilon_n$); it is carried over as-is from the ported code (`process_QS.py`). Flagged here as a point to verify rather than interpreted.

Strain rate (forward finite differences, both true and nominal):

$$\dot{\varepsilon}_{t,i} = \frac{\varepsilon_{t,i+1} - \varepsilon_{t,i}}{t_{i+1} - t_i}, \qquad \dot{\varepsilon}_{n,i} = \frac{\varepsilon_{n,i+1} - \varepsilon_{n,i}}{t_{i+1} - t_i} \tag{2.9}$$

The output signals `full_disp`/`full_load` are simply `coeff_dd·displacement` and `coeff_load·load` (the uncertainty-corrected signal, but without geometric scaling).

2.3.2 Notched specimen — Shear-Compression/Tension (type_check == 3)

Code: lines 80–124. This kinematics does not use a direct stress = F/A, strain = $\Delta L/L$ formula, but a polynomial of fitted coefficients `k_value` (± `sk_value`, see §4). This branch requires a correctly configured notched geometry: `_require_notched_geometry(specimen, d)` (defined in `core/process_shpb.py`, shared with SHPB/DI) raises an explicit error if `specimen.re_pa[d]` (notch radius) is not strictly positive, to avoid a silent division by zero — see detail in §6.4.

See §5 for the complete description of the equations (they depend on `k_value`/`sk_value`, hence are treated together with them).

2.3.3 Shear (type_check == 4)

Code: lines 125–136. `shear_check` distinguishes three sub-geometries.

Hat-shaped (`shear_check == 1`):

$$S_{as} = \pi(R_1 + R_2)(H_2 - H_1), \qquad l_0 = R_2 - R_1 \tag{2.10}$$

Double shear (`shear_check == 2`) or single shear (any other value, in particular `shear_check == 3`, or the default value 0):

$$S_{as} = n_{planes} \cdot th_i \cdot L_0, \qquad n_{planes} = \begin{cases} 2 & \text{shear_check} = 2 \text{ (double)} \\ 1 & \text{otherwise (single)} \end{cases}, \qquad l_0 = w_i \tag{2.11}$$

Shear (engineering) strain and shear stress, common to the three sub-geometries:

$$\gamma_i = \frac{\text{coeff_dd} \cdot d_i}{1000\, l_0}, \qquad \tau_j = \frac{\text{coeff_load} \cdot F_j / S_{as}}{10^6} \text{ (MPa)} \tag{2.12}$$

$$\dot{\gamma}_i = \frac{\gamma_{i+1} - \gamma_i}{t_{i+1} - t_i} \tag{2.13}$$

Important: on the QS tab (`_QSTab`), `shear_check` is never explicitly passed in the call context (`qs_context` in `ui/pages/page_process.py`, lines ≈773–781) — see §6.2. It therefore always takes its default value 0, which falls into the "single shear" branch ($n_{planes} = 1$): hat-shaped and double shear (factor 2) are, as the code currently stands, not reachable from the QS tab.

2.3.4 Adiabatic temperature model (Taylor-Quinney), common to all 4 modes

Code: lines 140–144, applied after the computation of `t_s`/`t_stress`, regardless of `type_check`.

$$c_T = \frac{\xi}{c_p \rho} \qquad (\xi = \text{specimen.xi_pa}[d],\ c_p = \text{specimen.cp_pa}[d],\ \rho = \text{specimen.rho_pa}[d]) \tag{2.14}$$

$$T_{i+1} = T_i + c_T \cdot \sigma_{t,i} \cdot 10^6 \cdot (\varepsilon_{t,i+1} - \varepsilon_{t,i}), \qquad T_0 = t_0 \text{ (project.thermal.initial_temperature_k)} \tag{2.15}$$

The factor of 10^6 converts the stress (stored in MPa in `t_stress`) back to Pa, so that $c_T \cdot \sigma \cdot d\varepsilon$ is dimensionally a temperature (with c_p in J/(kg · K) and ρ in kg/m^3, $c_p\rho$ is in J/(m^3 · K), so $\sigma\, d\varepsilon$ in Pa = J/m^3 indeed gives kelvins). `xi` ("Taylor-Quinney Coefficient") is the fraction of plastic work converted into heat — an adiabatic-transformation assumption (no comment in the code justifying this assumption, but it is implicit in the "Taylor-Quinney" name given in the source comment: "Adiabatic temperature-rise model (Taylor-Quinney), same for every kinematics").

2.4 Specimen geometries (core/models.py)

`SPECIMEN_FIELD_ORDER` (lines 115–127) defines, for each (mode, geometry) pair, the exact order of the fields expected in a specimen file — verified by the authors "line by line" against `test_type.py: import_spec_file()`. The geometries listed in `COMPRESSION_GEOMETRIES`, `TENSION_GEOMETRIES`, `SHEAR_GEOMETRIES` (lines 106–108) are the only ones offered in the UI.

For each geometry below: parameters (exact code names), the resisting cross-section/area formula A_0 actually used by `qs_processing.py`, and specifics.

2.4.1 Cylindrical (Compression)

- Fields (`SPECIMEN_FIELD_ORDER[("Compression","Cylindrical")]`): `rho`, `D`, `L`, `e`, `nu`, `xi`, `cp`
- `Specimen` attributes: `D_pa` (diameter, m), `L_pa` (initial length, m), `nu_pa`, `xi_pa`, `cp_pa`, `rho_pa`, `e_pa`
- $A_0 = \pi(D_{pa}/2)^2$ — see §3.1 (branch `comp_check=1`)
- True compression strain $\varepsilon_t = -\ln(1-\varepsilon_n)$, with $\varepsilon_n = \Delta L/L_0$ (mm→m scaling by /1000)
- Constant cross-section throughout the computation (A_0 does not evolve with strain; only the nominal→true conversion changes, there is no recomputation of a "true" section of the type $A = A_0/(1-\varepsilon_n)$ in this file)

2.4.2 Square/Rectangular (Compression)

- Fields: `rho`, `thickness`, `L`, `width`, `e`, `nu`, `xi`, `cp`
- Attributes: `thi_pa` (thickness), `L_pa`, `wi_pa` (width)
- $A_0 = w_i \cdot th_i$ (the "otherwise" branch of the §3.1 formula)
- Not reachable from the QS tab as the code currently stands (see §6.1): `comp_check` is always 1 there, which forces the cylindrical formula $\pi(D/2)^2$ even when the configured geometry is "Square/Rectangular". `D_pa` then never being set for a square specimen, this can produce a zero/incorrect cross-section.

2.4.3 Axisymmetric (Tension)

- Fields: `rho`, `D`, `L`, `e`, `nu`, `xi`, `cp`
- $A_0 = \pi(D_{pa}/2)^2$ (branch `tens_check=1`, §3.1)
- True tension strain $\varepsilon_t = \ln(1+\varepsilon_n)$

2.4.4 Flat (Tension, "Flat")

- Fields: `rho`, `thickness`, `L`, `width`, `e`, `nu`, `xi`, `cp`
- $A_0 = w_i \cdot th_i$

- Same remark as in §4.2: on the QS tab, `tens_check` is always 1 as soon as `type_check=2`, so this rectangular branch is likewise in practice never used by the QS tab (see §6.1).

2.4.5 Notched specimen (Shear-Compression/Tension, Notched)

- Fields (no sub-geometry, key `None`): `rho`, `D`, `e`, `nu`, `xi`, `cp`, `effective_zone`, `disp_yield`, `notch_radius`, `total_length`, `notch_angle`
- `Specimen` attributes: `D_pa` (diameter), `th_pa` ("effective zone thickness"), `re_pa` (notch radius, "notch_radius"), `NoA_pa` (notch angle, "notch_angle", unit declared as `deg` in `Specimen.new()`), `dy_pa` (displacement at the yield threshold, "disp_yield")
- No simple "$A_0 = \ldots$" formula here: stress and strain are computed by a polynomial fit depending on `k_value` (see §5), using `re_pa`, `NoA_pa`, `th_pa`, `D_pa`, `dy_pa`
- The `total_length` field exists in `SPECIMEN_FIELD_ORDER` but has no dedicated Monte-Carlo attribute in `Specimen` for this specific geometry (no `*_pa` field of `Specimen` corresponds to it explicitly for the Notched mode — not to be confused with `L_pa`, which is documented as specific to double/single shear). This parameter appears nowhere in `qs_processing.py`; its possible use elsewhere in the code was not verified as part of this study.

2.4.6 Hat-shaped (Shear)

- Fields: `rho`, `e`, `nu`, `xi`, `cp`, `H1`, `H2`, `H3`, `R1`, `R2`, `R3`, `total_length`
- Attributes used by `qs_processing.py`: `R1_pa`, `R2_pa`, `H1_pa`, `H2_pa` only — see §3.3: $S_{as} = \pi(R_1 + R_2)(H_2 - H_1)$, $l_0 = R_2 - R_1$
- `H3_pa`, `R3_pa` and `total_length` are not used by `qs_processing.py`. A search of the rest of the code shows that `H3_pa`/`R3_pa` (and `leL_pa`, `wiL_pa`) are indeed used in `core/process_shpb.py` and `core/process_di.py` (computation of an equivalent cross-section / a geometric ratio s_0 for wave-based SHPB/Direct Impact reduction), but not in the quasi-static reduction.

2.4.7 Double shear

- Fields: `rho`, `thickness`, `L`, `width`, `e`, `nu`, `xi`, `cp`, `head_width`, `total_length`
- Attributes used by `qs_processing.py`: `thi_pa`, `L_pa`, `wi_pa` (see §3.3): $S_{as} = 2\,th_i\,L_0$, $l_0 = w_i$
- `head_width` ($\rightarrow$ `wiL_pa`) and `total_length` ($\rightarrow$ `leL_pa`) exist in `Specimen` but are not used by `qs_processing.py` (used elsewhere, in `process_shpb.py`/`process_di.py`, for an equivalent-cross-section computation specific to wave-based reduction).
- Not reachable alone (with the $\times 2$ factor) from the QS tab, see §3.3 and §6.2.

2.4.8 Single shear

- Same fields as 4.7
- $S_{as} = th_i\,L_0$ (a single shear plane), $l_0 = w_i$
- Same remarks as 4.7 regarding `head_width`/`total_length`
- This is the only shear sub-geometry actually reached by the QS tab (by default, `shear_check` not passed $\rightarrow$ "otherwise" branch with factor 1), see §6.2.

2.5 k / sk coefficients for the notched specimen

2.5.1 File format (core/k_value_io.py)

- `load_k_value_file(path)` reads a text file, one value per line (blank lines ignored), and returns `(k_value, sk_check, sk_value)`.
- 6 mandatory lines: `k_value[0..5]` (`N_K = 6`).
- Optionally, a `Split = True`/`Split = False` line (case-insensitive regex `split\s*=\s*(true|false)`) followed by 7 lines `sk_value[0..6]` (`N_SK = 7`).
- Two accepted formats: 6 lines alone (no split — `sk_check` and `sk_value` then fall back to `None`, not an arbitrarily chosen default value, so that the caller does not touch the settings already displayed on screen), or 14 complete lines (6+1+7). An intermediate number of lines (7 to 13) is treated as an explicit error ("partial split/sk section").
- `save_k_value_file()` always writes the complete 14-line format.

2.5.2 Kinematic role of k_value (no split, sk_check false)

Exact code (`core/qs_processing.py`, lines 90–102), with d_i = `displacement[i]` (raw):

$$u_i = \frac{\text{coeff_dd} \cdot d_i - d_y}{\text{denom}}, \qquad \text{denom} = \frac{2000 \cdot r_e}{\cos(\theta_{notch})} \tag{2.16}$$

(r_e = `specimen.re_pa[d]`, θ_{notch} = `specimen.NoA_pa[d]`, d_y = `specimen.dy_pa[d]` = displacement at the yield threshold)

$$\varepsilon_{t,i} = k_0 u_i + k_1 u_i^2 + k_2 u_i^3 \tag{2.17}$$

$$\sigma_{t,i} = \frac{k_3(1 - k_4 \varepsilon_{t,i}) \cdot (\text{coeff_load} \cdot |F_i|)^{k_5}}{1000 \cdot th \cdot 1000 \cdot D} \tag{2.18}$$

(th = `specimen.th_pa[d]`, D = `specimen.D_pa[d]`, both in m; the product $1000\,th \cdot 1000\,D$ is therefore equivalent to $th_{mm} \cdot D_{mm}$).

Remarks explicitly carried by the code (comment lines 84–86): the offset `- dy` (displacement at the yield threshold) only comes into play in the no-split branch (`sk_check` false) — a comment notes that this offset is applied in that branch precisely to match the behavior of `process_QS.py`.

Point not documented by the code: the `k_value[5]` exponent applied to $|F_i|$ makes the stress-load relationship nonlinear (a power law), unlike a simple stress = force / section; the physical justification for this law does not appear in any comment in the file and was not found elsewhere in the files examined — to be confirmed with the original author if needed.

2.5.3 "Split fit" mechanism (sk_check true)

Exact code (lines 103–118): for each sample i, the set of coefficients used is chosen individually based on a raw displacement threshold (not corrected by `coeff_dd`) compared to `sk_value[6]`:

$$\text{coeffs}_i = \begin{cases} k_value & \text{if } d_i \le \text{sk_value[6]} \\ sk_value & \text{if } d_i > \text{sk_value[6]} \end{cases} \tag{2.19}$$

With these coefficients (generically denoted $k_0..k_5$ above, taken from `k_value` or `sk_value` as appropriate), the strain formula does not subtract `dy` in this branch:

$$u_i = \frac{\text{coeff_dd} \cdot d_i}{\text{denom}}, \qquad \varepsilon_{t,i} = k_0 u_i + k_1 u_i^2 + k_2 u_i^3 \tag{2.20}$$

and the stress follows exactly the same form as in §5.2 (with the coefficients of the set chosen for sample i).

Point explicitly worth flagging (potential ambiguity): the comparison threshold uses the raw displacement `dd[i]` whereas the strain formula itself uses the corrected displacement `coeff_dd * dd[i]`. The code does not comment on this choice; it may be intentional (a fixed physical threshold independent of the Monte-Carlo uncertainty) or a minor inconsistency inherited from the original — not resolved here, flagged as-is.

In both branches (split or not), the final result is re-zeroed afterward, regardless of the path taken:

$$\varepsilon_{t,i} \leftarrow \varepsilon_{t,i} - \varepsilon_{t,0} \tag{2.21}$$

(line 120, applied after the if/else, hence including the "split" case).

2.5.4 Consistency with the "notched" geometry (§4.5)

`_require_notched_geometry()` (imported from `core/process_shpb.py`, reused as-is by `process_qs`) explicitly refuses to start the computation if `specimen.re_pa[d]` is not strictly positive (> 0), with a clear message, precisely to prevent `denom` (the denominator above) from being zero.

This function's comment explains the targeted error scenario: the kinematics (`type_check`) is not tied to the project's overall test mode on the QS tab side (see §6.1) — if the QS tab's "Kinematics" dropdown was left on "3 — Notched" for a different test (simple compression, with no notch radius configured), without this check the computation would produce a cascade of `RuntimeWarning` "divide by zero"/"invalid value" instead of a clear error message; the check intercepts this case and displays an explicit message.

2.6 Limitations, assumptions and explicitly flagged ambiguities

2.6.1 On the QS tab, the detailed geometry (cylindrical vs. square/rectangular, axisymmetric vs. flat) is not taken into account

In `ui/pages/page_process.py`, `_QSTab._on_run()` (lines ≈773–781), the context passed to `process_qs()` sets:

```
comp_check=1 if type_check == 1 else 0,
tens_check=1 if type_check == 2 else 0,
```

i.e. `comp_check`/`tens_check` depend solely on the "Kinematics" menu (Compression/Tension/Notched/Shear), never on the geometry actually configured for the specimen (`project.geometry`, "Cylindrical"/"Square-Rectangular" or "Axisymmetric"/"Flat"). Yet in `process_qs()`, it is precisely `comp_check`/`tens_check` that selects the cross-section formula (§3.1): the $\pi(D/2)^2$ branch is taken in every compression/tension case on this tab, the $w_i th_i$ branch (rectangular/flat) never

being reached from it. This contrasts with the SHPB and Direct Impact tabs of the same file, which do derive `comp_check`/`tens_check` from `project.geometry` (e.g. line 1598: `comp_check=1 if project.geometry == "Cylindrical" else 2`).

Practical consequence: to reduce a quasi-static test on a square/rectangular or flat specimen with this code, `D_pa` (not configured for these geometries) would be used instead of `wi_pa`/`thi_pa`, which would produce an incorrect (potentially zero) cross-section. This point deserves verification/confirmation with the developers — it might be a known and accepted limitation of the QS tab (perhaps intended to be used only with cylindrical/axisymmetric specimens) rather than a bug, but nothing in the code or the UI help text says so explicitly.

2.6.2 On the QS tab, the shear sub-type is not selectable

The `qs_context` dict built by `_QSTab._on_run()` does not contain a `shear_check` key (unlike the SHPB/DI tabs, which derive it from `project.geometry` via `shear_map = {"Hat-shaped": 1, "Double shear": 2, "Simple shear": 3}`, lines 1604–1605 and 2838–2839). `process_qs()` then uses its default value `shear_check=0`, which — per the logic of the `type_check==4` branch (§3.3) — always falls into the "single shear" case (factor 1, $l_0 = w_i$). Hat-shaped and double shear (factor 2) are therefore, as the code currently stands, inaccessible from the QS tab, even if the user has configured a "Hat-shaped" or "Double shear" specimen on the specimen configuration page.

2.6.3 Notch angle: unit "deg" declared, but used directly inside cos()

`Specimen.new()` (`core/models.py`, line 189) declares the `notch_angle` parameter with unit `"deg"`. Yet `core/qs_processing.py` (line 88, identical in `process_shpb.py` and `process_di.py`) computes `math.cos(specimen.NoA_pa[d])` with no degree-to-radian conversion visible in the code (also searched for in `core/mc.py`, which handles Monte-Carlo sampling of the parameters, without finding a unit conversion there either). Python's `math.cos()` expects an argument in radians. This is flagged as a potential ambiguity/inconsistency between the unit label shown to the user and the computation actually performed — not resolved as part of this study; to be checked whether the user is in fact expected to enter the angle in radians despite the "deg" label, or whether this is a bug inherited from the original code.

2.6.4 Geometry fields declared but unused in QS

As detailed in §4.5, 4.6, 4.7: `total_length` (notched), `H3`, `R3`, `total_length` (hat-shaped), `head_width`, `total_length` (double/single shear) are all present in `SPECIMEN_FIELD_ORDER` and in the corresponding `Specimen` attributes, but none is read by `core/qs_processing.py`. `H3_pa`/`R3_pa`/`wiL_pa`/`leL_pa` are, however, indeed used in `core/process_shpb.py` and `core/process_di.py` for an equivalent-cross-section computation specific to wave-based reduction — they are therefore unrelated to the quasi-static chain.

2.6.5 No cross-section recomputation at large strain

None of the four kinematics (`core/qs_processing.py`) recomputes the resisting cross-section A_0 as a function of the current strain (no necking/reduced-section correction). A_0 is computed once from the specimen's initial dimensions (`specimen.*_pa[d]`) and remains constant across the whole curve for a given Monte-Carlo draw; only the nominal→true conversion (logarithmic, §3.1) accounts for the evolution of strain.

2.6.6 Temperature model: adiabatic assumption not justified in the code

As noted in §3.4, the adiabaticity assumption (no heat loss during the test) underlies the applied Taylor-Quinney model, but is not supported by any explicit comment beyond the name given to the formula in the source comment.

2.7 File / function summary

Computation	File	Function / method
QS orchestration (1 MC draw)	`core/qs_processing.py`	`process_qs()`
Multi-process worker	`core/qs_processing.py`	`_process_qs_task()`
Cutting the beginning of the curve	`core/curve_cutting.py`	`cut_curve_start()`
Machine compliance correction	`core/machine_compliance.py`	`apply_compliance_correction()`
Log-domain safety guard (true strain)	`core/process_shpb.py` (imported)	`_clamp_nominal_strain()`
Notched-geometry safety guard	`core/process_shpb.py` (imported)	`_require_notched_geometry()`
Reading k/sk file	`core/k_value_io.py`	`load_k_value_file()`
Writing k/sk file	`core/k_value_io.py`	`save_k_value_file()`
Specimen/geometry definition	`core/models.py`	`Specimen`, `Specimen.new()`, `Specimen.expand()`, `SPECIMEN_FIELD_ORDER`
MC uncertainty generation (coeff_load/coeff_dd)	`ui/pages/page_process.py`	`_QSTab._on_run()`
Building the process_qs call context	`ui/pages/page_process.py`	`_QSTab._on_run()` (variable `qs_context`)
Multi-draw mean/standard-deviation aggregation	`ui/pages/page_process.py`	`_aggregate_quantities()`

3. SHPB Analysis (Classic Two-Bar Hopkinson Bars)

Main source file: `core/process_shpb.py` (function `process_shpb()`, 834 lines). **Related files:** `core/toe_correction.py`, `core/wave_alignment.py`, `core/specimen_sensor_alignment.py`. **User-facing vocabulary** (cards, checkboxes): `ui/pages/page_process.py`, class `_SHPBTab`.

Warning from the code itself (reproduced verbatim)
The module docstring of `core/process_shpb.py` (lines 1-27) states:

> *"Ported as literally as possible (same variable names, same branch order, same formulas) specifically to minimize transcription risk in code this dense — readability was sacrificed for traceability against the original on purpose. […] NOT YET independently re-validated against real experimental data or the original GUI's output for the full matrix of options (heat × platen × punch × friction × inertia × viscoelastic bars × 4 kinematics) — only exercised with synthetic signals on the simplest option combination. Cross-check important results against the original application before relying on this for a real analysis, especially with several corrections enabled at once."*

This warning must be carried through into the user-facing documentation: the computation engine has not yet been independently validated against real data for the full matrix of option combinations.

3.1 Physical principle and general computation chain

3.1.1 Classic assumptions

The setup consists of two 1D elastic bars (Input Bar, IB, and Output Bar, OB) instrumented with strain gauges, sandwiching the specimen. The fundamental assumption of 1D wave theory (dispersion-free propagation in a semi-infinite elastic bar) relates the gauge strain to the force and particle velocity at the bar/specimen interface:

$$F(t) = E\,A_{\text{bar}}\,\varepsilon(t), \qquad v(t) = \pm C_0\,\varepsilon(t), \qquad C_0 = \sqrt{E/\rho}$$

where the sign depends on the direction of wave propagation. The code never explicitly refers to the specimen's "dynamic equilibrium" in its comments, but the whole construction (three independent signals ε_i, ε_r, ε_t, plus the `balance_quality_*` indicators, see §1.6) follows the classic "three-wave" approach: force and velocity are computed independently at both interfaces (input and output), without a priori assuming $F_{input} = F_{output}$; the user then checks this equilibrium a posteriori via the balance-quality indicators.

3.1.2 Exact convention used in the code — verified line by line

From `process_shpb()` (`core/process_shpb.py`, all kinematics branches, e.g. lines 502-509 for compression/tension):

$$F_{input}(t) = \text{sign} \cdot E_{mat,IB}\,\pi \left(\frac{D_{B,IB}}{2}\right)^2 (\varepsilon_i(t) + \varepsilon_r(t))\,k_{IB}$$

$$F_{output}(t) = \text{sign} \cdot E_{mat,OB}\,\pi \left(\frac{D_{B,OB}}{2}\right)^2 \varepsilon_t(t)\,k_{OB}$$

$$F_{average}(t) = \frac{F_{input}(t) + F_{output}(t)}{2}$$

$$V_{input}(t) = \text{sign} \cdot C_{0\,mat,IB}\,(\varepsilon_i(t) - \varepsilon_r(t))\,k_{IB}$$

$$V_{output}(t) = \text{sign} \cdot C_{0\,mat,OB}\,\varepsilon_t(t)\,k_{OB}$$

where k_{IB} = `IB.CPG_pa[d]` and k_{OB} = `OB.CPG_pa[d]` are the strain-gauge coefficients ("`SG_coeff`", historical alias CPG, see `core/models.py` line 44 and 81-85), and $E_{mat,\cdot}/C_{0\,mat,\cdot}$ are the "reference" modulus and wave speed at the interface (§2: identical to the bar's ambient properties unless a thermal-gradient correction is applied).

This is therefore exactly the classic "three-wave" formulation:

- F_{input} uses the two input-side waves ($\varepsilon_i + \varepsilon_r$, the "2-wave" formula);
- F_{output} uses only the transmitted wave (the "1-wave" formula);
- the nominal strain rate (see §1.4) combines the velocities at both interfaces, hence all three waves ε_i, ε_r, ε_t simultaneously.

The final stress choice (§3) then selects either F_{input} OR F_{output} (never $F_{average}$, which is only returned for diagnostic/plotting purposes in `SHPBResult.force_average`).

The sign (`sign`) is:

- `type_check in (1,2)` (compression/tension): `sign = -1` in compression, `+1` in tension (line 503);
- `type_check == 3` (notched shear-compression/tension) and `type_check == 4` (shear): `sign` depends on `options.tenscomp_check` (`-1` if "compression-like", `+1` if "tension-like", lines 385 and 612) — `type_check` is not used directly in these two cases.

3.1.3 Time windowing of the three waves ("Cutting waves")

Before any conversion, `process_shpb()` cuts the three time windows out of the bars' raw signals (lines 240-287):

- Wave speeds of the specimen material and (if enabled) the platen:

  $C_{0\,\text{specimen}} = \sqrt{E_{\text{specimen}}/\rho_{\text{specimen}}}$, $\qquad C_{0\,platen} = \sqrt{E_{platen}/\rho_{platen}}$ (if Platens enabled)

- Incident window: indices `idx_i1`, `idx_i2` = position of `xd1`, `xd2` (bounds chosen by the user on the plot) in the time vector, found by searching for the nearest sample below (`np.searchsorted(..., side="right") - 1`). Window length: `ti_pulse = idx_i2 - idx_i1`.

- Reflected window, shifted by the round-trip time gauge → bar end → gauge:

  $$t_{r,1} = xd1 + \frac{2\,PG_{IB}}{C_{0\,IB}} \left(+\frac{2\,L_{platen}}{C_{0\,platen}} \text{ if Platens}\right)$$

  where PG_{IB} = `IB.PG_pa[d]`, the gauge-to-bar-end distance on the specimen side.

- Transmitted window, shifted by the one-way travel time input gauge → specimen → output gauge:

$$t_{t,1} = xd1 + \frac{PG_{OB}}{C_{0\,OB}} + \frac{L_{\text{specimen}}}{C_{0\,\text{specimen}}} + \frac{PG_{IB}}{C_{0\,IB}} \left(+ \frac{2\,L_{platen}}{C_{0\,platen}} \text{ if Platens} \right) + \Delta t_{offset}$$

  where Δt_{offset} = `output_window_offset`, a manual residual realignment shift, typically produced by the wave-alignment tool (§6).

The three arrays `incident`, `reflected`, `transmitted` are extracted from the raw signals (`IB.signal`, `OB.signal`) and then multiplied by `SG_unc_d[d]`, a Monte-Carlo uncertainty factor distinct from the gauge coefficient `CPG_pa` itself (both are applied successively: `SG_unc_d[d]` here, then `CPG_pa[d]` later in the force/velocity formulas, and again explicitly at the end of the block to produce the final waves of `SHPBResult`). The exact meaning of this double multiplication (the physical nature of the signal before `CPG_pa` is applied, vs. the role of `SG_unc_d`) is not spelled out in the code comments — flagged here as a point to clarify rather than interpreted.

If the "Use dispersion-corrected signal" option (`disp_ela` or `disp_visco`) is active, these three arrays are overwritten by arrays already cut and corrected upstream (`AA[0]`, `AA[1]`, `AA[2]`, presumably from `core/dispersion.py`); the manual `output_window_offset` shift is then re-applied as an equivalent sample shift to the transmitted wave only (lines 275-287).

3.1.4 Strain rate, nominal strain, true strain (compression/tension case)

For `type_check in (1, 2)` (`core/process_shpb.py`, lines 502-604):

$$\dot{\varepsilon}_{nom}(t) = \frac{V_{input}(t) - V_{output}(t)}{L_{\text{specimen}}}$$

Nominal strain by explicit integration (rectangle rule, forward-Euler integration):

$$\varepsilon_{nom}(t_{i+1}) = \varepsilon_{nom}(t_i) + \dot{\varepsilon}_{nom}(t_i)\,(t_{i+1} - t_i), \qquad \varepsilon_{nom}(t_0) = 0$$

(code: loop `int_sr += n_sr[i]*(time1[i+1]-time1[i])`, lines 524-528), with a safety clamp at every step to $[-0.99999, 0.99999]$ (see §1.7).

Relative displacement (in mm): $u(t_i) = 1000\,L_{\text{specimen}}\,\varepsilon_{nom}(t_i)$.

True (logarithmic) strain, computed separately (and not simply derived from ε_{nom} via one single generic formula — the code explicitly branches):

$$\varepsilon_{true} = -\ln(1 - \varepsilon_{nom}) \text{ (compression)}, \qquad \varepsilon_{true} = \ln(1 + \varepsilon_{nom}) \text{ (tension)}$$

3.1.5 Nominal and true stress (compression/tension case)

Specimen cross-section:

$$S = \pi \left(\frac{D_{\text{specimen}}}{2} \right)^2 \text{ (cylindrical/axisymmetric)}, \qquad S = w_i\,e_i \text{ (square/flat, with } D_{eq} = 2\sqrt{S/\pi})$$

Nominal (engineering) stress, from the selected force F_F (see §3):

$$\sigma_{nom}(t) = \frac{F_F(t)}{S} \times 10^{-6} \text{ [MPa]}$$

True stress, linear Poisson correction (no strict incompressibility assumption):

$$\sigma_{true} = \sigma_{nom}\,(1 - 2\nu\,\varepsilon_{nom}) \text{ (compression)}, \qquad \sigma_{true} = \sigma_{nom}\,(1 + 2\nu\,\varepsilon_{nom}) \text{ (tension)}$$

where ν = `specimen.nu_pa[d]` (specimen Poisson's ratio).

3.1.6 Force-balance quality indicators ("balance quality")

Function `_balance_quality()` (lines 746-769), shared by all three kinematics blocks. Three indicators are computed:

- `balance_quality_force` (bq_1): ratio of cumulative work (force-displacement integral) output/input —
$$bq_1(t) = \frac{\int F_{output}\, du}{\int F_{input}\, du}$$
(cumulative rectangle integration over displacement).
- `balance_quality_waves` (bq_2): ratio of the time integrals of the waves —
$$bq_2(t) = \frac{\int \varepsilon_t\, dt}{\int (\varepsilon_i + \varepsilon_r)\, dt}$$
- `balance_quality_force_instant` (bq_3): instantaneous point-by-point ratio —
$$bq_3(t) = \frac{F_{output}(t)}{F_{input}(t)}$$

These three quantities quantify the deviation from the dynamic-equilibrium assumption ($F_{input} \approx F_{output}$, or equivalently $\varepsilon_i + \varepsilon_r \approx \varepsilon_t$); they do not alter the result, they are used solely as diagnostics.

3.1.7 Numerical safeguards (not physical corrections)

- `_clamp_nominal_strain()` (lines 115-133): clamps the nominal strain to $[-0.99999, 0.99999]$ before any logarithm is taken ($-\ln(1-\varepsilon)$ in compression, $\ln(1+\varepsilon)$ in tension), only on the side where the formula diverges, to avoid a `math domain error` crash across an entire batch of Monte-Carlo draws at once, rather than producing a finite (though extreme) value the user can judge.
- `_robust_dt()` (lines 192-217): the time step used for the strain-rate derivatives, computed as the median of consecutive gaps in `time1`, which makes it robust to the case where two consecutive samples share the same timestamp (a plain `time1[2]-time1[1]` would be fragile in that case).

3.2 Kinematics variants of the same computation engine

`process_shpb()` handles four test types (`options.type_check`), selected via the "Test mode" on the *Test Setup* page (not per individual processing tab — `_require_notched_geometry`, lines 136-163, explicitly warns that changing the test mode affects all tabs already processed).

3.2.1 Compression / tension (type_check = 1 or 2)

Detailed in §1. This is the "classic Hopkinson bars" case in the strict sense.

3.2.2 Notched shear-compression/tension — "Notched" (type_check == 3, lines 383-497)

Force/velocity identical in form to the §1.2 formulas (same expressions, `sign` driven by `tenscomp_check`). However:

- Displacement is integrated directly in mm from the velocity difference, without going through an intermediate strain:

$$u(t_{i+1}) = u(t_i) + (V_{input}(t_i) - V_{output}(t_i))\ dt \times 1000$$

- The "strain" ε_{notch} is obtained from a cubic calibration polynomial in normalized displacement, specific to the notched specimen's geometry, with user coefficients `k_value[0..5]` (loadable from a file, "Notched specimen k-coefficients" card):

$$d_{denom} = \frac{2000\, r_e}{\cos(\theta_{notch})}, \qquad x = \frac{u - 1000\, d_y}{d_{denom}}$$

$$\varepsilon_{notch} = k_0 x + k_1 x^2 + k_2 x^3$$

$$\sigma_{notch} = \frac{k_3\,(1 - k_4\,\varepsilon_{notch})\,|F_F|^{k_5}}{1000\, e_{zone} \times 1000\, D_{\text{specimen}}}$$

  where r_e = `specimen.re_pa[d]` (notch radius), θ_{notch} = `specimen.NoA_pa[d]` (notch angle), d_y = `specimen.dy_pa[d]` (displacement at the elastic threshold), e_{zone} = `specimen.th_pa[d]` (effective-zone thickness). If the "Split fit at a displacement threshold (sk)" option (`options.sk_check`) is enabled, the coefficient set switches from `k_value` to `sk_value` once $u >$ `sk_value[6]`.

- There is no separate nominal/true strain or stress: `n_s = t_s`, `n_stress = t_stress` (explicit aliases, lines 487-497). The strain rate `t_sr` is computed by a direct finite difference (not `_robust_dt`, but `time1[i+1]-time1[i]` at every step).

3.2.3 Pure shear — "Shear" (type_check == 4, lines 609-723)

Effective length l_u = `H2_pa - H1_pa` ("hat"-shaped specimen) or `wi_pa` (double/single). Force/velocity formulas identical in form to §1.2 (`sign` from `tenscomp_check`), L_{specimen} replaced by l_u for strain rate and displacement.

Effective shear area S_{AS} depending on geometry (`options.shear_check`):

$$S_{AS} = \pi(R_1 + R_2)(H_2 - H_1) \text{ (hat)}, \qquad S_{AS} = 2\, e_i\, L \text{ (double)}, \qquad S_{AS} = e_i\, L \text{ (single)}$$

$$\sigma_{nom} = \sigma_{true} = \frac{F_F}{S_{AS}} \times 10^{-6}, \qquad \varepsilon_{true} = \varepsilon_{nom} \text{ (no logarithmic transform)}$$

This is confirmed independently by `core/toe_correction.py::infer_true_strain_relation()` (see §5.4), whose docstring explicitly states: "shear: true = nominal (SHPB shear processing uses no log transform at all — see process_shpb.py, t_s = list(n_s))".

3.3 Choice of the force used for stress (force_check)

A single parameter, `options.force_check` (the "Force used for stress" dropdown on the *Processing* page, default = output), controls, in each of the three kinematics branches, which force is used to compute the nominal stress, via a small shared selector (`_select_stress_force()`, lines 84-95):

```
def _select_stress_force(F_input, F_output, F_average, force_check: int):
    if force_check == 1:
        return F_input
    if force_check == 3:
        return F_average
    return F_output

# ... in each of the three branches (e.g. line 420 for the notched case,
# line 547 for compression/tension, line 650 for shear):
FF = _select_stress_force(F_input, F_output, F_average, options.force_check)
```

Three possible values for `force_check` (a three-choice dropdown as of this version — previously binary, only `F_input`/`F_output` were selectable):

1. `F_input` ("Input-bar force") — force on the input-bar side only.
2. `F_output` ("Output-bar force", the UI default) — force on the output-bar side only; this is the "one-wave" formula, the one most used in practice once the input/output dynamic equilibrium has been verified (contextual help in `ui/pages/page_process.py`).
3. `F_average` ("Average force (input & output)") — the average $(F_{input} + F_{output})/2$, already computed in all three branches (`F_average` is always built, whether this choice is selected or not — see below) but previously never selectable for the stress computation, only exposed for diagnostics (`SHPBResult.force_average`). A common alternative when the equilibrium between F_{input} and F_{output} is only imperfect or only briefly reached, since it weights both ends of the specimen equally rather than relying on a single one (contextual help, `ui/pages/page_process.py`, lines 1086-1092).

$F_{average} = (F_{input} + F_{output})/2$ is still computed in all three branches regardless of the `force_check` choice (it always feeds `SHPBResult.force_average`, for plotting/diagnostics even when not used for the stress) — the only thing that changed with this evolution is that it can now also serve as F_F for the $\sigma_{nom}/\sigma_{true}$ computation, on the same footing as F_{input}/F_{output}, without duplicating the logic in each of the three kinematics branches (hence the pass through `_select_stress_force()`, factored out once).

3.4 Available physical corrections

3.4.1 Bar thermal-gradient correction ("heat_check", heated specimen)

Purpose (module docstring): compensate for the fact that a heated specimen conducts heat into the bar ends, locally changing the elastic modulus (and hence the wave speed) near the gauge relative to the rest of the bar, which stays at ambient temperature.

Function `_bar_gradient_wave_speed_modulus()` (lines 84-112), called separately for IB and OB with the gauge temperature (`T_gages_value = (gage_temp_ib, gage_temp_ob)`, in K, user-entered on the "Bar-temperature-gradient parameters" card) and the specimen's initial temperature `T0_pa[d]` (*Test Setup* page):

Branch `type_bars_check` $\in \{1, 2\}$ (analytical RTE model) — uses two material constants `Tm_value`

`= (Tm0, Tm1):`

$$RTE(T) = \frac{1 - \dfrac{T}{T_{m0}} \exp\left(T_{m1}\left(1 - \dfrac{T_{m0}}{T}\right)\right)}{1 - \dfrac{293}{T_{m0}} \exp\left(T_{m1}\left(1 - \dfrac{T_{m0}}{293}\right)\right)}$$

$$E_{mat} = RTE(T_0) \cdot E_{bar,amb}, \qquad C_{0\,mat} = \sqrt{E_{mat}/\rho_{bar}}, \qquad E_{surface} = RTE(T_{gage}) \cdot E_{bar,amb}$$

$RTE(T)$ is a relative modulus factor, normalized to 1 at 293 K (reference ambient temperature), a function of two material parameters (T_{m0}, T_{m1}) whose exact physical meaning (e.g. melting temperature/shape parameter) is not documented in the code — to be clarified by the documentation author if needed.

Branch `type_bars_check == 3` (imported, temperature-dependent material tables): $E_{surface}$ = value of $e(T)$ interpolated at `gage_temp` (table `material_table_ib/_ob`), $E_{mat} = e(T_0)$, $\rho_{mat} = \rho(T_0)$, $C_{0\,mat} = \sqrt{E_{mat}/\rho_{mat}}$ (falling back to the bar's ambient values if the table does not cover the requested temperature).

Special case of heat + platens together (lines 332-337): when `plat_check` is also active, `E_mat`/`C0_mat` (on both sides) are replaced by the platen's properties — the code comment explains this: "the "material on the other side of the bar interface" is the platen here".

Transmission coefficients at the "heated bar surface" $\leftrightarrow$ "reference material" interface (ambient bar, or platen if `plat_check` + `heat_check`):

$$C_{0B} = \sqrt{E_{surface}/\rho_{bar}}, \qquad S_B = \pi\left(\frac{D_{bar}}{2}\right)^2$$

$$T_{BP} = \frac{2\,C_{0B}\,E_{surface}\,S_B}{C_{0B}\,E_{mat}\,S_B + C_{0\,mat}\,E_{surface}\,S_B}, \qquad T_{PB} = \frac{2\,C_{0\,mat}\,E_{mat}\,S_B}{C_{0B}\,E_{mat}\,S_B + C_{0\,mat}\,E_{surface}\,S_B}$$

(computed separately on the input side `TBP_input`/`TPB_input` and the output side `TBP_output`/`TPB_output`.)

Corrected waves (lines 350-352):

$$\varepsilon_i \mathrel{*}= T_{BP,input}, \qquad \varepsilon_r \mathrel{/}= T_{PB,input}, \qquad \varepsilon_t \mathrel{/}= T_{PB,output}$$

The modulus used further down in the punch correction (§4.3) is always the bar's own modulus after thermal correction (`EB_punchI, EB_punch0 = e_mat_ib, e_mat_ob`, line 357), even if `plat_check` substituted `E_mat_IB`/`E_mat_OB` with the platen's properties for the force/velocity formulas — indentation is indeed assumed to take place at the bar surface itself.

3.4.2 Platen correction (plat_check)

An important point, not trivial to see from the code: contrary to what one might expect, enabling "Platens" alone (without `heat_check`) does not change the wave amplitude through a dedicated transmission coefficient. In the absence of a thermal correction, `E_mat_IB`/`E_mat_OB`/`C0_mat_IB`/`C0_mat_OB` remain equal to the bar's own ambient properties (lines 319-323), strictly identical to the case with no correction at all (lines 359-361). Platens then only intervene through three channels:

1. Wave travel time (windowing, §1.3): `2*plat_Len/plat_C0` is added to the time shift of the reflected wave and of the transmitted wave, for the round trip through/crossing of the platen(s).

2. Punch diameter (§4.3): `punch_diameter_I/punch_diameter_O = plat_Dia[d]` (instead of the specimen diameter) when `plat_check` is active — indentation is then modeled against the platen rather than directly against the specimen.
3. An additive term in the punch correction (§4.3) representing the platens' own elastic compliance.

Only when `heat_check` is enabled together with `plat_check` are the platen's properties (modulus, wave speed) substituted into the force/velocity formulas themselves (§4.1). The diameter used for `F_input`/`F_output` (`DB_punchI`, `DB_punchO`) and for the `nuB_punch`/`EB_punch` properties of the punch correction is always that of the bar itself, never the (smaller) platen diameter (lines 289-312): using the platen diameter would underestimate `F_input`/`F_output` as soon as platens are enabled, which would produce an artificially lower stress.

3.4.3 Punch/indentation correction (punch_check)

Compensates for local (contact-indentation) deformation at the bar-specimen (or bar-platen) interfaces, which is not representative of the specimen's own strain.

Shared elliptic geometric factor, `_elliptic_punch_factor()` (lines 772-786), a function of the diameter ratio $m = D_{\text{specimen or platen}}/D_{bar}$ (must be < 1, otherwise an explicit `ValueError` is raised: "Punch/platen correction requires the punch or platen diameter to be larger than the specimen diameter"):

$$E(m) = \int_0^{\pi/2} \sqrt{1 - m^2 \sin^2 x}\, dx, \qquad K(m) = \int_0^{\pi/2} \frac{dx}{\sqrt{1 - m^2 \sin^2 x}}$$

$$H_x(m) = 2 - \left(m + \frac{1}{m}\right) E(m) - \left(m - \frac{1}{m}\right) K(m)$$

(complete elliptic integrals of the first/second kind, computed numerically by `scipy.integrate.quad`.)

Punch compliance per interface (input I and output O sides):

$$K_p = H_x(m) \cdot \frac{16}{3\pi^2} \cdot \frac{1 - \nu_{bar}^2}{D_{\text{punch}}\, E_{bar}}$$

with ν_{bar}, E_{bar} always the bar's own properties (§4.2), D_{punch} = the platen diameter if `plat_check`, otherwise the specimen's diameter (or equivalent diameter).

Strain correction subtracted from the measured strain, specimen area $A = \pi D_{\text{specimen}}^2/4$:

$$p_s = 10^6 \left(K_{p,I} + K_{p,O}\right) \sigma_{\text{ref}} \frac{A}{L_{\text{specimen}}} - \underbrace{2 \frac{\sigma_{\text{ref}}\, L_{platen}}{E_{platen}\, L_{\text{specimen}}}}_{\text{if Platens active}}$$

(three near-identical implementations depending on kinematics: `_punch_correction_generic` for compression/tension (lines 802-812), `_punch_correction_notched` for the notched specimen (lines 789-799, uses K_p with separate `punch_diameter_I/O`), `_punch_correction_shear` for shear (lines 815-825, denominator D_{eq} instead of `punch_diameter_I/O` in K_p). σ_{ref} = the nominal stress (`n_stress`), except in the notched case where the already-computed `t_stress` is passed instead.

Application: the correction is then subtracted from the nominal strain (`n_s -= p_s` in compression/tension and shear, `t_s -= p_s` for the notched specimen), and the true stress and strain are then recomputed from the corrected nominal strain.

A point worth flagging explicitly: in the compression/tension branch (lines 565-574), whenever `punch_check` is active, the code systematically recomputes

```
t_s.append(-math.log(1 - n_s[i]))
t_stress.append(n_stress[i] * (1 - 2 * specimen.nu_pa[d] * n_s[i]))
```

i.e. only the compression formula $(-\ln(1-\varepsilon),\ 1-2\nu\varepsilon)$, regardless of `type_check` (compression or tension) — whereas the branch without the punch correction (lines 560-563 and 549-550) correctly branches between compression and tension. This is an inconsistency between the two computation paths, to be verified/confirmed with the authors rather than an interpretation of my own: as it stands, enabling the punch correction on a tension test appears to erroneously apply the compression true stress/strain formulas.

In the shear branch (lines 666-676), the punch correction additionally introduces a division by 2 of the corrected strain (`t_s.append(n_s[i] / 2)`) that has no equivalent in the uncorrected path (`t_s = list(n_s)`, no factor of 2) — also worth flagging as an asymmetric treatment between the two paths.

3.4.4 Inertia correction (inertia_check)

Compensates for the specimen's own radial/axial inertia effects at high strain rates. Implemented separately per kinematics, with the same functional form in all three cases:

$$c_1 = \frac{\rho_{\text{specimen}}\, D_{eq}^2\,(s_0^2 - 3/16)}{12}, \qquad c_2 = \frac{3\,\rho_{\text{specimen}}\, D_{eq}^2}{64}$$

$$\sigma_{inertia}(t) = \frac{c_1\left(\dot{\varepsilon}(t)^2 + \varepsilon(t)\ddot{\varepsilon}(t)\right)}{10^6} + \frac{c_2\,\ddot{\varepsilon}(t)}{10^6}, \qquad \sigma_{true}\ -= \sigma_{inertia}$$

where s_0 is a slenderness ratio (L/D or equivalent depending on geometry) and $\ddot{\varepsilon}$ the second derivative of strain ("strain-rate-rate", noted `n_srsr`/`t_srsr` in the code, computed by finite differencing the strain rate). The correction is applied only to the true stress, never to the nominal stress.

- Compression/tension (lines 598-604): D_{eq} = diameter (or square/flat equivalent diameter) of the specimen, $s_0 = L_{\text{specimen}}/D_{eq}$, derivatives computed with `_robust_dt()`.
- Notched specimen (lines 474-486): its own geometry — a fixed 45° angle $\cos(\pi/4)$ (independent of the actual notch angle `NoA_pa`, worth flagging), $S = D_{\text{specimen}}/\cos(\pi/4) \times e_{zone}$, $D = 2\sqrt{S/\pi}$, $s_{0i} = 2(r_e/\cos(\pi/4))/D$, the same c_1, c_2 formulas with D in place of D_{eq}, derivatives computed by direct finite difference (not `_robust_dt`).
- Shear (lines 711-723): $D_{eq,2}$ recomputed from the exact geometry of the setup (hat/double/single), $s_0 = H_3/D_{eq}$ or L_{head}/D_{eq} depending on the setup.

3.4.5 Friction coefficient (friction)

A friction coefficient per Monte-Carlo draw (`friction[d]`), applied identically in all three kinematics branches whenever it is non-zero (e.g. lines 578-583 for compression/tension):

$$s_0 = \frac{L_{\text{specimen}}}{D_{\text{specimen}}}, \qquad m = 1 - \frac{f}{3\,s_0}$$

$$\sigma_{nom}\ *= m$$

$$\sigma_{true} \mathrel{*}= m \text{ (in place)}, \qquad \sigma_{friction} = \frac{f}{3\,s_0}\,\sigma_{true} \text{ (after } m\text{)}, \qquad \sigma_{true} \mathrel{-}= \sigma_{friction}$$

A point worth flagging: because `friction_stress` is computed after `t_stress` has already been multiplied by m, the factor actually applied to the final true stress is

$$\sigma_{true}^{final} = m \cdot \sigma_{true}^{orig} \cdot \left(1 - \frac{f}{3s_0}\right) = m^2 \cdot \sigma_{true}^{orig}$$

i.e. m^2 rather than m, whereas the nominal stress is only reduced by a factor of m (it is never touched by `friction_stress`). This is a direct observation from reading the code, not a re-interpretation: `friction_stress` is exposed separately in `SHPBResult.friction_stress` but uses the already-corrected value of `t_stress`, not the original value.

The usual physical justification for this type of correction is the apparent reduction in measured stress due to the radial confinement induced by friction at the bar/specimen interfaces (barreling effect); the code does not document a bibliographic reference for the exact form $m = 1 - f/(3s_0)$.

3.5 "Toe" correction of the curve (core/toe_correction.py)

Corrects a common seating artifact (faces not perfectly flat/parallel, residual play closing up) — a slow, low-stiffness initial rise before the curve reaches its true elastic slope. Independent of any issue with the waves' time alignment (module docstring, lines 1-12).

3.5.1 Automatic detection of the linear region — detect_linear_region() (lines 51-89)

- A sliding window of width `window = max(5, n * window_fraction)` (12% of the curve's total length by default), searched only within the first `search_fraction` (60% by default) of the points — the elastic zone is assumed to occur early, before yielding/nonlinearity.
- For each window position: a degree-1 `numpy.polyfit` linear regression, with R^2 computed by `_fit_line()` (lines 31-37):
$$R^2 = 1 - \frac{\sum(y - \hat{y})^2}{\sum(y - \bar{y})^2}$$
- Selection: among the windows reaching `r_squared_threshold` (0.995 by default), the one with the maximum R^2 is kept (not the steepest one — the docstring explicitly states that slope alone is an "easily fooled" criterion). If no window reaches the threshold, the fallback is the window with the maximum R^2 among all candidates.
- In the degenerate case (a curve too short to produce even one candidate window), a regression is run over the entire curve.

3.5.2 Manual adjustment — fit_linear_region() (lines 40-48)

The same linear regression, but over a range `[start_index, end_index)` chosen manually by the user (drag-and-drop on the plot, "Toe (Foot) Correction" card).

3.5.3 Origin realignment — apply_toe_correction() (lines 92-102)

The intersection point of the fitted line with the zero-stress axis gives the new strain zero:

$$\varepsilon_{offset} = -\frac{\text{y-intercept}}{\text{slope}}, \qquad \varepsilon_{\text{corrected}} = \varepsilon - \varepsilon_{offset}$$

followed (if `trim=True`, the default behavior) by removal of all points with negative corrected strain (the "toe" proper).

3.5.4 Recomputing true strain after realignment — infer_true_strain_relation( ) / true_strain_from_nominal() (lines 105-148)

The correction above applies directly to the nominal strain (a simple linear realignment). Because the true-strain/nominal-strain relation is not linear (compression/tension), true strain must be recomputed after realignment, not simply translated. `infer_true_strain_relation()` automatically determines (from data already produced by `process_shpb()`) which of the three relations applies, by comparing at a representative point (the midpoint of the nonzero indices):

- compression: $\varepsilon_{true} = -\ln(1 - \varepsilon_{nom})$
- tension: $\varepsilon_{true} = \ln(1 + \varepsilon_{nom})$
- shear: $\varepsilon_{true} = \varepsilon_{nom}$ (no transformation — consistent with §2.3)

`true_strain_from_nominal()` then applies the identified relation to the corrected nominal strain (with a safety `np.clip` at 0.999999 on the compression side, the same logic as `_clamp_nominal_strain` in `process_shpb.py`).

3.5.5 Consistency with raw Monte-Carlo draws and confinement quantities (ui/pages/page_process.py)

§5.3 describes `apply_toe_correction()`, the `core/toe_correction.py` primitive that shifts/trims a single (strain, stress) pair. Historically, the "Apply" button on the "Toe (Foot) Correction" card only called this primitive once, on the already-aggregated mean curve (`project.last_results["SHPB"]`): every array field with the same length as the strain had the same boolean mask subtracted from it, but none of the individual Monte-Carlo draws were touched — neither `project.last_raw_draws["SHPB"]` (which feeds the *Covariance Analysis* page), nor the per-draw confinement quantities (§5.8 of the Confinement chapter), which live inside each raw draw rather than in the aggregate. Once the correction was applied, these two sources therefore remained offset by a number of points equal to the "toe" that had been removed — a misalignment reported by the user and fixed by the following mechanism.

Current mechanism: `_on_toe_apply()` (`_SHPBTab`, lines 2314-2385; `_DITab`, lines 3575-3625, rigorously identical logic) no longer touches the aggregate directly. It re-applies the same adjusted offset to each individual Monte-Carlo draw, via `_apply_toe_correction_to_draws()` (module function, lines 147-212), and then re-aggregates:

```
new_per_draw, n_removed = _apply_toe_correction_to_draws(
    self._last_per_draw, strain_key, strain_offset, relation
)
quantities = _aggregate_quantities(new_per_draw)
self.project.last_results["SHPB"] = {"label": "SHPB", **quantities, "toe_n_removed": ...}
_store_raw_draws(self.project, "SHPB", new_per_draw)
```

`self._last_per_draw` is the list of per-draw dictionaries already produced by the last Monte-Carlo run (one dict per successful draw, exactly the ones passed to `_aggregate_quantities()`/ `_store_raw_draws()` at the end of processing) — no new simulation is re-run: the toe correction does not alter in any way the wave-propagation physics (dispersion, force balance, etc.) already computed for each draw by `process_shpb()`; it is a pure change of origin on the strain axis applied after the fact, so re-applying this change of origin to already-computed results is exact, not approximate — see also the Confinement chapter, §5.8.

`_apply_toe_correction_to_draws(per_draw, strain_key, strain_offset, relation)` processes each draw independently:

- Its own crossing point `(draw_strain - offset) >= 0` serves as the cut mask, applied to every field of that same draw sharing the length of its strain array — force, temperature, displacement, `sensor_{idx}_signal`, and the `confinement_*` quantities (§5.8 of the Confinement chapter) when they have been computed for that draw. The fields coming from the result dataclass (`SHPBResult`) are plain Python lists rather than `ndarray` for most quantities (only the quantities built on the UI side — confinement, cross-referenced sensors — are already `ndarray`): the function treats both types indiscriminately (`isinstance(values, (np.ndarray, list))`), a broader condition than the historical `isinstance(values, np.ndarray)` filter used by the aggregate correction (§5.3), which only applied to arrays already stacked by `_aggregate_quantities()`.
- True strain is recomputed (not simply translated) via `true_strain_from_nominal()`, using the relation from `infer_true_strain_relation()` inferred once on the current aggregated curve and reused for every draw (the compression/tension/shear relation is a fixed property of the test's kinematics, not something that varies draw to draw — see §5.4).
- A draw whose curve never crosses the adjusted offset (an outlier Monte-Carlo realization) is removed from the set rather than left uncorrected — mixing an uncorrected draw into an otherwise corrected aggregate would silently bias the mean/standard deviation.
- The function never mutates its arguments: it returns a new list of new dictionaries, which lets the caller keep the original `self._last_per_draw` as an exact backup for "Reset" (`_on_toe_reset()`, lines 2395-2413/3630; restores `self._last_per_draw`, then regenerates `last_results`/`last_raw_draws` using the same aggregation functions as above, guaranteeing a bit-for-bit restoration identical to the last Monte-Carlo run).
- The function also returns the median (not the count removed from the first draw alone, which is more sensitive to noise specific to a single draw) of the number of points trimmed from the start across all surviving draws — accumulated application after application into an integer `toe_n_removed`, stored as-is (not an array, hence never touched by the aggregation functions) at the root level of `project.last_results["SHPB"]`. This is the counter that `ui/pages/page_results_export.py::_refresh_confinement()` reads back to keep aligned the raw signal (not cross-referenced per draw) of a confinement-cell or laser sensor, when that sensor does not have its "Include this cut in the Monte-Carlo loop" checkbox ticked — see the Confinement chapter, §5.5 and §7, for this second part of the correction (`_refresh_confinement`, lines 922-995).

Several successive applications of "Apply" (without a "Reset" in between) chain onto `self._last_per_draw` as it stands at that moment — as the historical aggregate correction already did — while `self._toe_per_draw_backup` (assigned once, on the first application following a Monte-Carlo run) always keeps the original raw set for "Reset".

3.6 Wave-alignment verification tool (core/wave_alignment.py)

Purpose (module docstring): predict what the transmitted wave should look like if the specimen responded purely elastically, from the measured incident+reflected wave, using only 1D mechanical impedance and the classic transmission coefficients at each interface crossed: `Bar_in -> [Platen_in] -> Specimen -> [Platen_out] -> Bar_out`. Explicitly limited: ignores internal reverberations within the specimen (assumed thin) and any inelastic response — a first-order prediction, useful for temporally realigning the measured transmitted wave without any assumption about the specimen's actual (nonlinear) behavior.

3.6.1 Mechanical impedance and transmission coefficient

`mechanical_impedance()` (lines 25-29):

$$Z = \rho\, C_0\, A, \qquad C_0 = \sqrt{E/\rho}, \qquad A = \pi(D/2)^2$$

`transmission_coefficient()` (lines 32-35), the classic stress/force transmission coefficient from a medium of impedance Z_1 into a medium of impedance Z_2:

$$T_{1\to 2} = \frac{2Z_2}{Z_1 + Z_2}$$

3.6.2 Theoretical transmitted wave — predicted_transmitted_wave() (lines 45-73)

Chain of impedances crossed:

chain $= [Z_{IB}, Z_{OB}]$ (no platens), chain $= [Z_{IB}, Z_{platen}, Z_{\text{specimen}}, Z_{platen}, Z_{OB}]$ (with platens)

Total transmission coefficient, the product of the coefficients at every successive interface of the chain:

$$T_{total} = \prod_k T_{k\to k+1}$$

Theoretical transmitted wave (in force):

$$F_{t,\text{predicted}}(t) = T_{total} \cdot (F_i(t) + F_r(t))$$

where $F_i + F_r$ is the incident+reflected wave already converted to force (supplied as input by the caller on the UI side).

3.6.3 Cross-correlation auto-alignment — best_alignment_shift() (lines 88-129)

Since the local "ripples" of the theoretical wave are merely a scaled copy of the ripples of the incident+reflected wave (and generally do not match the measured transmitted wave's own ripples — a different physical origin: dispersion, reverberation, actual material response), a raw point-by-point comparison does not work; only the overall shape (rise/plateau/decay) is comparable. Method:

1. Smoothing of both signals by an edge-padded moving average (`_smooth_edge_padded()`, lines 76-85), window `window = max(1, n // 8)`.

2. Zero-mean, unit-energy normalization of each smoothed signal.

3. Exhaustive search over the integer shift `shift` in $[-N_{max}, +N_{max}]$ (`max_shift_samples`, bounded by `n-1`) that maximizes the dot product (normalized cross-correlation) between the shifted measured signal (`np.roll`, zero-padded at the exposed edges) and the theoretical signal:

$$\text{shift}^\star = \arg\max_{\text{shift}} \langle \text{measured}_{\text{norm}}(\cdot - \text{shift}),\ \text{theoretical}_{\text{norm}} \rangle$$

The shift found can then be applied either manually (slider) or automatically, then "committed" into `output_window_offset` (§1.3) to actually affect the processing.

3.7 Time alignment of a sensor mounted directly on the specimen (core/specimen_sensor_alignment.py)

A sensor glued directly onto the specimen ("Custom Sensors" page, "mounted directly on the specimen" option) does not start recording at the same physical instant as the bar gauges: the loading wave still has to travel the distance gauge → specimen (and possibly cross platens, then travel half the specimen's length if the sensor is assumed to be positioned at mid-length).

Formula (module docstring, reproduced in the code, lines 15-24, and functions on lines 39-60):

SHPB case — `specimen_sensor_delay_shpb()`:

$$\Delta t = \frac{x_{gauge,IB}}{C_{0\,IB}} + \frac{L_{platens}}{C_{0\,platens}}\ \text{(if Platens)} + \frac{L_{\text{specimen}}}{2\,C_{0\,\text{specimen}}}$$

broken down into three separately exposed terms (`DelayBreakdown`: `bar_term`, `platen_term`, `specimen_term`, `total`):

- `bar_term = input_gage_position / c0_input_bar`
- `platen_term = platens_length / c0_platens` (0 if no platens)
- `specimen_term = sample_length / (2 * c0_sample)`

Direct Impact case — `specimen_sensor_delay_direct_impact()`: a strictly identical formula, but using the output bar's position/wave speed (`output_gage_position`, `c0_output_bar`) instead of the input bar's — consistent with the Direct Impact configuration (a single instrumented bar, on the output side).

`sample_wave_speed()` (lines 63-66): $C_{0\,\text{specimen}} = \sqrt{E_{\text{specimen}}/\rho_{\text{specimen}}}$, used when the specimen does not have a directly measured wave speed.

The contextual help in `ui/pages/page_process.py` (lines 944-964) notes that this delay is computed once, from the average (non-Monte-Carlo) properties of the bars/platens/specimen — the Monte-Carlo uncertainty on this travel time is deemed negligible and is not propagated separately.

3.8 Summary of limitations, assumptions and points of ambiguity found in the code

1. No validation performed (module docstring of `process_shpb.py`): the engine has not been independently validated against real data, nor compared to the original application's output, for the full matrix of options; only tested on synthetic signals in the simplest option combination.

2. Platens without thermal correction: introduces no dedicated wave-transmission coefficient — only the time windowing and the punch correction are affected (§4.2). This can surprise a user expecting an impedance-type effect similar to the wave-alignment tool (§6).

3. Punch correction + tension (compression/tension, §4.3): the code path activated by `punch_check` applies the compression true-stress/strain formula even for a tension test — an inconsistency between the path with and without the punch correction, to be confirmed.

4. Punch correction + shear: introduces a division by 2 of the strain that does not exist in the uncorrected path.

5. Friction correction: the factor actually applied to the final true stress is m^2 (not m), due to the order of operations in the code, whereas the nominal stress is only reduced by a factor of m.

6. Inertia correction, notched case: uses a fixed 45° angle ($\cos(\pi/4)$), independent of the actual notch angle configured by the user (`specimen.NoA_pa`).

7. Viscoelastic branch (`disp_visco`): the complex frequency-domain modulus is computed solely from the input bar's rheological parameters (`IB.e11`, `IB.e22`, `IB.n22`); the output bar reuses the resulting real modulus (`EE11`) to compute its own wave speed — an implicit assumption that both bars share the same rheology, not explicitly flagged in the code as a deliberate assumption.

8. Viscoelastic branch, frequency/sample correspondence: the modulus $E(f)$ is indexed frequency by frequency directly onto the time samples (`EE11[i]` associated with the i-th time sample), rather than through a full forward/inverse Fourier transform of the signal — a non-standard treatment for a dispersion correction, to be validated.

9. Meaning of `SG_unc_d`: applied in addition to the gauge coefficient `CPG_pa` during wave cutting, without the exact nature of this double weighting (a distinct Monte-Carlo uncertainty vs. redundancy) being commented in the code.

10. Global test-mode choice: `type_check` (compression/tension/notched/shear) is determined by the global "Test mode" setting on the *Test Setup* page, not per tab — `_require_notched_geometry()` explicitly warns of the risk of processing a test with the geometry of a test mode different from the one actually set at computation time.

11. `toe_n_removed` is a median, not an exact per-draw count (§5.5): since the toe correction now re-cuts each Monte-Carlo draw individually, the number of points trimmed from the start can differ slightly from draw to draw (noise specific to that draw's bar/specimen property realization). The `toe_n_removed` counter, used by `page_results_export.py::_refresh_confinement()` to realign the raw (not per-draw cross-referenced) signal of a confinement/laser sensor, keeps the median of this count across all surviving draws — a good representative as long as this inter-draw variation stays small (the common case), but an approximation, not an exact draw-by-draw correspondence for this particular fallback path (the per-draw cross-referenced path, §5.8 of the Confinement chapter, remains exact since it cross-references THAT draw's actual signal directly).

12. The `toe_n_removed` realignment of the raw signal only corrects a leading offset, not a difference in sampling rate: the combination of a raw confinement/laser sensor with the stress-strain curve in `_refresh_confinement()` remains, apart from this now-corrected offset, a simple position-to-position truncation to the shortest common length (see the Confinement chapter, §7, point 8) — it still implicitly assumes both signals share the same sampling step, an assumption independent of the toe correction and not addressed by it.

4. "Direct Impact" (DI) Analysis

Central file: `core/process_di.py` (functions `process_di()` and `_process_di_task()`). **Options:** dataclasses `DIOptions` / result `DIResult` (same file).

4.1 Physical principle and differences from classic SHPB

In "classic" SHPB (*Split Hopkinson Pressure Bar*), the specimen is sandwiched between an input bar (instrumented, incident wave + reflected wave) and an output bar (instrumented, transmitted wave): the input-side impact velocity is derived from the incident wave measured on the input bar. In Direct Impact (DI), there is only one instrumented bar: the output bar (OB, Output Bar). The specimen is directly impacted (by a projectile/flyer) and its incident velocity is measured independently (typically by a laser gate — see `velocity_c` in the code, loaded from an "Incident velocity file" in the UI's Direct Impact tab). There is therefore:

- no incident or reflected wave to process (no wave separation on an input bar);
- no instrumented input bar: no notion of an "input/output bar choice" for the force computation (unlike what SHPB with two bars can allow); force can only be computed on the output-bar side;
- input-side kinematics that is directly the measured velocity profile (`velocity_c`), multiplied by a calibration factor, instead of being derived through 1D wave theory from a gauge signal.

The file `core/process_di.py` is explicitly described in its docstring as "the single-output-bar counterpart of process_shpb.py: instead of an incident+reflected wave pair on an input bar, the specimen's incident velocity comes from a separate measurement (velocity_c, e.g. a laser gate signal), and only the transmitted wave on the output bar is used." It is "ported as literally as possible (same variable names, branch order and formulas) to keep it traceable against the original line-for-line" — i.e. it faithfully reproduces a historical script (`process_DI.py` / `process_DI()`), including its structural choices.

4.2 Pipeline overview (process_di())

Signature: `process_di(options, OB, specimen, velocity_c, time_pulse, xd1, xd2, d, SG_unc_d, f_inc_velocity, T0_pa, friction, k_value=None, sk_value=None, plat_nu=None, plat_rho=None, plat_emod=None, plat_Dia=None, plat_Len=None, Tm_value=None, T_gages_value=None, material_table_ob=None, AA=None)` — called once per Monte-Carlo draw `d` (via `_process_di_task()`, the entry point used by `core/parallel.run_map` from the Direct Impact tab in `ui/pages/page_process.py`).

Steps, in the order they appear in the code:

1. Cutting the signals (window $[xd1, xd2]$ on the transmitted wave; a window of the same duration on the incident velocity) — `process_di()`, "Cutting waves" block.
2. Optional substitution with the dispersion-corrected signal (`options.disp_ela` or `options.disp_visco`) — the raw cut transmitted signal is replaced by `AA[0]` (the result of the *Dispersion Corrections* page).

3. ”Platen” correction, if `options.plat_check`.
4. Bar thermal-gradient correction (”heat”), if `options.heat_check` — reuses `core/process_shpb.py::_bar_gradient_wave_speed_modulus()`.
5. Branch by test type (`options.type_check`: 1 compression, 2 tension, 3 notched Shear-Compression/Tension, 4 shear): integration of velocity/transmitted wave → displacement, nominal strain, force, nominal stress, true stress.
6. Optional additional corrections, applied within each branch: punch (`punch_check`), friction (`friction[d] != 0`), inertia (`inertia_check`).
7. Adiabatic heating (temperature), computed after the branch, common to all 4 test types.
8. Final formatting of the displacement (mm) and return of a `DIResult`.

`options.type_check` is not chosen directly in the *Processing* tab: it is derived from the global, project-wide ”Test mode” set on the *Test Setup* page (`_test_mode_options()` in `ui/pages/page_process.py`) — an explicit warning about this is raised by `_require_notched_geometry()` (see §9).

4.3 Cutting the signals and the incident velocity

4.3.1 Time windowing

```
xdt = xd2 - xd1
i_i1 = nearest_index(time_pulse, xd1)
i_i2 = nearest_index(time_pulse, xd2)
i_it = nearest_index(time_pulse, xdt)
time1 = time_pulse[i_i1:i_i2] - xd1
transmitted = OB.signal[i_i1:i_i2] * SG_unc_d[d]
velocity = velocity_c[0:i_it]
```

— `core/process_di.py::process_di()`, ”Cutting waves” block, relying on `core/process_shpb.py::_index_of_nearest` (search for the point of the time vector nearest to the target; replaces an older exact-equality lookup `time_pulse.index(round(x,6))`, which broke as soon as the time step became sub-microsecond — see the docstring of `_index_of_nearest`).

Key points:

- `time1` is re-indexed to start at $t = 0$ at `xd1` (the start of the transmitted-wave window).
- `transmitted` = the output bar's raw signal, multiplied by `SG_unc_d[d]` — a per-Monte-Carlo-draw multiplicative uncertainty factor, but fixed at `1.0` for every draw by the UI (`sg_unc_d = np.ones(n)` in `_DITab` of `ui/pages/page_process.py`): the Monte-Carlo mechanism exists on the `process_di()` side but is not actually activated (no uncertainty distribution is really attached to it) from the current Direct Impact tab.
- `velocity = velocity_c[0:i_it]`: the incident-velocity array (already realigned by the UI, see §6) is truncated to the first `i_it` samples — `i_it` being the index of `time_pulse` nearest to the window duration `xdt = xd2 - xd1`, not an absolute instant. This implicitly assumes that `velocity_c` is sampled at the same time step as `time_pulse`, and that its sample at index 0 corresponds exactly to $t = xd1$ (this is precisely what the UI's ”cut”/”shift” mechanism is meant to guarantee — see §6).

4.3.2 Dispersion-corrected signal (disp_ela / disp_visco)

```
if options.disp_ela or options.disp_visco:
    transmitted = AA[0][:]
```

— `process_di()`, right after the cutting step. If the user has checked ”Use dispersion-corrected signal” on the DI tab (`disp_ela = use_corrected` in `ui/pages/page_process.py::_DITab`), the raw cut transmitted signal above is entirely replaced by `AA[0]`, i.e. `result.transmitted_corrected` produced by the *Dispersion Corrections* page (computed for Direct Impact mode, i.e. `corr["direct_impact"] == True`). Note that `time1` itself still derives from `xd1`/`xd2`/`time_pulse` as in §3.1 — it is the time vector of the window chosen on the Corrections side (`corr["xd1"]`, `corr["xd2"]`, `corr["time_pulse"]`) that must remain consistent, otherwise `time1` and `AA[0]` would become desynchronized (controlled by the UI, which reuses `corr["xd1"]`/`corr["xd2"]` as `xd1`/`xd2` whenever ”Use dispersion-corrected signal” is checked).

4.3.3 ”Platen” correction

```
if options.plat_check:
    plat_C0 = sqrt(plat_emod[d] / plat_rho[d])
    SBO    = pi * (OB.D_pa[d] / 2) ** 2
    S_pla = pi * (plat_Dia[d] / 2) ** 2
    TPB_output = 2*plat_C0*plat_emod[d]*S_pla / (OB.C0_pa[d]*plat_emod[d]*S_pla + plat_C0*
    OB.e_pa[d]*SBO)
    transmitted = transmitted / TPB_output
    E_mat_OB, C0_mat_OB, rho_mat_OB = plat_emod[d], plat_C0, plat_rho[d]
else:
    rho_mat_OB = OB.rho_pa[d]
    E_mat_OB = C0_mat_OB = None          # defined further below
```

— `process_di()`, ”Platen correction” block. In mathematical notation, with $S_{OB} = \pi(D_{OB}/2)^2$ (output-bar cross-section) and $S_{plat} = \pi(D_{plat}/2)^2$ (platen cross-section):

$$T_{PB} = \frac{2\,C_{0,plat}\,E_{plat}\,S_{plat}}{C_{0,OB}\,E_{plat}\,S_{plat} + C_{0,plat}\,E_{OB}\,S_{OB}}, \qquad \varepsilon^{\text{corrected}}_{transmitted} = \frac{\varepsilon_{transmitted}}{T_{PB}}$$

This is an impedance transmission coefficient at the bar/platen interface (1D wave theory across a two-material interface): the strain measured on the bar is divided by this coefficient to recover the actual strain at the platen/specimen interface. If `plat_check` is active, the ”interface material” properties (`E_mat_OB`, `C0_mat_OB`, `rho_mat_OB`) used further down for the force and displacement become the platen’s, not the bar’s — subject to the interaction with `heat_check` described in §3.5.

4.3.4 Bar thermal-gradient correction (”heat”)

```
if options.heat_check:
    gage_t_ob = T_gages_value[1] if T_gages_value else 293.0
    e_mat_ob, c0_mat_ob, e_surface_ob = _bar_gradient_wave_speed_modulus(
        OB, d, gage_t_ob, T0_pa[d], options.type_bars_check, Tm_value, material_table_ob)
    if not options.plat_check:
        E_mat_OB, C0_mat_OB = e_mat_ob, c0_mat_ob
    C0OB = sqrt(e_surface_ob / OB.rho_pa[d])
    SBO    = pi * (OB.D_pa[d] / 2) ** 2
```

```
    TPB_output = 2*C0_mat_OB*E_mat_OB*SBO / (C0OB*E_mat_OB*SBO + C0_mat_OB*e_surface_ob*SBO
    )
    transmitted = transmitted / TPB_output
    EB_punch = e_mat_ob        # always the bar's own modulus, heat-corrected

if not options.heat_check:
    E_mat_OB, C0_mat_OB = OB.e_pa[d], OB.C0_pa[d]
```

— `process_di()`, ”Bar-temperature-gradient (heat) correction” block, relying on `core/process_shpb.py::_bar_gradient_wave_speed_modulus()` (a shared function, not duplicated, between SHPB and DI, as the file's module docstring explicitly states: ”The bar-temperature-gradient helper is shared with core/process_shpb.py since it's already per-bar.”).

`_bar_gradient_wave_speed_modulus(bar, d, gage_temp, t0, type_bars_check, tm_value, table)` computes, depending on `type_bars_check`:

- 1 or 2 (RTE model — *Relative Thermo-Elastic*):

$$RTE(T) = \frac{1 - \dfrac{T}{T_m^{(0)}} \exp\left(T_m^{(1)}\left(1 - \dfrac{T_m^{(0)}}{T}\right)\right)}{1 - \dfrac{293}{T_m^{(0)}} \exp\left(T_m^{(1)}\left(1 - \dfrac{T_m^{(0)}}{293}\right)\right)}$$

  applied once at the gauge temperature T_{gage} (gives RTE_{bar}, the ”bar-surface” modulus, for the transmission-coefficient correction), and once at the specimen's initial temperature T_0 (gives RTE_{mat}, the ”interface material” modulus, for the force/strain conversion):

$$E_{mat} = RTE_{mat} \cdot E_{OB}, \qquad C_{0,mat} = \sqrt{E_{mat}/\rho_{OB}}, \qquad E_{surface} = RTE_{bar} \cdot E_{OB}$$

- 3 (imported property table as a function of temperature): interpolation of E and ρ in `material_table_ob` at temperatures T_{gage} (for $E_{surface}$) and T_0 (for E_{mat}, ρ used in $C_{0,mat} = \sqrt{E_{mat}/\rho(T_0)}$).

Then, in `process_di()`: the transmission coefficient at the (hot/cold) bar-platen/specimen interface,

$$T_{PB} = \frac{2\,C_{0,mat}\,E_{mat}\,S_{OB}}{C_{0\,surface}\,E_{mat}\,S_{OB} + C_{0,mat}\,E_{surface}\,S_{OB}}, \qquad C_{0\,surface} = \sqrt{E_{surface}/\rho_{OB}}$$

divides `transmitted` in turn. `EB_punch` (used only in the punch correction, §5.1) is always set to `e_mat_ob` (the bar's modulus, thermally corrected), whether or not a platen is present — explicit code comment: ”EB_punch is always the bar's own (heat-adjusted) modulus, regardless of plat_check”.

4.3.5 Caution — plat_check / heat_check interaction on E_mat_OB / C0_mat_OB

Reading the order of the blocks precisely (§3.3 then §3.4), one observes the following: the final lines

```
if not options.heat_check:
    E_mat_OB, C0_mat_OB = OB.e_pa[d], OB.C0_pa[d]
```

execute unconditionally whenever `heat_check` is false, including when `plat_check` is true. Yet it is `plat_check` (block §3.3, line `E_mat_OB, C0_mat_OB, rho_mat_OB = plat_emod[d], plat_C0, plat_rho[d]`) that had set `E_mat_OB`/`C0_mat_OB` to the platen's properties. Consequence: if `plat_check=True` and `heat_check=False`, `E_mat_OB` and `C0_mat_OB` are overwritten with the bar's own properties (`OB.e_pa[d]`, `OB.C0_pa[d]`), while `rho_mat_OB` remains the platen's (`plat_rho[d]`, never overwritten by this block). These three variables then feed into the force computation (`F_output = (DB_punch/2)² · rho_mat_OB · C0_mat_OB² · output_m`, compression/tension/shear branches — §4.1/4.2) and into the displacement/strain computation (`... - C0_mat_OB · int_eps ...`), which amounts — in this specific case (`plat_check=True`, `heat_check=False`) — to mixing the platen's density with the bar's own modulus and wave speed, rather than a consistent single-material triplet (E, ρ, C_0). This point is flagged as observed in the code — the documenting agent cannot determine, from reading this file alone, whether this is deliberate behavior or a porting oversight; it is flagged explicitly for verification.

4.3.6 Diameter/properties used for F_output vs. the punch correction

```
DB_punch = OB.D_pa[d]                                    # always the BAR's diameter, never
    the platen's
nuB_punch, EB_punch = OB.nu_pa[d], OB.e_pa[d]            # (EB_punch overwritten with
    e_mat_ob if heat_check, §3.4)
bar_diameter       = OB.D_pa[d]
punch_diameter = plat_Dia[d] if options.plat_check else None   # None -> specimen/D_eq
    diameter at the punch
```

Two code comments spell out the physical reasoning behind this:

> *"F_output is computed from strain measured by a gage mounted ON THE BAR, using 1D wave theory (F = E_bar * A_bar * strain) — that relationship holds at the bar's own cross-section regardless of what's beyond the bar's end, so DB_punch (the area term) must always be the bar's own diameter, whether or not a platen is used."*
>
> *"Punch (indentation) correction always happens at the BAR surface […] so nuB_punch/EB_punch (used ONLY in the punch correction, never in F_output above) are always the bar's OWN properties; only the "punch diameter" (specimen vs platen) changes […]"*

In other words: in this implementation, the cross-section used for $F = E \cdot A \cdot \varepsilon$ is always that of the output bar, never that of the platen; and the modulus/Poisson's ratio used in computing the punch factor K_p are always the bar's — only the contact diameter (`punch_diameter`) varies between platen and specimen depending on `plat_check`.

4.3.7 Viscoelastic modulus and wave speed (_viscoelastic_modulus_and_speed, disp_visco)

```
def _viscoelastic_modulus_and_speed():
    ft = time1[1] - time1[0]
    NN = 2 * len(transmitted) - 1
    freqi = np.fft.rfftfreq(NN, d=ft)
    E1, E2, n2 = OB.e11[d], OB.e22[d], OB.n22[d]
    # E(f) = (1/E1 + E2/(E2**2 + (2*pi*f*n2)**2) - 1j*(2*pi*f*n2)/(E2**2 + (2*pi*f*n2)**2))
     ** -1
    # C0(f) = sqrt(Re[E(f)] / OB.rho_pa[d])
```

— `process_di.py::_viscoelastic_modulus_and_speed()` (a local nested function). This is a standard-linear-solid model in complex frequency, with E_1, E_2, η_2 = `OB.e11`, `OB.e22`, `OB.n22` (viscoelastic bar material parameters):

$$E(f) = \left(\frac{1}{E_1} + \frac{E_2}{E_2^2 + (2\pi f \eta_2)^2} - i \, \frac{2\pi f \eta_2}{E_2^2 + (2\pi f \eta_2)^2} \right)^{-1} \qquad \text{(real part kept)}$$

$$C_0(f) = \sqrt{\mathrm{Re}[E(f)]/\rho_{OB}}$$

The frequency vector `freqi = rfftfreq(2·N_transmitted - 1, dt)` is indexed element-by-element against the time samples of `transmitted` — i.e. the time sample at index i is directly assigned $E(\texttt{freqi}[i])$, with no actual FFT of the signal (this is not a classic frequency-by-frequency dispersion correction, but a direct association of a frequency index with each time index). This technique remains non-standard for this type of correction and its physical rigor has not been independently validated.

This function is only called if `options.disp_visco` is true: it replaces, in each branch, the constant modulus/wave speed `E_mat_OB`/`C0_mat_OB` with sample-by-sample varying arrays `EE11[i]`/`CC22[i]`, and fully recomputes `F_output`, `displacement`, `n_s`/`t_s` using these values.

Important for the user: in the current Direct Impact UI tab (`ui/pages/page_process.py::_DITab`), `DIOptions(disp_ela=use_corrected, ...)` is built without ever passing `disp_visco=True` — no checkbox in the DI tab drives this field (it stays at the dataclass's default value of `False`). This viscoelastic branch of the code is therefore, as of today, not reachable from the Direct Impact graphical interface; it can only be exercised by calling `process_di()` directly with `disp_visco=True`.

4.4 Strain / strain-rate / force / stress chain

Notation: CPG = `OB.CPG_pa[d]` (the output bar's gauge coefficient, in 1/V — converts the raw signal into strain); f_v = `f_inc_velocity[d]` (the incident-velocity calibration factor, §6); $\Delta t_k = t_{k+1} - t_k$ (the actual, non-uniform step of the `time1` vector).

4.4.1 Compression / Tension (type_check 1 or 2)

`process_di()`, branch `elif options.type_check in (1, 2):`.

Sign: $s = -1$ (compression) or $+1$ (tension) — `sign = -1 if options.type_check == 1 else 1`.

$$\varepsilon_t^{raw}(k) = s \cdot \varepsilon_{transmitted}(k) \cdot CPG \quad (\texttt{output_m}), \qquad V_{in}(k) = v_{incident}(k) \cdot f_v \quad (\texttt{V_input})$$

Integration (explicit Euler, actual step `time1[i+1]-time1[i]`, in meters):

$$u(k) = \sum_{i=0}^{k} V_{in}(i)\,\Delta t_i, \qquad I_\varepsilon(k) = \sum_{i=0}^{k} \varepsilon_t^{raw}(i)\,\Delta t_i$$

Nominal (engineering) strain, the classic "one-wave" form (output side only):

$$\varepsilon_{nom}(k) = \frac{u(k) - C_{0,mat}^{OB}\, I_\varepsilon(k)}{L_0}$$

(`n_s`, `core/process_di.py::process_di()`), then clamped to $(-0.99999, 0.99999)$ by `core/process_shpb.py::_clamp_nominal_strain()` (shared with SHPB), to avoid a logarithm domain error at the next step in the event of an extreme value.

Nominal strain rate and true strain:

$$\dot{\varepsilon}_{nom}(h) = \frac{\varepsilon_{nom}(h+1) - \varepsilon_{nom}(h)}{t_1(h+1) - t_1(h)}, \qquad \varepsilon_{true}(h) = -\ln\left(1 - \varepsilon_{nom}(h)\right)$$

(the same logarithmic formula regardless of compression or tension — the sign is already carried by `sign` upstream).

Force on the output bar (1D wave theory, impedance $\rho C_0^2 = E$):

$$F(k) = \pi \left(\frac{D_{OB}}{2}\right)^2 \cdot \rho_{mat}^{OB} \cdot \left(C_{0,mat}^{OB}\right)^2 \cdot \varepsilon_t^{raw}(k)$$

Specimen cross-section S: $\pi(D/2)^2$ if cylindrical (`comp_check`/`tens_check == 1`), otherwise $w_i \cdot th_i$ (rectangular, with equivalent diameter $D_{eq} = 2\sqrt{S/\pi}$).

Nominal and true stress:

$$\sigma_{nom}(k) = \frac{F(k)}{S} \cdot 10^{-6}\ [\text{MPa}], \qquad \sigma_{true}(k) = \begin{cases} \sigma_{nom}(k)\,(1 - 2\nu\,\varepsilon_{nom}(k)) & \text{compression (type_check=1)} \\ \sigma_{nom}(k)\,(1 + 2\nu\,\varepsilon_{nom}(k)) & \text{tension (type_check=2)} \end{cases}$$

This is an approximate "true"-section correction weighted by the specimen's Poisson's ratio ν (rather than the classic incompressibility assumption $\sigma_{true} = \sigma_{nom}(1 + \varepsilon_{nom})$).

If `options.disp_visco`: this entire block (displacement, ε_{nom}, $\dot{\varepsilon}_{nom}$, ε_{true}, F) is recomputed, replacing $C_{0,mat}^{OB}$ with `CC22[i]` and $\rho_{mat}^{OB} \cdot (C_{0,mat}^{OB})^2$ with `rho_mat_OB` · `CC22[i]`^{2} (algebraically = `EE11[i]`).

4.4.2 Shear (type_check == 4: hat-shaped / double / single)

`process_di()`, branch `else:` (the final branch of the if/elif/else cascade).

The same velocity/transmitted-wave integration as §4.1, but the shear strain is normalized by a shear-band width ℓ_u instead of the length L_0:

$$\ell_u = \begin{cases} R_2 - R_1 & \text{hat-shaped (`shear_check`=1)} \\ w_i & \text{double/single (`shear_check`=2 or 3)} \end{cases}, \qquad \gamma(k) = \frac{u(k) - C_{0,mat}^{OB}\, I_\varepsilon(k)}{\ell_u}$$

Here $t_s(h) = \gamma(h)$ directly (no logarithmic conversion — the shear "true strain" is taken equal to the nominal strain).

Shear area S_{AS}:

$$S_{AS} = \begin{cases} \pi(R_1 + R_2)(H_2 - H_1) & \text{hat-shaped} \\ 2\,th_i\,L & \text{double} \\ th_i\,L & \text{single} \end{cases}$$

Force identical to §4.1: $F(k) = \pi(D_{OB}/2)^2\,\rho_{mat}^{OB}\,(C_{0,mat}^{OB})^2\,\varepsilon_t^{raw}(k)$; nominal stress = true stress = $F/S_{AS}/10^6$ (no Poisson correction for shear).

Caution — Incomplete viscoelastic recomputation for double/single shear. The `if options.disp_visco:` recomputation in this branch (lines ∼410-425) systematically uses `specimen.R2_pa[d] - specimen.R1_pa[d]` as the width ℓ_u and the area formula `SAS = (R1+R2)(H2-H1)` ("hat-shaped" geometry) without testing `options.shear_check`, whereas the corresponding non-visco computation (just above) correctly branches on `shear_check` (1 → hat, 2/3 → `wi_pa`). In other words, if `disp_visco=True` and `shear_check` is 2 or 3 (double/single), the viscoelastic recomputation would use `R1_pa`/`R2_pa`/`H1_pa`/`H2_pa` (quantities not relevant to a double/single shear specimen) instead of `wi_pa`/`thi_pa`/`L_pa`. This point is flagged as observed — and is in any case currently unreachable from the DI UI, since `disp_visco` is never enabled there (§3.7).

4.4.3 Notched specimen — Shear-Compression/Tension (type_check == 3)

`process_di()`, branch `if options.type_check == 3:`.

Prerequisite: `core/process_shpb.py::_require_notched_geometry(specimen, d)` — raises an explicit error if the notch radius `specimen.re_pa[d]` is not configured (≤ 0), to avoid a silent division-by-zero further down (a function shared with SHPB). The error message reminds the user that the "Test mode" (which determines `type_check`) is project-global, not specific to a Processing tab.

$$F(k) = s \cdot E_{mat}^{OB} \cdot \pi \left(\frac{D_{OB}}{2}\right)^2 \cdot \varepsilon_{transmitted}(k) \cdot CPG, \qquad \varepsilon_t^{raw}(k) = s \cdot \varepsilon_{transmitted}(k) \cdot CPG$$

with $s = -1$ if `tenscomp_check==1` (compression) otherwise $+1$. Note: here F uses E_{mat}^{OB} directly (not the ρC_0^2 form of §4.1/4.2). The two forms are mathematically identical for a homogeneous, self-consistent material ($E = \rho C_0^2$), but they diverge in the special case flagged in §3.5 (`plat_check=True`, `heat_check=False`), where `E_mat_OB` and `rho_mat_OB·C0_mat_OB²` no longer correspond to the same property triplet.

Integration (here in millimeters, with `dtime` fixed once and for all to `time1[1]-time1[0]`, unlike the variable actual step of §4.1/4.2):

$$u_{mm}(k) = \sum_{i=0}^{k} V_{in}(i) \cdot dtime \cdot 1000, \qquad I_\varepsilon(k) = \sum_{i=0}^{k} \varepsilon_t^{raw}(i) \cdot dtime$$

$$d(k) = u_{mm}(k) - 1000 \cdot C_{0,mat}^{OB} \cdot I_\varepsilon(k) \qquad (\texttt{displacement}, \text{in mm})$$

Shear/notch strain, a calibrated cubic polynomial (coefficients `k_value[0..2]`), normalized by

$$\text{denom} = \frac{2000 \cdot r_e}{\cos(\theta_{notch})}$$

(r_e = `re_pa`, notch radius; θ_{notch} = `NoA_pa`, notch angle):

$$\gamma(k) = k_0 \cdot \xi(k) + k_1 \cdot \xi(k)^2 + k_2 \cdot \xi(k)^3, \qquad \xi(k) = \frac{d(k) - 1000\, d_y}{\text{denom}}$$

(d_y = `specimen.dy_pa[d]`, displacement at the yield threshold), then γ is rebased to start at 0: $\gamma(k) \mathrel{-}= \gamma(0)$.

Stress, empirical calibration (coefficients `k_value[3..5]`):

$$\sigma(k) = \frac{k_3 \left(1 - k_4\, \gamma(k)\right) \cdot |F(k)|^{k_5}}{10^6 \cdot th \cdot D}$$

(th = `specimen.th_pa[d]`, effective-zone thickness; D = specimen diameter, both converted m→mm via ×1000).

Two-coefficient-set calibration (`sk_check`): if enabled, each sample uses `k_value` if $d(k) \leq$ `sk_value[6]`, otherwise `sk_value` — a displacement threshold switches to a second set of coefficients (`k0..k5` + threshold `sk_value[6]`), to represent a bi-linear/bi-regime notch behavior.

At the end of the branch: `n_s, n_stress = t_s, t_stress` — there is no nominal/true distinction for the notched specimen (unlike the compression/tension branches).

Caution — Ambiguous final-displacement unit for type_check == 3. Right after computing σ (line `displacement = [v / 1000 for v in displacement]`), the code divides `displacement` by 1000 (mm → m). Yet the end-of-function comment states: "displacement is in mm already for type_check == 3 (matches the original, which multiplies by 1000 there)", and the final value exposed in `DIResult.displacement` (`displacement_mm = displacement if options.type_check == 3 else`

`[1000 * v for v in displacement]`) applies no reconversion for `type_check==3`. Strictly following the code's units (accumulation in mm in the "integration" step above, confirmed by the comparison against `1000*dy_pa`), the value stored in `displacement_mm` for a notched specimen therefore appears to be in meters, despite its field name. This point literally contradicts the adjacent comment in the code itself; it is flagged explicitly for the user to verify (a numerical comparison against the original application is recommended), rather than corrected or reinterpreted by this agent.

4.5 Additional physical corrections

The four corrections below are mathematically identical in mechanism to SHPB — the same functions/formulas — but in Direct Impact they only apply on the output-bar side, since there is no instrumented input bar in DI (no averaging/choice between input-side and output-side force, unlike what a two-bar SHPB can allow).

4.5.1 Punch/indentation correction (punch_check)

Elliptic geometric factor shared with SHPB — `core/process_shpb.py::_elliptic_punch_factor(d_ratio)`:

$$E(x) = \int_0^{\pi/2} \sqrt{1 - x^2 \sin^2\theta}\, d\theta, \qquad K(x) = \int_0^{\pi/2} \frac{d\theta}{\sqrt{1 - x^2 \sin^2\theta}}$$

$$H_x(x) = 2 - \left(x + \frac{1}{x}\right) E(x) - \left(x - \frac{1}{x}\right) K(x)$$

with $x = d_{ratio} = d_{punch}/d_{bar}$, valid only for $0 < x < 1$ (otherwise an explicit `ValueError` — the punch/platen diameter must be strictly greater than the specimen diameter).

Contact stiffness:

$$K_p = H_x(d_{ratio}) \cdot \frac{16}{3\pi^2} \cdot \frac{1 - \nu_{bar}^2}{d_{punch} \cdot E_{bar}}$$

where ν_{bar} = `nuB_punch` and E_{bar} = `EB_punch` are always the output bar's own properties (§3.6), and d_{punch} = `punch_diameter` equals the platen diameter if `plat_check`, otherwise the specimen's diameter (or equivalent diameter).

Application (schematically, with slightly different forms per branch — coefficients and denominators adapted to each test type's geometry):

- Compression/tension (§4.1): $p_s(k) = 2 \times 10^6\, K_p\, \sigma_{nom}(k)\, \dfrac{\pi D_{eq}^2/4}{L_0}$, then ε_{nom} −= p_s (re-clamped), and $\varepsilon_{true}/\sigma_{true}$ recomputed — but see the caution below.
- Notched (§4.3): $p_s(k) = 2 \times 10^6\, K_p\, \sigma(k)\, \dfrac{\pi D^2/4}{D}$, then γ −= p_s.
- Shear (§4.2): the same form with D_{eq} specific to the shear geometry.

Caution — Recomputation of t_stress with a fixed sign (compression/tension branch), regardless of type_check. In the post-punch recomputation block of branch 4.1 (`if options.punch_check:`), the true stress is recomputed via `t_stress.append(n_stress[i] * (1 - 2 * specimen.nu_pa[d] * n_s[i]))` — always the "compression" form $(1 - 2\nu\varepsilon)$, whereas the initial computation of `t_stress` (before the punch correction, a few lines earlier in the same branch) correctly branches on $(1-2\nu\varepsilon)$ in compression and $(1+2\nu\varepsilon)$ in tension. In other words, if `punch_check=True` and `type_check==2` (tension), the final true stress would use the "compression" formula instead of the "tension" formula. This point is flagged as observed in the code, without further correction or interpretation from this agent.

4.5.2 Friction (friction[d] != 0)

The classic (Bertholf/Cowper-type) correction, with s_0 = the specimen's slenderness (length / diameter) and f = the friction coefficient:

$$m = 1 - \frac{f}{3s_0}$$

- Compression/tension (§4.1) and shear (§4.2): $\sigma_{nom} \mathrel{*}= m$, $\sigma_{true} \mathrel{*}= m$, then $\sigma_{friction} = \frac{f}{3s_0} \cdot \sigma_{true}^{\text{corrected}}$ is computed for information/display purposes only (not subtracted a second time).
- Notched (§4.3): the same m and $\sigma_{true} \mathrel{*}= m$, but the code additionally subtracts $\sigma_{friction}$ from σ_{true} (`t_stress = [t_stress[i] - friction_stress[i] ...]`, absent from the other two branches). Since $\sigma_{friction} = \frac{f}{3s_0} \cdot (m \cdot \sigma_{true}^{raw})$, this amounts — for the notched branch alone — to $\sigma_{true}^{final} = m^2 \cdot \sigma_{true}^{raw}$ (an implicit application of the factor m twice), instead of $m \cdot \sigma_{true}^{raw}$ for the other test types. This asymmetry between branches is flagged as observed in the code, without presuming which of the two forms is the physically intended formula.

4.5.3 Radial/axial inertia (inertia_check)

The classic two-term formulation (radial + axial acceleration), with $s_0 = L/D$ (or D_{eq}):

$$c_1 = \frac{\rho\, D^2\, (s_0^2 - 3/16)}{12}, \qquad c_2 = \frac{3\,\rho\, D^2}{64}$$

$$\sigma_{inertia}(k) = \frac{c_1 \left(\dot{\varepsilon}(k)^2 + \varepsilon(k)\ddot{\varepsilon}(k)\right)}{10^6} + \frac{c_2\, \ddot{\varepsilon}(k)}{10^6}, \qquad \sigma_{true} \mathrel{-}= \sigma_{inertia}$$

$\dot{\varepsilon}$ and $\ddot{\varepsilon}$ (true-strain rate and "acceleration") are computed by finite differences over a robust time step (`core/process_shpb.py::_robust_dt()`, shared with SHPB: the median of consecutive gaps in `time1`, rather than a fixed step `time1[2]-time1[1]` that breaks if two consecutive samples share the same timestamp) — except in the notched branch (§4.3), which uses `dtime = time1[1] -time1[0]` fixed once and for all (not `_robust_dt`), for all of its $\dot{\gamma}/\ddot{\gamma}$ computations.

For the notched case, the geometry is projected at 45° (`cos_a = cos( /4)`), reflecting the characteristic shape of a shear-compression/tension specimen loaded at 45°.

4.5.4 Adiabatic heating (temperature)

Common to all 4 test types, computed after the test-type branch (`process_di()`, end of function):

$$\text{coeff}_{temp} = \frac{\xi}{c_p\, \rho}, \qquad T(k+1) = T(k) + \text{coeff}_{temp} \cdot \sigma_{true}(k) \cdot 10^6 \cdot (\varepsilon_{true}(k+1) - \varepsilon_{true}(k))$$

with $T(0) = T_0$ (`T0_pa[d]`), ξ = the specimen's Taylor-Quinney coefficient (`specimen.xi_pa[d]`), c_p = specific heat (`specimen.cp_pa[d]`), ρ = the specimen's density. This is an incremental integration of the adiabatic heating due to plastic work (adiabatic assumption, a fraction ξ of the plastic work converted to heat).

4.6 Realignment of the incident-velocity profile (”Cut”/”shift”) and calibration factor

This mechanism is entirely on the UI side (`ui/pages/page_process.py::_DITab`) — `process_di()` itself only consumes the already-realigned `velocity_c` array (see §3.1: `velocity = velocity_c[0:i_it]`, index 0 assumed synchronized with `xd1`).

4.6.1 Cutting/shifting (”Cut velocity profile at selected start” / ”shift by (s)”)

`_DITab._apply_velocity_cut(cut_time)`:

```
time_arr = np.asarray(self._time_pulse)
below = np.where(time_arr < cut_time)[0]
idx = int(below[-1]) if len(below) else 0
self._velocity_c = self._velocity_c[idx:]
```

— finds the index `idx` of the last `time_pulse` sample strictly before `cut_time`, then trims the first `idx` samples from the incident-velocity array. The resulting array therefore restarts its index 0 at instant `cut_time` (approximately, to the resolution of `time_pulse`), realigning the incident velocity onto the bar signal's time axis — exactly the assumption that `process_di()` then makes (`velocity_c[0]` $\leftrightarrow t = xd1$).

Two ways to call this function:

- ”Cut velocity profile at selected start” (`_on_cut_velocity`): uses the starting point of the ”Velocity start cut” window selected with the mouse on the plot (`self.picker.get_window("velocity_cut")[0]`).
- ”shift by (s)” + Apply (`_on_apply_velocity_shift`): uses directly the numeric value entered in the field (in seconds) as `cut_time`.

Cuts are cumulative: `self._cumulative_velocity_idx` totals the samples removed across multiple calls, and `project.di.velocity_cut_time` stores a single equivalent cut instant, expressed on the original (uncut) time axis, so that the cumulative cut can be reapplied in one go after a session reload (`self.velocity_cut_time` is restored and reapplied in `_DITab.__init__`/when the velocity file is loaded). The ”Reset cut” button (`_on_reset_velocity_cut`) reloads the raw file from disk and resets `_cumulative_velocity_idx = 0`.

4.6.2 Multiplicative calibration factor for the incident velocity

UI field ”Incident-velocity calibration factor” (`self.f_inc_velocity_edit`, default 1.0) → `f_inc_velocity` → expanded into a constant array `f_inc_arr = np.full(n, f_inc_velocity)` (one value per Monte-Carlo draw, but identical across all draws — no uncertainty distribution attached to this factor in the current UI, as with `SG_unc_d`, §3.1) → passed to `process_di(..., f_inc_velocity=f_inc_arr, ...)`.

Applied strictly multiplicatively, in every branch of `process_di()`:

$$V_{in}(k) = v_{incident}(k) \cdot f_v[d] \qquad (\texttt{V_input = velocity * f_inc_velocity[d]})$$

This is a global scale factor on the entire measured incident velocity — intended to compensate for a systematic calibration bias of the velocity sensor (e.g. a poorly calibrated laser gate), without altering the time-domain shape of the velocity profile.

4.6.3 Auxiliary helper for custom sensors mounted on the specimen

Not directly related to the incident-velocity profile itself, but in the same tab: the ”Auto-cut using wave travel time” button (for a ”Custom Sensor” mounted on the specimen, `_DITab._on_align_sensor`) automatically computes a cut point for that sensor via

$$t_{\text{cut}} = t_{\text{transmitted-wave start}} - \Delta t_{total}$$

where Δt_{total} = `core/specimen_sensor_alignment.py::specimen_sensor_delay_direct_impact()` — see §7 below — because in Direct Impact, the output bar's gauge sees the wave after it has already traveled through the bar + half the specimen (+ possibly the platen) since the instant the specimen began loading.

4.7 Reuse of toe_correction.py and specimen_sensor_alignment.py

These two modules are reused as-is by Direct Impact, with no DI-specific variant in their computation logic:

- `core/toe_correction.py` (`detect_linear_region`, `fit_linear_region`, `apply_toe_correction`): identical to SHPB, applied downstream of `process_di()` in the UI (`_DITab`) to remove the ”toe” (progressive contact engagement) from the stress-strain curve, using a calibration curve recorded on a rigid/reference specimen. Since the fix described in the SHPB chapter, §5.5, `_DITab._on_toe_apply()` (lines 3575-3625) no longer relies on a single call to `apply_toe_correction(strain, stress, offset, trim=True)` on the aggregated mean curve: it re-applies the same offset to every individual Monte-Carlo draw via `_apply_toe_correction_to_draws()` (a function shared with `_SHPBTab`, `ui/pages/page_process.py`, lines 147-212), and then re-aggregates — guaranteeing that `project.last_raw_draws["DI"]` (*Covariance Analysis* page) and the per-draw confinement quantities (Confinement chapter, §5.8, active on this tab under the same conditions as on SHPB) remain consistent with the corrected curve, instead of staying offset by the old ”toe” that was only removed from the aggregate. The mechanism, its guarantees (no new `process_di()` simulation is re-run — it is a pure change of origin on the strain axis, applied to already-computed results) and its limitations (the `toe_n_removed` counter = an inter-draw median, see SHPB chapter §8, points 11-12) are rigorously identical between `_SHPBTab` and `_DITab` — refer to the SHPB chapter, §5.5, for the full detail rather than duplicating it here.
- `core/specimen_sensor_alignment.py`: contains two parallel functions, `specimen_sensor_delay_shpb()` (two terms: input bar + half-specimen [+ platen]) and `specimen_sensor_delay_direct_impact()` (used by DI):

```
def specimen_sensor_delay_direct_impact(
    output_gage_position, c0_output_bar, sample_length, c0_sample,
    use_platens=False, platens_length=0.0, c0_platens=0.0,
):
    bar_term = output_gage_position / c0_output_bar
    specimen_term = sample_length / (2 * c0_sample)
    platen_term = platens_length / c0_platens if use_platens and c0_platens else 0.0
    return DelayBreakdown(bar_term, platen_term, specimen_term, total=bar_term+
    platen_term+specimen_term)
```

  Mathematically identical to the SHPB version (the same three terms: gauge position / bar wave speed, half-specimen length / specimen wave speed, platen length / platen wave

speed), the only difference being that DI has only one ”bar” term (the output bar’s) since there is no input bar to account for.

Neither module is called from inside `process_di.py` itself: the toe correction applies downstream on the already-produced results (stress-strain curve), and sensor/specimen alignment is a setup-assistance tool for the cut window, used upstream by the UI (`_DITab`), not a step in the `process_di()` pipeline.

4.8 Equation ↔ code correspondence table

Quantity / step	**File :: function (or block)**
Cutting the transmitted window + incident velocity	`core/process_di.py::process_di()` (”Cutting waves” block)
Nearest-index search	`core/process_shpb.py::_index_of_nearest()`
Dispersion-corrected signal substitution	`core/process_di.py::process_di()` (`disp_ela`/`disp_visco`)
Platen correction (transmission coefficient)	`core/process_di.py::process_di()` (”Platen correction” block)
Bar thermal-gradient correction (RTE model / table)	`core/process_shpb.py::_bar_gradient_wave_speed_modulus()` (shared)
Diameter/properties for F vs. the punch correction	`core/process_di.py::process_di()` (`DB_punch`, `nuB_punch`, `EB_punch`, dedicated comments)
Viscoelastic modulus/wave speed (per-sample frequency)	`core/process_di.py::_viscoelastic_modulus_and_speed()` (internal to `process_di()`)
Compression/tension branch	`core/process_di.py::process_di()` (`elif options.type_check in (1, 2):`)
Shear branch	`core/process_di.py::process_di()` (final `else:`)
Notched Shear-Comp./Tens. branch	`core/process_di.py::process_di()` (`if options.type_check == 3:`)
Notched-geometry guard	`core/process_shpb.py::_require_notched_geometry()` (shared)
Nominal strain clamping	`core/process_shpb.py::_clamp_nominal_strain()` (shared)
Elliptic punch factor H_x	`core/process_shpb.py::_elliptic_punch_factor()` (shared)
Robust time step (strain velocity/acceleration)	`core/process_shpb.py::_robust_dt()` (shared)
Adiabatic heating	`core/process_di.py::process_di()` (end of function, `coeff_temp`/`temperature`)
Final formatting (mm) and return	`core/process_di.py::process_di()` (`displacement_mm`, `DIResult`)
Monte-Carlo worker entry point	`core/process_di.py::_process_di_task()`
Incident-velocity profile cutting/shifting	`ui/pages/page_process.py::_DITab._apply_velocity_cut()`, `_on_cut_velocity()`, `_on_apply_velocity_shift()`, `_on_reset_velocity_cut()`
Incident-velocity calibration factor	`ui/pages/page_process.py::_DITab` (field `f_inc_velocity_edit`) → `process_di(f_inc_velocity=...)`

Quantity / step	File :: function (or block)
Custom-sensor alignment (auto-cut)	`ui/pages/page_process.py::_DITab._on_align_sensor()` + `core/specimen_sensor_alignment.py::specimen_sensor_delay_direct_impact()`
Toe correction (downstream, post-processing, re-cuts every Monte-Carlo draw)	`core/toe_correction.py::apply_toe_correction()`, `true_strain_from_nominal()` (shared) + `ui/pages/page_process.py::_apply_toe_correction_to_draws()` (shared) via `_DITab._on_toe_apply()` — see SHPB chapter §5.5

4.9 Limitations, warnings and points to verify

Explicit warning at the top of the file (`core/process_di.py`, module docstring):

> *”NOT YET independently re-validated against real experimental data across the full option matrix (heat x platen x punch x friction x inertia x viscoelastic bar x 4 kinematics) — exercised with synthetic and real signals only on the simplest option combination so far. Cross-check important results against the original application before relying on this for a real analysis.”*

That is: only the simplest option combination has been tested with real/synthetic signals; the full cross-matrix of heat × platen × punch × friction × inertia × viscoelastic bar × 4 test kinematics has not been independently re-validated. This must be clearly documented to the end user as a major methodological caveat on this module.

Other points noted in the code itself or inferred from a close reading, for the user to verify:

1. §3.5 — When `plat_check=True` and `heat_check=False`, `E_mat_OB`/`C0_mat_OB` are overwritten with the bar's properties while `rho_mat_OB` remains the platen's: a potentially inconsistent (E, ρ, C_0) triplet used in the force and displacement computations.

2. §4.3 — The final-displacement formula `displacement_mm` for the notched specimen (`type_check==3`) appears, from a strict tracking of units in the code, to be expressed in meters despite its name and despite the adjacent comment asserting it is ”already in mm”.

3. §4.2 — The viscoelastic recomputation (`disp_visco`) of the shear branch unconditionally uses the ”hat-shaped” geometry (R1/R2/H1/H2), even for `shear_check` = double or single. Without practical consequence as long as `disp_visco` remains unreachable from the DI UI (point 6 below).

4. §5.1 — The true-stress recomputation after the punch correction (compression/tension branch) always uses the ”compression” formula $(1 - 2\nu\varepsilon)$, even in tension (`type_check==2`).

5. §5.2 — The friction correction subtracts `friction_stress` a second time only in the notched branch (equivalent to applying the factor m squared), unlike the compression/tension and shear branches.

6. §3.7 / §4.2 — `options.disp_visco` is never enabled from the current Direct Impact UI tab (`ui/pages/page_process.py::_DITab`): only `disp_ela` (the ”Use dispersion-corrected signal” checkbox) is exposed there. The viscoelastic branch of the code is present and functional as an API but currently unreachable by an application user.

7. §3.1 — `SG_unc_d` and `f_inc_velocity` are per-Monte-Carlo-draw parameters on the `process_di()` side, but the DI UI always assigns them a constant value (`np.ones(n)` / `np.full(n, f_inc_velocity)`): neither the uncertainty on the output bar's gauge gain, nor that on the incident-velocity calibration factor, is currently sampled per draw.

8. `_require_notched_geometry` explicitly reminds that `type_check` depends on the project-global ”Test mode” (*Test Setup* page), not a per-*Processing*-tab choice: processing a notched test, then, without changing the Test mode, a compression/tension/shear test on another tab, silently reuses the notched branch with irrelevant specimen parameters — except that the guard now raises an explicit error if the notch radius is zero.

9. §7 — The per-draw re-cutting of the toe correction (`_apply_toe_correction_to_draws()`) inherits the same two nuances as on the SHPB tab (SHPB chapter, §8, points 11-12): the `toe_n_removed` counter used to realign a confinement/laser sensor left as a raw signal is a median across all draws, not an exact per-draw count; and this realignment only corrects a leading offset, not any difference in sampling rate between the sensor and the stress-strain curve.

These nine points are reported as observed in the source code, without re-interpretation or correction by this agent: they must be verified (and, where applicable, confirmed or disproved) by comparison with the original application before being presented as established defects in the final documentation.

5. Quasi-Oedometric (Confined) Test: Radial Stress, Hydrostatic Pressure and Deviator

Main source file: `core/confinement.py` (482 lines, read in full). Context files: `ui/pages/page_special_tests.py` (user configuration), `ui/pages/page_process.py` (SHPB / Direct Impact / Quasi-Static Monte-Carlo loops, which themselves compute the per-draw confinement quantities — see §5.8), `ui/pages/page_results_export.py` (the "Confinement / Triaxial Stresses" card, which prefers this real per-draw data when it exists), `ui/pages/page_covariance.py` (the Covariance Analysis page, same preference), `core/project.py` (the `ConfinementSettings` dataclass, together with the constants in `corr_file.py`).

5.1 Physical principle of the test

The quasi-oedometric test (or confined test) is a compression test (SHPB, direct impact or quasi-static) in which the specimen is placed inside a rigid confinement cell that prevents (or strongly limits) its free radial deformation. The cell itself is instrumented with a strain gage bonded to its outer wall, which measures the cell's hoop (circumferential) strain under the pressure exerted by the specimen as it tends to expand radially during axial compression.

This hoop displacement/strain of the cell is then converted, through a calibration (linear or nonlinear, see §2-3), into a radial stress applied by the specimen onto the cell wall — which, by the assumption of the setup (cylindrical confinement), is taken to equal the radial stress (= hoop stress, by axial symmetry) prevailing inside the specimen.

From this pair (axial stress, radial stress) — the axial stress coming from the standard SHPB/ Direct Impact/Quasi-Static reduction already performed on another tab — the usual stress invariants of a triaxial path are computed: the hydrostatic pressure p and the stress deviator q (§5), together with the radial, volumetric and equivalent (von Mises) strains of the specimen (§4-5).

This module groups the function `confine_show()` (and the "laser stress" block) together with the calibration constants (in particular `corr_file.nonlinear_check`, which selects the linear/nonlinear cell model) — see `core/confinement.py`, lines 1-21.

In the UI (`ui/pages/page_special_tests.py`, "Confinement (Quasi-Oedometric Test)" card), the confined test requires:

- checking "This is a quasi-oedometric (confined) test" (→ `project.confinement.enabled`);
- assigning the "Confinement cell strain gage" role to a Custom Sensor (Custom Sensors page) — this sensor supplies the `cell_deflection`/`cell_defl` signal used in all the calculations below;
- selecting the cell model ("Linear" or "Nonlinear", `project.confinement.nonlinear_enabled`) and entering the corresponding calibration constants.

5.2 Linear cell model

5.2.1 Formula

Implementation: `core/confinement.py::linear_cell_radial_stress()` (lines 40-54).

```
def linear_cell_radial_stress(
    axial_strain, cell_deflection, cell_modulus, C1, C2, ref_strain,
):
    e_cell_deff = cell_modulus * cell_deflection
    blend = (eps / ref_strain) * C2 + (1 - eps / ref_strain) * C1
    return e_cell_deff * blend / 1e6
```

Writing ε_a for the axial strain (`axial_strain`, actually the nominal/engineering strain taken from the SHPB/DI/QS result — see `strain_key = "nominal_strain" if "nominal_strain" in result else "strain"` in `page_results_export.py` line 814), δ_{cell} the calibrated hoop deflection/displacement signal of the cell (`cell_deflection`), E_v the cell modulus (`cell_modulus`, in Pa), ε_{ref} the reference strain (`ref_strain`), the radial stress (in MPa, from the division by 10^6) is:

$$E_{cell,deff} = E_v \cdot \delta_{cell} \tag{5.1}$$

$$\sigma_r = \frac{E_{cell,deff}}{10^6}\left[\frac{\varepsilon_a}{\varepsilon_{ref}}C_2 + \left(1 - \frac{\varepsilon_a}{\varepsilon_{ref}}\right)C_1\right] \tag{5.2}$$

or, in a single block:

$$\sigma_r(\varepsilon_a) = \frac{E_v\,\delta_{cell}}{10^6}\left[C_1 + \frac{\varepsilon_a}{\varepsilon_{ref}}(C_2 - C_1)\right] \tag{5.3}$$

5.2.2 Interpretation — C1, C2 and the transition mechanism

- C_1 ("low-strain calibration constant", UI: "C1 (low-strain calibration constant)") is the calibration coefficient applied in the low axial-strain regime ($\varepsilon_a \to 0$);
- C_2 ("high-strain calibration constant") is the coefficient applied in the high axial-strain regime ($\varepsilon_a \to \varepsilon_{ref}$);
- E_v ("Cell modulus Ev (Pa)") is an effective cell modulus which, multiplied by the deflection signal `cell_deflection`, yields a term with the dimensions of a stress, `E_cell_deff` (named `Ecell_deff` in the original code);
- ε_{ref} ("Reference strain") is the axial strain at which the second calibration curve (see §3) was established; here it serves as the pivot of the linear transition between C_1 and C_2.

The term `blend = (eps/ref_strain)*C2 + (1-eps/ref_strain)*C1` is thus a linear (affine) interpolation as a function of $\varepsilon_a/\varepsilon_{ref}$ between C_1 (weight 1 at $\varepsilon_a = 0$) and C_2 (weight 1 at $\varepsilon_a = \varepsilon_{ref}$). The code imposes no clamp: for $\varepsilon_a > \varepsilon_{ref}$, `blend` keeps extrapolating linearly beyond C_2 (extrapolation, no saturation), and likewise for $\varepsilon_a < 0$. This is not explicitly documented as a limitation in the code, but deserves to be flagged: the formula implicitly assumes the test stays within the calibration range $[0, \varepsilon_{ref}]$.

The `axial_strain` and `cell_deflection` vectors are truncated to the shortest common length (`n = min(len(axial_strain), len(cell_deflection))`) before any computation.

5.3 Nonlinear cell model

5.3.1 Overview

Implementation: `core/confinement.py::nonlinear_cell_radial_stress()` (lines 143-160), with the helper function `_eval_poly_no_constant()` (lines 90-97).

This function is structured into two separate steps:

- `fit_nonlinear_calibration()` (lines 100-119): fits the two degree-6 calibration polynomials (what step 1 below does) and returns them (`poly_ini`, `poly_esp`);
- `nonlinear_cell_radial_stress_precomputed()` (lines 122-140): evaluates the model (step 3 below) from polynomials that have already been fitted, without redoing the `np.polyfit`.

`nonlinear_cell_radial_stress()` itself (lines 143-160) chains the two — `fit_nonlinear_calibration(...)` then `nonlinear_cell_radial_stress_precomputed(...)` — and gives the same result for every caller (`_refresh_confinement`, `triaxial_synthetic_draws`). The point of this split: a Monte-Carlo loop that evaluates this model hundreds of times per draw (§5.8) fits the two polynomials only once, before the loop, instead of redoing a `np.polyfit` (the most expensive operation of the model) at every draw.

The nonlinear model relies on two numerical calibration curves (typically obtained from numerical simulations of the confinement cell), each relating the hoop strain of the cell to the radial stress:

- an "ini" curve: calibration at the specimen's initial height (L_{ini}) — files *Orthoradial displacement, initial (.txt) / Radial stress, initial (.txt)* on the Special Tests page, corresponding to the parameters `ortho_disp_ini`, `radial_ini`;
- an "esp" curve: calibration at the reference height (L_{esp}, the specimen height when its axial strain equals ε_{ref}) — files *Orthoradial displacement, at reference strain (.txt) / Radial stress, at reference strain (.txt)*, parameters `ortho_disp_esp`, `radial_esp`.

5.3.2 Step 1 — Normalization to hoop strain and polynomial fit

```
r_cell = cell_diameter / 2
ortho_def_ini = np.asarray(ortho_disp_ini) / r_cell
ortho_def_esp = np.asarray(ortho_disp_esp) / r_cell
radial_ini_mpa = np.abs(np.asarray(radial_ini)) / 1e6
radial_esp_mpa = np.abs(np.asarray(radial_esp)) / 1e6

ff1 = np.polyfit(ortho_def_ini, radial_ini_mpa, 6)
ff2 = np.polyfit(ortho_def_esp, radial_esp_mpa, 6)
```

The cell diameter D_{cell} (`cell_diameter`, parameter "Cell diameter Dcell (m)") is used solely to convert the measured/tabulated hoop displacement (in meters) into a dimensionless hoop strain, by dividing by the cell radius:

$$r_{cell} = \frac{D_{cell}}{2}, \qquad \varepsilon_{ortho}^{ini} = \frac{u_{ini}}{r_{cell}}, \qquad \varepsilon_{ortho}^{esp} = \frac{u_{esp}}{r_{cell}} \tag{5.4}$$

The tabulated radial stresses of the two calibration curves are taken in absolute value and then converted to MPa (division by 10^6):

$$\sigma_r^{ini} = \left|\sigma_{r,\text{file}}^{ini}\right| / 10^6, \qquad \sigma_r^{esp} = \left|\sigma_{r,\text{file}}^{esp}\right| / 10^6 \tag{5.5}$$

Each pair (hoop strain, radial stress) is then fitted by a degree-6 polynomial (`np.polyfit(..., 6)`), giving two sets of 7 coefficients `ff1` ("ini" curve) and `ff2` ("esp" curve).

5.3.3 Step 2 — Evaluating the polynomial without its constant term

`_eval_poly_no_constant()` (lines 57-64) evaluates the degree-6 polynomial while deliberately omitting its constant term (`coeffs[6]`):

$$P(x) = c_0x^6 + c_1x^5 + c_2x^4 + c_3x^3 + c_4x^2 + c_5x \tag{5.6}$$

The docstring explicitly states that this forces the curve through the origin (zero hoop strain → zero radial stress), judged "physically sensible" for this calibration — this is an explicit assumption/choice made by the port, to be flagged as such in the documentation.

5.3.4 Step 3 — Interpolating between the two curves according to the specimen's current height

```
l_ini = specimen_initial_length
l_esp = (1 - ref_strain) * l_ini
dhv = (1 - strain) * l_ini          # hauteur courante de l'éprouvette, déformation
    nominale (technique)

drad1 = _eval_poly_no_constant(ff1, defl) * (dhv - l_esp) / (l_ini - l_esp)
drad2 = _eval_poly_no_constant(ff2, defl) * (l_ini - dhv) / (l_ini - l_esp)
return drad1 + drad2
```

Notation: L_{ini} = initial height (length) of the specimen (`specimen_initial_length`), ε_{ref} = reference strain, $\varepsilon_a(t)$ = current nominal axial strain (`axial_nominal_strain`), $\delta_{cell}(t)$ = current cell deflection signal (`cell_deflection`).

Reference height (specimen height at $\varepsilon_a = \varepsilon_{ref}$):

$$L_{esp} = (1 - \varepsilon_{ref})\, L_{ini} \tag{5.7}$$

Current specimen height (computed from the engineering/nominal strain, hence positive in compression under this convention — `# current specimen height, from the nominal (engineering) strain`):

$$L(t) = (1 - \varepsilon_a(t))\, L_{ini} \tag{5.8}$$

The radial stress at time t is then a weighted linear interpolation between the two calibration polynomials P_{ini} (via `ff1`) and P_{esp} (via `ff2`), both evaluated at the same current deflection signal $\delta_{cell}(t)$, with weights that depend on the position of $L(t)$ between L_{esp} and L_{ini}:

$$\sigma_r(t) = P_{ini}(\delta_{cell}(t)) \cdot \frac{L(t) - L_{esp}}{L_{ini} - L_{esp}} + P_{esp}(\delta_{cell}(t)) \cdot \frac{L_{ini} - L(t)}{L_{ini} - L_{esp}} \tag{5.9}$$

Remarks:

- When $L(t) = L_{ini}$ (start of the test, $\varepsilon_a = 0$), the weight of P_{ini} equals $\frac{L_{ini}-L_{esp}}{L_{ini}-L_{esp}} = 1$ and that of P_{esp} equals 0: the result falls back exactly onto the "initial" calibration curve.
- When $L(t) = L_{esp}$ (i.e. $\varepsilon_a = \varepsilon_{ref}$), the weight of P_{ini} equals 0 and that of P_{esp} equals 1: the result falls back exactly onto the "reference-height" calibration curve.
- For ε_a outside $[0, \varepsilon_{ref}]$, the weighting is a linear extrapolation (the two weights always sum to 1, but one becomes negative) — the same caveat as for the linear model (§2.2): no saturation/clamp is applied in the code.
- Both polynomials P_{ini}, P_{esp} are evaluated at the same signal `defl` (current cell deflection), so it is only the weighting between the two curves that varies with the current height — not their argument.
- The `axial_nominal_strain` and `cell_deflection` arrays are truncated to the shortest common length before the computation (`n = min(...)`, line 93).

This function (or its precompute/evaluate pair) is called from four places: `ui/pages/page_results_export.py::_refresh_confinement()` (line 933, `nonlinear_cell_radial_stress()`, for the displayed mean curve) with `specimen_length = self.project.specimen.params["L"].mean` as `specimen_initial_length`; `triaxial_synthetic_draws()` (§5.7, synthetic fallback approximation) and `nonlinear_confinement_envelope()`, likewise; and `core/confinement.py::per_draw_confinement_quantities()` (§5.8), which uses the `fit_nonlinear_calibration()`/`nonlinear_cell_radial_stress_precomputed()` pair to evaluate this model once per Monte-Carlo draw without re-fitting the polynomials each time.

5.4 Constants alpha00 and alpha0$^{l/2}$ — radial-strain calibration

Implementation: `core/confinement.py::radial_strain()` (lines 219-240).

5.4.1 Implicit relation

```
a_term = (2/3) * (1 - eps_axial*(1 + eps_axial/2)) * alpha00 * eps_cell
b_term = alpha0_l2 * eps_cell / 3
eps_rad = np.zeros(n)
for _ in range(n_iter):             # n_iter = 50 par défaut
    eps_rad = a_term + b_term * (1 + eps_rad) ** 2
```

Writing ε_a for the (signed, see §4.3) axial strain, ε_{cell} for the cell gage signal (`cell_strain`, here in fact the `cell_deflection`/`cell_defl` parameter passed through unchanged — the function's parameter name is `cell_strain`), α_{00} (`alpha00`) and $\alpha_0^{l/2}$ (`alpha0_l2`) the two calibration constants (derived, according to the docstring, "from a numerical simulation of the confinement cell", entered on the Special Tests page), the radial strain ε_r is defined implicitly (it appears on both sides of the equation):

$$\varepsilon_r = \frac{2}{3}\left[1 - \varepsilon_a\left(1 + \frac{\varepsilon_a}{2}\right)\right]\alpha_{00}\,\varepsilon_{cell} + \frac{\alpha_0^{l/2}}{3}(1+\varepsilon_r)^2\,\varepsilon_{cell} \tag{5.10}$$

5.4.2 Numerical resolution

The code in `core/confinement.py` solves this equation by a simple fixed-point iteration ("fixed-point iteration", comment at lines 220-222), initialized at $\varepsilon_r^{(0)} = 0$ and repeated `n_iter = 50` times (default value, configurable):

$$\varepsilon_r^{(k+1)} = \underbrace{\frac{2}{3}\left[1 - \varepsilon_a\left(1 + \frac{\varepsilon_a}{2}\right)\right]\alpha_{00}\,\varepsilon_{cell}}_{\text{constant term}} + \frac{\alpha_0^{l/2}\,\varepsilon_{cell}}{3}(1 + \varepsilon_r^{(k)})^2 \tag{5.11}$$

No explicit convergence criterion is checked (no tolerance/residual test between iterations): the code unconditionally performs 50 iterations and keeps the final result, with no formal guarantee of convergence built into the code itself — to be flagged as an implicit limitation (no check of the numerical stability of the fixed point is performed).

5.4.3 Sign convention — an explicit point of caution in the code

The docstring and the calling code (`page_results_export.py`, lines 848-857) explicitly flag a sign convention flip between the rest of the application and these radial/volumetric/equivalent strain formulas:

> *"This app's own strain/stress convention keeps compression positive, but the radial/volumetric/equivalent strain formulas (Special Tests page) use the opposite sign convention: axial strain negative in compression, radial strain positive (the specimen widens) — negate axial strain going into them only, leaving the stress-side calculations above untouched."*

Concretely, before calling `radial_strain()`, `volumetric_strain()` and `equivalent_strain()`, the code does:

```
axial_strain_signed = -strain           # (ou -axial_nominal_strain dans
    triaxial_synthetic_draws)
```

i.e. $\varepsilon_a^{signed} = -\varepsilon_a^{compression>0}$. This does not apply to the radial-stress computation (`linear_cell_radial_stress`/`nonlinear_cell_radial_stress`) nor to the stress-invariant computation (`stress_invariants`), which use the application's standard "compression positive" convention, with no sign change.

5.5 Derived quantities: radial stress, pressure, deviator, strains

5.5.1 Stress invariants: hydrostatic pressure and deviator

Implementation: `core/confinement.py::stress_invariants()` (lines 108-114), returning a `StressInvariants(pressure, deviator)` dataclass (lines 102-105).

Writing σ_a for the axial stress (`axial_stress`, from the already-computed SHPB/DI/QS result) and σ_r for the radial stress computed in §2 or §3:

$$p = \frac{\sigma_a + 2\sigma_r}{3} \qquad \text{(hydrostatic pressure)} \tag{5.12}$$

$$q = |\sigma_a - \sigma_r| \qquad \text{(stress deviator)} \tag{5.13}$$

The arrays are truncated to the shortest common length before computation (`n = min(len(axial_stress), len(radial_stress))`). Note that q is defined here as a simple absolute difference $|\sigma_a - \sigma_r|$, not the full generalized von Mises invariant $q = \sqrt{\frac{3}{2}s_{ij}s_{ij}}$ — in the axisymmetric case with two distinct principal stresses (σ_a, σ_r repeated twice), this expression does coincide with $q_{vM} =$

$|\sigma_a - \sigma_r|$, so it is consistent, but this deserves to be stated explicitly in the documentation since it is not written in the code in the classic $\sqrt{3/2}\, s : s$ form.

5.5.2 Radial strain

See §4 — `core/confinement.py::radial_strain()`, an implicit formula solved by fixed point.

5.5.3 Volumetric strain

Implementation: `core/confinement.py::volumetric_strain()` (lines 243-248).

$$\varepsilon_v = (1 + \varepsilon_a)(1 + \varepsilon_r)^2 - 1 \tag{5.14}$$

where ε_a is the signed axial strain (negative-in-compression convention, §4.3) and ε_r the radial strain computed in §4. This formula is the classical definition of the finite volumetric strain from the principal elongations $1 + \varepsilon_a$ (axial) and $(1 + \varepsilon_r)$ repeated twice (the two radial directions), under the assumption of axial symmetry.

5.5.4 Equivalent (von Mises) strain

Implementation: `core/confinement.py::equivalent_strain()` (lines 251-256).

$$\varepsilon_{eq} = \frac{2}{3} \left| \ln(1 + \varepsilon_a) - \varepsilon_r \right| \tag{5.15}$$

where $\ln(1 + \varepsilon_a)$ is the "true" (logarithmic) axial strain derived from the signed engineering strain ε_a, and ε_r the radial strain (§4). This formula mixes a "true" (logarithmic) axial-strain measure with an engineering radial strain — the mixed log/engineering usage is taken as-is from the code, with no additional justification in the current docstring beyond the formula itself.

5.5.5 Summary — full computation chain

According to `ui/pages/page_results_export.py` (method `_refresh_confinement`, lines 865 onward), the computation sequence for a confined test is:

1. Read the nominal axial strain/stress of the selected SHPB/DI/QS result (`nominal_strain`/`nominal_stress`, or `strain`/`stress` as a fallback) — lines 911-915.
2. Read the signal of the sensor holding the `confinement_cell` role, via `_sensor_signal_for_result()` — which already prefers the signal cut/resampled per Monte-Carlo draw (`sensor_{idx}_signal`, see §5.8) when this result was computed with "Include this cut in the Monte-Carlo loop" checked for that sensor, and otherwise falls back to the raw/manually-cut signal — line 922. Post-toe-correction realignment (lines 936-940): when this signal falls back to the raw one (sensor not re-cut per draw), it was itself never touched by any toe correction (SHPB chapter, §5.5) applied to the strain/axial curve read in step 1 — without intervention, its leading points would then stay offset by the number of points the toe correction removed from the head of that curve. `_refresh_confinement()` re-reads the integer `toe_n_removed` (accumulated by `_on_toe_apply()`, see SHPB chapter §5.5) stored at the root of the result, and removes that same number of points from the head of the raw cell signal (`cell_defl = cell_defl[cut:]`, with `cut = min(toe_n_removed, len(cell_defl))`) before the call in step 3

below — only when `cell_used_mc` is false, since the per-draw-cut path (§5.8) is already correctly aligned by construction (see SHPB chapter §5.5). The same realignment is applied independently to the optional laser signal (step 6 below, lines 987-993) whenever it too comes from the raw channel.

3. Compute the mean radial stress σ_r via `linear_cell_radial_stress()` (§2) or `nonlinear_cell_radial_stress()` (§3) depending on `conf.nonlinear_enabled` — lines 942-963 (from the MEAN curves above, independently of the per-draw computation of §5.8).
4. Compute the invariants p, q via `stress_invariants()` (§5.1) — line 966.
5. Sign-flip the axial strain (`axial_strain_signed = -strain`) then compute ε_r, ε_v, ε_{eq} via `radial_strain()`, `volumetric_strain()`, `equivalent_strain()` (§4-5.3-5.4) — lines 974-977.
6. Laser option: if a sensor with role `laser_velocity` is present, an independent axial stress via `laser_stress()` (§5.6), then invariants p_{laser}, q_{laser} recomputed with the same σ_r — lines 979-1002, around 1069-1075 (see limitation 9 in §7: the laser remains, by explicit design choice, outside the Monte-Carlo loop computation of §5.8).
7. Uncertainty band for quantities 3-5 (gage): real if the current result was computed with §5.8 (`confinement_radial_stress_std` present), otherwise synthetic via `triaxial_synthetic_draws()` (§5.7) — around lines 1020-1046. Uncertainty band for the laser variant (6): always synthetic, via `triaxial_synthetic_draws()`, regardless of the case above.

The four plots displayed ("Confinement / Triaxial Stresses" card) are: confinement stresses vs. axial strain, radial/volumetric/equivalent strains vs. nominal axial strain, stress path (p, q), and pressure p vs. volumetric strain ε_v (oedometric analogue). Only the origin of the uncertainty band shown as a dotted/transparent envelope on each of them varies (real or synthetic, per point 7 above); the mean curves and the plotting stay the same in both cases. The message shown below the plots (`confine_status`) explicitly states, for the gage-based part, whether the uncertainty comes from a true per-draw propagation or from the synthetic approximation — see §5.8.

5.5.6 Alternative axial stress via laser velocimetry

Implementation: `core/confinement.py::laser_stress()` (lines 29-37) — independent of the output bar gage signal, useful as a complement/cross-check:

$$\sigma_a^{laser} = \frac{D_{ob}^2}{D_{spe}^2} \cdot \frac{1}{2}\, \rho_{ob}\, C_{0,ob}\, \frac{v_{laser}}{10^6} \qquad \text{(MPa)} \tag{5.16}$$

with D_{ob} the output-bar diameter (`ob_diameter`), D_{spe} the specimen diameter (`specimen_diameter`), ρ_{ob} the output-bar density (`ob_rho`), $C_{0,ob}$ the wave speed in the output bar (`ob_c0`), v_{laser} the surface velocity measured by the laser (`laser_velocity`). The docstring states that this formula matches exactly `(D0**2/Dspe**2) * 0.5 * rho0 * C0 * laser_velocity / 1e6` from the original implementation.

5.5.7 Synthetic fallback envelope ("imitation") — triaxial_synthetic_draws

Implementation: `core/confinement.py::confinement_envelope()` (linear, lines 201-240), `nonlinear_confinement_envelope()` (nonlinear, lines 243-284), and `triaxial_synthetic_draws()` (all quantities, individual draws, lines 398-482).

`triaxial_synthetic_draws()` serves as the fallback for the uncertainty band shown on the Confinement card: it is used only when no real per-draw data is available for the quantity in question (see §5.8) — which remains systematically the case for the three "laser" quantities, by explicit design choice (see limitation 9 in §7).

`confinement_envelope/nonlinear_confinement_envelope` (the `ConfinementEnvelope` dataclass, lines 186-198) are, themselves, dead code: they are called nowhere in the application (verified by exhaustive search) — only `triaxial_synthetic_draws` is used in practice, both by `page_results_export.py` and by `page_covariance.py`.

Principle (`n_draws = 200` draws by default): at each synthetic draw, the following are perturbed:

- the axial stress, by Gaussian noise whose standard deviation equals its already-reported uncertainty (`axial_stress_std`),
- the cell deflection, by Gaussian noise whose standard deviation equals the mean of its reported uncertainty (`cell_deflection_std`),
- and, for the linear model only, the cell modulus E_v, by Gaussian noise whose standard deviation equals `cell_modulus_std_fraction * cell_modulus` (2 % by default, `cell_modulus_std_fraction`, matching the comment "matching the original's E_cd = np.random.normal(E_c, 0.02*E_c, N)"),

after which σ_r, p, q, ε_r, ε_v, ε_{eq}, and the "laser" variants of σ_a, p, q, are recomputed for that synthetic draw. `triaxial_synthetic_draws` returns the individual draws (`list_of_per_draw_arrays` per quantity), in the same format as the real computation of §5.8 (`project.last_raw_draws`) — which lets `page_results_export.py/page_covariance.py` handle the two sources interchangeably.

Why this is an approximation, not a true uncertainty propagation: each synthetic draw perturbs the axial stress and the cell deflection with fresh, independent Gaussian noise, with no link to any real Monte-Carlo draw of the SHPB/DI/QS reduction — whereas in reality, the axial stress AND the cell signal (when the latter is cut per draw, §5.8) both stem from the same random realization of the bar/specimen properties/sensor alignment for a given draw, and are therefore correlated. `triaxial_synthetic_draws` destroys this real correlation by drawing two independent noises — hence describing this approach as an "imitation" rather than a genuine uncertainty propagation, which is what motivates the real computation described in §5.8.

Robustness note implemented in `triaxial_synthetic_draws()` (lines ∼421-427): if the first nominal-strain point is very close to zero ($< 10^{-6}$ in absolute value) without being exactly 0, it is forced to 0.0 — the code comment explains that this prevents a floating-point residue from upstream processing from stopping the Covariance Analysis page from finding a draw that starts exactly at the "Reference (start) strain = 0.0" requested by the user. The same treatment is applied identically to the real draws of §5.8, on the `page_covariance.py` side.

5.5.8 Real computation in the Monte-Carlo loop (SHPB / Direct Impact / Quasi-Static)

Implementation: `core/confinement.py::per_draw_confinement_quantities()` (lines 327-395); wired in from `ui/pages/page_process.py` (`_prepare_confinement()`, lines 92-126, shared by the three tabs; used from `_SHPBTab._on_run`, `_DITab._on_run`, `_QSTab._on_run`).

Principle: rather than relying solely on σ_r, p, q, ε_r, ε_v, ε_{eq} computed once from the result's MEAN curves (in `_refresh_confinement`, §5.5) with an uncertainty approximated by independent synthetic draws (§5.7), these six quantities are computed once per Monte-Carlo draw, right inside the SHPB/Direct Impact/Quasi-Static reduction loop (`ui/pages/page_process.py`), from the real axial stress/strain of THAT draw combined with the real confinement cell signal of that same draw (when the latter exists — see below). The generic aggregation mechanism already used for every other per-draw quantity (`_aggregate_quantities()`/`_store_raw_draws()`, lines 71-101 of `page_process.py`) then automatically takes care of the mean, the standard deviation AND the raw set of draws, with no extra dedicated code — exactly as for `strain`, `stress`, or an existing `sensor_{idx}_signal`.

`per_draw_confinement_quantities(axial_strain, axial_stress, cell_deflection, conf, specimen_initial_length, poly_ini=None, poly_esp=None)` computes, for a single draw: the radial stress (`linear_cell_radial_stress()` or `nonlinear_cell_radial_stress_precomputed()` depending on `conf.nonlinear_enabled` — see §3.1 for the precomputation of the `poly_ini`/`poly_esp` polynomials), the invariants (`stress_invariants()`), then — with the same sign flip as in §4.3 — the three derived strains. It returns a dict with keys prefixed by `confinement_` (`confinement_radial_stress`, `confinement_pressure`, `confinement_deviator`, `confinement_radial_strain`, `confinement_volumetric_strain`, `confinement_equivalent_strain`), merged directly into the calling loop's `draw` dict.

`_prepare_confinement(project)` (lines 92-126 of `page_process.py`, shared by the three tabs) determines whether this computation is possible for this run: confinement enabled (`project.confinement.enabled`), an enabled Custom Sensor with role `confinement_cell` present, and — for the nonlinear model — valid calibration curves (otherwise it returns `None` and the per-draw computation is simply skipped for this run). When the nonlinear model is used, it also calls `fit_nonlinear_calibration()` once before the loop (see §3.1), rather than re-fitting the two polynomials at every draw.

Availability of the real per-draw cell signal — three cases, depending on the tab:

- **SHPB / Direct Impact**: the computation of §5.8 is active ONLY if the sensor with role `confinement_cell` ALSO has its "Include this cut in the Monte-Carlo loop" box checked ("Specimen-Mounted Sensor Alignment" mechanism, see the SHPB/Direct Impact chapter) — this is what gives this sensor a signal genuinely re-cut/resampled for each draw (`draw[f"sensor_{idx}_signal"]`), from the bar/specimen properties actually drawn for that particular draw. If the box is unchecked, `_prepare_confinement` detects the absence of this per-draw signal and the computation of §5.8 is simply disabled for this run — the Results & Export tab then falls back onto the synthetic envelope of §5.7, with an explicit message inviting the user to check the box.
- **Quasi-Static (QS)**: this tab has — and needs — no per-draw re-cutting mechanism (`cut_in_montecarlo`): there is no wave to propagate in a quasi-static test, hence no propagation delay to realign draw by draw — the cell sensor is already, by construction, on the same time base as the load/displacement. The computation of §5.8 is therefore active as soon as confinement is enabled and a `confinement_cell` sensor exists, but it combines the REAL axial stress/strain of each draw (already Monte-Carlo-varied through this tab's Cell Load/Measurement uncertainties) with the raw cell signal, identical for all draws (no Monte-Carlo variation specific to this signal on this tab — an explicit design choice, see limitation 10 in §7). The resulting uncertainty is therefore real but partial: it captures the variability on the axial side, not that of the cell channel itself.

In every case, the message shown at the end of processing (`status_label`, on the Processing page) explicitly states whether the computation of §5.8 was performed for this run, and why.

Likewise, `_refresh_confinement()` (§5.5) and the Covariance Analysis page (`_triax_draws()`, `ui/pages/page_covariance.py` lines 336-436) inspect the presence of `confinement_radial_stress_std`/`confinement_*` in the result/raw draws to determine, one quantity at a time, whether they can use this real data or must fall back on the synthetic approximation of §5.7 — and state this honestly in their respective status messages.

Consistency with a toe correction applied after the fact: the six `confinement_*` quantities computed by §5.8 live inside the `draw` dictionary of each Monte-Carlo draw, at the same length as the axial strain of that draw — exactly the generic format that the toe correction already resizes for every other per-draw quantity (`ui/pages/page_process.py::_apply_toe_correction_to_draws()`, see SHPB chapter §5.5). A toe correction applied after a Monte-Carlo run with §5.8 active ("Include this cut in the Monte-Carlo loop" checked) therefore automatically re-cuts these six quantities, draw by draw, with no additional dedicated code — `project.last_raw_draws["SHPB"/`

`"DI"]` and the aggregate `project.last_results[...]` then remain mutually consistent, and the Covariance Analysis page inherits the same correction. This was not the case before this fix (the toe correction only touched the aggregate): this is precisely the alignment defect ("offset") documented and fixed in the SHPB chapter, §5.5. The raw fallback path (sensor not re-cut per draw, `cell_used_mc` false in `_sensor_signal_for_result()`) remains, for its part, outside this per-draw mechanism — this is what is separately fixed by the `toe_n_removed` realignment described in §5.5 above, inside `_refresh_confinement()`.

5.6 Summary of function ↔ quantity correspondences

Quantity / computation	**Function (file `core/confinement.py`)**
Radial stress, linear model	`linear_cell_radial_stress()`
Radial stress, nonlinear model (fit + evaluation in one call)	`nonlinear_cell_radial_stress()` (uses `_eval_poly_no_constant()`)
Radial stress, nonlinear model — fitting the calibrations ALONE, once per run	`fit_nonlinear_calibration()` (lines 100-119)
Radial stress, nonlinear model — evaluation ALONE from already-fitted polynomials	`nonlinear_cell_radial_stress_precomputed()` (lines 122-140)
Axial stress from laser (optional)	`laser_stress()`
Hydrostatic pressure p, deviator q	`stress_invariants()` → `StressInvariants`
Uncertainty envelope, linear model	`confinement_envelope()` → `ConfinementEnvelope` (dead code, never called in the application)
Uncertainty envelope, nonlinear model	`nonlinear_confinement_envelope()` → `ConfinementEnvelope` (dead code, never called in the application)
Radial strain (alpha00, alpha0$^{l/2}$)	`radial_strain()`
Volumetric strain	`volumetric_strain()`
Equivalent (von Mises) strain	`equivalent_strain()`
Synthetic fallback Monte-Carlo draws (all quantities, including laser; for Covariance Analysis and the `_refresh_confinement` fallback)	`triaxial_synthetic_draws()` (§5.7)
Six confinement quantities for ONE real Monte-Carlo draw (radial stress, p, q, ε_r, ε_v, ε_{eq})	`per_draw_confinement_quantities()` (lines 327-395, §5.8)

UI entry point that orchestrates everything: `ui/pages/page_results_export.py::_refresh_confinement()` (the "Confinement / Triaxial Stresses" card) — computes the final display from data already aggregated (real when available, otherwise synthetic). Upstream, on the processing side, `ui/pages/page_process.py::_prepare_confinement()` (lines 92-126, shared by the three Processing tabs) is the entry point that decides whether the real per-draw computation (§5.8) is possible for this run and precomputes the nonlinear polynomials when applicable. User configuration of the constants (C_1, C_2, E_v, ε_{ref}, D_{cell}, α_{00}, $\alpha_0^{l/2}$, nonlinear calibration curves): `ui/pages/page_special_tests.py`. Parameter data structure: `core/project.py::ConfinementSettings` (dataclass, lines 108-134).

5.7 Limitations, assumptions and warnings found in the code

1. **No clamp at the calibration bounds**: in both models (linear §2.2 and nonlinear §3.4), the weighting between regimes/curves is an unbounded linear interpolation; outside the range $[0, \varepsilon_{ref}]$ it becomes an extrapolation. This is not flagged as an explicit warning in the code, but it is a direct consequence of the formula and must be flagged to the user.
2. **Nonlinear calibration polynomial forced through the origin**: `_eval_poly_no_constant( )` deliberately eliminates the constant term of the degree-6 `np.polyfit` fit, which forces $\sigma_r(\varepsilon_{ortho} = 0) = 0$ — an assumption justified in the docstring as "physically sensible", but one that degrades the fit quality relative to a free 7-coefficient fit.
3. **Fixed-point iteration convergence not checked**: `radial_strain()` unconditionally performs `n_iter=50` iterations with no convergence test (no early-stopping criterion, no check that the residual decreases) — an implicit limitation of the code, not to be presented as a method that guarantees convergence.
4. **Reversed sign convention for the derived strains**: the functions `radial_strain()`, `volumetric_strain()`, `equivalent_strain()` expect a negative-in-compression axial strain (opposite to the "compression positive" convention used by the rest of the application), hence the explicit negation (`axial_strain_signed = -strain`) performed by the caller before invoking them — explicitly documented in a comment in `page_results_export.py` (lines 848-853). This does NOT apply to the stress computations (σ_r, p, q), which remain in the "compression positive" convention.
5. **Deviator** $q = |\sigma_a - \sigma_r|$: a simplified two-principal-stress form (axisymmetric), correct in that setting but not written in the general $\sqrt{3/2\, s_{ij} s_{ij}}$ form — to be clarified in the documentation to avoid any confusion with a general 3D computation.
6. **Mixed log/engineering equivalent strain**: $\varepsilon_{eq} = \frac{2}{3}|\ln(1 + \varepsilon_a) - \varepsilon_r|$ combines a true (logarithmic) axial strain with an engineering radial strain — a combination carried over as-is from the original code, with no further justification in the current docstring.
7. **Cell modulus perturbed at a fixed 2 %**: in the linear model's Monte-Carlo envelopes, the standard deviation of E_v is fixed at 2 % of its value (`cell_modulus_std_fraction`, the default value of `ConfinementSettings`), configurable but with this default value explicitly matched to the original (`E_cd = np.random.normal(E_c, 0.02*E_c, N)`). In the nonlinear model, no equivalent uncertainty on the calibration curves is modeled (only the axial stress and the cell deflection are perturbed).
8. **Systematic truncation to the minimum length**: every function in `core/confinement.py` silently truncates its input arrays to `min(len(...))` with no warning if the lengths differ — a behavior to keep in mind when interpreting results in case of signals of differing lengths (axial stress vs. cell signal vs. laser signal). This truncation remains position-to-position, not time-based: it implicitly assumes the same sampling step between the two combined signals. For the raw fallback path (sensor not re-cut per draw), `_refresh_confinement()` now corrects the leading offset that a toe correction would introduce (`toe_n_removed`, see SHPB chapter §5.5 and §5.5 above) before this truncation — but a difference in sampling rate between the sensor and the stress-strain curve is, itself, neither detected nor corrected, with or without a toe correction.
9. **Real per-draw computation (§5.8) gated, on SHPB/Direct Impact, by an independent checkbox**: the Monte-Carlo-loop computation of the confinement quantities is enabled on these two tabs only if the `confinement_cell` sensor separately has its "Include this cut in the Monte-Carlo loop" box (Specimen-Mounted Sensor Alignment mechanism) checked. Nothing technically prevents leaving confinement enabled without checking this box: in that case, the per-draw computation is silently skipped (no error), and the whole downstream chain (processing status, `_refresh_confinement`, Covariance Analysis) falls back onto the synthetic approximation of §5.7 — with an explicit message, but the user must

read that message to notice it.

10. **Cell signal not Monte-Carlo-varied on the Quasi-Static tab**: unlike SHPB/Direct Impact, the per-draw computation on QS combines a genuinely drawn axial stress/strain with a fixed confinement cell signal, identical for every draw of this run (an explicit choice: there is no per-draw re-cutting mechanism specific to QS, see §5.8). The resulting uncertainty for the confinement quantities on this tab is therefore real but partial: it does not capture the variability intrinsic to the cell channel (measurement noise, sensor calibration uncertainty), only that transmitted through the variability of the axial stress/strain.
11. **Laser-velocimetry quantities always synthetic, never computed in the loop**: `axial_stress_laser`, `pressure_laser` and `deviator_laser` are, by explicit design choice, never computed by `per_draw_confinement_quantities()` in the Monte-Carlo loop (§5.8) — even when the six other confinement quantities are, for that same run. They systematically come from the synthetic envelope of §5.7, regardless of the availability of real per-draw data for the cell-gage-based quantities.
12. `toe_n_removed` **is a median across draws, not an exact count**: since the toe correction re-cuts each Monte-Carlo draw individually (SHPB chapter, §5.5), the number of points removed from the head can slightly differ from one draw to another. The `toe_n_removed` counter re-read by `_refresh_confinement()` (§5.5 above) to realign a confinement/laser sensor that remained a raw signal takes the median of this count across all surviving draws — a reasonable approximation as long as this inter-draw variation stays small, but not an exact match for this specific fallback path (the per-draw-cut path of §5.8, for its part, remains exact since it re-cuts directly the real signal specific to each draw).

6. Digital Image Correlation (DIC)

The DIC (Digital Image Correlation) module of `shpb_app` tracks, over an image sequence or a video, characteristic points of a speckle pattern applied to the specimen, in order to derive displacement, strain, velocity and acceleration fields, together with a dynamic force for the region of interest (ROI). The module also includes a predictive estimator of measurement uncertainty.

Relevant source files:

- `core/dic_preprocessing.py` — image preprocessing (crop, 8-bit scaling, downscale, temporal decimation).
- `core/dic_correlation.py` — defines the ROI and the correlation mesh (despite its name, this file does not contain the tracking algorithm itself: `ROI`, `Ellipse`, `MeshGrid`, `build_mesh*`).
- `core/dic_processing.py` — this is the file that contains the actual tracking/correlation algorithm (`run_dic`, `_track_subset`, `_subpixel_peak`, `_grid_strain`, `_apply_neighbor_consistency_filter`) and the result structures (`DICFrameResult`, `DICResult`).
- `core/dic_kinematics.py` — kinematic derivation (velocity, acceleration, von Mises equivalent strain, strain rate, dynamic force) and conversion to true strain (`nominal_to_true_strain`).
- `core/dic_uncertainty.py` — the full, predictive DIC measurement-uncertainty model.
- `core/dic_result_io.py` — re-reads a CSV export (`_on_export_results`) back into a `DICResult`, to reopen a past run without recomputing it (§4.8).
- `core/dic_render.py` — graphical rendering of a DIC field onto an image (Delaunay triangulation), shared by the live display and the video export.
- `core/camera_clock.py` — a separate ("Camera / Video Sync") tool for reducing a camera clock file; it does not directly feed into the DIC computation but is related to the topic of time synchronization.
- `core/project.py` (the `DICSettings` dataclass, lines ∼170–299) — all persisted parameters (default values quoted below).
- `ui/pages/page_image_correlation.py` — UI orchestration, vocabulary, effective-time formulas (`_effective_fps`, `_dic_time_s`).

General module warning (`core/dic_processing.py`, header docstring): this is not a full deformable-subset DIC solver (unlike commercial software such as VIC-2D or Ncorr, which also optimize the local rotation/shear/stretch of each subset during matching). Each subset here is tracked as a rigid template in pure translation, located by `cv2.matchTemplate`, with a 3-point parabolic sub-pixel refinement. This is judged sufficient for the moderate, predominantly uniaxial deformations typical of an SHPB / direct impact / quasi-static test, at a much lower implementation cost than a full subset-shape optimization.

6.1 Pipeline overview

1. Spatial preprocessing (`core/dic_preprocessing.preprocess_frame`): crop → conversion to a displayable 8-bit image → downscale.
2. Temporal decimation (`core/dic_preprocessing.decimated_frame_indices`): selecting one frame out of every N.
3. Defining the ROI and the correlation mesh (`core/dic_correlation.py`): rectangle / polygon

/ ellipse, a regular grid of points spaced by `step`, each the center of a `subset_size` × `subset_size` subset.
4. Tracking/correlation (`core/dic_processing.run_dic`): matching by normalized cross-correlation (NCC), frame by frame, in "Fixed" or "Moving" mode, with quality filters (correlation threshold, neighbor consistency).
5. Local strain field (`core/dic_processing._grid_strain`): gradient of the displacement field by finite differences on the regular grid.
6. Derived kinematics (`core/dic_kinematics.compute_kinematics`): velocity, acceleration, von Mises equivalent strain/strain rate, ROI dynamic force.
7. Conversion to true strain (`core/dic_kinematics.nominal_to_true_strain`), optional, when publishing to Results & Export.
8. Camera/gage time correction (`time_scale`, `time_offset_s`, handled in `page_image_correlation.py`) — no automatic synchronization.
9. Uncertainty model (`core/dic_uncertainty.py`), optional, propagated from displacement all the way to the dynamic force.

6.2 Preprocessing (core/dic_preprocessing.py)

Three independent, optional steps, always applied in this order (module docstring):

1. Crop (`crop_frame`) to an ROI rectangle (defined in `core/dic_correlation.py`) drawn on the raw (full-resolution) image, clamped to the image bounds (`ROI.clamped`).
2. Contrast stretching to a displayable/analyzable 8-bit image (`frame_source.to_display_uint8`).
3. Downscale (`downscale_frame`) by an integer factor `factor` ≥ 1 (1 = no downscale), by area averaging (`cv2.INTER_AREA`) — the appropriate interpolation for shrinking an image without aliasing.

Orchestration function: `preprocess_frame(raw_frame, crop, downscale_factor)`, which returns a contiguous uint8 array. Important point: the ROI and the correlation mesh, together with the whole correlation engine, work entirely in the pixel coordinates of this preprocessed image — never in those of the raw image.

Temporal decimation (processing 1 frame out of every N rather than all of them) is an independent concept, handled by:

```
decimated_frame_indices(n_frames, reference_frame, step)
```

which returns the list `[reference_frame, reference_frame+step, reference_frame+2*step, ...]` until the end of the sequence. The reference frame (undeformed state, zero displacement) is always included first; DIC only tracks deformation forward in time — no frame prior to `reference_frame` is ever processed.

Default values (`core/project.py`, `DICSettings`): `downscale_factor = 1`, `frame_step = 1`.

6.3 ROI and correlation mesh (core/dic_correlation.py)

This file only builds and validates the grid of points to be correlated; the correlation itself is a separate, later processing step (`core/dic_processing.py`).

6.3.1 ROI shapes

- **ROI**: a rectangle $(x, y, \text{width}, \text{height})$ in the pixel coordinates of the (preprocessed) reference frame. `clamped()` constrains it within a `frame_width` × `frame_height` frame.
- **Ellipse**: an axis-aligned oval (center (c_x, c_y), semi-axes r_x, r_y) — suited to a cylindrical specimen viewed in profile, so as not to waste correlation points on corners outside the specimen. `bounding_roi()` gives the smallest enclosing rectangle.
- **Polygon**: a list of vertices (x, y), for specimen shapes that a rectangle does not fit well (dogbone, etc.).

6.3.2 Building the grid — build_mesh(roi, subset_size, step)

Points spaced `step` pixels apart, centered `subset_size`/2 pixels in from the ROI edge (so that each subset stays entirely inside the ROI):

$$x_0 = \text{roi.x} + \frac{\text{subset_size}}{2}, \qquad x_1 = \text{roi.x} + \text{roi.width} - \frac{\text{subset_size}}{2} \tag{6.1}$$

(likewise for y), then:

```
xs = np.arange(x0, x1 + 1e-9, step)
ys = np.arange(y0, y1 + 1e-9, step)
```

If $x_1 < x_0$ or $y_1 < y_0$ (ROI too small for even a single subset), an empty grid (`n_points == 0`) is returned — it is up to the caller to detect this case and warn the user, rather than treating it as a silent error.

6.3.3 Non-rectangular ROIs

- `build_mesh_from_polygon(vertices, subset_size, step)`: first builds the regular grid over the polygon's bounding box (`build_mesh`), then masks (`MeshGrid.mask`) every point whose center falls outside the polygon (test via `matplotlib.path.Path.contains_points`).
- `build_mesh_from_ellipse(ellipse, subset_size, step)`: the same principle over `ellipse.bounding_roi()`, with the membership test

$$\left(\frac{x - c_x}{r_x}\right)^2 + \left(\frac{y - c_y}{r_y}\right)^2 \leq 1 \tag{6.2}$$

Explicit warning from the code: this masking only tests the center of each point — a subset near the boundary can therefore overhang slightly outside the ROI; the mask is only used to decide which points count as "inside the ROI" for statistics/results, not to precisely clip the outline of each subset.

A masked point (outside the ROI) is treated by `core/dic_processing.py` exactly like a point whose subset falls outside the image: excluded from tracking, NaN in every result field.

6.3.4 MeshGrid structure

- `xs`, `ys`: 1D coordinates of the grid's columns/rows.
- `subset_size`, `step` (in pixels).
- `mask`: a boolean array (or `None` for a simple rectangular ROI, which never has holes).

- `n_points = n_points_x * n_points_y` (grid of the bounding box), `n_valid_points` = the number of points actually inside the ROI (`np.count_nonzero(mask)` if a mask exists, otherwise = `n_points`).

Default values (`DICSettings`): `subset_size_px = 21`, `step_size_px = 7`.

6.4 Tracking/correlation algorithm (core/dic_processing.py)

6.4.1 General principle

Each mesh point is tracked as a rigid template (a `subset_size` × `subset_size`-pixel subset, cut once and for all from the reference frame, never re-cut from a later frame — this prevents the correlation error of one step from propagating/drifting over the long run). This template is searched for in every processed frame by normalized cross-correlation.

6.4.2 Normalized cross-correlation and search window

Key function: `_track_subset(frame, template, center_x, center_y, search_radius)`.

A search window is defined around the center $(center_x, center_y)$, extended by `search_radius` pixels beyond the template on each side:

$$sx_0 = \text{round}\left(center_x - \frac{t_w}{2} - r\right), \qquad sx_1 = \text{round}\left(center_x + \frac{t_w}{2} + r\right) \tag{6.3}$$

$$sy_0 = \text{round}\left(center_y - \frac{t_h}{2} - r\right), \qquad sy_1 = \text{round}\left(center_y + \frac{t_h}{2} + r\right) \tag{6.4}$$

(with t_w, t_h the template dimensions, r = `search_radius`), then clipped to the frame bounds. If the clipped window is smaller than the template in either direction ($sx_1 - sx_0 < t_w$ or $sy_1 - sy_0 < t_h$), the point is declared lost for this frame: returns $(center_x, center_y, \text{NaN})$.

The correlation itself is computed by OpenCV:

```
corr_map = cv2.matchTemplate(search_window, template, cv2.TM_CCOEFF_NORMED)
```

i.e. the normalized cross-correlation coefficient (NCC), defined by OpenCV as:

$$R(x,y) = \frac{\sum_{x',y'}(T(x',y') - \bar{T})(I(x+x',y+y') - \bar{I}(x,y))}{\sqrt{\sum_{x',y'}(T(x',y') - \bar{T})^2 \cdot \sum_{x',y'}(I(x+x',y+y') - \bar{I}(x,y))^2}} \tag{6.5}$$

where T is the template (reference subset), $\bar{T}$ its mean, I the search window, $\bar{I}(x,y)$ the local mean of the patch of I at the candidate position (x,y). $R \in [-1,1]$, 1.0 = a perfect match. The peak (position and maximum value) is found by `cv2.minMaxLoc(corr_map)`.

6.4.3 Sub-pixel refinement — _subpixel_peak

A 3-point parabolic interpolation, independently in x and in y, around the integer peak (p_y, p_x):

$$\delta_x = \frac{1}{2} \cdot \frac{c_0 - c_2}{c_0 - 2c_1 + c_2} \tag{6.6}$$

with c_0, c_1, c_2 the correlation values at abscissas $p_x - 1, p_x, p_x + 1$ (likewise in y for δ_y), the offset only applied if $|c_0 - 2c_1 + c_2| > 10^{-12}$ (avoids dividing by ~ 0). The code falls back to the integer peak position (zero offset) if the peak sits on the edge of the correlation map (no neighbor on one side to fit a parabola).

The new position of the subset's center in the current frame is:

$$x_{new} = sx_0 + p_x + \delta_x + \frac{t_w}{2}, \qquad y_{new} = sy_0 + p_y + \delta_y + \frac{t_h}{2} \tag{6.7}$$

Documented limitation (`core/dic_uncertainty.py`, section "WHAT THIS MODULE DOES NOT MODEL"): this 3-point parabolic refinement has a slight interpolation bias near integer-pixel displacements, on the order of a few hundredths of a pixel in the worst case — a bias not modeled in the uncertainty estimate.

6.4.4 "Fixed" vs. "Moving" tracking modes

The `tracking_mode` parameter of `run_dic` (`"fixed"` by default, `"moving"` otherwise — any other value is treated as `"fixed"`). What changes is only where the search window of each frame is centered:

- `"fixed"`: always centered on the point's original position in the reference frame (xx_0, yy_0). Simple and immune to the propagation of an earlier bad match, but `search_radius` alone must cover the point's total displacement over the whole processed sequence.
- `"moving"`: centered on the position found in the previously processed frame (cur_x, cur_y). Lets a narrow `search_radius` follow a large cumulative displacement, at the cost that a bad match in one frame shifts the search window (and hence every following frame) far from the point's true position.

Default values: `search_radius_px = 15`, `tracking_mode = "fixed"`.

6.4.5 Correlation quality filter — correlation_threshold

If `correlation_threshold > 0`, a point is permanently dropped ("dropped for good", NaN for this frame and all subsequent ones) as soon as its peak NCC in some frame falls below this threshold:

```
if correlation_threshold > 0 and not (c >= correlation_threshold):
    active[iy, ix] = False
```

The test `not (c >= threshold)` rather than `c < threshold` also catches the case $c = \text{NaN}$ (search window entirely off the image). By default `correlation_threshold = 0.0`: no point is ever dropped for quality reasons alone.

Typical documented cause of a point being lost mid-sequence: necking of the specimen cross-section, which causes a point originally on the specimen to end up on the background, so the reference template no longer matches it at all.

6.4.6 Neighbor consistency filter — _apply_neighbor_consistency_filter

A numerical analogue of finite-element "hourglass control" (the term used in the docstring). For each point still active in a given frame, its displacement $(u, v) = (x_{new} - x_0, y_{new} - y_0)$ is compared to the median of the displacements of its active immediate neighbors (4-connectivity, up/down/left/right on the grid):

$$\text{deviation} = \sqrt{(u - \text{median}(u_{neighbors}))^2 + (v - \text{median}(v_{neighbors}))^2} \tag{6.8}$$

If `deviation > max_neighbor_deviation_px`, the point is permanently removed (`active = False`, position and correlation set to NaN for this frame). An isolated point (all its neighbors already excluded) is compared to nothing and is never rejected by this filter. Disabled by default (`max_neighbor_deviation_px = 0.0`).

Documented purpose: to catch the case of a point that matched well (so that `correlation_threshold` does not reject it) but has "latched onto" a similar-looking neighboring speckle pattern, slightly offset from the true one — more likely when the search window is large. Such an isolated point visibly makes the deformed mesh "jagged" all around it.

6.4.7 Main loop — run_dic

1. Checks that `frame_indices` is not empty and that `mesh.n_points > 0`.
2. Takes xx_0, yy_0 = `mesh.grid_xy()` as reference positions.
3. Cuts a `subset_size` × `subset_size` template centered on each valid point (`mesh.mask`) of the reference frame; excludes (`valid = False`) any point whose subset overhangs the frame.
4. Computes the uncertainty ingredients (`estimate_noise_sigma`, `subset_sssig_xy`, see §7) once, on the reference subsets — a silent failure (best-effort, never interrupts an otherwise successful run; the error is logged in `uncertainty_error`).
5. `active = valid.copy()`: a mask that only loses points, never regains any, as the sequence proceeds.
6. For each processed frame (in order, sequentially — parallelization via `n_workers > 1` applies only within a single frame, never between frames, since "moving" mode centers each frame on the result of the previous one):
   - Reference frame: position/correlation fixed at $(xx_0, yy_0, 1.0)$.
   - Any other frame: tracking of every active point (`_track_subset`, possibly spread across multiple threads via `core/parallel.run_map` — threads because `cv2.matchTemplate` releases the GIL), followed by the correlation threshold, then the neighbor consistency filter.
   - $u, v = new_x - xx_0, new_y - yy_0$.
   - Computation of the local strain field `_grid_strain` (§5).
7. Returns a `DICResult` (mesh, frame indices, list of `DICFrameResult`, plus the uncertainty ingredients of the reference frame).

6.4.8 Exporting and re-importing results (CSV) — core/dic_result_io.py

`DICResult` is never persisted in the session file (docstring of `DICSettings`, `core/project.py`): a DIC run is kept in memory on the Image Correlation page itself, "re-run on demand", to keep the session file lightweight. The only way this module offers to get a run out of the page is the "Export results (.csv)…" button (`page_image_correlation.py::_on_export_results`): a file with one row per (frame, grid point), with columns `frame_index`, `time_s`, `point_row`, `point_col`, `x0`, `y0`, `x`, `y`, `u`, `v`, `u_mm`, `v_mm`, `exx`, `eyy`, `exy`, `correlation`.

The "Import results (.csv)…" button, next to it, re-reads this same format to rebuild a `DICResult` without having to rerun the correlation — useful for comparing several runs, resuming a reduction on another machine, or simply reopening an old result after closing the session. `core/dic_result_io.load_dic_result_csv(path)` performs the reading; `page_image_correlation.py::_on_import_results` then calls exactly the same `_on_run_finished(result)` as a real correlation run, to benefit from the entire existing display/export/publish chain without duplicated code.

What is reconstructed identically (no loss, same units, the same NaN-for-excluded-point convention as the original): the regular point mesh (`x0/y0` → `mesh.xs/mesh.ys`, with `step` recovered exactly from the grid's median spacing), and every frame — tracked position, displacement, correlation, and the strain fields (`exx/eyy/exy`, an entire array falling back to `None` if the whole frame was empty at export time — the same convention as `DICFrameResult`).

What is not in the file, and is therefore absent after re-import: the run's settings (subset size, ROI shape/location, search radius, tracking mode, quality thresholds, crop/downscale/decimation preprocessing) and the raw uncertainty ingredients (`noise_sigma`, `sssig_x`, `sssig_y`). A re-imported result therefore systematically has `noise_sigma = sssig_x = sssig_y = None` — exactly the state that `compute_uncertainty()` already treats as "no uncertainty estimate available" for any other reason (§9.9), so no uncertainty envelope is offered for a re-imported result, with no special-case code needed. `mesh.mask` is left as `None` (a point masked by a non-rectangular ROI and a point that simply was never tracked are both NaN across the whole sequence in the export — indistinguishable, and treated identically downstream); `mesh.subset_size` receives `step` as an inert placeholder value (`core/dic_uncertainty.py` never reads `mesh.subset_size` without first checking `noise_sigma is not None`, a condition always false here).

Recovery of two calibrations, when the file contains them (non-empty columns at export time):

- **Pixel size** (`mm_per_working_px`): the `u_mm`/`v_mm` columns are only written if a pixel size was entered at export time (`scale = self._mm_per_working_px()`). When present, the ratio `u_mm / u` (and `v_mm / v`) recovers this same scale factor — already "mm per working pixel", i.e. already including the downscale of the original run. The value kept is the median of this ratio, grouped over both axes and over every available point/frame (rather than a single row, which the export's `%.6g`/`%.4g` rounding could bias).
- **Effective camera frequency** (`fps`): the `time_s` column is only written if a camera frequency was entered (`fps = self._effective_fps()`); `frame_index` itself is always exact. The ratio `(frame_index - ref_index) / time_s`, over every frame other than the reference, recovers this same effective frequency (camera frequency × `time_scale`), here too by taking the median over every available row.

In both cases, only the product of two settings is recoverable, never their original decomposition (`pixel_size_mm` × `downscale_factor`; `frame_rate_hz` × `time_scale`) — because it is this product, and this product alone, that enters every downstream calculation (§8). On import, the page therefore places the recovered value into a single field and neutralizes the other factor (`downscale = 1`; `time_scale = 1.0`, `time_offset_s = 0.0`), which reproduces exactly the same computations regardless of the original decomposition. A nonzero `time_offset_s` at export time is not recoverable at all (it is an additive constant, invisible in a ratio) and is assumed to be zero. When a calibration column is absent or empty, the corresponding value is not recovered (`None`) rather than approximated — the status message shown after import says so explicitly, rather than suggesting a calibration exists when it does not.

What import never touches: the page's subset size, step, frame step and search radius sliders — their change handlers all invalidate the current result (`_invalidate_results()`), which would immediately erase the result that was just imported. Import moreover needs no video/sequence loaded beforehand: the page works equally well from a blank page (the background image preview on the result plots is then simply absent — a "nice-to-have", never required to display the fields themselves).

6.5 Local strain field — _grid_strain (core/dic_processing.py, lines 188–202)

```
def _grid_strain(xx0, yy0, u, v):
    dx = np.gradient(xx0, axis=1)
    dy = np.gradient(yy0, axis=0)
    dudx = np.gradient(u, axis=1) / dx
    dudy = np.gradient(u, axis=0) / dy
    dvdx = np.gradient(v, axis=1) / dx
    dvdy = np.gradient(v, axis=0) / dy
    return dudx, dvdy, 0.5 * (dudy + dvdx)              # exx, eyy, exy
```

That is, on the regular grid (nominal spacing = `step`, in both x and y):

$$\varepsilon_{xx} = \frac{\partial u}{\partial x}, \qquad \varepsilon_{yy} = \frac{\partial v}{\partial y}, \qquad \varepsilon_{xy} = \frac{1}{2}\left(\frac{\partial u}{\partial y} + \frac{\partial v}{\partial x}\right) \tag{6.9}$$

where u, v are the components of the displacement field (x - reference x, y - reference y). The derivatives are computed by centered finite differences (`np.gradient`), with a one-sided scheme at the grid edges (native `np.gradient` behavior). ε_{xy} above is the tensorial shear strain (half the engineering shear $\gamma_{xy} = 2\varepsilon_{xy}$) — a convention explicitly restated in `core/dic_kinematics.py`.

Nature of the approximation — the code's own explicit statement (docstring of `_grid_strain`):

> *"First-order (small-strain) engineering strain field from the displacement field [...] A quick, local estimate — good for spotting where/when strain concentrates — not a substitute for a proper large-deformation DIC strain computation."*

This means it is:

- a first-order (linearized, small-strain) estimate of the displacement gradient, hence structurally close to a nominal/engineering strain (small relative displacements between neighboring grid points), not the logarithmic/true strain used by the application's other reduction engines (standard SHPB/QS engine);
- a local finite-difference method on a regular mesh, with no global reconstruction (no smoothing/regularization by finite elements or polynomials as in some commercial DIC software);
- a NaN point in a frame (lost track, excluded by the quality or neighbor filter) stays NaN in `exx`/`eyy`/`exy` at that point; since `np.gradient` uses a centered scheme, losing one point only contaminates the gradients of its immediate neighbors (the centered difference touching that point), not the whole grid (as stated in the docstring of `core/dic_kinematics.py`).

`u.shape[0] < 2` or `u.shape[1] < 2` → returns `(None, None, None)` (grid too small to compute a gradient).

6.5.1 Von Mises equivalent strain — core/dic_kinematics.von_mises_strain

$$\varepsilon_{eq} = \sqrt{\frac{2}{3}(\varepsilon_{xx}^2 + \varepsilon_{yy}^2) + \frac{4}{3}\varepsilon_{xy}^2} \tag{6.10}$$

This is the standard formula $\varepsilon_{eq} = \sqrt{\frac{2}{3}\varepsilon_{ij}^{dev} : \varepsilon_{ij}^{dev}}$ reduced to the plane-strain case with engineering shear $\gamma_{xy} = 2\varepsilon_{xy}$.

Plane-strain assumption made explicit (docstring of the `dic_kinematics.py` module): the out-of-plane strain ε_{zz} is taken as zero, since a single 2D camera never measures it directly (a true 3D/

stereo DIC setup would give ε_{zz} directly). This is the simplest and most standard assumption in 2D DIC, applied as-is and documented at the point of use rather than hidden.

The same formula applied to the strain-rate tensor (`exx_rate`, `eyy_rate`, `exy_rate`, obtained by time differentiation, see §6) gives the von Mises equivalent strain rate.

6.6 Derived kinematics — core/dic_kinematics.compute_kinematics

6.6.1 Time axis

```
t_s = [(fi - ref_index) / fps for fi in frame_indices]
```

`fps` = the number of frames actually processed per second (camera frequency already divided by the `frame_step` decimation, see §2); `ValueError` if `fps` is not strictly positive.

6.6.2 Velocity and acceleration

Optional pixel → mm conversion via `mm_per_px` (otherwise everything stays in px). Then time differentiation by finite differences (centered inside, one-sided at the edges), via `np.gradient( stack, t_s, axis=0)` (function `_gradient_over_time`), which handles a non-uniform time step directly from the `t_s` coordinate array:

$$v_x = \frac{dx}{dt}, \quad v_y = \frac{dy}{dt}, \quad v_{mag} = \sqrt{v_x^2 + v_y^2} \tag{6.11}$$

$$a_x = \frac{dv_x}{dt}, \quad a_y = \frac{dv_y}{dt}, \quad a_{mag} = \sqrt{a_x^2 + a_y^2} \tag{6.12}$$

Units: mm/s, mm/s^2 if `mm_per_px` is supplied, otherwise px/s, px/s^2 (`Kinematics.length_unit`).

NaN propagation (module docstring): since `np.gradient` is a centered-difference scheme, a NaN value in one frame only contaminates the point(s) immediately adjacent in time (its centered-difference neighbors), never the whole sequence, nor other spatial points.

6.6.3 ROI dynamic force — constant-section assumption

```
a_mag_m_s2 = a_mag_stack * 0.001          # mm/s^2 -> m/s^2
dynamic_force_stack = rho_kg_m3 * area_m2 * a_mag_m_s2                # F = rho * A * a, en
    Newtons
```

that is:

$$F_{dyn} = \rho \cdot A \cdot a_{mag} \tag{6.13}$$

with ρ in kg/m^3, A (`area_m2`) in m^2, a_{mag} converted to m/s^2. Dimensional analysis made explicit in the code:

$$\left[\frac{kg}{m^3}\right] \times [m^2] \times \left[\frac{m}{s^2}\right] = \left[\frac{kg \cdot m}{s^2}\right] = [N] \tag{6.14}$$

`dynamic_force` is only computed if `rho_kg_m3`, `area_m2` and `mm_per_px` are all three supplied (a physical density/area needs a physical acceleration to combine with).

Physical interpretation and constant-section assumption (docstring of `compute_kinematics`): $\rho \times A$ represents the "mass per unit length" of a prismatic specimen of constant section A (for instance $A = \text{width} \times \text{thickness}$ for a flat specimen — see the calibration fields on the Image Correlation page: `specimen_width_mm`, `specimen_thickness_mm`, combined in `core/project.py`, comment near line 210-213: "F = rho * (w * e) * a"). This is the direct application of Newton's second law $F = m \cdot a$, with the mass expressed as density × cross-sectional area.

Limitations explicitly noted:

- The cross-section is assumed constant along the specimen and does not shrink locally (no account taken of necking).
- a_{mag} is evaluated pointwise (the field therefore varies across the ROI); the most meaningful reading is the ROI mean, interpreted as the inertial force of a uniform-section segment accelerating as the ROI average.
- It is nevertheless presented as a "real", dimensionally consistent force, not an arbitrary combination of units.

6.6.4 Smoothing — smooth_series

A centered, NaN-aware moving average on a 1D series (typically an ROI mean per frame — used to smooth acceleration/dynamic force before plotting): `window` is rounded up to the next odd number if even; each output point averages the finite values of its `window` centered neighbors, so that a NaN range never extends beyond its own coverage. `window <= 1` returns an unchanged copy.

6.7 Conversion to true strain — nominal_to_true_strain (core/dic_kinematics.py, lines 204–229)

$$\varepsilon_{true} = \ln(1 + \varepsilon) \tag{6.15}$$

implemented via `np.log1p(e)` for numerical stability.

Validity conditions, made explicit in the docstring:

- This is the standard 1D conversion (stretch ratio $\lambda = 1 + \varepsilon$, $\varepsilon_{true} = \ln \lambda$), exact for a genuinely 1D uniaxial elongation/shortening.
- An approximation in every other case: a 2D/3D strain state has no single, well-defined scalar "true strain" like a simple 1D change in length. Applying it to a normal strain component (`exx`, `eyy`), or to the always-nonnegative equivalent/effective strain `eps_eq`, is a standard practice that reduces to the exact 1D case in this last instance.
- Never applied to `exy` (shear strain): shear has no stretch-ratio interpretation on which this formula relies. Callers therefore never convert `exy` (see `ui/pages/page_image_correlation.py`, `_publish_dic_strain`, and `core/project.py`: `strain_convert_to_true` is ignored/never applied when `strain_definition == "exy"`).
- Undefined (NaN) wherever $1 + \varepsilon \leq 0$ (more than 100 % nominal shortening), rather than raising an exception — physically, this point has been crushed to zero or negative length, which no strain measure can meaningfully describe.

Associated rate formula, `nominal_to_true_strain_rate(rate, nominal_strain)`:

$$\dot{\varepsilon}_{true} = \frac{d}{dt}\left[\ln(1+\varepsilon)\right] = \frac{\dot{\varepsilon}}{1+\varepsilon} \tag{6.16}$$

(same domain of validity and same out-of-domain NaN behavior as above).

By default, `DICSettings.strain_convert_to_true = True` — documented behavior (`core/project.py`): this default was chosen because an unconverted comparison against the standard engine's "Strain (true)" would be the more surprising/less useful of the two options for the "Multi-Strain Comparison" plot.

6.8 Time scale / Time offset — camera-gage synchronization

There is no automatic camera/bar-signal synchronization in the application — this is explicitly stated in several places in the code and the interface (`ui/pages/page_image_correlation.py`, e.g. lines ∼906–913, ∼2514–2518): "there is no camera-to-bar-signal synchronization in this app".

Mechanism (`core/project.py`, `DICSettings.time_scale`/`time_offset_s`, lines 261–284):

- The camera and the gage/bar acquisition (DAQ) are two independent, unsynchronized clocks. Moreover, the camera's actual capture rate may differ slightly from the nominal value entered by the user (`frame_rate_hz`).
- `time_scale` corrects this last point:

  $$\text{effective_fps} = \text{frame_rate_hz} \times \text{time_scale} \tag{6.17}$$

  Default value 1.0 (entered frequency unchanged); e.g. 1.02 means the camera actually ran 2 % faster than the entered value.
- `time_offset_s` then shifts the DIC time (added after scaling, so it does not affect the time derivatives — a constant offset does not change a derivative):

  $$t_{DIC}(i) = \frac{i - i_{ref}}{\text{effective_fps}} + \text{time_offset_s} \tag{6.18}$$

  implemented in `_dic_time_s` (`page_image_correlation.py`, line ∼1584–1597) and, for the video burn-in, in `_format_timestamp` (line ∼3121–3137).
- Both parameters must be set manually by the user, by matching a feature visible in both time series (for instance the start and duration of the same event) — no automatic detection is provided.
- They apply everywhere this page builds a time axis: every plot against time, velocity/acceleration/strain rate (which depend on `dt`, so `time_scale` also corrects the physical `dt` used for these derivatives), the published "DIC Strain" series, any CSV export with a `time_s` column, and the timestamp burned into the exported video.
- Warning (comment near line ∼943-947): if the camera recording covers a duration shorter or longer than the time that actually elapsed, `time_scale` can also compensate for that.

Side note: the `core/camera_clock.py` module (the separate "Camera / Video Sync" tool) provides functions to reduce an external camera clock file (exposure pulse-train detection `detect_pulse_train`, trigger rising-edge detection `detect_rising_edge`, frame-time extrapolation `estimate_frame_times`, trigger-frame localization `locate_trigger_by_signal` / `locate_trigger_by_frame_number`) — but this is a separate file-analysis tool, not automatically wired into `time_scale`/`time_offset_s` of the DIC pipeline itself; synchronization with the bar signals remains, within the DIC pipeline, a manual user setting.

6.9 DIC measurement-uncertainty model (core/dic_uncertainty.py)

6.9.1 What this model IS and is NOT

The module's header docstring is very explicit and should be reproduced in full in the documentation:

> *"There is no single, universally 'correct' way to state a DIC measurement's uncertainty […] DIC's uncertainty depends on the speckle pattern's own texture, the camera's noise, the correlation mesh's own settings (subset size, step), and how much the material actually deforms during the test."*

The model combines three distinct error mechanisms, drawing on the DIC metrology literature cited in the code: Sutton et al. (the reference DIC textbook); Bornert et al., "Assessment of Digital Image Correlation Measurement Errors: Methodology and Results", Exp Mech 2009; Wang, Sutton et al., "Quantitative Error Assessment in Pattern Matching", Strain 2009; Lecompte et al. 2006 (effects of the speckle pattern / subset size).

What this module is NOT (section "THE RIGOROUS ALTERNATIVE THIS DOES NOT REPLACE"): for a validated, metrology-grade uncertainty budget (e.g. for a report that must state a certified uncertainty), the standard practice is an EMPIRICAL calibration that this module does not replace: run the identical DIC pipeline (same subset size, same step, same speckle pattern) on (a) two images of the same undeformed specimen (a "zero"/rigid-body test) to directly measure the true noise floor including all real effects at once, and/or (b) a reference image synthetically deformed with a known displacement/strain field, to measure the real bias and scatter against ground truth. This yields a single empirical number (replacing terms 1-2 below) validated for that precise setup, whereas everything that follows is a predictive model with no ground truth to check itself against.

What this module does NOT model (section "WHAT THIS MODULE DOES NOT MODEL"): lens distortion/calibration, out-of-plane (3D) motion in a 2D DIC setup, vibration/motion blur beyond what already shows up in a lower correlation coefficient, and the fixed sub-pixel interpolation bias that certain correlation schemes exhibit near integer-pixel displacements (this application's 3-point parabolic fit has such a bias, on the order of a few hundredths of a pixel in the worst case — see `_subpixel_peak`, §4.3). `_MIN_SIGMA_PX = 0.005 px` is a floor set precisely to acknowledge that no real system goes below a small fraction of a pixel, whatever the speckle quality.

6.9.2 Mechanism 1 — Sub-pixel registration precision floor (photometric)

Image noise estimation — `estimate_noise_sigma(image)`: the fast Immerkaer noise estimator (J. Immerkaer, "Fast Noise Variance Estimation", CVIU 1996): convolution with a discrete Laplacian kernel chosen so that the image's smooth content cancels out, leaving essentially only noise:

```
kernel = [[1, -2, 1], [-2, 4, -2], [1, -2, 1]]
conv = convolve2d(img, kernel, mode="valid")
sigma = sqrt(pi/2) * mean(|conv|) / 6
```

that is:

$$\sigma_{noise} = \sqrt{\frac{\pi}{2}} \cdot \frac{\text{mean}(|I * K|)}{6} \tag{6.19}$$

Inexpensive (a single small convolution), requires no calibration image (dark-frame/flat-field), at the cost of being an approximate estimate — a genuinely textured speckle pattern has real high-frequency content that this estimator cannot perfectly distinguish from noise, which tends to make it a slightly pessimistic (too high) rather than optimistic estimate — a direction judged safer for an uncertainty floor. Returns 0.0 if the image is smaller than 3×3 pixels.

SSSIG (Sum of Squared Subset Intensity Gradients) — `subset_sssig_xy(patch)`, computed on the reference subset only:

$$\text{SSSIG}_x = \sum (\partial I/\partial x)^2, \qquad \text{SSSIG}_y = \sum (\partial I/\partial y)^2 \tag{6.20}$$

(gradients obtained by `np.gradient`). A rich, high-contrast speckle subset gives a high SSSIG (a sharp, well-localized correlation peak), a flat/low-contrast subset gives a low SSSIG (a shallow, poorly localized peak).

Cramér-Rao-type floor — `crlb_sigma_px(noise_sigma, sssig)`:

$$\sigma_{u,floor} = \max\left(\sqrt{\frac{\sigma_{noise}}{\text{SSSIG}_x}}, \sigma_{min}\right), \qquad \sigma_{v,floor} = \max\left(\frac{\sigma_{noise}}{\sqrt{\text{SSSIG}_y}}, \sigma_{min}\right) \tag{6.21}$$

with σ_{min} = `_MIN_SIGMA_PX` = 0.005 px. NaN wherever `sssig` is itself NaN (excluded/untracked point). This is the same closed-form formula used by industrial DIC software (e.g. Correlated Solutions' VIC-2D) to publish a per-point "u/v" error map. It is a floor (a lower bound), not the whole story — hence terms 2 and 3.

6.9.3 Mechanism 2 — Correlation-quality degradation

`correlation_inflation(correlation, corr_floor=0.5, exponent=2.0)`: a multiplicative inflation factor applied to the registration floor, growing as that frame's peak NCC moves away from a perfect match (1.0):

$$\text{inflation} = \left(\frac{1}{\max(\text{corr_floor}, \min(1, C))}\right)^{\text{exponent}} \tag{6.22}$$

where C is the frame's NCC correlation, clipped between `corr_floor` (0.5 by default) and 1.0. `exponent = 2.0` is a modeling choice (not derived from first principles, explicitly stated in the docstring): keeps the inflation modest for a good-but-imperfect match (NCC = 0.98 → inflation ≈ 1.04) while visibly penalizing a poor match (NCC = 0.7 → inflation ≈ 2).

Explicit distinction from `correlation_threshold`: this is a smooth degradation model for a point that is still being tracked, not a substitute for the `correlation_threshold` cutoff of `run_dic` (§4.5), which drops a point outright once its match has become unreliable.

Mechanism 2 — physical motivation: the real mechanisms that degrade a match but that the photometric floor of mechanism 1 does not intrinsically capture: out-of-plane motion, lighting change, partial decorrelation, motion blur.

6.9.4 Mechanism 3 — Subset distortion (intra-subset averaging)

`distortion_term_px(local_strain_mag, subset_size_px, k=0.5)`:

$$\sigma_{distortion} = k \cdot |\varepsilon_{local}| \cdot \text{subset_size_px} \tag{6.23}$$

Since each subset is tracked as a single rigid template in translation (§4.1), if the true displacement field varies appreciably within a subset (local strain large relative to the subset size), the single position reported by the correlation is a blend across that variation, not the exact position of a single material point. $k = 0.5$ approximates a linear displacement gradient across the subset: the match stabilizes near the subset's center, so the worst-case discrepancy at the subset's edge, $|\varepsilon| \times \text{subset_size}/2$, is a reasonable order-of-magnitude estimate of the resulting position error — a modeling choice, not a rigorous derivation (explicitly stated).

`local_strain_mag` (computed in `compute_uncertainty`, line 432):

$$|\varepsilon_{local}| = \sqrt{\varepsilon_{xx}^2 + \varepsilon_{yy}^2 + 2\varepsilon_{xy}^2} \tag{6.24}$$

(the Frobenius norm of the local strain tensor from `_grid_strain`, or 0 if strains are not available for this frame).

This is the model's direct answer to the question "how much element distortion is there" — the classic rule of the speckle-pattern literature: "keep the subset small relative to the strain gradient".

6.9.5 Combining the three terms in quadrature

`combine_displacement_uncertainty(sigma_floor_px, inflation, distortion_px)`:

$$\sigma_u = \sqrt{(\sigma_{u,floor} \times \text{inflation})^2 + \sigma_{distortion}^2} \tag{6.25}$$

(likewise for σ_v with $\sigma_{v,floor}$). An independence assumption, explicitly discussed as a simplification (header docstring, point "All three are combined in quadrature"): the three terms are assumed uncorrelated — a simplification, since mechanisms 1 and 2 in particular are not fully independent in reality — but treating them as such yields a usable envelope that is, if anything, on the conservative side (the terms do not partially cancel each other out).

6.9.6 Propagation to the strains

`strain_uncertainty_from_displacement(sigma_u_px, sigma_v_px, step_px)` — propagation via the same centered-difference formula as `_grid_strain` (`np.gradient` on a regular grid of spacing `step_px`):

For an interior point, $\varepsilon_{xx} = \dfrac{u_{i,j+1} - u_{i,j-1}}{2\,\text{step}}$; if the two samples of u each carry independent noise of variance σ_u^2:

$$\text{Var}(\varepsilon_{xx}) = \frac{2\sigma_u^2}{(2\,\text{step})^2} = \frac{\sigma_u^2}{2\,\text{step}^2} \quad \Rightarrow \quad \sigma_{\varepsilon_{xx}} = \frac{\sigma_u}{\text{step}\sqrt{2}} \tag{6.26}$$

and likewise $\sigma_{\varepsilon_{yy}} = \sigma_v/(\text{step}\sqrt{2})$. For shear, $\varepsilon_{xy} = \frac{1}{2}(\partial u/\partial y + \partial v/\partial x)$ combines a y-derivative of u and an x-derivative of v, each carrying that point's own σ_u/σ_v (a point's registration uncertainty

does not depend on the direction in which it is later differentiated), added in quadrature and divided by two:

$$\sigma_{\varepsilon_{xy}} = \frac{1}{2} \cdot \frac{\sqrt{\sigma_u^2 + \sigma_v^2}}{\text{step}\sqrt{2}} \tag{6.27}$$

Documented approximation: the formula uses the nominal spacing `step_px` everywhere, including at the mesh edges where `np.gradient` actually uses a one-sided difference (not the two-sided formula assumed here) — an approximation that slightly underestimates the real uncertainty of edge points (a one-sided difference over the same step is noisier, not less noisy, than the two-sided version).

Uncertainty of the von Mises equivalent strain — `von_mises_strain_uncertainty`: a linearized (first-order) error propagation through the `von_mises_strain` formula:

$$\sigma_{\varepsilon_{eq}}^2 = \left(\frac{\partial \varepsilon_{eq}}{\partial \varepsilon_{xx}}\right)^2 \sigma_{\varepsilon_{xx}}^2 + \left(\frac{\partial \varepsilon_{eq}}{\partial \varepsilon_{yy}}\right)^2 \sigma_{\varepsilon_{yy}}^2 + \left(\frac{\partial \varepsilon_{eq}}{\partial \varepsilon_{xy}}\right)^2 \sigma_{\varepsilon_{xy}}^2 \tag{6.28}$$

with $\frac{\partial \varepsilon_{eq}}{\partial \varepsilon_{xx}} = \frac{2\varepsilon_{xx}}{3\varepsilon_{eq}}$, $\frac{\partial \varepsilon_{eq}}{\partial \varepsilon_{yy}} = \frac{2\varepsilon_{yy}}{3\varepsilon_{eq}}$, $\frac{\partial \varepsilon_{eq}}{\partial \varepsilon_{xy}} = \frac{4\varepsilon_{xy}}{3\varepsilon_{eq}}$. Explicit simplifying assumption: $\varepsilon_{xx}, \varepsilon_{yy}, \varepsilon_{xy}$ are assumed to have independent errors, whereas in reality they share the same underlying u/v noise — the same kind of approximation used everywhere else in the module. NaN (rather than a division by zero) where $\varepsilon_{eq} = 0$.

6.9.7 Temporal propagation (rate, velocity, acceleration, force)

`time_derivative_uncertainty(sigma_value, dt_s)` — the same principle, differentiating in time instead of space, assuming a roughly uniform `dt_s`:

$$\sigma_{\dot{x}} = \frac{\sigma_x}{\text{dt}\sqrt{2}} \tag{6.29}$$

Explicitly flagged simplification: chaining this call twice (position → velocity → acceleration) treats the noise of each stage as independent of the previous one, which is not exactly true (consecutive stages reuse overlapping position samples) — flagged in the code, not corrected.

Used for σ_{v_x}, σ_{v_y} (from σ_u, σ_v), then σ_{a_x}, σ_{a_y} (from σ_{v_x}, σ_{v_y}), and for the strain-rate uncertainties $\sigma_{\dot{\varepsilon}_{xx}}$, $\sigma_{\dot{\varepsilon}_{yy}}$, $\sigma_{\dot{\varepsilon}_{xy}}$.

Dynamic-force uncertainty — the same mm/s² → m/s² conversion (×0.001) that `compute_kinematics` applies to a_{mag} before $F = \rho A a$:

$$\sigma_F = \rho \cdot A \cdot \sigma_{a_{mag}} \times 0.001 \tag{6.30}$$

(a direct linear propagation, ρ and A treated as exact).

Uncertainty of "magnitude" quantities — `_hypot_uncertainty(x, y, sigma_x, sigma_y)`, used for the magnitude of displacement, velocity and acceleration: propagation through $\text{mag} = \sqrt{x^2 + y^2}$:

$$\sigma_{mag} = \sqrt{\left(\frac{x}{\text{mag}}\sigma_x\right)^2 + \left(\frac{y}{\text{mag}}\sigma_y\right)^2} \tag{6.31}$$

(falls back to 0 at the degenerate point mag = 0).

6.9.8 Effective point count and ROI-mean uncertainty

`effective_independent_count(n_active_points, roi_area_px2, subset_size_px)`: neighboring DIC points are not independent measurements — their subsets overlap when `step < subset_size` (the usual setting), and even without overlap, neighboring points on a continuous material are physically linked, so their tracking errors tend to move together rather than cancel out through averaging the way independent noise would.

The model approximates the correlation length as equal to one subset width, i.e. treats the ROI as covered by:

$$n_{eff} = \frac{A_{ROI}}{\text{subset_size}^2} \tag{6.32}$$

independent "patches" — a standard (if coarse, explicitly stated) correction for spatially correlated noise when averaging over an area (cf. the "effective sample size" concept in spatial statistics). Bounded to $[1, n_{active_points}]$, so that this calculation never claims a precision better than the raw number of tracked points, nor a zero/negative effective count.

`roi_mean_uncertainty(sigma_field, n_eff)`: the standard error of an ROI mean, combining the RMS of the pointwise uncertainties with the standard $1/\sqrt{n}$ reduction, but using the effective count rather than the raw point count:

$$\sigma_X = \frac{\sqrt{\frac{1}{N}\sum_i \sigma_i^2}}{\sqrt{n_{eff}}} \tag{6.33}$$

(the RMS of the finite pointwise sigmas, the correct way to combine several different sigmas into one representative value for points of varying quality, divided by $\sqrt{n_{eff}}$). NaN if no point has a finite sigma.

6.9.9 Orchestration — compute_uncertainty

Builds a complete `DicUncertainty` object from a `DICResult` (needs `noise_sigma`/`sssig_x`/`sssig_y`) and, optionally, an already-computed `Kinematics` (for the rate/velocity/acceleration/force fields). Returns `None` if `result` has no noise/SSSIG estimate (the best-effort case, see §4.7). The same `mm_per_px`/`rho_kg_m3`/`area_m2` used for `compute_kinematics` must be passed in, so that the uncertainty bands genuinely match the values they bracket.

`DicUncertainty` structure — one array per processed frame, the same layout as `DICFrameResult`/`Kinematics` (`sigma_u`, `sigma_v`, `sigma_disp_mag`, `sigma_exx`, `sigma_eyy`, `sigma_exy`, `sigma_eps_eq`, then, if `kin` is supplied: `sigma_vx`/`vy`/`v_mag`, `sigma_ax`/`ay`/`a_mag`, `sigma_exx_rate`/`eyy_rate`/`exy_rate`/`eps_eq_rate`, `sigma_dynamic_force`).

Exposed tuning parameters (passed to the UI, `core/project.py DICSettings`): `show_uncertainty` (bool, default `True`), `uncertainty_sigma_k` (the multiple of the 1-σ shown as the envelope, default 1.0). The code's explicit comment on defaulting `show_uncertainty = True`:

> *"a DIC result plotted with no indication of its own uncertainty invites over-reading it"*

And on `uncertainty_sigma_k`: $1.0 \to \pm 1\sigma$ ($\sim$68 % coverage if the underlying errors were Gaussian, which is not verified — explicitly stated), $2.0 \to \pm 2\sigma$ ($\sim$95 %).

6.10 Visual rendering (core/dic_render.py) — brief summary

A single function `render_field(fig, ax, background, x, y, values, field_label, vmin, vmax, cmap, timestamp_text, text_color, lost_track_color, max_edge_px)`, shared between the live display (Image Correlation page) and the video export (`core/dic_video_export.py`).

- Draws on a fresh Delaunay triangulation of the points' real positions in the current frame (`ax.tricontourf`), rather than on the reference frame's fixed quad grid (`contourf` on `mesh.grid_xy()`) — a documented motive: keeping a fixed quadrilateral's connectivity while its four corners move independently can make it fold over onto itself as soon as neighboring points diverge, even slightly (very visible once a specimen necks strongly — a jagged, self-overlapping, pointed patch of color instead of a smooth field). A fresh Delaunay triangulation of a point cloud never has this problem by construction.
- A point with no reliable position in a given frame (lost track, masked, or rejected by a quality filter) is simply omitted from the triangulation, rather than kept at a substitute position — a substitute position was itself a source of the same folding artifact.
- If fewer than 3 valid points (or a degenerate set, e.g. collinear points), falls back to a plain colored point cloud (`ax.scatter`) instead of the triangulation.
- Explicit `vmin`/`vmax` fix the color scale (used for frame-to-frame consistency in an exported video); left as `None`, the scale auto-adjusts to each frame's data (corresponds to `color_scale_mode = "auto"` in `DICSettings`, the alternative being `"fixed"` with `color_scale_min`/`color_scale_max`).
- Masking "bridge" triangles over an excluded zone (`max_edge_px`, `_long_edge_mask`): a Delaunay triangulation only ever triangulates the convex hull of the points supplied, with no notion of a hole inside it. A non-rectangular ROI (polygon/ellipse) or a manually excluded zone leaves a genuine concave gap with no tracked point inside it (`MeshGrid.mask`) — but the points just on either side of the gap remain valid, so a naive triangulation still connects them with long, artificial triangles, which `tricontourf` then colors as if they were real data (a smooth but entirely fictitious gradient filling the excluded notch/hole). When `max_edge_px` is supplied, every triangle with an edge longer than this length is removed from the triangulation before plotting (`Triangulation.set_mask`), so the excluded zone appears as a true void rather than a fabricated colored area. The recommended value, `default_max_edge_px(mesh.step) = mesh.step` $\times$ `3` (used by both callers — the live Results canvas and the video export), leaves a clear margin above the diagonal of an undeformed regular grid triangulation ($\sim$1.41 $\times$ step) and of a moderately deformed zone (up to $\sim$2 $\times$ step), while still eliminating a bridge that spans an entire notch/hole. Known and accepted limitation: right at the reentrant (concave) corner of an excluded zone, a small triangle can still have every edge under this threshold while still covering a fragment of the excluded zone — rounding a concave corner takes a much shorter edge than bridging the entire hole. This masking eliminates massive bridging over the interior of a hole, not this visually minor rounding right against its edge.

This module is essentially graphical and neither modifies nor recomputes any physical quantity.

6.11 Summary of file → function correspondences

Topic	File	Function(s)
Crop / contrast / downscale	`core/dic_preprocessing.py`	`crop_frame`, `downscale_frame`, `preprocess_frame`
Temporal decimation	`core/dic_preprocessing.py`	`decimated_frame_indices`

Topic	File	Function(s)
Rectangle / ellipse / polygon ROI	core/dic_correlation.py	ROI, Ellipse, bounding_roi
Correlation mesh	core/dic_correlation.py	build_mesh, build_mesh_from_polygon, build_mesh_from_ellipse, MeshGrid
NCC tracking + sub-pixel	core/dic_processing.py	_track_subset, _subpixel_peak
Full tracking loop, Fixed/Moving, filters	core/dic_processing.py	run_dic, _track_chunk, _apply_neighbor_consistency_filter
Local strain field	core/dic_processing.py	_grid_strain
Export / re-import of results (CSV)	core/dic_result_io.py, ui/pages/page_image_correlation.py	load_dic_result_csv, _on_export_results, _on_import_results
Velocity / acceleration	core/dic_kinematics.py	compute_kinematics, _gradient_over_time
Von Mises equivalent strain/rate	core/dic_kinematics.py	von_mises_strain
ROI dynamic force	core/dic_kinematics.py	compute_kinematics (the dynamic_force block)
Conversion to true strain	core/dic_kinematics.py	nominal_to_true_strain, nominal_to_true_strain_rate
Temporal smoothing	core/dic_kinematics.py	smooth_series
Time scale / offset	core/project.py (DICSettings), ui/pages/page_image_correlation.py	_effective_fps, _dic_time_s, _format_timestamp
Image noise / SSSIG	core/dic_uncertainty.py	estimate_noise_sigma, subset_sssig_xy
Registration floor (mech. 1)	core/dic_uncertainty.py	crlb_sigma_px
Quality degradation (mech. 2)	core/dic_uncertainty.py	correlation_inflation
Subset distortion (mech. 3)	core/dic_uncertainty.py	distortion_term_px
Quadrature combination	core/dic_uncertainty.py	combine_displacement_uncertainty
Propagation to strains	core/dic_uncertainty.py	strain_uncertainty_from_displacement, von_mises_strain_uncertainty
Temporal propagation (rate, v, a, F)	core/dic_uncertainty.py	time_derivative_uncertainty, _hypot_uncertainty
Effective points / ROI mean	core/dic_uncertainty.py	effective_independent_count, roi_mean_uncertainty
Uncertainty orchestration	core/dic_uncertainty.py	compute_uncertainty, DicUncertainty
Field rendering (Delaunay)	core/dic_render.py	render_field
Masking bridge triangles over an excluded zone	core/dic_render.py	default_max_edge_px, _long_edge_mask

Topic	File	Function(s)	
Camera clock (separate tool)	`core/camera_clock.py`	`detect_pulse_train`, `trigger_by_signal`, `trigger_by_frame_number`, `estimate_frame_times`	`locate_` `locate_`

6.12 Summary of limitations and warnings found in the code

1. **Rigid subset tracking in translation** (no deformable subset) — appropriate for moderate, predominantly uniaxial deformations (`core/dic_processing.py`, header).
2. `_grid_strain`: a local, small-strain, first-order estimate — not a rigorous large-deformation DIC computation (`_grid_strain`, docstring).
3. The conversion $\varepsilon_{true} = \ln(1+\varepsilon)$ is only exact in 1D uniaxial; an approximation in 2D/3D; never applied to the shear `exy` (`nominal_to_true_strain`, docstring).
4. ROI dynamic force: constant-section assumption (no necking), evaluated pointwise (`compute_kinematics`, docstring).
5. No automatic camera/gage synchronization — manual setting via `time_scale`/`time_offset_s` (`ui/pages/page_image_correlation.py`, several locations).
6. A predictive uncertainty model, not empirically calibrated — does not replace a "zero"/rigid-body test or a known synthetic deformation for a certified uncertainty (`core/dic_uncertainty.py`, header).
7. The uncertainty model does not cover: optical distortion, out-of-plane motion, vibration/blur beyond what the NCC already reflects, the sub-pixel interpolation bias of the parabolic fit (`core/dic_uncertainty.py`, header).
8. The three uncertainty terms are combined in quadrature assuming independence — an acknowledged simplification, tending to be conservative (`core/dic_uncertainty.py`, header).
9. Error propagation (strain, rate, force) assumes independent errors between components that in reality share the same underlying u/v noise — an assumed simplification (`strain_uncertainty_from_displacement`, `von_mises_strain_uncertainty`, `time_derivative_uncertainty`).
10. Effective point count: a coarse approximation of the correlation length (= one subset width) to correct the ROI averaging for spatially correlated noise (`effective_independent_count`, docstring).
11. $\pm1\sigma/\pm2\sigma$ coverage assumed Gaussian but unverified (`core/project.py`, comment on `uncertainty_sigma_k`).
12. This DIC uncertainty model is distinct from the formal uncertainty computed by the "Crack & Wave Analysis" tool (Chapter 10) for the stress-intensity factors K_I/K_{II}: the latter is a least-squares covariance (exact OLS for the standard DIC ring fit, Gauss-Newton asymptotic for X-FEM-DIC — Chapter 10, §4.2 and §6.7), applied to an after-the-fact near-tip field fit or to a direct re-correlation of the images, not an extension or refinement of this chapter's predictive model.
13. A result re-imported from a CSV export (§4.8) never has an uncertainty envelope (`noise_sigma`/`sssig_x`/`sssig_y` are not recoverable from the file) and loses the original run's settings (subset size, ROI, search radius, thresholds, preprocessing) — only the mesh, the per-frame tracked fields, and, when the corresponding columns were populated at export time, the pixel size and the effective camera frequency are recovered (`core/dic_result_io.py`, module docstring).
14. Masking bridge triangles over an excluded zone (§10, `max_edge_px`) eliminates massive bridging over the interior of a hole (non-rectangular ROI, excluded zone), but a small triangle hugging a concave corner of that zone can still survive the masking — rounding a concave corner takes a shorter edge than the threshold, unlike a bridge spanning the whole hole

(`core/dic_render.py::_long_edge_mask`, docstring).

7. Wave Dispersion Correction — Elastic Regime

Reference source file: `core/dispersion.py` (858 lines). Module header (lines 1–24):

> *"Pochhammer-Chree dispersion correction. Ported from FFT_dispersion.py / FFT_dispersion_ID.py / FFT_dispersion_visco.py / FFT_dispersion_visco_ID.py."*

The module handles four combinations (two independent axes), all within the same file:

	Elastic bars	**Viscoelastic bars**
Two-bar setup (classic SHPB)	`elastic_correction()`	`viscoelastic_correction()`
Output-bar-only setup (Direct Impact, "_ID")	`elastic_correction_direct_impact()`	`viscoelastic_correction_direct_impact()`

This chapter covers exclusively the "Elastic bars" column (`elastic_correction` and `elastic_correction_direct_impact`, plus the shared functions `_bar_elastic_wave_speed` and `_solve_pochhammer_branch`/`_solve_pochhammer_branches`). The viscoelastic case (Standard Linear Solid-type rheological model, complex modulus, `core/rheology_models.py`, `_solve_visco_branch`, `_visco_phase_correction`, `viscoelastic_correction*`) is covered in the "viscoelastic dispersion" chapter; it is mentioned here only to explain how the application switches between the two regimes.

A warning from the module (lines 19–23) must be reproduced as-is in the user-facing documentation:

> **Warning from the code itself:** "NOTE: this module is numerically faithful to the original but has not been re-validated against real experimental data (no test data / no way to run sympy's nsolve root search in this environment) — treat the physics as 'ported, not re-verified' and sanity-check against a known-good original run before relying on it for a real analysis."

In other words: the port faithfully preserves the original numerical algorithm, but has not been re-verified against a known experimental case in the porting environment.

7.1 Physical principle: why a finite bar disperses the wave

The elementary bar theory (1D theory, the "Bernoulli/pure compression wave" theory) assumes a uniform axial strain wave over the whole bar cross-section, propagating without change of shape at the constant celerity

$$c_0 = \sqrt{E/\rho} \tag{7.1}$$

where E is the Young's modulus of the bar material and ρ its density. This is the relation used in the application for every "non-dispersive" calculation (time of flight, standard reduction) — see `core/models.py` and the `C0_pa` field of a `Bar`.

In reality, a bar has a finite radius a. The displacement field in a cylindrical bar of finite cross-section is not uniform over the section: the propagation of a harmonic compression wave necessarily involves radial motion (dynamic Poisson effect, radial inertia). This axial/radial coupling means that the real phase velocity $C_p(\omega)$ of a harmonic component of angular frequency ω depends on frequency and on the radius-to-wavelength ratio: this is the phenomenon of geometric dispersion (not to be confused with the material dispersion of viscoelastic bars, covered elsewhere).

Practical consequence for Hopkinson bars: a strain pulse measured by a gauge bonded at a distance z from the bar/specimen interface does not have the same temporal shape as the pulse that actually reaches the interface (or that would reach any other point of the bar), because each frequency component of the pulse propagated at a slightly different phase velocity and accumulated a different phase shift over the distance z. The larger the gauge-to-interface distance and/or the more high-frequency content the pulse contains (steep front, shock), the larger this parasitic distortion (oscillations, spreading) becomes. Dispersion correction therefore consists in recomputing, for each frequency component of the measured signal, the phase shift (and the amplitude attenuation/amplification) it would have undergone if it had been "rewound" or "advanced" by exactly the distance z along the bar, using the bar's true dispersion relation $C_p(\omega)$ — not the constant velocity c_0 of 1D theory — and then recomposing a corrected signal representative of the desired point (typically the interface with the specimen).

7.2 Dispersion theory implemented: the Pochhammer-Chree equation

The name explicitly used in the code (module docstring, line 1, and the docstring of `_solve_pochhammer_branch`, `elastic_correction`) is "Pochhammer-Chree", the exact theory of propagation of longitudinal axisymmetric waves in an infinite isotropic elastic cylinder (exact solution of 3D elastodynamics in cylindrical coordinates, fundamental longitudinal mode L(0,1)). No reference to Bancroft or Love appears in the code itself; it is indeed the Pochhammer-Chree characteristic equation, in its classic form involving the Bessel functions J_0 and J_1, that is solved.

7.2.1 Exact variables and notation from the code (core/dispersion.py::_solve_pochhammer_branch, lines 162–197)

For each "branch" (incident, reflected or transmitted), with:

- $a =$ `radius`: radius of the corresponding bar ($D_{pa}/2$, where D_{pa} is the bar's mean diameter),
- $c_0 =$ `c0`: 1D elastic celerity of the bar, $c_0 = \sqrt{E/\rho}$ (from `_bar_elastic_wave_speed`, see §2.3),
- $\nu =$ `nu`: bar Poisson's ratio,
- $f =$ `freq`: frequency vector (Hz) from the FFT (see §4),
- $C_p =$ `Cp`: unknown phase velocity to solve for, at the current frequency.

the code sets:

$$\beta = \frac{1-2\nu}{1-\nu} \tag{7.2}$$

$$x = \left(\frac{C_p}{c_0}\right)^2 (1+\nu) \tag{7.3}$$

$$\eta = \frac{2\pi f}{C_p} \quad \text{(this is the wavenumber } k = \omega/C_p\text{, named } \texttt{nl} \text{ in the code)} \tag{7.4}$$

$$h = \eta\sqrt{\beta x - 1}, \qquad k = \eta\sqrt{2x-1} \tag{7.5}$$

(note: in the code, the variable `k_expr`/`k` above is not the axial wavenumber — it is one of the two radial quantities characteristic of the Pochhammer-Chree equation, analogous to the classic q from the literature; h corresponds to the classic p).

7.2.2 Characteristic equation solved

The dispersion equation solved numerically, frequency by frequency, is exactly (extracted literally from `_solve_pochhammer_branch`, lines 179–182):

$$(x-1)^2\, ah\, \frac{J_0(ah)}{J_1(ah)} - (\beta x - 1)\left[x - ak\,\frac{J_0(ak)}{J_1(ak)}\right] = 0 \tag{7.6}$$

with J_0, J_1 the Bessel functions of the first kind of order 0 and 1 (`sympy.besselj`). This is the standard form of the Pochhammer-Chree characteristic equation for the fundamental axisymmetric longitudinal mode of an infinite isotropic elastic cylinder, written in reduced variables x (proportional to the square of the velocity ratio) and h, k (radial quantities depending on $\eta = \omega/C_p$, on a and on ν through β). The unknown solved for at each frequency is C_p (via x, h, k, which all depend on it): this is indeed the dispersion relation $C_p(\omega)$, or equivalently $\omega(k)$, of the real (finite-radius) bar, as opposed to the non-dispersive relation $C_p = c_0 = \text{const}$ of 1D theory.

7.2.3 Bar wave speed c0 and Poisson's ratio: _bar_elastic_wave_speed()

`_bar_elastic_wave_speed()` (lines 113–159) provides the pair (c_0, ν) used as the starting point of the solve:

- Standard case (`heat_check=False`): $c_0 = C0_{pa}$ and $\nu = \nu_{pa}$ are simply the averages (over the Monte-Carlo draw) of the bar properties entered by the user — no additional computation.
- Case with thermal correction (`heat_check=True`): if a temperature gradient along the bar is active (tests at non-ambient temperature), the module recomputes an effective Young's modulus E_{mat} (and hence $c_0 = \sqrt{E_{mat}/\rho}$) from an exponential temperature profile along the bar,

$$T(z) = T_{amb} + (T_0 - T_{amb})\exp\left(-2\sqrt{\frac{h}{k_{therm} D}}\, z\right), \qquad z \in [0, PG],\ h = 20 \tag{7.7}$$

(h here is a hardcoded constant convective exchange coefficient, not to be confused with the Pochhammer-Chree h of §2.1), followed by either a Ramberg-Osgood-like "rte" empirical law (bar types 1/2, with thermal conductivity $k_{therm} = 40$ or 220 W/m/K depending on the material), or an imported temperature-dependent property table (bar type 3, via `table.value("e"/`

`"rho"/"nu", ...`)). This mechanism, although present in `core/dispersion.py`, belongs to bar thermal behavior rather than to elastic dispersion proper; it only affects the dispersion calculation through the values (c_0, ν) it feeds into `_solve_pochhammer_branch`.

7.3 Numerical solution method: frequency-by-frequency root search

7.3.1 Algorithm (_solve_pochhammer_branch, lines 162–197)

For each "branch" (incident, reflected or transmitted wave), the solve proceeds by a continuation method:

1. Initialization at $f = 0$ (DC): `cp_sol = [c0]`, `m1 = [1.0]`, `m2 = [1.0]` — at zero frequency, the Pochhammer-Chree phase velocity tends exactly to c_0 (low-frequency limit of 1D theory), and the correction factors are taken equal to 1 (no correction).
2. Loop over the positive frequency bins `i = 1 …len(freq)-1` (the bin `freq[0]=0` is skipped since it is already covered by the initialization):
   - The characteristic equation of §2.2, at frequency `freq[i]`, is solved for the unknown C_p by `sympy.nsolve` — a numerical root solver (sympy's multidimensional/scalar Newton-Raphson, possibly with arbitrary-precision arithmetic), initialized (`x0`) with the solution found at the previous frequency, `cp_sol[-1]`.
   - This is therefore a continuation ("marching") method: each solution seeds the next, assuming that $C_p(\omega)$ varies continuously (no abrupt jump between adjacent frequency bins) along the dispersion branch being followed. No explicit bound (search interval) is provided to `nsolve` — only a starting point; it is the physical continuity of the dispersion curve, combined with Newton's good local convergence behavior, that guarantees (in practice) that the root found stays on the same branch (the fundamental mode) from one bin to the next.
   - The retained solution is taken in absolute value: `cp_sol.append(float(Abs(dsol)))` — the result of `nsolve` may be slightly complex (numerical residual), and only the modulus is kept as the physical phase velocity.
3. Correction factors `m1`, `m2`: once C_p is found for the current frequency, the code recomputes x_a, η_a, h_a, k_a from that solution and then evaluates:

$$M_1 = \frac{2\left(1 + \dfrac{1-\beta x}{x-1}\right)}{ah\,\dfrac{J_0(ah)}{J_1(ah)} + ak\,\dfrac{J_0(ak)}{J_1(ak)} \cdot \dfrac{1-\beta x}{x-1}} \tag{7.8}$$

(real part kept: `float(complex(dm1).real)`), and

$$M_2 = \left(\frac{C_p}{c_0}\right)^2 \tag{7.9}$$

The ratio M_2/M_1 is then used to correct the amplitude of each frequency component (§4). The code does not explicitly document the physical origin of M_1: structurally, this form corresponds to a correction factor relating the axial strain measured at the bar's surface (what a gauge bonded on the outside of the bar actually reads) to a cross-section-averaged quantity — a type of correction classically associated with Bancroft/Davies in the Hopkinson-bar dispersion

literature — but this interpretation is not confirmed by any code comment and must be treated as a cautious reading, not a certainty.

A `progress_cb(i, total)` callback is invoked after each frequency is solved (used by the UI for the progress bar, cf. `page_corrections.py`, message "Running dispersion correction… (this involves a root-search per frequency and can take a while)").

7.3.2 Parallelization across branches (_solve_pochhammer_branches, lines 209–239)

The incident/reflected/transmitted branches are independent of each other (each has its own radius a, possibly its own c_0, ν for the input bar vs. the output bar), but each branch is intrinsically sequential internally (continuation from one frequency to the next). `_solve_pochhammer_branches` allows, if `n_workers > 1`, running the (up to 3) branches in parallel in separate processes (via `core/parallel.run_map`) — a gain bounded at best to $\sim 3\times$, never a frequency-by-frequency parallelism. With `n_workers <= 1` (the default), it is a plain sequential call equivalent to the original code, with per-frequency progress tracking.

7.4 Full signal correction algorithm (FFT → per-bin correction → IFFT)

Implemented in `elastic_correction()` (two-bar setup, lines 309–404, internal function `_corrected` lines 373–385) and, identically (a single branch), in `elastic_correction_direct_impact()` (lines 407–449).

7.4.1 Time-domain window cutting (before the FFT)

The measured signal is first cut into time windows corresponding respectively to the incident, reflected (input bar IB) and transmitted (output bar OB) waves, from the bounds `xd1,xd2` that the user positions on the "Pulse window(s)" plot (`ui/pages/page_corrections.py`). The reflected and transmitted windows are positioned automatically from the incident window by adding the corresponding times of flight (same duration `ti_pulse = idx2-idx1` as the incident window for all three):

$$t_{\text{reflected},1} = t_{d1} + \frac{2\,PG_{IB}}{C0_{IB}} \left[+ \frac{2L}{C0_{plateau}} \text{ if a plateau/pulse-shaper is active} \right] \tag{7.10}$$

$$t_{\text{transmitted},1} = t_{d1} + \frac{PG_{OB}}{C0_{OB}} + \frac{L_{\text{specimen}}}{c_{0,\text{specimen}}} + \frac{PG_{IB}}{C0_{IB}} \left[+ \frac{2L}{C0_{plateau}} \right] \tag{7.11}$$

with $c_{0,\text{specimen}} = \sqrt{E_{\text{specimen}}/\rho_{\text{specimen}}}$ (elastic celerity of the specimen, used only for the time-of-flight estimate, not for the stress-strain analysis). PG denotes the mean gauge position (gauge-to-interface distance) of the bar in question. This is a non-dispersive time of flight (based on constant C_0), which is consistent: it only serves to roughly position the windows, the actual dispersion correction being applied afterwards in the frequency domain. A guard, `_require_window()` (lines 44–74), raises an explicit error if a resolved window is empty or too narrow (less than 2 samples apart), rather than letting a cryptic `IndexError` propagate further into the FFT/root-search computation.

7.4.2 Moving to the frequency domain (FFT)

For a window of length n and sampling step dt:

```
yi = np.fft.rfft(incident)                    # same for reflected -> yr, transmitted ->
    yt
freq = np.fft.rfftfreq(n, d=dt)
ai, pi = np.abs(yi), np.angle(yi)             # amplitude and phase per frequency bin
```

This is a real FFT (`rfft`), so only the positive frequencies (DC to Nyquist) are represented, each bin being characterized by an amplitude `ai[i]` and a phase `pi[i]`.

7.4.3 Per-frequency-component correction: _corrected() (lines 373–385)

For each frequency bin f_i, with z the signed propagation distance to correct (see §4.4), c_0 the 1D celerity of the bar in question, $C_p(f_i)$ the Pochhammer-Chree phase velocity solved in §3, and $M_1(f_i)$, $M_2(f_i)$ the associated correction factors:

Phase correction:

$$\phi'(f_i) = \phi(f_i) - \left(2\pi f_i \frac{z}{c_0}\right)\left(\frac{c_0}{C_p(f_i)} - 1\right) \tag{7.12}$$

(code: `phase[i] - (2*cmath.pi*freq[i]*z/c0) * (c0/cp_sol[i] - 1)`)

Interpretation: $\dfrac{\omega z}{c_0}$ is the phase shift a non-dispersive wave (1D theory, constant velocity c_0) would accumulate over the distance z; multiplying it by $\left(\dfrac{c_0}{C_p} - 1\right)$, the term subtracted is exactly the difference between the real dispersive phase shift ($\omega z / C_p(\omega)$) and the non-dispersive phase shift ($\omega z / c_0$) over that same distance — in other words $\phi' = \phi - \omega z \left(\dfrac{1}{C_p(\omega)} - \dfrac{1}{c_0}\right)$. The measured phase is thus "re-projected" to the shifted position z along the bar, taking into account the true dispersive physics.

Amplitude correction and spectral reconstruction:

$$\hat{X}'(f_i) = \frac{M_2(f_i)}{M_1(f_i)} \, |X(f_i)| \, e^{i\phi'(f_i)} \tag{7.13}$$

(code: `(m2[i]/m1[i]) * amp[i] * np.exp(1.0j * n_phase[i])`)

Return to the time domain (IFFT):

```
sig = np.fft.irfft(fft_new, n=n)
```

The explicit `n=n` argument (the length of the original window, potentially odd since the windows are freely positioned by the user and not forced to a fixed size) is essential: `np.fft.irfft` cannot infer an odd length from the real spectrum alone, and assumes an even length by default — which, for an odd-length window, would produce a numerically wrong reconstructed signal over its entire length (not merely shifted by one sample). The code documents this explicitly in a comment (lines 376–383) and points to the identical mechanism in `core/visco_calibration.py::_predict_pulse` for a detailed numerical example.

7.4.4 Signed distance z and sign convention of the three waves

The three actual calls in `elastic_correction` (lines 387–389) are:

```
new_incident    = -np.absolute(_corrected(ai, pi, freq,  z1, c01, cp_i, m1i, m2i))
new_reflected   =  np.absolute(_corrected(ar, pr, freq, -z1, c01, cp_r, m1r, m2r))
new_transmitted = -np.absolute(_corrected(at, pt, freq, -z2, c02, cp_t, m1t, m2t))
```

where $z_1 = PG_{IB}$, $z_2 = PG_{OB}$ (mean gauge-to-interface distances of the input bar and the output bar). Two distinct mechanisms are worth noting:

1. The sign of z in the phase correction distinguishes the direction of propagation being corrected: $+z_1$ for the incident wave (measured before it reaches the interface, hence "advanced" towards the interface), $-z_1$ for the reflected wave (measured after a round trip, so the correction is made in the opposite direction), $-z_2$ for the transmitted wave (measured after already having travelled from the interface to the output gauge). This choice of sign is consistent with the construction of the time windows in §4.1 (same distances, same implicit propagation signs).
2. The final sign forced after `np.absolute(...)` ($-$, $+$, $-$ respectively for incident/reflected/transmitted) enforces the expected physical sign convention of each pulse (typically: incident and transmitted in compression, reflected in tension — or the opposite depending on the project's gauge wiring, but in every case a fixed, consistent convention) — independently of any global sign ambiguity the FFT amplitude/phase reconstruction might introduce (an amplitude $|X|$ combined with a reconstructed phase does not necessarily preserve the original sign of the time-domain signal). This is a purely algorithmic sign-correction mechanism, not a physical dispersion phenomenon in itself.

After the IFFT, if the corrected signal comes out with one sample fewer than the original window (truncation residual of rfft/irfft on an odd size), the last sample is duplicated to compensate (lines 391–396): `new_incident = np.concatenate((new_incident, [new_incident[-1]]))`, etc.

7.4.5 "Output bar only" case (Direct Impact): elastic_correction_direct_impact()

Structurally identical, but with a single branch (transmitted), a single time window (positioned directly by the user, not derived from an incident window), and a single correction applied with $z = -z_2$ (lines 407–449):

```
n_phase = [pt[i] - (2*cmath.pi*freq[i]*(-z2)/c02) * (c02/cp_t[i] - 1) for i in range(len(pt
    ))]
fft_new = [(m2t[i]/m1t[i]) * at[i] * np.exp(1.0j*n_phase[i]) for i in range(len(n_phase))]
new_transmitted = -np.absolute(np.fft.irfft(fft_new, n=n))
```

Here window selection is done by looking up the nearest sample in an explicit `time_pulse` vector (`_index_of_nearest`, lines 77–96), rather than by `np.searchsorted` on `timet` as in the two-bar setup — a difference purely tied to the input data format of the Direct Impact setup, with no impact on the correction physics itself.

7.5 Frequency windowing and bandwidth bounding of the correction

7.5.1 Absence of time-domain windowing (apodization) before the FFT — always true

A careful reading of the file shows no Hann/Hamming/Tukey-type time-domain windowing before the FFT, and no explicit detection/guard against a divergence of the root search at high frequency. Both points remain true regardless of the mechanism of §5.2 below (which acts downstream, on the already-corrected spectrum, not on the raw signal upstream of the FFT):

- The only mechanism that could limit the impact of high-frequency noise on the root-search side is indirect: since `nsolve` is initialized by continuation from the previous solution, a divergence or misbehavior of the root search at high frequency (a region where the Pochhammer-Chree dispersion curve becomes strongly nonlinear, near the Rayleigh limit) could cause the continuation to fail or drift — but no detection or guard against this case is coded in `_solve_pochhammer_branch` (unlike `_solve_visco_branch`, which does have a try/except `ValueError` fallback — see the viscoelastic chapter).
- Nor is there any time-domain windowing (apodization) applied to incident/reflected/transmitted before `rfft`: the raw, cut signal (rectangular, with no smooth transition at the edges) is transformed directly. This can introduce spectral leakage if the window edges do not coincide with zeros of the signal — the code neither mentions nor corrects this point.

7.5.2 Bounding the correction's own bandwidth — _bandwidth_rolloff_weight() (lines 92–153)

Motivation, as documented in the code (`core/dispersion.py`, docstring of `_bandwidth_rolloff_weight`): the correction's own gain (the factor $M_2(f)/M_1(f)$ of §4.3) is not bounded by 1 — at high frequency it commonly exceeds 1, since it exists to cancel the bar's geometric dispersion, not to suppress anything. A real gauge signal always carries broadband electronic/quantization noise, negligible at the frequencies that actually matter for the pulse (low ones) but not strictly zero at high frequencies — a correction gain above 1 in that region therefore amplifies this noise rather than any signal actually transmitted by the bar, a mechanism documented as the cause of the high-frequency oscillation bursts that a Pochhammer-Chree correction can produce on real data. The code explicitly states that the C_p/C_0 curve (§7, "Result" diagnostic) stays smooth over the same range: it is therefore not the root search itself diverging or misbehaving, the noise amplification is inherent to this type of correction, regardless of its implementation — not a bug in the solve.

Mechanism: a smooth weight (raised cosine), between 0 and 1, multiplied into the already-corrected spectrum (not the raw signal) just before the IFFT — i.e. downstream of §4.3, not an upstream filtering as mentioned (and still absent) in §5.1:

```
def _bandwidth_rolloff_weight(freq, radius, c0_ref, max_normalized_freq, rolloff_fraction
    =0.25):
    if max_normalized_freq is None or max_normalized_freq <= 0 or radius <= 0 or c0_ref <=
    0:
         return np.ones_like(freq)                    # disabled -- original behavior,
    unbounded gain
    f_cut = max_normalized_freq * c0_ref / radius
    f_start = max(0.0, 1.0 - rolloff_fraction) * f_cut
    w = np.ones_like(freq)                             # 1.0 below f_start
    in_taper = (freq > f_start) & (freq < f_cut)
```

```
    w[in_taper] = 0.5 * (1.0 + np.cos(np.pi * (freq[in_taper] - f_start) / (f_cut - f_start
    )))
    w[freq >= f_cut] = 0.0                                  # 0.0 beyond f_cut
    return w
```

$$w(f) = \begin{cases} 1 & f \le f_{start} \\ \dfrac{1 + \cos\left(\pi \dfrac{f - f_{start}}{f_{cut} - f_{start}}\right)}{2} & f_{start} < f < f_{cut} \\ 0 & f \ge f_{cut} \end{cases} \qquad f_{cut} = f_{max} \cdot \frac{c_0}{a}, \quad f_{start} = (1 - 0.25)\, f_{cut} \tag{7.14}$$

with f_{max} = "Max normalized frequency (f · radius/C0)" (`max_normalized_freq`, "Dispersion Corrections" page, "Correction bandwidth limit" card), a the radius of the bar in question, c_0 its celerity — the same normalized quantity $f \cdot a/c_0$ already plotted on the C_p/C_0 curve's abscissa (§7), hence directly readable on the same diagnostic plot already used for the correction. The transition fraction (`rolloff_fraction`, 0.25 by default, not exposed in the UI) gives a gradual cutoff (raised cosine) rather than a hard threshold, so as not to reintroduce a new spectral discontinuity (a classic source of Gibbs-type ringing) in place of the old, unbounded behavior.

Wiring into `elastic_correction()`/`elastic_correction_direct_impact()` (lines 366–473/475–517, new parameter `max_normalized_freq: float = 0.6` on both functions): the weight is computed separately for each branch (incident, reflected, transmitted — or transmitted only in Direct Impact), with the radius/celerity specific to THAT branch (input bar for incident/reflected, output bar for transmitted), then multiplied into the reconstructed spectrum just before the IFFT, inside `_corrected()` itself (§4.3):

```
fft_new = np.array([(m2[i] / m1[i]) * amp[i] * np.exp(1.0j * n_phase[i]) for i in range(len
    (n_phase))])
fft_new = fft_new * _bandwidth_rolloff_weight(np.asarray(freq), radius, c0,
    max_normalized_freq)
sig = np.fft.irfft(fft_new, n=n)
```

UI setting (`ui/pages/page_corrections.py`, "Correction bandwidth limit" card, lines 161–201): checkbox "Limit correction bandwidth" (`self.bandwidth_limit_check`, checked by default — `project.dispersion_bandwidth_limit_enabled: bool = True`, `core/project.py` line 353) and numeric field "Max normalized frequency (f · radius/C0)" (`self.bandwidth_limit_edit`, default value `project.dispersion_bandwidth_limit = 0.6`, line 358). Unchecking the box amounts to passing `max_normalized_freq=0.0` to all the correction functions — `_bandwidth_rolloff_weight` then returns an array of 1 everywhere, i.e. exactly the original behavior (unbounded gain, §5.1 above unchanged). `_max_normalized_freq_for_run()` (lines 487–494) encapsulates this switch and feeds all four correction calls (elastic/viscoelastic × two-bar/Direct Impact, lines 870/883/908/920) — `no_correction`/`no_correction_direct_impact` are, logically, not affected (no dispersion physics to bound in the absence of correction). Persisted in a session (`core/session.py`, lines 128–129/191–193) and in a setup profile (`core/setup_profile.py`, lines 77–78/144–147), alongside `dispersion_correction_mode`/`dispersion_setup_mode`.

The default of 0.6 is a pragmatic choice, not a bound derived from first principles — the docstring of `_bandwidth_rolloff_weight` states this explicitly: a round value, chosen to sit comfortably beyond the region where the real energy of a typical SHPB pulse has already died out, while remaining within the range usually inspected on the C_p/C_0 curve itself — a starting point, not

a proven optimum for every bar/pulse combination; to be adjusted (editable field) case by case, lower if ringing persists, higher only for a specific reason to trust the correction further out in the spectrum.

Complementary to, not redundant with, the low-pass filter on the raw signal (`Bar.lowpass_enabled`/`lowpass_cutoff_hz`, "Input/Output Bar low-pass filter" checkboxes on the same page, disabled by default, 300 kHz by default — `core/lowpass.py`): that filter acts before the correction, on the raw gauge signal, at a fixed cutoff frequency in Hz chosen by the user; the bandwidth bound above acts afterwards, on the correction's own gain, at a normalized cutoff frequency $f \cdot a/c_0$ automatically scaled to each bar's geometry/material. The two can be combined — one does not make the other redundant, since they bound different things (the content of the measured signal, versus the gain the correction itself can apply).

7.6 Switching between elastic and viscoelastic within the module

The choice between the four combinations in the table at the head of the chapter is made outside `core/dispersion.py`, in `ui/pages/page_corrections.py` (contextual help quoted: "Bar material model": No dispersion correction, Elastic bars (standard Pochhammer-Chree), Viscoelastic bars; and "Setup": two-bar setup vs. Direct Impact). Depending on the choices `project.dispersion_correction_mode`/`project.dispersion_setup_mode`, the page calls directly one of the following functions of `core/dispersion.py` (see `page_corrections.py`, method `_on_run`, lines ≈660–740):

- `no_correction`/`no_correction_direct_impact` (no dispersion physics, just window cutting),
- `elastic_correction`/`elastic_correction_direct_impact` (this chapter),
- `viscoelastic_correction`/`viscoelastic_correction_direct_impact` (separate chapter — Standard Linear Solid-type rheological model or other `RheologyModel`, complex modulus $E^*(\omega)$, attenuation, etc. → see the viscoelastic dispersion chapter).

The module therefore does not "switch" internally between the two physics through a single parameter or central `if`: these are distinct functions, selected upstream by the UI, which share only common utilities (window cutting, `_require_window`, `_index_of_nearest`, the `DispersionResult` structure, and the general FFT → per-bin correction → IFFT principle). The characteristic equation itself differs fundamentally between the two ("real" Pochhammer-Chree, with constant ν and c_0, for elastic; "complex" Pochhammer-Chree with modulus $E^*(\omega)$ depending on the rheological model, for viscoelastic — see `_solve_visco_branch`, outside the scope of this chapter).

7.7 Exposed results (DispersionResult, lines 99–110)

For diagnostics/plotting (the "Dispersion Corrections" page), in addition to the raw and corrected signals, the module exposes:

- `freq_over_c`: the reduced frequency parameter $f \cdot a/c_0$ for each branch (the classic dimensionless quantity of Pochhammer-Chree dispersion curves, often noted Ω or fa/c_0 in the literature) — line 401: `[f * a1 / c01 for f in freq]`.
- `phase_velocity_ratio`: the ratio $C_p(f)/c_0$ for each branch — line 402: `[c / c01 for c in cp_i]`.

Together these two quantities form the normalized dispersion curve C_p/c_0 as a function of fa/c_0, a standard representation for visualizing how far the correction departs from the 1D assumption ($C_p/c_0 = 1$) over the frequency band of the measured pulse.

- `temperature_profile`: the profile $(z, T(z))$ along each bar, only if `heat_check` is active (see §2.3) — with no direct link to geometric dispersion itself, but influencing the input c_0, ν.

7.8 Summary of code correspondences

Theoretical element	File / function
1D bar speed $c_0 = \sqrt{E/\rho}$, ν, optional thermal correction	`core/dispersion.py::_bar_elastic_wave_speed()`
Pochhammer-Chree characteristic equation (Bessel J_0/J_1)	`core/dispersion.py::_solve_pochhammer_branch()`, lines 179–182
Continuation root search (`sympy.nsolve`, seed = previous solution)	`core/dispersion.py::_solve_pochhammer_branch()`, lines 178–196
Amplitude correction factors M_1, M_2	`core/dispersion.py::_solve_pochhammer_branch()`, lines 188–194
Branch parallelization, incident/reflected/transmitted	`core/dispersion.py::_solve_pochhammer_branches()`
Window time cutting, non-dispersive times of flight	`core/dispersion.py::elastic_correction()` (lines 329–352) / `_cut_two_bar_windows()`
FFT of the windowed pulse	`core/dispersion.py::elastic_correction()`, lines 354–359
Phase + amplitude correction per bin	`core/dispersion.py::elastic_correction()::_corrected()`, lines 373–385
IFFT (reconstruction, n=n critical)	`core/dispersion.py::_corrected()`, line 384
Sign convention forced per wave	`core/dispersion.py::elastic_correction()`, lines 387–389
Direct Impact case (output bar only)	`core/dispersion.py::elastic_correction_direct_impact()`, lines 407–449
Elastic/viscoelastic/none switch (UI)	`ui/pages/page_corrections.py`, "Bar material model" combo, method `_on_run`

7.9 Limitations and warnings identified in the code

- Not numerically re-verified: explicit warning at the top of the module (§above), to be repeated in the user documentation — the port is judged faithful to the original algorithm but could not be re-tested against a known reference case.
- No guard on `nsolve` convergence in the high-frequency elastic regime (unlike the viscoelastic path, which does have a try/except fallback): a poorly found root or a divergence of the continuation at high frequency would not be detected automatically.
- No spectral filtering/windowing: all frequencies up to Nyquist are corrected and recomposed; no attenuation of noisy high frequencies.
- Physical interpretation of M_1 undocumented in the code: its form is consistent with a Bancroft/Davies-type amplitude correction factor (surface vs. cross-section average), but this is not confirmed by a code comment — to be treated with caution in the final documentation (proposed wording: "empirical correction factor of Pochhammer-Chree theory, whose role is to correct the amplitude of each frequency component in addition to its phase").
- Non-dispersive time of flight for window cutting: positioning of the reflected/transmitted

windows (§4.1) uses constant C_0 even in the corrected elastic case — only the spectral correction itself is dispersive, not the placement of the time windows.

- The `n=n` parameter of `np.fft.irfft` is flagged in the code itself (lines 376–383) as a critical, non-optional correction point for odd-length windows (a common case here, since windows are freely positioned by the user) — omitting it would produce a numerically wrong reconstructed signal over its entire length, not merely a shifted one.

8. Viscoelastic dispersion correction, Bayesian calibration and damping

This chapter covers three closely related code blocks:

1. the rheological models (`core/rheology_models.py`) that supply the complex modulus $E^*(f)$ of a bar made of a viscoelastic material (polymer) ;
2. the calibration of these models from two free-decay pulses measured at two points on the same bar, either in Bayesian-search mode (`core/bayes_visco_calibration.py`, with its Gaussian-Process surrogate `core/gaussian_process.py`) or in fixed-parameter "Test parameters" mode (`core/visco_calibration.py`), together with automatic alignment of the two pulses (`core/pulse_alignment.py`) ;
3. the standalone "Free-Decay Damping" tool (`core/decay.py`), which estimates a damping coefficient using the half-power bandwidth method.

The general FFT-based dispersion-correction mechanism (frequency decomposition, per-bin phase/amplitude correction, IFFT) is covered in the chapter on elastic dispersion and is not repeated here. This chapter focuses on how $E^*(f)$ is computed for the selected rheological model, and on the calibration/analysis algorithms that build on it.

This file covers sections 8.1 to 8.4 (rheological models, pulse propagation and Bayesian calibration) ; the Gaussian Process surrogate (8.5), pulse auto-alignment (8.6), the free-decay damping tool (8.7) and the end-of-chapter summaries (8.8, 8.9) are covered elsewhere.

> **Warning from the code itself:** the fixed-parameter calibration module carries the following warning in its own docstring (`core/visco_calibration.py`, lines 43-46): "NOT YET independently re-validated against real experimental data or the original GUI's output [...] ported as literally as possible from the original for traceability; cross-check results against the original application before relying on this for a real calibration." It applies to this whole chapter, which relies entirely on `predict_pulse`/`pulse_correlation` from this module (§8.3).

8.1 Why a viscoelastic model?

In the standard "elastic" dispersion correction (metal bars), the bar's Young's modulus E is a real constant, independent of frequency: only the geometry (radius, Poisson ratio) introduces dispersion, via the Pochhammer-Chree equation.

For bars made of a viscoelastic material (polymers such as PMMA, polycarbonate, nylon – used in particular to test soft, low-impedance materials), the elastic modulus is no longer a constant but a frequency-dependent complex modulus:

$$E^*(\omega) = E'(\omega) + i\,E''(\omega), \qquad \omega = 2\pi f \tag{8.1}$$

where $E'(\omega)$ (real part, "storage modulus") represents the instantaneous elastic stiffness and $E''(\omega)$ (imaginary part, "loss modulus") represents viscous dissipation. This frequency dependence feeds directly into the complex phase velocity used by the dispersion solver,

$$c(\omega) = \sqrt{E^*(\omega)/\rho} \tag{8.2}$$

(see `core/visco_calibration.py::_complex_modulus_and_speed`, lines 88-97), which then feeds into exactly the same Pochhammer-Chree root search (Bessel functions) as the elastic correction, but with complex shear/Lame moduli $\mu(\omega)$, $\lambda(\omega)$ instead of real ones. That root search now relies on the new shared module `core/pochhammer_solver.py` (§8.3.2), rather than on a symbolic solver duplicated inside the calibration module.

`core/rheology_models.py`'s docstring points out that only $E^*(f)$ changes from one model to the next: the per-frequency-bin Pochhammer-Chree root search, the FFT/IFFT pulse propagation and the derived Lame moduli are generic and do not otherwise depend on which rheological model was chosen ("that seam is what makes it possible to offer several models here without duplicating the (expensive, delicate) dispersion physics per model", lines 9-11).

The bar's Poisson ratio ν is deliberately kept separate from each rheological model's own parameters: it is a property that is always present on the bar, independent of which relaxation model is used for its modulus (`core/rheology_models.py`, lines 13-15).

The dispersion correction actually applied on the Dispersion Corrections page – updated from an earlier version of this documentation. Contrary to what an earlier version of this text stated (describing the viscoelastic dispersion corrector as hard-coded to the Standard Linear Solid model alone), `core/dispersion.py::viscoelastic_correction` and `viscoelastic_correction_direct_impact` now take a generic `RheologyModel` and a `params` dict and call `model.complex_modulus(freq, params)` (`core/dispersion.py`, lines 741-791 for the two-bar function), exactly like `core/visco_calibration.py`. `core/rheology_models.py`'s own docstring (lines 45-54) confirms this explicitly: "[...] `viscoelastic_correction`/ `viscoelastic_correction_direct_impact` [...] are generic over all four models here, via `model.complex_modulus(freq, params)`."
Point directly verified in the code rather than taken from a single docstring: the calibration page's own docstring (`ui/pages/page_visco_calib.py`, lines 10-13) and the contextual help for its "Rheological model" card (lines 324-328) still state the old behaviour: "A found/tested Standard Linear Solid set can be pushed directly to the Dispersion Corrections page with one click – that page's physics only understands that one model, so the button is disabled for the other three." This text is no longer accurate: the `apply_button` starts disabled (line 557) and is disabled while a run is in progress (line 985), but is re-enabled *unconditionally with respect to the active model* as soon as a result is available (`self.apply_button.setEnabled(True)`, line 1107, in `_on_run_finished`) – no code path leaves it disabled for Kelvin-Voigt, Generalized Maxwell or Burgers. There is therefore an inconsistency between the documentation embedded in the code (module docstring, page-level contextual help) and the behaviour actually observed in the source: worth checking with the author before relying on it for a design decision, but as written today, "Apply to Dispersion Corrections" does work for all four models, not just Standard Linear Solid.

Separately, the Monte-Carlo bar expansion (`core/models.py::Bar.expand_viscoelastic`) remains a separate code path unrelated to this generic mechanism: it stays hard-coded to the SLS module and is not covered by this chapter (`core/rheology_models.py`, docstring note, lines 56-61). The corresponding project storage (`core/project.py::ViscoelasticSettings`) has also been generalized: it now carries a generic `model_key` plus `params`/`params_std` dicts, instead of the older fixed `e1`/`e2`/`n2` fields; a `migrate_viscoelastic_dict` function upgrades any older session/profile file saved before this change.

Bandwidth limiting of the correction (`_bandwidth_rolloff_weight`): `viscoelastic_correction` and `viscoelastic_correction_direct_impact` receive, like their elastic counterparts, a `max_normalized_`

`freq: float = 0.6` parameter – see the "elastic dispersion" chapter §5.2 for the full mechanism (a raised-cosine $0 \to 1$ taper multiplied into the reconstructed spectrum before `irfft`, motivated by avoiding high-frequency noise amplification, tunable on the Corrections page). The only difference in the viscoelastic case is the choice of the reference speed c_0 that normalizes the frequency: a viscoelastic bar has no single, constant wave speed the way an elastic bar does, so each branch (incident, reflected, transmitted) uses its own zero-frequency speed derived from that branch's complex modulus, `abs(c_i[0])` / `abs(c_r[0])` / `abs(c_t[0])`, rather than a single shared c_0. Concretely (`core/dispersion.py`, lines 818-820):

```
w_i = _bandwidth_rolloff_weight(freq, a1, abs(c_i[0]), max_normalized_freq)
w_r = _bandwidth_rolloff_weight(freq, a1, abs(c_r[0]), max_normalized_freq)
w_t = _bandwidth_rolloff_weight(freq, a2, abs(c_t[0]), max_normalized_freq)
```

with each weight then multiplied into the corresponding spectrum right before `np.fft.irfft(.. ., n=n)`. In `viscoelastic_correction_direct_impact` (a single transmitted branch), only `w_t` exists (line 877), built the same way from `abs(c_t[0])`. The setting (checkbox + value, Dispersion Corrections page) is shared with the elastic case: the same `project.dispersion_bandwidth_limit_enabled` / `project.dispersion_bandwidth_limit` and the same `_max_normalized_freq_for_run()` feed all four correction functions (elastic two-bar, elastic Direct Impact, viscoelastic two-bar, viscoelastic Direct Impact) – see the "elastic dispersion" chapter §5.2 for the full UI wiring; it is not documented or exposed separately for the viscoelastic case.

Finally, `core/dispersion.py` does not use the new `core/pochhammer_solver.py` solver: its own per-frequency-bin root search still calls `sympy.nsolve` directly (`core/dispersion.py`, lines 33-34, 236, 559, 574), and its own module docstring says so explicitly (lines 19-23): "this module is numerically faithful to the original but has not been re-validated against real experimental data (no test data / no way to run sympy's `nsolve` root search in this environment)". The replacement by the fast solver described in §8.3.2 has, to date, only been carried out in `core/visco_calibration.py`, not in `core/dispersion.py`.

8.2 The four rheological models (core/rheology_models.py)

The module defines a generic `RheologyModel` structure (dataclass, line 102) carrying a list of `ParamSpec` entries (key, label, unit, Bayesian-search bounds, default value for "Test parameters" mode) and a `complex_modulus(freq, params) -> E*(f)` function. Four models are registered in the `MODELS` dictionary (line 199, keys `MODEL_KELVIN_VOIGT`, `MODEL_SLS`, `MODEL_GENERALIZED_MAXWELL_2`, `MODEL_BURGERS`), listed in order of increasing complexity in the module docstring (lines 17-43): Kelvin-Voigt (2 parameters) → Standard Linear Solid (3 parameters) → Generalized Maxwell 2 branches (5 parameters) → Burgers (4 parameters). This file has not changed since the previous version of this documentation.

8.2.1 Kelvin–Voigt (_kelvin_voigt_modulus, line 120)

Rheological scheme: a spring E mounted in parallel with a dashpot η (`short_formula = "E || eta"`).

Parameters: E (Pa), η (Pa·s). Complex modulus:

$$E^*(\omega) = E + i\,\omega\,\eta \tag{8.3}$$

Code (lines 120-123):

```
def _kelvin_voigt_modulus(freq, params: dict) -> np.ndarray:
    e, eta = params["e"], params["eta"]
    w = 2 * np.pi * np.asarray(freq, dtype=float)
    return e + 1j * w * eta
```

Limitation noted in the code (`formula_html`, lines 209-216): this model's damping grows without bound as frequency increases, and it does not distinguish a separate instantaneous/relaxed modulus; it is described as "a rough baseline rather than a good fit for a real polymer's relaxation spectrum."

8.2.2 Standard Linear Solid / Zener (_sls_modulus, line 126) – the only model wired into the Dispersion Corrections page prior to this revision

Rheological scheme: a spring E_1 in series with a Kelvin-Voigt cell (E_2 in parallel with n_2) – the "3-parameter Kelvin" representation (`short_formula = "E1 -- (E2 || n2)"`).

Traceability of the formula (module docstring, lines 25-30): the formula was "re-derived and checked numerically against both classical SLS topologies to pin down which one it actually is, since the app's own existing code comments describe the other one." `ui/rheology_diagram.py::draw_sls` draws this "series-Kelvin" topology, consistent with what is actually calibrated (`_sls_modulus`'s own docstring, lines 127-131).

Parameters: E_1 (Pa), E_2 (Pa), n_2 (Pa·s). Constitutive equation (compliance $J(\omega)$, then modulus $E^*(\omega) = 1/J(\omega)$):

$$J(\omega) = \frac{1}{E_1} + \frac{E_2}{E_2^2 + (\omega n_2)^2} - i\,\frac{\omega n_2}{E_2^2 + (\omega n_2)^2}, \qquad E^*(\omega) = \frac{1}{J(\omega)} \tag{8.4}$$

Code (lines 132-136):

```
def _sls_modulus(freq, params: dict) -> np.ndarray:
    e1, e2, n2 = params["e1"], params["e2"], params["n2"]
    w = 2 * np.pi * np.asarray(freq, dtype=float)
    j1 = (1.0 / e1) + (e2 / (e2 ** 2 + (w * n2) ** 2))
    j2 = (w * n2) / (e2 ** 2 + (w * n2) ** 2)
    return 1.0 / (j1 - 1j * j2)
```

This is a single-relaxation-time (Debye) model. The model's `polymer_note` warns that this single spectrum is "often too narrow for PMMA/PC/nylon's real spectrum", whose loss spectrum is broader and less symmetric. This is the model the application originally shipped with, and – as corrected in §8.1 above – it is no longer, as of today, the only model `core/dispersion.py` is able to apply.

8.2.3 Generalized Maxwell / Wiechert, 2 branches (_generalized_maxwell_2_modulus, line 139)

Rheological scheme: a long-term spring E_∞ in parallel with two Maxwell arms (E_1–n_1 and E_2–n_2), each tuned to its own relaxation time $\tau_k = n_k/E_k$ (`short_formula = "Einf || (E1-n1) || (E2-n2)"`).

Parameters: E_∞ (Pa), E_1 (Pa), n_1 (Pa·s), E_2 (Pa), n_2 (Pa·s). Constitutive equation:

$$\tau_k = \frac{n_k}{E_k} \quad (k = 1, 2), \qquad E^*(\omega) = E_\infty + \sum_{k=1,2} E_k \cdot \frac{i\,\omega\,\tau_k}{1 + i\,\omega\,\tau_k} \tag{8.5}$$

Code (lines 140-147):

```
def _generalized_maxwell_2_modulus(freq, params: dict) -> np.ndarray:
    e_inf, e1, n1, e2, n2 = params["e_inf"], params["e1"], params["n1"], params["e2"],
    params["n2"]
    w = 2 * np.pi * np.asarray(freq, dtype=float)
    tau1, tau2 = n1 / e1, n2 / e2
    return (
        e_inf
        + e1 * (1j * w * tau1) / (1 + 1j * w * tau1)
        + e2 * (1j * w * tau2) / (1 + 1j * w * tau2)
    )
```

With two relaxation times, this model gives a broader, asymmetric spectrum, described in the `polymer_note` as "generally a closer match to real amorphous/semi-crystalline polymers than a single Debye term."

8.2.4 Burgers (_burgers_modulus, line 150)

Rheological scheme: a Maxwell element (spring E_1 in series with dashpot n_1) mounted in series with a Kelvin-Voigt cell (spring E_2 in parallel with dashpot n_2) – a direct combination of the two simplest models (`short_formula = "(E1-n1) -- (E2 || n2)"`).

Parameters: E_1 (Pa), n_1 (Pa·s), E_2 (Pa), n_2 (Pa·s). Constitutive equation (compliance, then inversion):

$$J(\omega) = \frac{1}{E_1} + \frac{1}{i\,\omega\,n_1} + \frac{1}{E_2 + i\,\omega\,n_2}, \qquad E^*(\omega) = \frac{1}{J(\omega)} \tag{8.6}$$

Code (lines 182-191):

```
def _modulus_at(f):
    w = 2 * np.pi * f
    j_maxwell = (1.0 / e1) + 1.0 / (1j * w * n1)
    j_kelvin_voigt = 1.0 / (e2 + 1j * w * n2)
    return 1.0 / (j_maxwell + j_kelvin_voigt)

e_complex = _modulus_at(_safe_freq(freq))
if freq.size > 1 and freq[0] == 0.0:
    e_complex[0] = _modulus_at(freq[1]).real
return e_complex
```

Singularity at $f = 0$ and special handling: unlike the other three models, Burgers' static (zero-frequency) modulus is exactly zero (the Maxwell arm "flows freely under any sustained stress"). This causes a problem documented in detail in `_burgers_modulus`'s own docstring (lines 151-178):

- `core/visco_calibration.py::_predict_pulse` computes a reference speed $c[0] = \sqrt{E(0)/\rho}$ that it reuses, unchanged, in its phase-correction formula for every frequency bin, and its final step (`cmath.rect`) requires an exactly real result – even a tiny nonzero imaginary part triggers a crash ("can't convert complex to float") ;

- a zero-frequency reference is not physically relevant anyway for a millisecond-scale SHPB pulse (the material never actually sustains a DC load during the test).

Implemented solution: the DC bin (`freq[0] == 0.0`) is evaluated not at $f = 0$ but at the pulse's own lowest nonzero fundamental frequency, `freq[1]` (already used elsewhere in `_predict_pulse` as the root-search seed), keeping only the real part of that evaluation as the reference stiffness and dropping its imaginary (loss) part. The docstring notes that, numerically, this lands "within a fraction of a percent of `|E(freq[1])|`" (the convention `_predict_pulse` uses for every other bin) – a well-conditioned stand-in, not a degenerate one.

`_safe_freq` (lines 115-117) replaces any zero frequency with `_EPS_FREQ = 1e-6` Hz to avoid a literal division by zero in $1/(i\,\omega\,n_1)$ – but this guard is applied only inside `_burgers_modulus`, never to the other three models, whose DC modulus is already finite and exactly real; the module explicitly notes (lines 73-88) that applying this nudge to the other models would cause the same complex-to-real conversion crash ("this was tried once and caused exactly that crash").

This model has a genuinely zero long-term modulus, which lets it represent instantaneous elasticity + retarded (creep) elasticity + genuine viscous flow – relevant for thermoplastics such as nylon under sustained or repeated loading (`polymer_note`), a combination none of the other three models (all with a finite long-term modulus) can represent.

8.2.5 Summary table

Model	**Key (`MODEL_*`)**	**Parameters (units)**	$E^*(\omega)$
Kelvin–Voigt	`kelvin_voigt`	E (Pa), η (Pa·s)	$E + i\omega\eta$
Standard Linear Solid	`standard_linear_solid`	E_1, E_2 (Pa), n_2 (Pa·s)	$1/\left[\frac{1}{E_1} + \frac{E_2}{E_2^2 + (\omega n_2)^2} - i\frac{\omega n_2}{E_2^2 + (\omega n_2)^2}\right]$
Generalized Maxwell (2 branches)	`generalized_maxwell_2`	E_∞, E_1, E_2 (Pa), n_1, n_2 (Pa·s)	$E_\infty + \sum_{k=1,2} E_k \frac{i\omega\tau_k}{1 + i\omega\tau_k}$, $\tau_k = n_k/E_k$
Burgers	`burgers`	E_1, E_2 (Pa), n_1, n_2 (Pa·s)	$1/\left[\frac{1}{E_1} + \frac{1}{i\omega n_1} + \frac{1}{E_2 + i\omega n_2}\right]$

8.3 Pulse propagation through the viscoelastic bar model (core/visco_calibration.py)

This function is the shared core of both calibration modes (Bayesian and "Test parameters"): it predicts pulse 2 (measured further along the bar) from the measured pulse 1 and a candidate parameter set. Since the previous version of this documentation, this function's internal root search no longer relies on `sympy.nsolve` but on the new shared module `core/pochhammer_solver.py` (§8.3.2), with no change to the underlying physics.

8.3.1 Complex modulus, speed and Lame moduli (_complex_modulus_and_speed, line 88)

For each frequency f in pulse 1's FFT spectrum:

$$\mu(f) = \frac{E^*(f)}{2(1+\nu)}, \qquad \lambda(f) = \frac{\nu\, E^*(f)}{(1+\nu)(1-2\nu)}, \qquad c(f) = \sqrt{\frac{E^*(f)}{\rho}} \tag{8.7}$$

where ν is the Poisson ratio (a separate parameter, cf. §8.1) and ρ the bar's density. Code (lines 93-97):

```
e_complex = model.complex_modulus(freq, params)
mu = [e / (2 * (1 + nu)) for e in e_complex]
ram = [nu * e / ((1 + nu) * (1 - 2 * nu)) for e in e_complex]
c = [cmath.sqrt(e / density) for e in e_complex]
return mu, ram, c
```

8.3.2 Per-frequency-bin Pochhammer-Chree root search (_predict_pulse, line 100, via core/pochhammer_solver.py)

Pulse 1 is transformed with a real FFT (`np.fft.rfft`), giving amplitude and phase per bin. For each nonzero frequency f_v ($v = 1, \dots$), with a the bar radius, the code numerically solves (unknown B):

$$p = \sqrt{\frac{\rho\,\omega_v^2}{\lambda_v + 2\mu_v} - B^2}, \qquad q = \sqrt{\frac{\rho\,\omega_v^2}{\mu_v} - B^2} \tag{8.8}$$

$$\frac{2p}{a}(q^2 + B^2)\, J_1(pa)\, J_1(qa) - (q^2 - B^2)^2\, J_0(pa)\, J_1(qa) - 4B^2 pq\, J_0(qa)\, J_1(pa) = 0 \tag{8.9}$$

(J_0, J_1: Bessel functions of the first kind). The complex root found gives the wavenumber $k = \Re(B)$ (used for the phase velocity $c_p = \omega_v/k$) and the attenuation $\mathrm{atn} = |\Im(B)|$.

The solver (`core/pochhammer_solver.py`, an entirely new module). Before this revision, this root search was performed directly inside `core/visco_calibration.py` by `sympy.nsolve`, which symbolically re-derives and evaluates the equation (arbitrary-precision `mpmath` arithmetic) on every call – for a pulse window of a few hundred to a few thousand samples, that meant as many *independent* symbolic root searches per forward-model evaluation, and `core/bayes_visco_calibration.py`'s own docstring explicitly calls this cost the dominant one for the whole Bayesian search (§8.4.1). The code has been extracted into this new shared module and rewritten as a complex secant iteration (`complex_secant_solve`) built on `scipy.special.jv` (a compiled, complex-argument Bessel function), evaluating *exactly* the same two equations `_predict_pulse` always solved (`pochhammer_chree_residual`/`pochhammer_chree_fallback_residual` below), with the same per-bin continuation convention (each bin's solve starts from the previous bin's own converged root, never from scratch), and the same exception contract (`RootNotFoundError` is a `ValueError` subclass, specifically so every existing `except ValueError:` around a root-search call keeps working unchanged).

The module's docstring stresses that this is a *faithful re-implementation*, not a different numerical method that happens to find "a" root: a secant method is exactly what `mpmath`'s `findroot` (sympy's own `nsolve` backend) itself falls back to for a scalar equation given one starting point and no explicit derivative, and `scipy.special.jv` evaluates bit-for-bit the same Bessel function of the first kind as `sympy.besselj` – they are the same mathematical function, evaluated by a compiled, decades-validated (AMOS/Cephes-derived) numeric routine instead of one re-derived symbolically on every call.

Primary residual code (`pochhammer_chree_residual`, lines 137-158):

```
def pochhammer_chree_residual(b: complex, omega: float, density: float, ram_v: complex,
    mu_v: complex,
                                radius: float) -> complex:
    p = cmath.sqrt(density * omega ** 2 / (ram_v + 2 * mu_v) - b ** 2)
    q = cmath.sqrt(density * omega ** 2 / mu_v - b ** 2)
    return (
        (2 * p / radius) * (q ** 2 + b ** 2) * jv(1, p * radius) * jv(1, q * radius)
        - (q ** 2 - b ** 2) ** 2 * jv(0, p * radius) * jv(1, q * radius)
        - 4 * (b ** 2) * p * q * jv(0, q * radius) * jv(1, p * radius)
    )
```

Call from `_predict_pulse` (lines 127-148):

```
for v in range(total):
    omega = 2 * cmath.pi * freq[v + 1]
    try:
        dsol = complex_secant_solve(
            lambda b: pochhammer_chree_residual(b, omega, density, ram[v + 1], mu[v + 1],
    radius),
            k_sol[-1],
        )
        b_sol.append(dsol)
        k_sol.append(float(dsol.real))
        cp_sol.append(2 * math.pi * freq[v + 1] / k_sol[-1])
        atn.append(float(abs(dsol.imag)))
    except RootNotFoundError:
        dsol = complex_secant_solve(
            lambda cp: pochhammer_chree_fallback_residual(cp, omega, c[v + 1], nu, beta,
    radius),
            cp_sol[-1],
        )
        cp_sol.append(float(dsol.real))
        k_sol.append(2 * math.pi * freq[v + 1] / cp_sol[-1])
        atn.append(atn[-1])
        b_sol.append(k_sol[-1] - atn[-1] * 1j)
    if progress_cb:
        progress_cb(v + 1, total)
```

Fallback (`except RootNotFoundError`, shown above): if the primary formulation's search fails, the code solves a second form of the Pochhammer-Chree equation, this time for the unknown c_p directly, with:

$$\beta = \frac{1-2\nu}{1-\nu}, \qquad \xi = \left(\frac{c_p}{c_v}\right)^2 (1+\nu), \qquad k = \frac{\omega_v}{c_p} \tag{8.10}$$

$$h_e = k\sqrt{\beta\xi - 1}, \qquad k_e = k\sqrt{2\xi - 1} \tag{8.11}$$

$$(\xi-1)^2\, a\, h_e \frac{J_0(a h_e)}{J_1(a h_e)} - (\beta\xi - 1)\left(\xi - a k_e \frac{J_0(a k_e)}{J_1(a k_e)}\right) = 0 \tag{8.12}$$

Fallback residual code (`pochhammer_chree_fallback_residual`, lines 161-176):

```
def pochhammer_chree_fallback_residual(cp: complex, omega: float, c_v: complex, nu: float,
    beta: float,
```

```
                                        radius: float) -> complex:
    xi = (cp / c_v) ** 2 * (1 + nu)
    nl = omega / cp
    h_e = nl * cmath.sqrt(beta * xi - 1)
    k_e = nl * cmath.sqrt(2 * xi - 1)
    return (
        ((xi - 1) ** 2) * radius * h_e * (jv(0, radius * h_e) / jv(1, radius * h_e))
        - (beta * xi - 1) * (xi - radius * k_e * (jv(0, radius * k_e) / jv(1, radius * k_e)
    ))
    )
```

In this fallback case, attenuation is not recomputed: it is carried over from the previous bin (`atn.append(atn[-1])`), and the equivalent complex wavenumber is reconstructed as $B = k - i \cdot \text{atn}$. Neither the original module nor `core/pochhammer_solver.py` explicitly documents the physical derivation of this second formulation or the precise conditions that trigger the fallback; it is reported here as implemented, with no further interpretation.

The first nonzero frequency bin ($v = 0$) is seeded by simple linear extrapolation: `k_sol = [0, ( 2*pi*freq[1])/c[1]]` (line 122), used as the seed (`k_sol[-1]`) for the first call to `complex_secant_solve`.

Effect of replacing `sympy.nsolve` – a performance gain, not a physics change, but one new, handled failure mode. `core/pochhammer_solver.py`'s module docstring notes several points:

- convergence of `complex_secant_solve` is judged by the *relative* size of the last secant step (`|step| <= tol * max(1, |x|)`), not by how small the residual itself is – these equations mix Bessel-function ratios and polynomial terms with no natural common scale, so an absolute-residual tolerance would be either too loose or too tight depending on the bin; this scale-free criterion is close to what `mpmath.findroot` itself effectively converges on ;
- **a new failure mode, absent from the original sympy implementation**: during a Bayesian search's "blind" exploration (typically the initial Latin Hypercube Sampling design, §8.4.3, which samples the whole search box with no regard for physical plausibility), an extreme parameter combination can overflow the range of an IEEE-754 `double` in the complex arguments of `pochhammer_chree_residual`/`pochhammer_chree_fallback_residual`. `sympy.nsolve` never saw this case, because `mpmath`'s arbitrary-precision arithmetic has a far larger exponent range than a `double` – so this genuinely is "a new failure mode this faster implementation can hit that the original could not." It is handled exactly like any other convergence failure: it raises `RootNotFoundError` (a `ValueError`), caught by `_predict_pulse`'s existing fallback to the second equation (or, if that also fails, propagated up to `core/bayes_visco_calibration.py`'s existing handling, which scores the candidate point 0.0, cf. §8.4.2) – not a crash, and not a silent `inf`/`nan` ;
- the performance gain quoted by `core/bayes_visco_calibration.py`'s own docstring (§8.4.1) is "roughly two to three orders of magnitude faster per call" ;
- practical consequence noted in `core/visco_calibration.py` (lines 55-57): this module no longer has an "un-installable-in-this-sandbox" dependency (sympy), which finally makes it directly testable in this environment (see `test_visco_calibration.py`).

No numerical bug in the old duplicated code is reported as fixed by this refactor: it is an extraction/speed-up with equivalent numerical behaviour, with this one new failure mode (floating-point range overflow), itself handled cleanly by the already-existing fallback mechanism.

8.3.3 Phase, attenuation and damping correction, then time-domain reconstruction

For each bin w:

$$\Delta\varphi_w = \varphi_w - \frac{2\pi f_w z_1}{c_0}\left(\frac{c_0}{|c_{p,w}|} - 1\right) \tag{8.13}$$

$$\hat{Y}_w = \exp\left(-\frac{C_{\text{damp}}}{c_0} z_1\right) \cdot A_w \cdot \exp(-\text{atn}_w z_1) \cdot \exp(i\,\Delta\varphi_w) \tag{8.14}$$

where z_1 is the distance between the pulse's two measurement points, C_{damp} the bar's damping factor ("Damping factor/C", in rad/m – typically obtained from the Free-Decay Damping tool, §8.7), $c_0 = c(f{=}0)$ the reference speed, and A_w/φ_w pulse 1's spectral amplitude/phase. Code (lines 159-164):

```
c0_real = c[0].real
new_phase = [phase[a] - (2 * cmath.pi * freq[a] * z1 / c0_real) * ((c0_real / abs(cp_sol[a
    ]) - 1)) for a in range(len(phase))]
new_pulse_fft = [
    cmath.exp(-(damp / c[0]) * z1) * amplitude[w] * cmath.exp(-atn[w] * z1) * cmath.rect(1,
     new_phase[w])
    for w in range(len(new_phase))
]
```

The predicted pulse 2 is obtained by a real inverse FFT with the length explicitly forced (`np.fft.irfft(new_pulse_fft, n=n)`, line 182) – the comment (lines 165-181) explains that without this explicit `n=n`, an odd-length pulse window (a common case, since the window comes from a user drag-and-drop) would produce a numerically wrong reconstructed signal, not merely a truncated one (a measured discrepancy of up to 2 in a signal of order 1, for a 139-sample window), because of `irfft`'s implicit assumption about the parity of the original length – this one-sample shortfall had originally surfaced as a plotting crash ("x and y must have same first dimension").

8.3.4 Comparison score: Pearson vs R²(SCORE_FUNCS, lines 233-236)

The predicted pulse and the measured pulse 2 are first aligned over their common interval (`_align_pulses`, line 186, windowing rule `time2 >= 0 / time1 < max(time2)`, unchanged since the original implementation). Two metrics are offered (`SCORE_ORDER = ("r2", "correlation")`, default `DEFAULT_METRIC = "r2"`):

Pearson correlation (`_pearson_score`, line 211):

$$r = |\text{corr}(\text{measured}, \text{predicted})| = \left|\frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_i (x_i - \bar{x})^2}\sqrt{\sum_i (y_i - \bar{y})^2}}\right| \tag{8.15}$$

(via `scipy.stats.pearsonr`, absolute value taken). The module's docstring stresses that this measure is invariant to an affine transform of either signal – precisely its blind spot here: a candidate whose shape is correct but whose absolute amplitude (damping, overall modulus) is wrong can still score close to 1.

Coefficient of determination R^2(`_r2_score`, line 217), the default criterion:

$$SS_{\text{res}} = \sum_i (y_i - \hat{y}_i)^2, \qquad SS_{\text{tot}} = \sum_i (y_i - \bar{y})^2, \qquad R^2 = \text{clip}\left(1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}}, -1, 1\right) \tag{8.16}$$

(y = measured pulse, $\hat{y}$ = predicted pulse). The code explicitly notes that $R^2$is clipped to $[-1, 1]$ (whereas a classical $R^2$is unbounded below) "so that an unbounded score would let one pathological candidate dominate the Gaussian Process surrogate's fitted length scale during Bayesian optimization" (lines 221-225); -1 already means "twice as wrong as just predicting the mean", which is plenty bad for the optimizer to reject that region.

The public functions `predict_pulse` (line 252) and `pulse_correlation` (line 261) are the entry points reused by `core/bayes_visco_calibration.py`; `test_parameters` (line 269) is the entry point for the page's "Test parameters" mode: it evaluates a single fixed $(\nu, \text{model}, \text{params})$ set chosen by the user.

Explicit warning from the module (docstring, lines 43-46): "NOT YET independently re-validated against real experimental data or the original GUI's output [...] ported as literally as possible from the original for traceability; cross-check results against the original application before relying on this for a real calibration."

8.4 Bayesian calibration (core/bayes_visco_calibration.py)

8.4.1 Why a Bayesian search rather than a grid

The module docstring (lines 7-16) justifies this choice: a grid search over a model's n parameters (up to 6 dimensions for Generalized Maxwell, ν included) would cost $\text{count}_1 \times \text{count}_2 \times \cdots \times \text{count}_n$ evaluations, each involving a root search per frequency bin – an expensive operation. Bayesian optimization replaces this with: (1) an initial Latin Hypercube Sampling (LHS) design, (2) a cheap-to-fit/evaluate Gaussian Process surrogate, (3) an iterative proposal of the next point maximizing an acquisition strategy over the surrogate (§8.4.4), (4) early stopping via patience.

A historical note in the module (lines 18-30) states that forward-model evaluations used to be "originally" far more expensive still: `core.visco_calibration._predict_pulse` used to solve each frequency bin's root via `sympy.nsolve` (symbolically re-derived on every call), since replaced by the scipy-based secant iteration described in §8.3.2, "roughly two to three orders of magnitude faster per call." The Bayesian-optimization approach remains the right one regardless of this speed-up – the forward model is still a non-differentiable black box (no gradient-based sampler, e.g. PyMC's NUTS, can be pointed at it directly) and is still far more expensive than the GP surrogate proposing where to evaluate it next – but the original "thousands of evaluations would be prohibitively slow" framing is less extreme than when this module was first written.

Two direct benefits for this problem: far fewer expensive forward-model calls are needed to reach a given precision (tens rather than thousands, regardless of how many parameters the chosen model has), and parameter values are continuous rather than snapped to `linspace` steps, so the search converges to the true optimum rather than to the nearest grid node.

8.4.2 Evaluating a candidate point, priors and hyperpriors

A candidate point starts as a vector in the unit hypercube $[0, 1]^d$ (the working space of the GP and of the acquisition search). Since the previous version of this documentation, converting that

vector into physical parameters no longer follows a simple fixed uniform/log-uniform rule per parameter: every parameter (the Poisson ratio included) now carries its own `PriorSpec` (dataclass, line 269), of one of four kinds (`PRIOR_FIXED`, `PRIOR_UNIFORM`, `PRIOR_GAUSSIAN`, `PRIOR_LOGNORMAL`):

Prior	Fields	Role
`"fixed"`	`value`	pinned to a single value, dropped from the search (one fewer dimension).
`"uniform"`	`lo`, `hi`	flat between `lo` and `hi` – the implicit default; log-uniform for moduli/viscosities, linear for ν.
`"gaussian"`	`mean`, `std`	a Normal(mean, std) prior, in physical units – e.g. a literature value with its uncertainty.
`"lognormal"`	`mean` (median), `std` (factor $F > 1$)	a Log-Normal prior: about 68% of the mass falls between median/F and median$\times F$ – suited to a modulus/viscosity known only to within an order of magnitude, while staying strictly positive.

The module docstring's "PRIORS" section (lines 80-95) explains why this matters: without an explicit prior, the posterior estimate itself (§8.4.5) draws its dense sample uniformly across the whole box, which leaves a reported uncertainty nearly as wide as the box itself whenever the score surface is fairly flat over a large part of it – exactly what the parameter interdependence matrix is warning about when it reports values near ± 1 (§8.4.5). A Gaussian/Log-Normal prior grounded in real prior knowledge (literature, a datasheet, an earlier calibration) then narrows the reported posterior wherever the two pulses alone cannot constrain the parameter, instead of letting it spread back out across the whole box.

Hyperpriors: any field of a `PriorSpec` (`lo`/`hi`, `mean`/`std`, the median/factor pair) can itself be either a plain number or a `(lo, hi)` pair – a *hyperprior*: instead of fixing, say, a Gaussian prior's mean, you give a plausible range for it and the same Bayesian search locates the best value within that range, jointly with everything else. Each such field adds one search dimension (`_build_layout`, line 433): a *spread* field (`std`, or the Log-Normal factor) is always explored log-uniformly, a *location* field (`lo`/`hi`/`mean`/the median) follows the same convention as the parameter itself (log-uniform for a modulus/viscosity, linear for ν) – this is a real cost: a Gaussian prior with a hyperprior can cost up to 3 search dimensions (mean, std, value) instead of 1.

`_assemble` (line 476) expands a (reduced-dimension) unit-hypercube point or batch into physical values: hyperprior fields are resolved first (each its own column, via `_sample_scale`, line 401), then the parameter's own value is drawn from the resulting distribution – the inverse-CDF trick (`scipy.stats.norm.ppf`) is what lets Latin Hypercube Sampling's stratification carry over into a stratified sample of those distributions, not just of a flat box:

```
def _sample_scale(lo, hi, u, log_scale: bool):
    if log_scale:
        return np.exp(np.log(lo) + u * (np.log(hi) - np.log(lo)))
    return lo + u * (hi - lo)
```

`eval_point(u)` (a closure inside `calibrate_bayesian`, lines 794-807) converts u into real parameters via `_assemble`, calls `predict_pulse(pulse1, time1, nu, model, params, density, radius, z1, damp)` (§8.3.2-8.3.3) and then scores the result with `pulse_correlation(predicted, pulse2, time1, time2,`

`metric)` (§8.3.4). Any exception is caught and the point is scored 0.0 rather than excluded ("scored (not excluded) so the GP still learns ävoid this region"), so the surrogate also learns the shape of unevaluable regions – this is the path that receives, in particular, a `RootNotFoundError` caused by a floating-point overflow in `core/pochhammer_solver.py` (§8.3.2) whenever the fallback itself also fails. The function keeps track (`nonlocal`) of the best predicted pulse seen so far (`best_pulse_seen`) and its score (`best_corr_seen`), to avoid one more expensive call at the end of the search (see §8.4.6).

8.4.3 Initial sampling (Latin Hypercube Sampling)

`latin_hypercube(n, dim, rng)` (`core/gaussian_process.py`, line 169) generates `n_init` initial points in $[0,1]^{\text{dim}}$: each dimension is split into n equal intervals, each hit exactly once with a random jitter inside it, and the intervals are independently shuffled per dimension:

```
for d in range(dim):
    cut = (np.arange(n) + rng.uniform(size=n)) / n
    rng.shuffle(cut)
    samples[:, d] = cut
```

This spreads the first, most informative evaluations evenly across the whole search box, without needing the $n_{\text{per_dim}}^{n_{\text{dims}}}$ points a grid would need, and without the large gaps that purely uniform-random sampling leaves for small n.

These `n_init` points can be evaluated in parallel if `n_workers > 1` (via `core/parallel.run_map` and the `_eval_point_task` worker, line 545): they are independent of one another, unlike every point in the adaptive loop that follows. An optional `cancel_check` (`calibrate_bayesian`'s docstring, lines 735-742) allows cooperative cancellation – checked once per LHS point when `n_workers == 1`, and once per adaptive-search iteration after that; an in-flight parallel LHS batch (`n_workers > 1`) can only be interrupted once the whole batch has completed.

8.4.4 Adaptive loop: Gaussian Process surrogate + acquisition strategy

For each iteration (up to `n_iter`, line 851):

1. A new `GaussianProcess` (§8.5) is fit on every (U, y) point evaluated so far (`gp.fit(U, y)`).
2. `_propose_next(gp, best_so_far, dim, rng, acquisition=...)` (line 613) searches for the point in $[0,1]^{\text{dim}}$ that maximizes the chosen acquisition strategy.
3. The proposed point u_{next} is evaluated with the true, expensive forward model (`eval_point`), and the score is appended to (U, y).
4. Convergence/patience criterion: the `no_improve` counter is reset to 0 if `new_best - best_so_far > convergence_tol` (default 10^{-4}), otherwise incremented; the search stops as soon as `no_improve >= convergence_patience` (default 10 iterations without improvement), or after `n_iter` iterations (default 40) if patience is never reached.

Acquisition strategies (`acquisition`, one of four constants `ACQ_EI` (default), `ACQ_PI`, `ACQ_UCB`, `ACQ_THOMPSON`) – new compared to the previous version of this module, which only offered Expected Improvement:

- `ACQ_EI` – **Expected Improvement** (`_expected_improvement`, line 571), the default strategy:

$$\text{EI}(x) = (\mu(x) - f_{\text{best}} - \xi)\,\Phi(z) + \sigma(x)\,\varphi(z), \qquad z = \frac{\mu(x) - f_{\text{best}} - \xi}{\sigma(x)} \tag{8.17}$$

  where $\mu(x)$, $\sigma(x)$ are the GP's predicted mean and standard deviation, $f_{\text{best}} = \max(y)$ the best score observed so far, Φ/φ the standard normal CDF/pdf, and $\xi = 0.01$ a small

exploration margin ("without it, EI can get stuck refining right around the current best instead of also checking still-uncertain regions"). $\sigma(x)$ is floored at 10^{-9}. It is the balanced, general-purpose default, weighing "how much better" together with "how likely."

- `ACQ_PI` – **Probability of Improvement** (`_probability_of_improvement`, line 586): $\mathrm{PI}(x) = \Phi\left(\frac{\mu(x)-f_{\mathrm{best}}-\xi}{\sigma(x)}\right)$ – only the *probability* of any improvement, however small. Greedier/ more exploitative than EI (it stops caring once a region looks unlikely to beat the current best, even if a genuine improvement remains plausible there) – tends to converge in fewer evaluations once the search is already fairly well localized (e.g. by informative priors), but is more prone to settling on a local optimum when the search box is wide or the model is weakly identified.
- `ACQ_UCB` – **Upper Confidence Bound** (`_upper_confidence_bound`, line 597): $\mathrm{UCB}(x) = \mu(x)+\kappa\,\sigma(x)$ with $\kappa = 2$ (`_UCB_KAPPA`) – explores purely by how uncertain the surrogate still is, without reference to the current best at all. Systematically visits under-explored regions more than EI/PI do, which suits a wide search box or a model prone to multiple competing optima (e.g. Generalized Maxwell/Burgers), at the cost of a few more evaluations before settling on the best one.
- `ACQ_THOMPSON` – **Thompson Sampling** (`_propose_next_thompson`, line 660): draws one random continuous function from the surrogate's *joint* posterior (`GaussianProcess.sample_y`, not just its pointwise mean/std, §8.5.3) and proposes that sample's own optimum. A stochastic middle ground between EI and UCB – since each iteration's proposal comes from a different random draw, it naturally avoids repeatedly proposing near-duplicate points, which is useful when a search seems stuck.

The first three strategies share the same maximization mechanics (`_propose_next`, lines 613-657): a broad, cheap random scan (`n_random = max(2000, 400*dim)` candidates, since this only calls the GP, never the expensive forward model), followed by local polishing via `scipy.optimize.minimize` (L-BFGS-B, bounds $[0,1]^{\mathrm{dim}}$) from the `n_restarts = max(8, dim+6)` best candidates. `ACQ_THOMPSON` is handled separately: having no smooth acquisition surface to hand to a gradient optimizer (a single random draw evaluated on a discrete candidate set), it draws `n_candidates = clip( 250*dim, 500, 1500)` candidates and takes the sample's argmax – a smaller pool than the other three strategies' random scan, since sampling a joint posterior needs an $O(n^3_{\mathrm{candidates}})$ Cholesky factorization.

`calibrate_bayesian`'s docstring (lines 725-733) explicitly states that only the initial LHS phase is parallelizable: the adaptive loop is not, because proposing several points from the same, not-yet-updated GP would change which points the search actually visits, not merely how fast it runs.

8.4.5 Posterior mean, standard deviation and parameter correlation

Once the search is finished, one last GP is fit on every evaluated point (lines 901-902). The posterior is then estimated by cheap, dense sampling of the surrogate's predicted mean (never of the expensive forward model), lines 903-938:

1. `n_posterior = 20000` points U_{post} drawn uniformly in $[0,1]^{\mathrm{dim}}$; `mean_post, _ = gp.predict(U_ post)`.
2. Weighting by a softmax of the predicted mean, with an adaptive "temperature":

$$T = \max(\mathrm{std}(\mu_{\mathrm{post}}), 10^{-6}) \times 0.3, \qquad w_i = \frac{\exp((\mu_{\mathrm{post},i} - \max_j \mu_{\mathrm{post},j})/T)}{\sum_k \exp((\mu_{\mathrm{post},k} - \max_j \mu_{\mathrm{post},j})/T)} \tag{8.18}$$

3. The points are converted back to physical units via `_assemble` – which, since priors were introduced (§8.4.2), is no longer a plain uniform draw: for a `"gaussian"` or `"lognormal"`

prior, the dense sample is drawn from *that* distribution (by inverse CDF), not uniformly across the box, which makes the reported posterior a self-normalized-importance-sampling estimate of posterior $\propto$ prior $\times$ likelihood, rather than just the likelihood restricted to a box. Weighted mean/std per parameter:

$$\bar{x}_k = \sum_i w_i x_{i,k}, \qquad \sigma_k = \sqrt{\sum_i w_i (x_{i,k} - \bar{x}_k)^2} \tag{8.19}$$

4. Parameter correlation matrix (`param_correlation`, lines 927-938) – the weighted Pearson correlation between every pair of parameters over that same posterior sample:

$$\mathrm{Cov}_{jk} = \sum_i w_i (x_{i,j} - \bar{x}_j)(x_{i,k} - \bar{x}_k), \qquad \rho_{jk} = \mathrm{clip}\left(\frac{\mathrm{Cov}_{jk}}{\sigma_j \sigma_k}, -1, 1\right) \tag{8.20}$$

(diagonal forced to 1). Restricted to physical parameters: hyperparameters (below) are excluded from it.

> The module docstring (lines 68-78) spells out what this diagnostic means: a value near ± 1 between two parameters means the measurement (the two pulses) mostly constrains some combination of them (e.g. their product) rather than either one individually, and no amount of extra search budget fixes that – only a different or additional measurement can. "It says nothing about the parameters being physically coupled, only that this dataset cannot tell them apart."

The overall mean/std score (`score_mean`, `score_std`) is the weighted mean/std of `mean_post` itself (lines 945-946).

Hyperparameters (§8.4.2, "HYPERPRIORS"): whenever at least one prior field was given as a `(lo, hi)` range, this same posterior sample also yields a weighted mean/std for every active hyperparameter (`ViscoBayesianResult.hyper_names`, each named `"<param>__<field>"`, e.g. `"e1__mean"`), lines 916-925 – but, as noted above, they stay excluded from `param_correlation`, which remains scoped to the physical parameters this module's diagnostics are actually about.

The final result also exposes the raw trace of every point actually evaluated (`evaluated_params`, `evaluated_scores` – distinct from the dense synthetic posterior sample) and the history of the best score per evaluation (`history_best`, for the convergence plot).

8.4.6 Predicted pulse returned: reused, not recomputed

`predicted_pulse` (used for the comparison plot) is not recomputed at the posterior mean – the code's own comment (lines 948-967) explains that this mean point (a weighted average in linear space even for log-scaled parameters) can land in a region where the Pochhammer-Chree root search converges very slowly, particularly for Generalized Maxwell/Burgers, which would give the appearance of the interface hanging (the progress bar already at 100% while an untracked call is still running, since such a call would not be wired to `progress_cb`).

The predicted pulse of the best point actually evaluated during the search (`best_pulse_seen`, cf. §8.4.2) is reused instead (line 968); it is free (already computed), guaranteed to have already succeeded once, and needs nothing beyond the already-cheap GP/array math once the loop has reached 100%. A recompute (lines 969-973) only happens if absolutely every evaluation failed (`best_pulse_seen is None`).

8.5 The surrogate Gaussian Process (core/gaussian_process.py)

A "from scratch" implementation, deliberately light on dependencies (numpy + scipy only), documented as "both sufficient and easy to audit/traceable" for a problem with a few dozen points in at most a few dimensions (docstring, lines 1–11).

8.5.1 Covariance kernel

Anisotropic squared-exponential kernel (ARD — Automatic Relevance Determination), one correlation length ℓ_d per dimension:

$$k(x, x') = \sigma_f^2 \exp\left(-\frac{1}{2}\sum_d \left(\frac{x_d - x'_d}{\ell_d}\right)^2\right) \tag{8.21}$$

(`_kernel`, line 56–61). The training covariance matrix adds an i.i.d. noise term and a numerical stability "jitter":

$$K = k(X, X) + (\sigma_n^2 + 10^{-10})\, I \tag{8.22}$$

8.5.2 Hyperparameter fitting (marginal likelihood maximization)

The target y is standardized before fitting: $y_c = (y - \bar{y})/\mathrm{std}(y)$. The hyperparameters $\theta = (\ln \ell_1, \ldots, \ln \ell_d, \ln \sigma_f, \ln \sigma_n)$ are found by minimizing the negative log marginal likelihood:

$$\mathrm{NLL}(\theta) = \frac{1}{2} y_c^\top K^{-1} y_c + \frac{1}{2}\ln|K| + \frac{n}{2}\ln(2\pi) \tag{8.23}$$

(`_neg_log_marginal_likelihood`, line 63–76), solved via a Cholesky factorization (`scipy.linalg.cho_factor`/`cho_solve`) for stability and cost. The optimization uses `scipy.optimize.minimize` (L-BFGS-B) with `n_restarts` (default 6) random initializations within fixed bounds: $\ln \ell_d \in [-3, 2]$, $\ln \sigma_f \in [-2, 2]$, $\ln \sigma_n \in [-4, 0]$ (commented at line 86–92: the noise can go up to the full standard deviation of the standardized signal, because the per-frequency `sympy.nsolve` solver can vary slightly between neighboring parameter sets, and a bit of noise margin keeps the GP from overfitting this "wobble").

8.5.3 Prediction (posterior mean/variance)

For new points X_* (`predict`, line 117–126), with $K_s = k(X_*, X)$ and $\alpha = K^{-1} y_c$ (already computed at fitting time via Cholesky):

$$\mu_*(X_*) = K_s \alpha \cdot \sigma_y + \bar{y} \tag{8.24}$$

$$\sigma_*^2(X_*) = \max\Big(\sigma_f^2 - \mathrm{diag}(K_s K^{-1} K_s^\top),\ 10^{-12}\Big) \cdot \sigma_y^2 \tag{8.25}$$

(the code exploits $k(x_*, x_*) = \sigma_f^2$ for a stationary kernel, line 124). These are the classic Gaussian Process regression formulas, with a final de-standardization back to the score's original scale. `latin_hypercube` (§8.4.3) is also defined in this same module.

8.6 Sub-sample-accurate pulse auto-alignment (core/pulse_alignment.py)

8.6.1 Why

The docstring (lines 1–39) explains the problem: manually dragging the two pulse windows is imprecise exactly where it matters most — the starting point of the pulse 2 window. An imprecision of a few samples on this point does not change the analysis of pulse 1 itself (its window only sets the FFT's frequency resolution) but directly corrupts any comparison with the actually measured pulse 2: the Bayesian search would then "compensate" for this arbitrary offset by biasing the fitted parameters, with no relation to any real material property.

8.6.2 Algorithm (align_pulse2, line 50)

The width of the pulse 2 window as dragged by the user is kept identical (a legitimate user choice); only its starting point is searched automatically:

1. A template signal is extracted from the pulse 1 window (absolute value of the raw signal, consistent with the construction of `pulse1`/`pulse2` in the page's `_on_run`). `ref = template[:m]` with $m = \min(\text{width}_1, \text{width}_2)$ in samples — when the two widths differ, the comparison is made on the common leading portion, consistent with how `core.visco_calibration._align_pulses` already reconciles differing lengths elsewhere.
2. A search margin around the starting point indicated by the user (`margin`, by default half the width of the pulse 1 window) defines a candidate index interval $[lo, hi]$.
3. For each candidate integer offset i_2 in this interval, the score is systematically the Pearson correlation (`SCORE_FUNCS["correlation"]`, i.e. `_pearson_score`, §8.3.4) between the candidate and the `ref` template — and not the criterion (R^2 or Pearson) chosen for the calibration itself. The docstring justifies this choice (lines 28–39): pulse 2 is expected to be attenuated relative to pulse 1 (this is precisely the dispersion/damping physics the calibration seeks to measure), so judging alignment by absolute amplitude matching (R^2) would wrongly penalize the correct offset simply because pulse 2 is physically smaller — exactly the opposite of the reason R^2 was preferred over correlation for the calibration objective itself. Here, the scale invariance of correlation is, on the contrary, the right tool: it ignores the expected attenuation and focuses solely on shape matching.
4. The best integer offset `best_i2` (maximum score) is then refined to sub-sample precision by a 3-point parabolic interpolation around the integer optimum:

$$\text{offset} = \frac{1}{2} \cdot \frac{y_0 - y_2}{y_0 - 2y_1 + y_2} \tag{8.26}$$

where y_0, y_1, y_2 are the scores at offsets best_i2 − 1, `best_i2`, best_i2 + 1. The refinement is applied only if $|\text{offset}| < 1$ ("a real peak refines by less than one whole sample") and if the best offset is not at the edge of the search interval (a 3-point parabola through an optimum at the edge would not make sense).

The new start is `new_start = time[best_i2] + offset*dt`, the end being shifted by the same amount to preserve the original width.

The function returns `None` if the neighborhood of the indicated point does not contain enough signal to perform the search (e.g. too close to the edge of the recording).

8.7 The "Free-Decay Damping" standalone tool (core/decay.py)

This tool (single function `compute_decay(ftime, fmeasure)`, line 31) estimates a bar's damping coefficient from a free-decay signal (e.g. an impact/hammer-strike recording), using the half-power (–3 dB) bandwidth method on the peak of the FFT spectrum. The docstring states that the logic is a port of `decay.py`, without its original Tkinter graphical interface.

8.7.1 Precise localization of the peak and the half-power points

1. Real FFT of the signal: `yi = rfft(fmeasure)`, `freq = rfftfreq(n, dt)`, `amplitude = |yi|`.
2. Cubic interpolation of the amplitude spectrum (`scipy.interpolate.interp1d(freq, amplitude, kind="cubic")`) on a fine grid `freq_fine` (5× more points than the native FFT grid), to locate the peak and the half-power points with a precision finer than the FFT's native resolution ("exactly as in the original implementation", line 49–50).
3. Peak: `peak_idx = argmax(amp_fine)` → natural frequency f_n = `freq_fine[peak_idx]`, max amplitude A_{max}.
4. Half-power threshold (−3 dB):

$$A_{\text{half-power}} = \sqrt{\frac{A_{max}}{2}} \tag{8.27}$$

5. Search for the two frequencies flanking the peak where the spectrum crosses this threshold: `lower_idx` = first index (scanning from the start of the array) where the amplitude exceeds the threshold; `upper_idx` = first index after `lower_idx` where the amplitude falls back below the threshold (or the last index of the array if never reached). Hence f_l = `freq_fine[lower_idx]`, f_u = `freq_fine[upper_idx]`.

Note: this search scans the entire spectrum from index 0, not just a neighborhood of the peak — it is therefore sensitive to a noisy or multi-peak spectrum (see the UI warning below), with no explicit guard in the code against a spurious threshold crossing before the main peak.

8.7.2 Damping factor

With the relative bandwidth ratio = $(f_u - f_l)/f_n$:

$$\zeta = \sqrt{0.5 - \sqrt{\frac{1}{4 + 4\,\text{ratio}^2 - \text{ratio}^4}}} \tag{8.28}$$

(`damping_ratio`, line 70 — this is the dimensionless damping ratio, "curve damping factor"). The full damping factor, expressed in the units used elsewhere in the application for the "Damping factor/C" parameter (`core/visco_calibration.py`'s `damp`, cf. §8.3.3), is:

$$C = \zeta \cdot 2\pi f_n \tag{8.29}$$

(`full_damping_factor`, line 78). Note on units: the comment on the `full_damping_factor` field in `DecayResult` (line 23) annotates it "rad/m, per original units convention", whereas dimensionally $\zeta \cdot 2\pi f_n$ (with f_n in Hz) is an angular frequency, i.e. in rad/s, not rad/m. The code documents no further conversion via a wave speed at this point; the `full_damping_factor` value computed here is reused as-is as the `damp` parameter (rad/m) in `core/visco_calibration.py` (§8.3.3) and in `core/dispersion.py`. This is flagged as an ambiguous point in the code, unresolved by the source

documentation, to be verified before any rigorous quantitative use — this is not an extrapolation on our part but an apparent dimensional inconsistency between the comment and the calculation as written.

The fitted and plotted exponential decay envelope (`exponential_envelope`) is:

$$s_{env}(t) = \max(|s|) \cdot \exp(-\zeta \cdot 2\pi f_n \cdot t) \tag{8.30}$$

8.7.3 Downstream use

`ui/pages/page_decay.py` automatically saves `full_damping_factor` into `project.viscoelastic.damping_factor` (line 131), reusable via the "Use value from Free-Decay Damping tool" button on the Dispersion Corrections page and on the viscoelastic calibration page (`ui/pages/page_visco_calib.py`, `_on_use_decay_damping`).

The page's contextual help (`page_decay.py`, lines 80–84) warns: "A narrow, clean peak gives a more reliable estimate than a noisy or split one — if the spectrum looks off, check that the recorded decay signal is long enough and free of extra vibration sources."

8.8 Summary of file/function cross-references

Topic	File	Function/method
4 rheological models, $E^*(f)$	`core/rheology_models.py`	`_kelvin_voigt_modulus`, `_sls_modulus`, `_generalized_maxwell_2_modulus`, `_burgers_modulus`
Modulus/speed/Lamé coefficients per frequency	`core/visco_calibration.py`	`_complex_modulus_and_speed`
Pulse FFT/Pochhammer-Chree/IFFT propagation	`core/visco_calibration.py`	`_predict_pulse` / `predict_pulse` (public wrapper)
Pearson / R^2 scores	`core/visco_calibration.py`	`_pearson_score`, `_r2_score`, `SCORE_FUNCS`
Time realignment before scoring	`core/visco_calibration.py`	`_align_pulses`
Fixed-parameter evaluation	`core/visco_calibration.py`	`test_parameters`
Full Bayesian search	`core/bayes_visco_calibration.py`	`calibrate_bayesian`
Unit hypercube → parameter conversion	`core/bayes_visco_calibration.py`	`_from_unit`
Expected Improvement	`core/bayes_visco_calibration.py`	`_expected_improvement`
Next-point proposal	`core/bayes_visco_calibration.py`	`_propose_next`
GP kernel, fit, prediction	`core/gaussian_process.py`	`GaussianProcess._kernel`, `.fit`, `.predict`
LHS sampling	`core/gaussian_process.py`	`latin_hypercube`
Pulse 2 auto-alignment	`core/pulse_alignment.py`	`align_pulse2`

Topic	File	Function/method
Free-decay analysis / half-power bandwidth	`core/decay.py`	`compute_decay`
Applied viscoelastic dispersion correction (SLS only)	`core/dispersion.py`	`viscoelastic_correction`, `viscoelastic_correction_direct_impact`
Bandwidth bound (per-branch reference speed)	`core/dispersion.py`	`_bandwidth_rolloff_weight` (mechanism detailed in the "elastic dispersion" chapter, §5.2)

8.9 Summary of limitations and warnings found in the code

- Only the Standard Linear Solid model is currently wired into the dispersion correction on the Dispersion Corrections page; Kelvin-Voigt, Generalized Maxwell (2 branches) and Burgers are available only for calibration (`core/rheology_models.py`, lines 30–54).
- The viscoelastic bar Monte-Carlo expansion (`core/models.py::Bar.expand_viscoelastic`) remains hardwired to the SLS module, independently of the four models described here (`core/rheology_models.py`, lines 56–61).
- The SLS modulus formula corresponds to the "spring in series with a Kelvin-Voigt cell" topology, as documented in the code (`core/rheology_models.py`, lines 127–138).
- The Burgers model has an exactly zero static modulus, which requires special handling of zero frequency (evaluation at `freq[1]`, only the real part kept) to avoid a complex-to-real conversion crash, documented as actually reproduced (`core/rheology_models.py`, lines 71–88, 158–184).
- In `_predict_pulse`, a second formulation of the Pochhammer-Chree equation is used as a fallback when the first fails to converge; in that case the attenuation is not recomputed but carried over from the previous bin, with no detailed physical justification in the code (`core/visco_calibration.py`, lines 133–146).
- `core/visco_calibration.py` carries an explicit non-validation warning: the module has not yet been independently re-validated against real experimental data or the original application's output (docstring, lines 43–47).
- The Bayesian search's adaptive loop is deliberately not parallelized (unlike the initial LHS phase), because doing so would change which points are actually visited (`core/bayes_visco_calibration.py`, lines 218–226).
- The posterior parameter correlation matrix indicates a non-identifiability of the measurement, not a real physical coupling between parameters, and no additional search budget can resolve it (`core/bayes_visco_calibration.py`, lines 51–61).
- The final predicted pulse is not recomputed at the posterior mean (risk of very slow root-solver convergence in that region), but reused from the best point actually evaluated during the search (`core/bayes_visco_calibration.py`, lines 380–401).
- In `core/pulse_alignment.py`, the alignment criterion is always Pearson correlation, independently of the criterion chosen for the calibration itself (R^2 by default), by a deliberate, documented choice (lines 28–39).
- In `core/decay.py`, the annotated unit for `full_damping_factor` ("rad/m") does not dimensionally match the formula $\zeta \cdot 2\pi f_n$ (which yields rad/s for f_n in Hz) — an inconsistency noted here with no further conversion visible in the surrounding code; a point to verify with the author/user before quantitative use.
- In `core/decay.py`, the search for the half-power points scans the entire spectrum from

index 0 (not just a neighborhood of the peak), with no guard against a noisy/multi-peak spectrum; the page's contextual help warns that a narrow, clean peak is needed for a reliable estimate.

- The bandwidth bound (`_bandwidth_rolloff_weight`, §8.1) uses, for each viscoelastic branch, its own zero-frequency speed (`abs(c_i[0])`/`abs(c_r[0])`/`abs(c_t[0])`) as the normalization reference, for lack of a single wave speed in a viscoelastic medium; the default value (0.6) remains, as in the elastic case, a pragmatic heuristic not validated by a dedicated parametric study.

9. Standalone tools (SGBC, camera/ video sync, batch analysis, Bayesian calibration)

This chapter documents the tools that are independent of the main analysis pipeline, accessible separately in the application: the strain-gage bridge coefficient calculator (SGBC), camera/ video synchronization, batch analysis, and Bayesian calibration of constitutive laws (MAMBA port, §7). It also briefly covers the downsampling utility shared by the whole application, the video/image-sequence frame-reading abstraction, and the annotated DIC video export. The free-decay damping tool (`core/decay.py`) is covered in the chapter on viscoelastic dispersion and is not repeated here.

9.1 Strain-gage bridge coefficient calculator (SGBC)

Source file: `core/cpg_calc.py` — UI page: `ui/pages/page_cpg_calc.py`

9.1.1 Context and role

The CPG coefficient ("Coefficient Pont de Gauge", i.e. strain-gage bridge coefficient) converts the raw voltage measured at the output of a strain-gage bridge (after amplifier gain) into strain:

$$\varepsilon = CPG \times V_{bridge} \tag{9.1}$$

CPG is the value stored by the Monte-Carlo parameter `SG_coeff` (alias `CPG_pa`) of a bar (`Bar`, see `core/models.py`). The module `core/cpg_calc.py` computes this coefficient — and its uncertainty — from the physical quantities of the bridge setup, for the three usual Wheatstone bridge configurations. The code is explicitly documented as as literal a port as possible of the original application (`test_type.py`, functions `CPG_comp`/`CPG_comp_value`), not independently revalidated against the original application's output (module docstring, lines 18-20).

Two optional corrections, specific to the gage itself rather than to the bridge setup, can be enabled in addition to the base calculation: the transverse-sensitivity correction (§1.7) and the thermal-output correction (§1.8).

Three bridge types are supported (`BridgeType`, `core/cpg_calc.py` lines 29-39):

- **Quarter-bridge**: 1 active gage, 3 fixed completion resistors.
- **Half-bridge**: 2 active gages (axial + lateral), exploiting the bar's Poisson ratio for a lateral-strain compensation term.
- **Full-bridge**: 4 active gages (2 axial + 2 lateral).

Only Half- and Full-bridge use the bar's Poisson ratio (`USES_POISSON_RATIO`, lines 43-47) — Quarter-bridge has no lateral gage to compensate.

9.1.2 Physical inputs

All inputs are Monte-Carlo parameters (mean + standard deviation), read by `compute_cpg()` (`core/cpg_calc.py` lines 58-99):

Symbol	Meaning	Unit
GF	Gage Factor	dimensionless
V_{ex}	Bridge excitation voltage (Input Tension)	V
$Gain$	Amplifier gain	dimensionless
T	Operating temperature of the gage	K
$SG_{T,\%}$	Gage-factor temperature coefficient, entered in %/K	%/K
ν_b	Poisson ratio of the bar (Half/Full only)	dimensionless

A `ValueError` exception is raised if GF, V_{ex}, $Gain$ or T is zero (line 75-76), and for Half/Full if ν_b is zero (line 88-89, since ν_b appears in the denominator).

The reference temperature at which the gage factor is specified is hard-coded:

$$T_{ref} = 293 \text{ K} \quad (\texttt{_REFERENCE_TEMPERATURE}, \text{line } 49) \tag{9.2}$$

9.1.3 Common thermal term

The gage-factor temperature coefficient is first converted from %/K to a fraction (line 78):

$$SG_T = \frac{SG_{T,\%}}{100} \tag{9.3}$$

A "thermal" term common to the three bridge types is then computed (`compute_cpg`, line 79):

$$V_{th} = GF \cdot [1 + SG_T \cdot (T - T_{ref})] \cdot V_{ex} \tag{9.4}$$

This term represents the effective bridge voltage, corrected for the gage factor's thermal drift.

9.1.4 CPG coefficient formula by bridge type

Quarter-bridge (`core/cpg_calc.py`, lines 81-82):

$$CPG_{quarter} = \frac{4}{V_{th} \cdot Gain} \tag{9.5}$$

Half-bridge (factor = 4, lines 90-91):

$$CPG_{half} = \frac{4}{V_{th}(1 + \nu_b)\, Gain} \tag{9.6}$$

Full-bridge (factor = 2, lines 90-91):

$$CPG_{full} = \frac{2}{V_{th}(1 + \nu_b)\, Gain} \tag{9.7}$$

These three formulas have units of inverse volts (1/V), displayed as such in the interface (`page_cpg_calc.py` line 416: `"Coefficient = {value:.6g} ± {std:.6g} (1/V)"`).

9.1.5 Uncertainty propagation

The standard deviation of the CPG coefficient is obtained by quadrature error propagation over the relative standard deviations of each input (implicit assumption of statistical independence of the inputs and of first-order linearization — not explicitly documented in the code, but standard for this kind of calculation).

Quarter-bridge (lines 83-86):

$$\left(\frac{\sigma_{CPG}}{CPG}\right)^2 = \left(\frac{\sigma_{GF}}{GF}\right)^2 + \left(\frac{\sigma_{Vex}}{V_{ex}}\right)^2 + \left(\frac{\sigma_{Gain}}{Gain}\right)^2 + \left(\frac{\sigma_T}{T}\right)^2 \tag{9.8}$$

Half-/Full-bridge (lines 92-96), with an additional term in ν_b:

$$\left(\frac{\sigma_{CPG}}{CPG}\right)^2 = \left(\frac{\sigma_{GF}}{GF}\right)^2 + \left(\frac{\sigma_{Vex}}{V_{ex}}\right)^2 + \left(\frac{\sigma_{Gain}}{Gain}\right)^2 + \left(\frac{\sigma_T}{T}\right)^2 + \left(\frac{\sigma_{\nu_b}}{\nu_b}\right)^2 \tag{9.9}$$

In both cases (line 98):

$$\sigma_{CPG} = CPG \cdot \sqrt{\sum \left(\frac{\sigma_{x_i}}{x_i}\right)^2} \tag{9.10}$$

Point worth noting (potential code ambiguity): the temperature term directly uses σ_T/T (the relative standard deviation of the temperature itself), whereas the dependence of CPG on T goes through the factor $[1 + SG_T(T - T_{ref})]$ and is therefore not strictly proportional to T (except if $SG_T = 0$). The formula applied is a simplified "multiplicative-factor" quadrature propagation, consistent with a literal port of the original code, but which does not correspond to a rigorous error propagation through the term $[1 + SG_T(T - T_{ref})]$ (which would give a factor $\dfrac{SG_T\,\sigma_T}{1 + SG_T(T - T_{ref})}$ rather than σ_T/T). This is not documented as an intentional approximation in the code; it is most likely a direct legacy of the original application's formula. Likewise, the uncertainty on $SG_{T,\%}$ itself is not taken into account (no `sg_t_std` input): the gage factor's temperature coefficient is treated as exact.

9.1.6 Interface / synchronization

The page `ui/pages/page_cpg_calc.py` lets the user select the bridge type (showing/hiding the "Bar Poisson Ratio" field according to `USES_POISSON_RATIO`), enter the inputs above (mean + standard deviation), compute via `compute_cpg()`, then — via the "Sync to bar's Strain Gage Bridge Coefficient" button — write the result directly (mean, dispersion, NORMAL distribution) into the `SG_coeff` parameter of the project's input or output bar (`_on_sync`, lines 472-488).

9.1.7 Transverse-sensitivity correction (optional)

Function: `transverse_sensitivity_correction()` (`core/cpg_calc.py`)

A strain gage responds, to a lesser degree, to strain perpendicular to its measurement axis — its transverse sensitivity K_t, given on the manufacturer's data sheet (typically around 0.5-1 %, the "TRANSVERSE SENSITIVITY" value on the package). This sensitivity biases the indicated strain relative to the true axial strain.

For a single axial gage under a uniaxial stress field — the usual case for this application (gage bonded along the axis of a bar) — the correction relies on two relationships from Vishay technical note TN-509 ("Errors Due to Transverse Sensitivity in Strain Gages"):

1. The gage factor announced by the manufacturer F is itself calibrated on a uniaxial tension specimen, where the transverse strain equals $-\nu_0\varepsilon_a$ (ν_0 the calibration Poisson ratio, 0.285 for Micro-Measurements/Vishay gages): $F = F_a(1 - \nu_0 K_t)$, with F_a the "true" axial gage factor of the gage.
2. On the measured bar, under uniaxial stress with a real Poisson ratio ν (the "Bar Poisson Ratio" field already present on this page, reused as-is), the real transverse strain equals $-\nu\varepsilon_a$.

Combining the two, the real axial strain is derived from the indicated strain by:

$$\varepsilon_a = \varepsilon_{ind} \cdot \frac{1 - \nu_0 K_t}{1 - \nu K_t} \tag{9.11}$$

Since $\varepsilon = CPG \times V_{bridge}$, applying this same multiplicative factor directly to the CPG coefficient produces a corrected strain for any bridge voltage measured afterwards:

$$CPG_{corr} = CPG \cdot \frac{1 - \nu_0 K_t}{1 - \nu K_t} \tag{9.12}$$

The factor is applied identically regardless of the selected bridge type, as a first-order approximation (consistent with the rest of this module, documented but not independently revalidated). The "Bar Poisson Ratio" checkbox, normally hidden for Quarter-bridge (§1.1), becomes visible again as soon as this correction is enabled, since it reuses this same field for ν.

The uncertainty is propagated by combining, in relative quadrature, the uncertainty already propagated on CPG (§1.5) with that of the correction factor itself — the latter obtained by analytical differentiation of the factor with respect to K_t and ν (the calibration coefficient ν_0 is treated as an exact constant, with no uncertainty of its own):

$$\frac{\partial(\text{factor})}{\partial K_t} = \frac{\nu - \nu_0}{(1 - \nu K_t)^2} \qquad \frac{\partial(\text{factor})}{\partial \nu} = \frac{(1 - \nu_0 K_t)\, K_t}{(1 - \nu K_t)^2} \tag{9.13}$$

9.1.8 Thermal-output correction (optional, informative)

Function: `thermal_output_strain()` (`core/cpg_calc.py`)

Distinct from the gage factor's temperature coefficient (§1.2, which corrects the gage's SENSITIVITY), the "thermal output" is the apparent strain a gage reports purely due to a change in temperature, with no mechanical load at all — caused by the difference in thermal expansion between the gage grid and the specimen material. The manufacturer provides, for each gage/material combination (the gage being paired with a given material, e.g. 1018 steel), a temperature polynomial whose coefficients are printed on the package, typically of 3rd to 7th order.

The "Thermal Output Correction" card lets the user enter the polynomial degree (drop-down menu, 3 to 7), the temperature unit (°C or °F, to match the coefficient set used — the manu-

facturer usually provides both), the coefficients $a_0 \dots a_n$ themselves (in microstrain per unit^i), as well as the reference temperature (the one at which the bridge was balanced/zeroed) and the operating temperature (that of the test).

Per Vishay technical note TN-504-1 ("Strain Gage Thermal Output and Gage Factor Variation with Temperature"), equation 3, the apparent strain to correct is the polynomial value at the operating temperature MINUS its value at the reference temperature — not just its value at the operating temperature alone, since any thermal output already present at the moment of zeroing is already cancelled by that zeroing itself:

$$\varepsilon_{T/O} = P(T_{op}) - P(T_{ref}) \qquad P(T) = \sum_{i=0}^{n} a_i T^i \tag{9.14}$$

Unlike the transverse-sensitivity correction (§1.7), this correction acts on the strain itself (an additive offset), not on the CPG coefficient (a multiplicative factor): it is therefore not folded into the coefficient synchronized to `SG_coeff`. The result is displayed for information only (in microstrain, with its equivalent offset voltage $\varepsilon_{T/O} \times 10^{-6}/CPG$), to be applied manually by the user — typically relevant when the bridge was zeroed at a temperature different from that of the test itself.

This static correction assumes a fixed operating temperature throughout the test. For a case where the temperature genuinely varies during the test (adiabatic heating of a specimen during a fast dynamic test, or ambient-temperature drift during a slow quasi-static test), the Custom Sensors page offers a time-varying version of this same correction, applied directly to the sensor signal rather than merely reported — see chapter 1 §4.4.

9.2 Camera / video synchronization

Source file: `core/camera_clock.py` — UI page: `ui/pages/page_video_sync.py` — Video abstraction: `core/frame_source.py` (see §6)

9.2.1 Context

This tool locates, in a high-speed video or image sequence, the exact frame exposed at the instant the acquisition system triggers, and associates it with an absolute time on the same time base as the rest of the test's recorded signals. It relies on a "clock file" recorded in parallel by the camera, containing:

- an absolute time vector (same base as the test's other signals);
- the camera's exposure ("clock") signal: one pulse per frame;
- optionally, a trigger signal: an edge marking the instant the external trigger was received.

9.2.2 Reading the clock file

`read_clock_file()` (`core/camera_clock.py`, lines 36-88) reads a text file delimited by space/comma/semicolon/tab, automatically detecting:

- the delimiter (the one that splits the first 10 non-empty/non-commented lines into a constant, maximal number of fields — a delimiter that accidentally coincides with a decimal separator, e.g. a French comma, is thus discarded in favor of the delimiter that gives the most consistent columns, lines 57-65);

- a possible header line (if the first line is not entirely numeric);
- the French decimal-comma format (replacing , → . before conversion to float, line 101).

Non-numeric lines (other than the header) are simply ignored rather than raising an error. `decimate_for_display()` (lines 107-142) reduces the number of points displayed (20000 by default) for plotting, keeping the min and max of each bin rather than uniform subsampling — this is so as not to miss narrow pulses on a flat background when displaying. This reduction only affects the plot: peak and trigger detection always runs on the full-resolution arrays (docstring, lines 118-120).

9.2.3 Detecting the exposure pulse train

Function: `detect_pulse_train()` (`core/camera_clock.py`, lines 151-174).

The camera's exposure signal is a periodic pulse (one per frame). Detection uses `scipy.signal.find_peaks` with two criteria:

Minimum distance between accepted peaks — computed as a number of samples from a median sampling interval dt (`_estimate_dt`, lines 145-148, median of the positive time deltas):

$$d_s = \begin{cases} 0.5 \times T_{expected} & \text{if an expected period } T_{expected} \text{ is supplied} \\ 2 \times dt & \text{otherwise} \end{cases} \qquad d_{samples} = \max\left(1, \text{round}\left(\frac{d_s}{dt}\right)\right) \tag{9.15}$$

The expected period $T_{expected}$ (`expected_period_s`), when known (recording frequency entered, fps of the loaded video, or duration of the clock file divided by the number of loaded frames — see `_expected_period_s()` in `page_video_sync.py`, lines 502-525), sets the minimum gap between two accepted peaks to half this period. Without it, only a safety minimum gap ($2\,dt$) is imposed, which exposes noisy signals to over-detection (docstring, lines 158-161).

Minimum prominence — if not supplied, set to 30 % of the signal's peak-to-peak range (lines 169-171):

$$\text{prominence} = 0.3 \times (\max(signal) - \min(signal)) \tag{9.16}$$

The sample indices of the detected peaks and their corresponding times are returned.

9.2.4 Estimating the absolute time of each frame

Function: `estimate_frame_times()` (lines 177-207).

For each frame $0..n_{frames} - 1$ of the loaded video/sequence:

1. The $n_{known} = \min(n_{peaks_detected}, n_{frames})$ first frames directly receive the time of the corresponding detected exposure peak (association by rank, frame i <-> peak i).
2. Beyond that (frames with no matching detected pulse, generally at the end of the sequence), the time is extrapolated at a constant period:
   - the period used is $T_{expected}$ if supplied, otherwise the median of the gaps between the times of the detected peaks themselves:
   $$T_{period} = \text{median}\,(\text{diff}(peak_times[:\, n_{known}])) \tag{9.17}$$
   - for each frame $i \geq n_{known}$:
   $$t_i = t_{n_{known}-1} + (i - (n_{known} - 1)) \times T_{period} \tag{9.18}$$

3. If no period is available (no peak detected, and no period supplied), the times that cannot be determined remain NaN.

It is this function that feeds the "time ≈ …" marker shown under the current frame and the vertical line drawn on each displayed measurement channel.

9.2.5 Detecting a rising edge (single-step trigger signal)

Function: `detect_rising_edge()` (lines 210-221): the first sample where the signal moves from "at or below" to "above" a threshold (by default, the midpoint between the signal's min and max) — useful for a trigger signal that switches only once (a step), as opposed to a periodic pulse signal.

$$\text{threshold} = \frac{\min(signal) + \max(signal)}{2} \quad \text{(if not explicitly supplied)} \tag{9.19}$$

Returns the index of the first sample above the threshold (via `np.argmax` on the boolean mask), or `None` if the signal never exceeds the threshold.

9.2.6 Method 1 — Locating the trigger via a trigger signal

Function: `locate_trigger_by_signal()` (lines 253-280).

1. Detection of the exposure pulse train (§2.3) on the exposure signal.
2. Detection of the rising edge of the trigger signal (§2.5), giving `trigger_time`.
3. Association of the trigger's sample index (`trigger_idx`) to one of the detected exposure peaks' sample indices (`peak_indices`), via `_nearest_index()` (lines 224-236), according to the `match` parameter (three choices, exposed in the UI as "Match the trigger edge to"):
   - `"nearest"` (default): the exposure peak whose index is closest to `trigger_idx` (comparison of the two immediately-before/after candidates via binary search `np.searchsorted`, selecting the closer one in absolute value).
   - `"after"`: the first exposure peak whose index is $\geq$ `trigger_idx` (clamped to the last frame if the trigger occurs after the last detected peak).
   - `"before"`: the last exposure peak whose index is $\leq$ `trigger_idx` (clamped to the first frame if the trigger occurs before the first peak).

   Implementation note: the matching is done in sample-index space (`peak_indices`, `trigger_idx` — integers, not times), which implicitly assumes roughly regular sampling of the clock file so that proximity in index correctly reflects proximity in time; this is not explicitly checked by the code.

If no rising edge is found in the trigger signal, or if no exposure pulse is detected, a warning is added and `frame_index`/`frame_time` remain `None` (lines 268-276). A warning is also issued if the number of detected peaks differs from the number of actually loaded frames (lines 261-265), inviting the user to visually check the detection on the plot.

9.2.7 Method 2 — Locating the trigger via a known frame number + frequency

Function: `locate_trigger_by_frame_number()` (lines 283-335).

Used when the clock file has no trigger column, but the triggered frame number is directly known (reported by the camera itself), along with its recording frequency f (Hz).

- If `expected_period_s` is not supplied but f is, it is derived: $T_{expected} = 1/f$ — used as a hint for peak detection (§2.3).
- The targeted exposure pulse is `target = frame_number + frame_offset`, where `frame_offset` (default 0) compensates for any pulses recorded before the first frame actually loaded (e.g. an unsaved pre-trigger circular buffer): the pulse used for frame `frame_number` is thus the `target`-th detected pulse, not necessarily the `frame_number`-th.

Case 1 — the target pulse is within the detected range ($0 \leq target < n_{peaks}$):

$$t_{frame} = peak_times[target] \qquad \text{(method: “exposure pulse count”)} \tag{9.20}$$

Case 2 — the target pulse falls outside the detected pulses, and f is known — extrapolation at a constant frequency:

$$t_{base} = \begin{cases} peak_times[0] - \dfrac{frame_offset}{f} & \text{if at least one peak was detected} \\ time[0] & \text{otherwise} \end{cases} \tag{9.21}$$

$$t_{frame} = t_{base} + \frac{target}{f} \quad \text{(method: “frequency extrapolation”)} \tag{9.22}$$

A warning is always added in this case to flag that the time was extrapolated rather than read directly off the time vector (lines 319-323).

Case 3 — out of range and frequency not supplied: no time can be computed; a warning invites the user to enter the camera frequency (lines 326-329), and `frame_time` remains `None`.

9.2.8 Returned result

`TriggerResult` (lines 239-250) gathers: `frame_index`, `frame_time`, `trigger_time` (raw edge of the trigger signal, when applicable), `n_peaks_detected`, full `peak_times`/ `peak_sample_indices`, the method used (a free-form string: `"trigger signal"`, `"exposure pulse count"` or `"frequency extrapolation"`), and a list of warnings — displayed as-is in the interface so the user can judge the reliability of the result.

9.2.9 Limitations noted in the code

- The trigger<->peak matching (signal method) is done in index space, not real time — an implicit assumption of sampling regularity (not verified by assertion).
- Without an expected period, peak detection can over-detect on a noisy signal (docstring of `detect_pulse_train`).
- `estimate_frame_times` returns NaN for any frame whose time can neither be read nor extrapolated.
- A mismatch between the number of detected peaks and the number of loaded frames never blocks the calculation, only triggers a warning (the user must check the plot).

9.3 Batch analysis

Source file: `core/batch_analysis.py` — UI page: `ui/pages/page_batch_analysis.py`

9.3.1 Context

Combines several already-analyzed tests (each exported as CSV/TXT from the "Results & Export" page, any computed quantity being a column named after its internal name) into a single statistical summary over a common X axis. A literal port of `analysis_resume.py` (`analysis_tests()`) — not independently revalidated against the original application (docstring, lines 35-37). X and Y are not fixed to strain/stress: any exported column can be chosen for either (e.g. `confinement_pressure` on X, `confinement_deviator` on Y).

Three variability bands are computed around the mean curve:

- "Experimental": RMS of the Monte-Carlo uncertainty specific to each test (column `<Y>_std`, if present in the exported file — otherwise zero).
- "Material": test-to-test scatter about the mean, fitted as a normal law on the ratio (this test's value / the mean value) at each X level — unless overridden by a fixed percentage imposed by the user.
- "Total": quadratic combination of the two previous ones.

9.3.2 Loading files

`load_export_header()` (lines 88-109) reads only the header line of an exported file (delimiter inferred from the extension: `,` for `.csv`, tab for `.txt` — `_sniff_delimiter`, lines 80-85), to build the UI's X/Y drop-down lists. `_load_export_columns()` (lines 112-136) loads the requested columns (x, y, and optional y_std) from a file. A `<y_column>_std` column, if present in the header, is automatically used as the "Experimental" uncertainty of that test on Y — no separate selector is needed (`run_batch_analysis_from_files`, lines 160-164).

9.3.3 Isothermal correction (adiabatic heating)

Parameters: `IsothermalSettings` (lines 50-55) — `xi` (Taylor-Quinney coefficient), `cp` (specific heat, $\mathrm{J/(kg \cdot K)}$), `rho` (density, $\mathrm{kg/m^3}$), `alpha` (temperature sensitivity, MPa/K). Applied, when enabled, to the raw Y of each test individually, before any resampling (`_combine_tests`, lines 240-248) — only relevant when Y is a stress and X a strain (module docstring, line 19).

For a test of length n (arrays x, y), the code computes (lines 243-247):

$$k = \frac{\xi}{c_p \rho} \tag{9.23}$$

Cumulative temperature rise ΔT, by left-rectangle step integration — $\Delta T_0 = 0$ then, for $g = 0..n-2$:

$$\Delta T_{g+1} = \Delta T_g + k \cdot y_g \cdot (x_{g+1} - x_g) \cdot 10^6 \tag{9.24}$$

(the factor 10^6 suggests a conversion of y, expressed in MPa, to Pa, to obtain an energy density in $\mathrm{J/m^3}$ consistent with $c_p\rho$ in $\mathrm{J/(kg \cdot K) \cdot kg/m^3}$ — not explicitly commented in the code).

Isothermal correction applied to Y:

$$y_g^{corr} = y_g + \alpha \cdot \Delta T_g \qquad \text{for } g = 0..n-2 \tag{9.25}$$

and for the test's last point (index $n-1$):

$$y_{n-1}^{corr} = y_{n-2} + \alpha \cdot \Delta T_{n-1} \tag{9.26}$$

Point worth flagging explicitly: the corrected value of the last point reuses y_{n-2} (the raw, uncorrected second-to-last value) instead of y_{n-1} (the raw last value). This follows from line 247 as written (`[... for g in range(len(y)-1)] + [y[-2] + iso.alpha*dT[-1]]`) and most likely originates from a literal port of a quirk (a bug, or an undocumented deliberate choice) in the original code; it is not explained in the comments and has not been reinterpreted here. This should be checked with the original author if clarification is needed for the final documentation.

9.3.4 Resampling onto a common grid

Decided test by test according to the monotonicity of X (`_backtrack_fraction`, lines 178-190): the fraction of X's range over which X drops back below its own running maximum.

$$f_{backtrack} = \frac{\max_i \left(\text{running_max}(x)_i - x_i\right)}{\max(x) - \min(x)} \tag{9.27}$$

If $f_{backtrack} \leq 0.15$ (`_BACKTRACK_TOLERANCE`, line 197) for all tests, X is deemed "nearly monotone" (typical measurement noise of a few %) and the `"target_x"` method is used; otherwise (e.g. a stress path with unload/reload), the `"index_fraction"` method is used.

`"target_x"` method (X behaves like a strain, lines 253-268):

- A common grid of n_{min} points (n_{min} = length of the shortest test), over 95 % of the smallest test's maximal X (the smallest of the maximal X values across all tests):

$$x_{max}^{target} = 0.95 \times \min_{\text{tests}} \left(\max(x)\right) \qquad x_i^{target} = i \cdot \frac{x_{max}^{target}}{n_{min}}, \quad i = 0..n_{min} - 1 \tag{9.28}$$

- For each test and each target point x^{target}, the code looks for the last index whose X is strictly below the target (a simple search, no linear interpolation — faithfully reproducing the original application's "last index below target" algorithm, docstring lines 255-256):

$$\text{index retained} = \max\{i : x_i < x^{target}\} \tag{9.29}$$

Special case noted in the code: if no X index is strictly below the target (typically for $x^{target} = 0$, the first target), index 1 is used by default (`below[-1] if len(below) else 1`, line 266) rather than index 0 — this fallback value is not commented in the code and is potentially a quirk inherited from the original application rather than a deliberately documented choice. The Y value (and Y_std) kept at each target point is directly that of the sample at the found index, rounded to 6 decimals (`round(..., 6)`) — it is therefore not a linear interpolation but a "step down" nearest-neighbor.

`"index_fraction"` method (non-monotone X, lines 269-284, via `_resample_by_fraction`, lines 200-213):

- Each test is resampled onto n_{min} equally-spaced points along its own recording order (index fraction 0..1), by linear interpolation of X and Y against this fraction:

$$\text{frac}_i = \frac{i}{n-1}, \; i = 0..n-1 \qquad \text{target_frac}_j = \frac{j}{n_{min}-1}, \; j = 0..n_{min}-1 \tag{9.30}$$

$$x_j^{res} = \text{interp}(\text{target_frac}_j; \text{frac}, x) \qquad y_j^{res} = \text{interp}(\text{target_frac}_j; \text{frac}, y) \tag{9.31}$$

- Every point remains within the range actually recorded for that test (no extrapolation).
- The common reported X axis is then the point-by-point average, across all tests, of the x^{res}:

$$x_j^{target} = \frac{1}{N_{tests}} \sum_k x_{k,j}^{res} \tag{9.32}$$

9.3.5 Mean curve and variability bands

Once all tests have been resampled onto the common grid (matrices y_{arr}, `y_std_arr` of shape (N_{tests}, n_x)) — `_combine_tests`, lines 287-317:

Mean curve:

$$\bar{y}(x) = \frac{1}{N_{tests}} \sum_{k=1}^{N_{tests}} y_k(x) \tag{9.33}$$

"Experimental" variability — RMS of the Monte-Carlo standard deviations specific to each test, at each grid point:

$$\sigma_{exp}(x) = \sqrt{\frac{1}{N_{tests}} \sum_k \sigma_{y,k}(x)^2} \tag{9.34}$$

"Material" variability — at each grid point, the ratio $r_k = y_k(x)/\bar{y}(x)$ is computed for each test k, then a normal law is fitted to the set of these ratios (`scipy.stats.norm.fit`), giving a standard deviation σ_{fit}; the "Material" variability is then:

$$\sigma_{mat}(x) = n_\sigma \cdot \sigma_{fit}(x) \cdot \bar{y}(x) \tag{9.35}$$

where n_σ is the sigma parameter (number of Monte-Carlo standard deviations of the project, `sigma=self.project.uncertainty_sigma` passed from the UI).

The "Fixed Y variability override" option (`fixed_pct`, if set by the user): entirely replaces the calculation above with a fixed percentage of the mean (line 303-304), useful when the number of tests is too small for the measured dispersion to be statistically meaningful (UI contextual help):

$$\sigma_{mat}(x) = \frac{fixed_pct}{100} \times \bar{y}(x) \tag{9.36}$$

"Total" variability — quadratic combination:

$$\sigma_{total}(x) = \sqrt{\sigma_{mat}(x)^2 + \sigma_{exp}(x)^2} \tag{9.37}$$

Relative variabilities (for the UI's "Relative Uncertainty" view), as a percentage of the mean:

$$\sigma_{total,\%}(x) = 100 \cdot \frac{\sigma_{total}(x)}{\bar{y}(x)}, \quad \sigma_{exp,\%}(x) = 100 \cdot \frac{\sigma_{exp}(x)}{\bar{y}(x)}, \quad \sigma_{mat,\%}(x) = 100 \cdot \frac{\sigma_{mat}(x)}{\bar{y}(x)} \tag{9.38}$$

(computed with `np.errstate(divide="ignore", invalid="ignore")`, so inf/NaN can silently appear if $\bar{y}(x) = 0$ at some point).

9.3.6 Export

`export_batch_result()` (lines 321-329) writes to a folder: `x.txt`, `mean_y.txt`, `material_variability_y_std.txt`, `total_y_std.txt`, `experimental_variability_y_std.txt`, and `axes.txt` (names of the X/Y columns used).

9.3.7 Limitations and remarks noted in the code

- Not independently revalidated against the original application.
- The monotone/non-monotone switch threshold (15 %) is an empirical choice in the code, not derived from a physical criterion.
- The `"target_x"` method uses a "last point below target" search (no interpolation), thus introducing a discretization error on the order of each test's sampling step — behavior deliberately reproduced identically from the original application.
- Special-case fallback to index 1 (rather than 0) when no X point is below the target (§3.4), not documented in the source code.
- The isothermal correction treats the curve's last point asymmetrically compared to the others (reuse of y_{n-2}, §3.3) — behavior noted here but not explained in the source code's comments.

9.4 Downsampling (core/downsampling.py)

Source file: `core/downsampling.py` (20 lines)

A single function `downsample(values, n)` (lines 14-20): simple decimation (keeps 1 sample out of n), with no averaging or anti-aliasing filtering. For an array of length L and a factor $n > 1$:

$$\text{count} = \left\lfloor \frac{L}{n} \right\rfloor \qquad \text{values}^{downsampled} = [\text{values}[i \cdot n] \text{ for } i = 0..\text{count} - 1] \tag{9.39}$$

If $n \leq 1$, the array is returned unchanged. Named `N_cut` in the original application (`process_t.py`), reproduced identically: `[arr[int(N*y)] for y in range(int(len(arr)/N))]` (docstring, lines 1-8). Applied uniformly to all of a test's raw signals (bar signals, time/load/ displacement, velocity, custom sensors) so that they remain synchronized with each other after reduction — used to speed up the processing of very large measurement files.

9.5 Video / image-sequence abstraction (core/frame_source.py)

Source file: `core/frame_source.py` (191 lines) — a technical module, with no physical calculation.

Provides a common interface (`FrameSource`: `n_frames`, `get_frame(i)`, `fps`) for two sources: `VideoFrameSource` (an MP4/MKV/AVI file decoded on the fly via OpenCV, with a fallback frame count obtained by scanning if the container's index does not provide it — `_count_frames_by_scanning`) and `ImageSequenceFrameSource` (a folder of TIFF/JPEG/PNG/BMP images, sorted in natural filename order via `_natural_sort_key` so that "frame2" precedes "frame10"). The function `to_display_uint8()`

converts any frame (8/16-bit, grayscale or RGB) into a displayable 8-bit image, with contrast stretching between the 0.5^{th} and 99.5^{th} percentiles of the frame's values by default (high-speed 16-bit sensors typically only use a small part of the full dynamic range). This module involves no physical analysis formula.

9.6 Annotated DIC video export (core/dic_video_export.py)

Source file: `core/dic_video_export.py` (179 lines) — a rendering module, with no new physical calculation (the displayed fields are those already computed by digital image correlation, see `core/dic_processing.py`/`core/dic_results.py`).

Generates an MP4 video (mp4v codec via OpenCV) from an already-processed DIC result sequence: each frame combines the camera background with the isocontours of the chosen field, drawn at the actually tracked position of each point (deformed mesh, not the fixed reference grid), on a white background and at a fixed resolution (1920×1440 px by default), with the option of a second panel showing, live, the time evolution of the field's ROI average. The color scale (vmin/vmax) and the time panel's axes are constants supplied by the caller (not recomputed per frame) so that consecutive frames remain visually comparable. Rendering uses a pure Agg matplotlib figure (not Qt), which makes it safe to run outside the main thread.

9.7 Bayesian calibration of constitutive laws — port of MAMBA (core/calib_*.py, ui/pages/page_bayes_calib.py)

9.7.1 Context: MAMBA and the porting decision

MAMBA is an earlier Tkinter + pymc3 program, independent of HYDRA, that fits a free-text constitutive-law equation (in strain e, strain rate v, temperature T, and up to 3 optional extra variables $a/b/d$) to experimental stress-strain data by Bayesian calibration — either simple ("pooled") or hierarchical (random effects per test, with partial pooling), with a multivariate-normal likelihood using a measured covariance matrix. The "Bayesian Calibration" tab ("Calibration" section of the sidebar) is a port of this into HYDRA, on PyMC 5 + PyTensor rather than MAMBA's older pymc3/Theano stack.

Unlike MAMBA, which loads folders of per-test `.txt` files, each variable of the equation (and the modeled/output quantity) is assigned, per calibration test, to a (grid, mean curve, covariance matrix) pair previously saved in the calibration library of the Covariance Matrix page (chapter 1, §3.5) — this is what makes it possible to build a multi-test hierarchical calibration set entirely within a single HYDRA session: process one test, save the quantities it needs, process the next, and so on.

Several limitations found in MAMBA's original code were corrected rather than reproduced as-is — covered in detail throughout this section and summarized in §7.10: a fragile equation parser (text split twice, inconsistently, then passed to `eval()`) replaced by a validated `ast` tree built once; a "validation" step that in fact never re-evaluated the model under the validation test's own conditions (it only overlaid the calibration fit); a likelihood covariance frozen at the cross-test mean even when a per-test covariance was available; a complete absence of convergence diagnostics (R-hat/ESS); a sampler `target_accept` hard-coded and hidden from the user.

The choice of PyMC 5 + PyTensor (rather than a lighter alternative) carried, as of the previous version of this document, an acknowledged but unresolved cost: heavy libraries, trickier PyInstaller packaging for producing a standalone executable. This has since been explicitly addressed

by dedicated build code and documentation, detailed in §7.5 and §7.10 — but without an actual compilation test on a Windows machine having yet been possible in the environment where this work was carried out (no network access to install the packages, and a PyInstaller build must in any case run on the target machine).

9.7.2 Equation parser and evaluator (core/calib_equation.py)

MAMBA parsed the equation text twice, independently: once via ad hoc string splitting to detect free parameter names (`def_model.py::recup_expr` — dead code, the sympy expression it built was never actually reused), and once via literal `str.replace()` substitution of each detected name with an indexed string, passed to `eval()` (`cal_MCMC`) — fragile (a parameter name that was a substring of another token could silently corrupt the substitution; scientific notation such as `1e-3` risked having its `e` confused with the strain variable) and dependent on an old Theano/ numpy interoperability that does not carry over to PyTensor.

`core/calib_equation.py` parses the text exactly once, into an `ast` tree (`ast.parse(expr, mode= "eval")`) restricted to an explicit whitelist — since this is free text typed by the user, this whitelist is a genuine security boundary, not merely a style choice: `^` is first translated to `**` (MAMBA's own convention, and the user manual's, where `^` is power and `log` denotes the natural logarithm, not base 10); only `Name`/`Constant`/`BinOp`/ `UnaryOp`/`Call`/`Load` nodes are accepted (`_validate_ and_collect`, lines 232-295); the only allowed binary operators are `+ - * /` and power (`ast.Add`/ `Sub`/`Mult`/ `Div`/`Pow`), the only allowed unary operators `+/-` (`_ALLOWED_BINOPS`/`_ALLOWED_UNARYOPS`, lines 121-126); a function call (`ast.Call`) must target a recognized name (`KNOWN_FUNCS`), with no keyword argument, and exactly one positional argument. Any other construct (subscripting, attribute access, lambda, comprehension, comparison, etc.) is rejected with an explicit error message.

Reserved variables (`RESERVED_VARS`, lines 105-108): e (strain), v (strain rate), T (temperature), and up to 3 optional variables $a/b/d$ — exactly MAMBA's own set. Recognized functions (`KNOWN_FUNCS`, lines 110-117, also exactly MAMBA's own set, named `lll` in the original code): `sqrt cos sin tan cosh sinh tanh acos asin atan erf exp log log10 asinh acosh atanh`, plus the constant `pi` (handled separately, automatically injected into the namespace if the equation uses it and the caller has not already supplied it, `evaluate`, lines 405-426). Free parameters (the quantities the Bayesian Calibration will actually fit) are detected automatically as whatever `Name` nodes remain once the reserved variables and `pi` are removed — a single source of truth, unlike MAMBA's two independent and sometimes inconsistent passes.

The evaluator (`evaluate(parsed, namespace, backend)`, lines 405-426, built on `_eval_node`, lines 353-402) is deliberately agnostic to the type of the values involved: the tree is re-evaluated by dispatching each arithmetic operator through ordinary Python operator overloading (`+ - * / **`), which works identically for a numpy array or for a PyTensor tensor variable, and each recognized function through a `backend` dictionary supplied by the caller — `NUMPY_BACKEND` (lines 206-217, `np.sqrt`, `np.arccos` for `acos`, `scipy.special.erf` for `erf` for lack of a native numpy equivalent, etc.) for the numeric preview, or the PyTensor equivalent built on the fly by `core/ calib_pymc_model.py::_build_pytensor_backend` (§7.5) for the calibration model itself. A single evaluation function, two interchangeable backends, no duplicated logic to keep in sync — this is the mechanism that lets the "Per-test calibration fit" card (§7.9) re-evaluate the model after the fact with no numerical code separate from what was used during sampling.

Domain guards (added after the initial port)

Several of the whitelisted functions are defined only on part of the real line: `sqrt`/`log`/ `log10` require a non-negative argument (`log`/`log10`: strictly positive), `acos`/ `asin` require an argument in $[-1, 1]$, `atanh` requires an argument in $(-1, 1)$, `acosh` requires an argument ≥ 1 — and the power operator `**` (written `^` in this grammar) is defined for a NEGATIVE base only when the exponent is an integer; a negative base with a non-integer exponent (for example a fitted hardening exponent n in `v**n`) has no real result.

> **Origin of the fix, as documented in the code itself:** the strain-rate channel of a real test had 1 to 2 slightly negative points (ordinary measurement noise near zero) feeding directly into an equation of the form `v**m`/`log(v)`, which produced a `NaN`/complex value during evaluation and broke the calibration (a numpy `RuntimeWarning` silently producing a `NaN`, or PyMC's initial-point check simply refusing to start sampling — either way, unusable).

Rather than letting one or two noisy points near zero fail an entire calibration, each restricted-domain function/operation below is now evaluated against its argument **clipped into its valid domain** (with a tiny `DOMAIN_EPS` margin away from an exact boundary, since several of these functions have a derivative that blows up right at the edge of their domain — a classic trap for gradient-based samplers) instead of the raw value:

```
DOMAIN_EPS = 1e-9

def _safe_sqrt(x):
    return np.sqrt(np.maximum(x, DOMAIN_EPS))

def _safe_acos(x):
    return np.arccos(np.clip(x, -1.0 + DOMAIN_EPS, 1.0 - DOMAIN_EPS))

def _safe_pow(base, exponent):
    return np.maximum(base, DOMAIN_EPS) ** exponent
```

(`_safe_sqrt`/`_safe_log`/`_safe_log10`/`_safe_acos`/`_safe_asin`/ `_safe_acosh`/`_safe_atanh`/`_safe_pow`, lines 139-174). This changes nothing for a point already inside the domain (clipping is a no-op there) and only affects the handful of points that would otherwise have been mathematically undefined — this does not, and cannot, turn aberrant data into a physically meaningful fit; it only keeps the computation running instead of crashing on it.

An important distinction is drawn between a **literal integer exponent** (`e**2`, `e**-1`, `T**2.0`, …) — always perfectly well-defined for any real base, including negative (for example $(-3.0)^2 = 9.0$) — and a variable or non-integer literal exponent, which cannot be proven safe for every future negative base (for example every MCMC draw of a fitted exponent). This distinction is detected at parse time (`_is_integer_exponent`, lines 328-350, which traces back through any leading unary `+`/`-`): only an exponent not proven safe goes through the guarded power; clipping an integer exponent would silently corrupt a negative input that is nonetheless physically legitimate (for example a sub-zero Celsius temperature, squared).

`evaluate_with_domain_report(parsed, namespace)` (lines 429-471) is the instrumented variant of `evaluate()` that, in addition to the value, returns a report `{function name (or "pow"): number of real points clipped}` — used only for a one-off diagnostic before sampling (`core/calib_run.py::_domain_guard_report_for_tests`, §7.6), never on `evaluate()`'s own hot path (the `_safe_*` functions are unconditional clips, with no branching) and never fed back into the PyTensor graph of the actual sampling.

9.7.3 Three prior treatments per free parameter (core/calib_priors.py)

`ParamPrior` (dataclass, lines 41-111) carries, for a given free parameter, one of MAMBA's three mutually exclusive treatments (`PRIOR_KINDS`, line 27):

- `fixed`: a non-calibrated constant (`fixed_value`), used as-is wherever the parameter appears in the equation.
- `uniform` (pooled — a single value shared across all calibration tests): `pm.Uniform(name, uniform_lo, uniform_hi)`.
- `normal_hier` (hierarchical — a genuine random effect per test, with partial pooling): MAMBA's Normal–inverse-Chi-squared hyperprior, reproduced formula for formula (`core/calib_pymc_model.py::build_model`, §7.5):

$$\mu \sim \mathcal{N}(\mu_0, s_0) \tag{9.40}$$
$$X \sim \chi^2_{\nu_0} \tag{9.41}$$
$$\sigma = \sqrt{\frac{s_0^2\,\nu_0}{X}} \tag{9.42}$$
$$\theta_j \sim \mathcal{N}(\mu, \sigma) \quad \text{for each test } j \tag{9.43}$$

with $\mu_0/s_0/\nu_0$ (`hier_mu0`/`hier_s0`/`hier_nu0`) the three numbers entered by the user. MAMBA took four numbers (`mu0, s0, S0, nu0`) but in fact fed the same `s0` back into both the Normal hyperprior on μ and the Chi-squared-derived computation of σ — a separate `S0` was therefore never an actually independent parameter; this redundant field has not been carried over here, a single `hier_s0` stands in for it.

`validate()` (lines 79-96) checks that `uniform_hi > uniform_lo` and that `hier_s0 > 0`/`hier_nu0 > 0` depending on the chosen kind; `validate_priors(dict)` (lines 114-123) additionally checks that each dictionary key actually matches its own `ParamPrior`'s `name`. `point_value()` (lines 98-111) supplies a single representative numeric value (the fixed point, the midpoint of the Uniform interval, or μ_0 for the hyperprior) used by the numeric preview path before any real sampling, as well as (with the same logic separately reproduced in `core/calib_run.py::_representative_param_value`, §7.6) by the domain-guard diagnostic.

This class is deliberately distinct from `core/bayes_visco_calibration.py::PriorSpec` (chapter 8, §8.4, used by the application's other Bayesian calibration page, the one for the viscoelastic bar model) — a different concept (bounds for a Gaussian-Process-surrogate optimizer, with no hierarchical treatment), deliberately not shared.

9.7.4 From the covariance library to a test dataset (core/calib_library_adapter.py)

A pure numpy/stdlib module (no PySide6/pymc dependency), which turns a row assignment of the "Calibration tests" card (one `CovarianceLibraryEntry` per variable role, plus one for the output quantity) into a `CalibTestDataset` (dataclass, lines 38-51: `label`, `grid`, `variables` — a role → interpolated-array dictionary —, `observed`, `covariance`, `is_validation`, and two traceability fields `output_entry_name`/`variable_entry_names`).

`build_test_dataset(label, variable_entry_map, output_entry, is_validation, zero_start_vars, zero_start_output)` (lines 61-168) interpolates each assigned variable (linearly, `np.interp`) onto the output quantity's own grid — never the other way round. This choice is deliberate: the likelihood (`pm.MvNormal`, §7.5) is built against the measured covariance matrix of the output quantity, which

only makes sense on the grid it was actually computed on (an $N \times N$ matrix cannot be resampled onto a different grid without recomputing it from the raw draws, which the library entry does not retain); every other variable is, by contrast, only a pointwise value needed by the equation at each point of that same grid. A `LibraryAdapterError` is raised if a variable's own grid does not fully cover the $[\min, \max]$ range of the output grid (interpolation would otherwise silently extrapolate), or if fewer than 2 grid points are available, or if the covariance matrix's shape does not match (N, N).

"Start at zero" translation per role (new setting). `zero_start_vars` (a set of role names, for example {"e", "T"}) and `zero_start_output` (boolean, for the output entry) independently control, role by role, whether the curve assigned to that role must be translated by its own first sample (`mean - mean[0]`) before interpolation, so that it starts exactly at 0 at its own native first grid point. Two separate controls, not a single global switch, because a user may want to correct, say, only a strain channel with a known gauge zero-drift, without touching temperature or the output. A pure translation does not change a curve's variance, so the output covariance matrix stays unchanged in every case.

Special case of role `e`: every `CovarianceLibraryEntry` in the application (Covariance Matrix page, chapter 1, §3.1) has its `grid` axis systematically pegged to the strain channel of the same test, regardless of which physical quantity is actually stored in its `mean` (stress, temperature, …) — so that `output_entry.grid` (returned as `CalibTestDataset.grid`, and used as the horizontal axis on every plot showing this dataset) IS numerically strain, the same physical quantity as the reserved variable "e". Checking "e" in `zero_start_vars` therefore *also* translates the grid returned, by its own first sample (`out_grid - out_grid[0]`), so that the strain axis itself starts at 0 — not only the separately assigned "e" curve's values (which feed the equation, not the horizontal axis). This is purely a display effect: interpolation has already taken place in the original, non-translated grid coordinates, and the equation evaluator never reads `dataset.grid` (only `dataset.variables`) — so there is no effect on the calibration itself, only on the starting point of the horizontal axis of the "Per-test calibration fit"/"Validation" plots (§7.9).

Covariance mode and regularization ("jitter"). `per_test_or_shared_covariance(tests, mode, jitter_frac)` (lines 243-277, `COVARIANCE_MODES = ("per_test", "shared_mean")`, lines 171-175) resolves the choice of likelihood covariance: `per_test` (the new default — fixes MAMBA's bug, which always used the cross-test mean even when a per-test covariance was available) returns each test's own matrix; `shared_mean` (MAMBA's old systematic behaviour, kept as an explicit option) requires all matrices to have exactly the same shape and returns their element-wise mean.

`jitter_frac` (default `DEFAULT_COVARIANCE_JITTER = 0.0`, line 177, no regularization) is passed to `regularize_covariance(cov, jitter_frac)` (lines 214-240), applied to every returned matrix (after per-test/shared-mean resolution, before it reaches the Cholesky factor of `pm.MvNormal`): it adds `jitter_frac * mean(diag(cov))` to the diagonal — a standard "nugget"/ridge regularization for a numerically ill-conditioned empirical covariance matrix.

`covariance_condition_number(cov)` (lines 180-211) computes the matrix's condition number (ratio of the largest to the smallest eigenvalue, `np.linalg.eigvalsh`) — a standard measure of the numerical reliability of its exact inverse, which is exactly what `pm.MvNormal(..., chol=...)` actually uses in its likelihood. A matrix built from Monte-Carlo draws (Covariance Analysis page, a limited number of draws spread very smoothly/correlated over many grid points) can exhibit one or two dominant modes and a long tail of near-zero-variance directions that are in fact nothing but finite-sample noise, not real structure.

Finding from a real case, documented in the code itself: a condition number in the tens of millions (confirmed on a stress covariance actually measured by a user, condition number $\approx 4.6 \times 10^7$) means the exact GLS/MvNormal criterion is dominated by an almost arbitrary, physically meaningless near-zero-variance direction rather than by the visually obvious shape of the data — which can pull a calibration's free parameters toward severely wrong values, sometimes pinned against a bound, even while the sampler keeps reporting an excellent R-hat (it has converged, but to the wrong answer, because that is indeed what the exact likelihood wants). Verified on this same pathological case: `jitter_frac=0.01` (1%) brings the condition number down to $\approx 2.3 \times 10^4$, and `jitter_frac=0.1`–`0.3` (10–30%) visually brings the resulting fit close to a simple unweighted least-squares fit of the same equation on the same data — confirming that this is a real, effective fix, not a merely cosmetic change of a number.

This setting is a genuine modelling decision (it slightly weakens the constraint that the exact structure of the measured covariance imposes on the fit), which is why it is exposed as an explicit, user-adjustable setting ("Sampler settings" card, "Covariance jitter" field, §7.9) rather than a silent, hard-coded epsilon. `core/calib_run.py` separately computes the RAW condition number (before any regularization) of each calibration test's covariance and reports it, if above a threshold, in the "Result summary" card (§7.6, §7.9).

9.7.5 Building and sampling the PyMC model (core/calib_pymc_model.py)

Every pymc/pytensor import in this module is deferred — inside function bodies, never at module level — so that importing `ui.main_window` (and thus starting the application) never requires these libraries to be installed. `pymc_available()` (lines 52-66) is a cached probe (importing both `pymc` and `pytensor` must succeed) that the rest of the application queries to proactively disable the Run button, rather than only failing after a click.

`build_model(parsed, priors, tests, covariances)` (lines 146-227) builds — without sampling — the `pm.Model`: one `pm.Uniform` per `uniform` parameter (reused identically in the evaluation of every test); for each `normal_hier` parameter, the Normal–inverse-Chi-squared hyperprior of §7.3, followed by `pm.Normal(name, mu=mu, sigma=sigma, shape=n_tests)` (a random effect per test); `fixed` parameters are plain Python constants merged into the namespace, never random variables. For each test j, the namespace (reserved variables already interpolated onto that test's output grid, plus the current fixed/pooled/hierarchical values) is passed to the same `core.calib_equation.evaluate()` function as the numeric preview path, but with the PyTensor backend built on the fly by `_build_pytensor_backend()` (lines 77-131, which maps each recognized function to its `pytensor.tensor` equivalent, e.g. `pt.arccos` for `acos`, and reproduces exactly the same domain guards as `NUMPY_BACKEND`, §7.2): this is the direct benefit of the single-backend equation parser described in §7.2 — a real test whose strain-rate channel has slightly negative points would otherwise make PyMC fail right at model construction ("Initial evaluation of model at starting point failed!"), not merely the numeric preview. Each test's measured covariance matrix is a fixed input, not a random variable — its Cholesky factor is computed in plain numpy (`np.linalg.cholesky`), once per test, outside the PyTensor graph — and the likelihood is `pm.MvNormal(f"obs_{j}_{label}", mu=phi_j, chol=chol_j, observed=test.observed)`.

Default NUTS sampler: nutpie (corrected since the previous version of this document)

`sample_model(built, draws, tune, chains, cores, target_accept, init, nuts_sampler, progress_cb, cancel_check)` (lines 258-317) runs NUTS sampling (`pm.sample(..., progressbar=False, return_`

`inferencedata=True)`, PyMC's native text progress bar being disabled so it does not write to standard output outside the application's control). Default values (lines 230-234): `DEFAULT_DRAWS=1000`, `DEFAULT_TUNE=1000`, `DEFAULT_CHAINS=4`, `DEFAULT_TARGET_ACCEPT=0.9` (PyMC's own default — MAMBA hard-coded 0.99 and never exposed it to the user), `DEFAULT_INIT="auto"`.

The previous version of this document noted, as a limitation (§7.10), that `pm.sample`'s `nuts_sampler` parameter was not certain to actually be wired end to end from the interface's sampler settings. That is no longer the case: `DEFAULT_NUTS_SAMPLER = "nutpie"` (line 255) is now defined in this module and *actually* passed by `sample_model` through to `pm.sample(nuts_sampler=...)` (lines 258-312, only when the value is non-empty — see below); `core/calib_run.py::run_calibration` (§7.6) in turn merges it into the default `sampler_settings` dictionary, so that a run that specifies nothing explicitly does use `nutpie`, not the historical `"pymc"` sampler.

> *"nutpie" (a Rust implementation of NUTS, https://github.com/pymc-devs/nutpie) instead of PyMC's built-in "pymc" sampler. WHY: the "pymc" sampler compiles each model's logp/gradient via PyTensor's C backend, which requires a working C/C++ compiler (g++ on Windows, via mingw-w64) present on the machine that actually runs it — no problem in this development sandbox or on a development machine with a compiler installed, but a real problem for a frozen PyInstaller executable handed to an end user with no compiler or development environment. "nutpie" instead compiles via PyTensor's Numba backend, and Numba's dependency (`llvmlite`) statically links LLVM into its wheel — no system compiler is needed on the end user's machine, only the (larger, but self-contained) numba/nutpie packages need to be bundled into the frozen application. This is also the choice currently recommended by PyMC itself (`pip install "pymc[nutpie]"`; `pm.sample` already prefers nutpie automatically when it is importable) — forcing this choice explicitly here is therefore "adopting the modern default PyMC itself recommends," not a workaround added only for packaging.*

(source-code comment, lines 236-254). The `nuts_sampler` parameter also accepts `"pymc"` (the historical C-backend sampler — works on a development machine with a compiler, but would fail on a frozen executable with no compiler), `"numpyro"` or `"blackjax"` (PyMC's other alternatives, not specifically supported/tested by this application); a falsy value (`None`/`""`) omits the argument entirely, falling back to whatever `pm.sample` would choose on its own — an explicit escape hatch for a machine where nutpie is genuinely not installed, rather than this function silently forcing an unavailable sampler. **This setting, however, is exposed nowhere on the interface's "Sampler settings" card (§7.9)** — it can only be changed by calling `run_calibration`/`sample_model` directly (for example from a script or a test), never from a control on the page.

Scope warning, to be flagged explicitly, unchanged since the previous version of this document: this development sandbox has no network access and has neither pymc, nor pytensor, nor arviz installed — `build_model`/`sample_model` were therefore written carefully against the documented PyMC 5/PyTensor API, but could never actually be run against the real libraries during their development, including for the `nuts_sampler="nutpie"` wiring described above. A manual test on the user's own machine (a simple synthetic 2-test/1-parameter case, checked by hand) remains necessary before this module can be considered numerically validated, beyond the mere fact that it compiles and that its wiring to the interface works (see §7.9/§7.10). All the other modules of this feature (`calib_equation.py`, `calib_priors.py`, `calib_library_adapter.py`), by contrast, are fully tested with real data in this same sandbox — this exception is isolated here precisely so it does not spread to the rest.

NOTE on cancellation: PyMC exposes no supported interruption point once `pm.sample` has started — `cancel_check` is accepted (for interface symmetry with every other calibration function the application's shared worker thread drives, §7.8) but is checked only BEFORE sampling starts, never during. `progress_cb` likewise is called only twice ((0, 1) before, (1, 1) after) — not genuine step-by-step progress.

9.7.6 Orchestrating a run (core/calib_run.py)

`run_calibration(equation_str, priors, test_assignments, covariance_mode, sampler_settings, zero_start_vars, zero_start_output, progress_cb, cancel_check)` (lines 154-306) is the single function the Run button calls (via the shared worker thread, §7.8) and that the interface's integration tests substitute to verify the Qt wiring without needing a real PyMC.

Order of steps: parse the equation (`parse_equation`) → check that a prior exists for every detected free parameter → `validate_priors` → cancellation checkpoint → build the test datasets (`_build_datasets`, lines 71-103, which separates "Validation only" rows from calibration rows, applies `zero_start_vars`/`zero_start_output` to each, and raises a `CalibRunError` if no row is supplied or if all rows are marked validation-only) → compute the RAW condition number (before regularization) of each calibration test's covariance (`covariance_condition_number`, §7.4, kept regardless of the mode/jitter actually used afterward) → compute the domain-guard diagnostic (`_domain_guard_report_for_tests`, lines 123-151, over both the calibration AND the validation tests, each evaluated once with a representative value per free parameter via `_representative_param_value`, lines 106-120 — a strictly informational diagnostic, never a reason to block a run: any evaluation error for a given test is silently ignored for that test) → merge `sampler_settings` with the defaults (`draws`, `tune`, `chains`, `cores=1`, `target_accept`, `init`, `covariance_jitter=DEFAULT_COVARIANCE_JITTER`, `nuts_sampler=DEFAULT_NUTS_SAMPLER`) → resolve the covariance mode, applying `covariance_jitter` (§7.4) → check `pymc_available()` (otherwise `PymcUnavailableError`) → cancellation checkpoint → `build_model` → `sample_model` (with `nuts_sampler` passed through, §7.5) → package a `CalibrationResult`.

`CalibrationResult` (dataclass, lines 41-63) carries: the source equation, the variables/parameters used, the priors, the covariance mode, the pooled/hierarchical/fixed parameter names, the posterior summary, `idata` (the full arviz `InferenceData`, kept for the diagnostics of §7.7), the lists of calibration and validation tests, the sampler settings actually used (`sampler_settings`, an echo of what was actually passed to `sample_model` — including `nuts_sampler`), `covariance_condition_numbers` ({test label: raw condition number}), `domain_guard_warnings` ({test label: {function name (or "pow"): number of points clipped}}), and `zero_start_vars`/`zero_start_output` (an echo of the translations applied).

Every internal exception (`EquationError`, `PriorError`, `LibraryAdapterError`) is systematically re-raised as `CalibRunError` (a `ValueError`), so that the page's single status label never needs to know the original exception type; `PymcUnavailableError` (a `RuntimeError`) stays distinct, since it is an environment problem, not a user-input one.

Saving/loading a calibration project to a file (core/calib_project_io.py) — a feature absent from the previous version of this document

A new module, with no counterpart in the previous version of this documentation (the file then only existed as a work-in-progress feature, not yet reliable enough to describe). It saves/reloads an entire Bayesian Calibration page configuration — equation, each free parameter's prior, the "Calibration tests" table's assignments, and the sampler settings — as a JSON+CSV file pair, following exactly the same convention already established by Results & Export's "Export full test" and Test Setup's "Load Test" (`ui/pages/page_results_export.py::_on_export_full_test`, `ui/pages/page_test_setup.py::_on_load_test`): *Save* asks for the `.csv` path and derives the `.json` path from it (`_json_path_for`, lines 100-102, `path.rsplit(".", 1)[0] + ".json"`); *Load* asks for the `.json` path and looks for the associated `.csv` alongside it (`_csv_path_for`, lines 104-105).

The two files play different roles, not two interchangeable halves of a single format: the **CSV** is the "Calibration tests" table itself (genuinely tabular — one row per test, one column per

equation variable role plus Output/Validation), while the **JSON** carries the equation text, each free parameter's complete prior (every field, not only the ones its own `kind` actually uses — the same "every field always present" convention as `core.calib_priors.ParamPrior`, §7.3), the per-column "Variable" quantity-selector choices, the covariance mode, and the sampler settings (including `covariance_jitter`, but NOT `nuts_sampler`, which in any case is exposed nowhere on the interface, §7.5).

This module deliberately does NOT store the covariance-library entries that a test column points to — those are saved/reloaded separately, as their own JSON+CSV pair, by `core/covariance_library_io.py` (chapter 1, §3.5.1) from the Covariance Matrix page. A calibration project file only ever stores library-entry NAMES; a name that no longer exists in `project.covariance_library` at reload time is left unresolved — it is up to the caller (the page, §7.9) to report precisely which ones, rather than this module guessing or silently dropping them.

A pure stdlib module (`json`/`csv`) plus `core.calib_priors` — no PySide6, no pymc/pytensor — hence fully testable with real data, the same reasoning as `core/calib_equation.py` and `core/calib_library_adapter.py`.

Exposed functions:

- `export_calibration_project(csv_path, equation, priors, role_quantities, output_quantity, covariance_mode, sampler_settings, tests, used_vars, zero_start_vars, zero_start_output) -> json_path` (lines 108-165): writes the CSV table (one row per entry in `tests`, columns `test_label`, one per role in `used_vars` (in that order), `output_entry`, `is_validation`) and everything else into the paired `.json` file (`format_version=FORMAT_VERSION`, line 46, currently 1). Does not catch write errors (`OSError` propagates unchanged, as for every other export in the application). Accepts a partially filled table (an empty string for a cell not yet assigned is preserved as-is rather than requiring every row to be complete before it can be saved — a work-in-progress configuration is a legitimate thing to save for resuming later).
- `import_calibration_project(json_path) -> CalibProjectData` (lines 168-239): reads back the file pair. Raises `CalibProjectIOError` (a `ValueError`) for a malformed JSON, an unknown prior kind, or a missing/malformed companion CSV — the latter is required (a project file without its test table is treated as incomplete, never silently accepted with zero tests, which would silently discard real configuration work if it were then saved again). Handles backward compatibility with this format's very first version, which had only a single global `"zero_start"` boolean (uniformly translating every role AND the output, with no per-column choice) rather than today's separate `zero_start_vars`/`zero_start_output` fields: if neither current field is present in the JSON, the old boolean (if present) is reproduced as-is rather than silently discarded (lines 217-227).

`CalibProjectData` (dataclass, lines 58-73) is `import_calibration_project`'s return value — everything needed to repopulate cards 1-4 of the page, except for resolving library-entry names against the CURRENT project's `covariance_library` (the page's own job, since only it knows which entries currently exist and how to report an unresolved one).

Interface-side wiring (`ui/pages/page_bayes_calib.py`): a "Calibration project (save/load)" card, placed before the Equation card (§7.9), with two Save/Load buttons. A notable robustness point, fixed in response to a real bug report: loading a project used to clear only the INPUT widgets (equation/priors/tests/settings) but never the RESULT of a previous run (`_last_result`/`_posterior_flat`) — so that every card depending on a result (Result summary, Per-test fit, Interdependence, Diagnostics, Validation) kept displaying whatever had actually been sampled before, silently stale and no longer matching the equation/priors now shown on screen, until Run was clicked again. `_on_load_calibration_project` now clears these cards first of all, before

even reading the file — after a load, they return to their own "no run yet" state, so that nothing on screen can be mistaken for that project's result until it has actually been run.

9.7.7 Posterior diagnostics and genuine cross-validation (core/calib_diagnostics.py)

Two levels of "real testability" deliberately coexist in this module: `posterior_predictive_at`/`posterior_correlation_matrix`/`forest_plot_rows`/`energy_diagnostic_arrays` take only plain numpy arrays/dictionaries (never an `arviz.InferenceData` object beyond a simple `idata.sample_stats["energy"]` access), hence are fully tested with real data in this sandbox; only `convergence_summary` remains a thin wrapper around `arviz.summary` (deferred import, the same pattern as the pymc/pytensor imports in `core/calib_pymc_model.py`) — genuinely impossible to test without arviz installed, hence covered only at the wiring level.

`posterior_predictive_at(parsed, param_draws, dataset)` (lines 134-169) is a genuine posterior predictive evaluation: the validated equation (§7.2) is re-evaluated, once per posterior draw, under the supplied `CalibTestDataset`'s own conditions (its own grid, its own e/v/T/a/b/d variables) — this directly fixes MAMBA's "validation" bug, which never did anything more than overlay the calibration fit on the validation plot, without ever actually re-evaluating the model under the validation test's own conditions. Returns the grid, the observed curve, the posterior-predictive mean, and a 95% band (2.5/97.5 percentiles across all draws).

`resolve_predictive_param_draws(free_params, priors, posterior_flat, test_index)` (lines 77-129) resolves, for each free parameter, which posterior draws to use: a `fixed` parameter is broadcast as a constant; a `uniform` (pooled) one is read directly; a `normal_hier` one, if `test_index` is supplied (the test in question genuinely was part of the fit — "Per-test calibration fit" card, §7.9), uses its own per-test value; otherwise (a validation test genuinely held out of the fit — "Validation" card, §7.9), falls back to the parameter's population hyper-mean (`mu_<name>`). This is a simplification documented explicitly, both in the code and in the Validation card's own help text: it predicts with the population's central tendency and ignores the cross-test spread that a genuinely new test would also be subject to — so the band displayed for a validation test is narrower than the true predictive uncertainty as soon as a hierarchical parameter is involved.

`posterior_correlation_matrix(posterior_flat)` (lines 172-191) computes an ordinary Pearson correlation matrix (`np.corrcoef`, no arviz) between the supplied named posterior draws (typically each pooled parameter directly, plus each hierarchical parameter's hyper-mean `mu_<name>`), names sorted alphabetically, requires at least 2 parameters. `convergence_summary(idata)` (lines 198-209) wraps `arviz.summary` — R-hat/ESS always computed (MAMBA never computed them at all, `forestplot(..., rhat=False)`).

Forest plot and Energy plot now drawn by hand (fix for a real arviz API bug)

An earlier version of this module drew the Forest/Energy diagnostic views directly via `az.plot_forest`/`az.plot_energy`. A real bug report, surfaced from a user's machine, showed these calls failing with an apparently unrelated error ("Keyword arguments ['r_hat', 'ax'] …have no active aesthetic mapped to them") — the installed arviz version no longer accepted the same `ax=`/`r_hat=` keyword-argument form this module had been written to use. arviz's PLOTTING API has gone through more than one overhaul; its underlying DATA structures (`InferenceData`, built on xarray — `idata.sample_stats["energy"]`, and the columns returned by `az.summary()`, both used below) have stayed considerably more stable across versions.

`forest_plot_rows(posterior_summary, var_names)` (lines 235-261) extracts exactly what a forest plot needs — point estimate, 94% HDI interval, R-hat — from `posterior_summary` (the like-named field

of `CalibrationResult`, itself the return value of `convergence_summary(idata)`, i.e. `az.summary()`'s default columns: mean, sd, hdi_3%/hdi_97%, r_hat, …). Returns a list of {name, mean, lo, hi, r_hat} dictionaries, one per name in `var_names` that is actually present in `posterior_summary` (a name that is absent — for example a fixed parameter, never sampled — is silently skipped, not an error). `lo`/`hi` fall back to mean±standard deviation when neither of the two HDI column spellings used across arviz versions (`"hdi_3%"`/`"hdi_97%"`, the historical default 94% interval, or an older/alternative spelling `"hdi_2.5%"`/`"hdi_97.5%"`) is present.

`energy_diagnostic_arrays(idata)` (lines 264-300) extracts directly the per-draw Hamiltonian energy values specific to NUTS from `idata.sample_stats["energy"]` (a simple xarray access — part of arviz's core DATA structures, not its plotting API, hence unaffected by the API change described above) and derives from it the two distributions the standard "energy plot" (BFMI diagnostic) usually overlays: `"marginal"` (every energy value from every draw/chain, flattened and recentred on its own mean) and `"transition"` (the first differences of energy WITHIN each chain, computed chain by chain so that a jump at the boundary between two chains is never counted, then flattened together). A transition distribution markedly narrower than the marginal one signals a sampler that struggled with this posterior's geometry (low BFMI) — something MAMBA never exposed to the user.

Both diagnostics are now built by hand, from this stable data, with plain numpy and matplotlib (drawn by the caller, `ui/pages/page_bayes_calib.py`, which already owns every other plot on this page in the same way) rather than by following a moving plotting API that this module cannot test without installing exactly the user's own arviz version. Secondary benefit: both are now actually testable here with fake data, with no monkeypatching — a real improvement in coverage, not merely a workaround.

9.7.8 Shared worker thread (core/qt_worker.py)

`CalibWorker(QObject)` (lines 41-67, signals `progress`/`finished`/`failed`/ `cancelled`) carries the execution of a long-running function off the interface thread — `run_calibration` here, but also `core/bayes_visco_calibration.py::calibrate` for the application's other Bayesian calibration page (chapter 8, §8.4). This class already existed, identically, in `ui/pages/page_visco_calib.py` (under the name `_CalibWorker`) before this port; it has been extracted here into a shared module rather than duplicated a second time, since it has nothing specific to viscoelastic calibration. `run()` (lines 57-67) catches `OperationCancelled` (signal `cancelled`) and any other `Exception` (signal `failed`, stringified message) — including, in fact, an exception raised inside a slot connected to the `finished` signal itself, since `finished.emit(...)` occurs inside the same `try` block.

9.7.9 Interface — the Bayesian Calibration page's cards

The page (`ui/pages/page_bayes_calib.py`) flattens MAMBA's 4-step wizard (Define the model → Set up the calibration → Run → Show the results) into always-visible cards on a scrolling page, following the Card/`make_scroll_page` convention already used everywhere else in the application — not a step-by-step wizard. The page now has **twelve** cards (versus eleven in the previous version of this document): a new "Calibration project (save/load)" card (§7.6.1), placed first of all, before the Equation card, plus the eleven cards already described previously — all updated below to reflect the actual additions found in the code (domain guards, covariance jitter, "start at zero" translation, measurement-uncertainty band, hand-rebuilt forest/energy plots).

1. **Calibration project (save/load)** (lines 174-210, §7.6.1): "Save project (.json+.csv)…" / "Load project (.json)…" buttons. The help text notes that *Save* works even if some rows of the test table are not yet fully assigned (a work-in-progress configuration is a legitimate

thing to save) and that *Load* REPLACES the equation/priors/tests/ sampler settings currently shown, it does not merge them with what is already there. A covariance-library entry referenced by name but not resolved on load is explicitly listed in the card's status label rather than silently dropped.

2. **Equation**: a free-text entry field (e.g. `A * (1 + B*e^n) * (1 + C*log(v)) * (1 - T^m)`, a Johnson-Cook-type form), a Parse button (or simply leaving the field). The help text lists the reserved variables and recognized functions directly from `RESERVED_VARS`/`KNOWN_FUNCS` (§7.2), never retyped by hand, so that the help text and the parser cannot drift apart, and now explicitly explains the domain guards (§7.2): a slightly negative point of a real channel feeding `log(v)` or `v^m` is clipped to a tiny positive value instead of crashing the run, and a test whose real data needed this clipping is flagged in the Result summary card.
3. **Free parameters**: one row per auto-detected free parameter, rebuilt on every successful parse. Each parameter starts as Normal (hierarchical) by default — the card's subtitle recalls the workflow recommended by MAMBA's own manual: run everything hierarchically first, inspect the parameter interdependence card (card 9) once diagnostics are available, then demote weakly identified or strongly cross-correlated parameters to Uniform (pooled).
4. **Calibration tests**: rebuilt only when the actual set of reserved variables used by the equation changes. A "Variable" row (one per column, plus one for Output) lets the user choose which physical quantity a column actually represents, among those saved in the calibration library (chapter 1, §3.5) — a reserved letter such as e therefore has no fixed meaning: it can just as well be assigned to a hydrostatic pressure or a volumetric strain as to a strain. Sensible but non-binding defaults: $e \rightarrow$ strain, $v \rightarrow$ strain rate, $T \rightarrow$ temperature, Output $\rightarrow$ stress, $a/b/d \rightarrow$ "(any quantity)" (no filter). Changing a column's quantity immediately refilters each row's dropdown for that column (an entry whose quantity is unknown — an empty `quantity_key` — is never hidden, for lack of known provenance); a selection already made that no longer matches the new filter is reset to "– choose –" rather than staying invisible but active. A new "Start at zero" row, below the Variable row, carries an INDEPENDENT checkbox per column (§7.4): checking a column translates every test's curve for THAT column only (subtracting its own first sample, to start exactly at 0), leaving every other column untouched — useful for a channel with a known zero offset (for example a strain-gauge drift) without shifting variables that do not need it; checking e (strain) additionally shifts the horizontal axis plotted on the Per-test calibration fit/ Validation plots, since every saved entry's grid is itself strain — a pure display effect, with no effect on the calibration's numbers. Each row can be checked "Validation only" to be excluded from the fit and reserved for a genuine cross-check (card 10).
5. **Sampler settings**: covariance mode (Per-test by default / Shared mean), Draws/Tune/ Chains/Cores (Cores initialized from `project.n_workers`), Target accept (0.9 by default), initialization method (auto/adapt_diag/jitter+adapt_diag/advi+adapt_diag), and a new **Covariance jitter** field (0 by default — no change — §7.4): the tooltip directly explains the real $\approx 4.6 \times 10^7$ condition-number case encountered in practice and recommends trying 0.01–0.1 if the Result summary card reports an ill-conditioned covariance, using the smallest value that makes the warning disappear — with a red message and the Run button disabled if `pymc_available()` is false. **The NUTS sampler choice (`nuts_sampler`, §7.5) does not appear on any control of this card** — the module's "nutpie" default (§7.5) systematically applies to every run launched from the page; only a direct call to `run_calibration` (script, test) can change it.
6. **Run**: Run/Stop/progress bar/status label. The Stop button's tooltip explicitly states that PyMC offers no supported way to interrupt sampling once it has actually started — Stop only takes effect if clicked before NUTS begins (§7.5, §7.10).
7. **Result summary**: posterior mean ± standard deviation per pooled parameter, or hypermean ± hyper-standard-deviation per hierarchical parameter (the population-level esti-

mate — each test's own value remains available in the full posterior but is not displayed here), with an explicit flag if R-hat exceeds 1.01. Recalls the covariance mode and the list of roles translated to zero for this run. Two new conditional warnings: a severely ill-conditioned likelihood covariance (raw condition number $> 10^5$, §7.4) is flagged per test, with its exact condition number and a reminder of any jitter applied; a test whose real data required domain-guard clipping (§7.2) is flagged with, for each function involved, the number of real points actually clipped.

8. **Per-test calibration fit**: a combo box of calibration tests (never "Validation only" rows), observed curve vs. posterior-predictive mean ± 95% band — a genuine re-evaluation of the equation under the chosen test's own conditions (§7.7), using, for a hierarchical parameter, that specific test's own value. New: an additional grey "Observed 95% band" now surrounds the observed curve itself, computed at $\pm 1.96\sigma$ from the diagonal of that test's own covariance matrix — distinct from the green posterior-predictive band (which describes the uncertainty of the FITTED model) and displayed alongside it, so the observed curve no longer gives the misleading impression of exact, noise-free data.
9. **Parameter interdependence**: the same heat-map presentation as `page_visco_calib.py` (chapter 8, §8.4.5: `imshow`, `RdBu_r` palette, annotated cells) — posterior correlation between every pair of calibrated parameters (pooled directly, hierarchical via their hyper-mean `mu_<name>`); a simple message replaces the heat map if fewer than 2 parameters are calibrated.
10. **Posterior diagnostics**: "Posterior density" (per-parameter histogram, always available, with no arviz) / "Forest plot" / "Energy plot" (these last two depend on `idata.sample_stats`/ `az.summary()` but are now drawn by hand rather than via `az.plot_forest`/`az.plot_energy`, §7.7 — an explicit, clean message if the required data is missing, rather than a crash, regardless of the installed arviz version). For a hierarchical parameter, the Forest plot now shows BOTH its hyper-mean (`mu_<name>`, the same value labelled "hyper-mean" in Result summary) AND the value actually sampled for each individual test (`<name>[<test index>]` , arviz's default naming convention for a pymc variable with `shape=(n_tests,)` declared with no explicit dims/coords) — useful for checking whether a specific test's own value has drifted far from the hyper-mean shown in Result summary (the Per-test calibration fit card uses the test's own value, not the hyper-mean).
11. **Validation**: appears only if at least one "Validation only" row was used in the last run — a genuine posterior-predictive cross-check (§7.7), not a mere overlay of the calibration fit as MAMBA did. The same grey measurement-uncertainty band as card 8 surrounds the observed curve. The caveat about falling back to the hyper-mean for a hierarchical parameter (§7.7) is explicitly repeated in the card's own help text.
12. **Export**: "Export all figures as PDF…" — a separate PDF file per currently populated plot, written to a user-chosen folder, named `bayes_calib_<card>.pdf` (`per_test_calibration_ fit`, `parameter_interdependence` if at least 2 parameters are calibrated, `posterior_diagnostics`, `validation` if at least one validation row exists); "Export posterior summary as .csv/.txt…" (the `convergence_summary` summary, §7.7, one row per arviz variable with all its statistic columns).

9.7.10 Limitations and warnings noted in the code

- Never run against a real PyMC/PyTensor/arviz during development (§7.5) — an **unchanged** finding since the previous version of this document, including for the more recent `nuts_sampler="nutpie"` wiring: this entire feature (model construction, NUTS sampling, arviz diagnostics) was written carefully against the documented API, but a manual numerical-validation test on the user's machine remains required before any use in real conditions.

- `nuts_sampler` **wiring — fixed since the previous version of this document:** the "nutpie" default (§7.5) is now actually passed end to end, from `core/calib_run.py::run_calibration` through to `pm.sample(nuts_sampler=...)` (`core/calib_pymc_model.py::sample_model`). One point remains unresolved, however: this setting is exposed on no control of the "Sampler settings" card (§7.9) — impossible to change without editing the code or calling `run_calibration` directly.
- **The PyInstaller packaging question — addressed since the previous version of this document, but still not verified end to end:** the `nuts_sampler="nutpie"` default (§7.5), the `PYTENSOR_FLAGS=cxx=` environment variable set by `main.py` before any import (so that no remaining PyMC/PyTensor code path ever tries to invoke a C/C++ compiler), the addition of `numba`/`nutpie` to `requirements.txt`, and a dedicated PyInstaller `hydra.spec` (`copy_metadata`/`collect_data_files`/`collect_submodules` for pymc/pytensor/arviz/nutpie/numba/llvmlite/xarray/pandas, including a specific workaround for the `arviz_base`/`arviz_stats`/`arviz_plots` sub-packages scanned via `pkgutil.iter_modules()` rather than by pip distribution name) together address the question explicitly left open in the previous version. `BUILD_WINDOWS.md` documents it in detail. By this guide's own admission, however (its own "Manual step required before considering the build 'done'" section): *none of these changes could be tested with a real PyMC/PyTensor/numba/ PyInstaller* in the environment where they were prepared (no network access to install them, and a PyInstaller build must in any case run on the target Windows machine) — an actual compilation test AND a functional test (running the generated exe and carrying out a real Bayesian Calibration computation end to end, ideally on a clean Windows machine with no Python/compiler installed, to confirm that no compiler is actually required) remain mandatory steps before any distribution, not merely recommended ones.
- No real cancellation while sampling is in progress: PyMC exposes no supported interruption point once `pm.sample` has started; the Stop button only takes effect if clicked beforehand, and `cancel_check` is checked only once, right before the call to `pm.sample` (`core/calib_pymc_model.py::sample_model`, lines 297-299).
- `progress_cb` is called only twice ((0, 1) before, (1, 1) after) — not genuine step-by-step progress, since PyMC's native text progress bar is disabled so it does not write to standard output outside the application's control.
- The domain guards (§7.2) are a deliberately transparent fix, not a silent mechanism: they clip a real out-of-domain point instead of letting it crash/produce a `NaN`, but they never make an equation applied to genuinely aberrant data correct. The diagnostic that reports them (`_domain_guard_report_for_tests`) is explicitly documented as a "best-effort" check before sampling: it does not replay every real MCMC draw exactly, only one evaluation per test with one representative value per free parameter — the absence of a warning in Result summary means "nothing detected by this check," not an absolute guarantee that no clipping ever occurred during actual sampling.
- Falling back to the hyper-mean for a hierarchical parameter in the Validation card (§7.7, §7.9, card 11): the predictive band displayed for a validation test ignores that parameter's own cross-test spread — it is therefore narrower than the true predictive uncertainty.
- A free parameter's prior kind is global, not per test: a parameter is entirely pooled, entirely hierarchical, or entirely fixed across all calibration tests of a given run — no partial mixing is possible within a single run.
- A library entry's `quantity_key` is never validated against the entry's actual content (chapter 1, §3.5): it is used only to filter the "Calibration tests" card's dropdown menus, never to check the data itself.

- The equation parser's whitelist is deliberately narrower than full Python (§7.2): no `%`, `//`, comparison, or logical operators — a deliberate security choice since the input is free text typed by the user; the validated tree is in any case never passed to `eval()`/`exec()`, only re-evaluated by `core/calib_equation.py`'s own controlled dispatcher.

9.7.11 Summary of file/function references

Topic	File	Function/method
Equation parsing/ whitelist	`core/calib_equation.py`	`parse_equation`, `_validate_and_collect`
Dual-backend evaluation (numpy/PyTensor)	`core/calib_equation.py`	`evaluate`, `_eval_node`, `NUMPY_BACKEND`
Domain guards (sqrt/log/ acos/…, power)	`core/calib_equation.py`	`_safe_*`, `_is_integer_exponent`, `evaluate_with_domain_report`
Three prior treatments per parameter	`core/calib_priors.py`	`ParamPrior`, `validate_priors`
Covariance library → test dataset	`core/calib_library_adapter.py`	`build_test_dataset`
"Start at zero" translation per role/output	`core/calib_library_adapter.py`	`build_test_dataset` (parameters `zero_start_vars`/ `zero_start_output`)
Covariance mode (per-test / shared mean)	`core/calib_library_adapter.py`	`per_test_or_shared_covariance`
Condition number & regularization (jitter) of the covariance	`core/calib_library_adapter.py`	`covariance_condition_number`, `regularize_covariance`
PyMC/PyTensor availability probe	`core/calib_pymc_model.py`	`pymc_available`
Model construction (priors, MvNormal likelihood)	`core/calib_pymc_model.py`	`build_model`, `_build_pytensor_backend`
NUTS sampling (nutpie default)	`core/calib_pymc_model.py`	`sample_model`, `DEFAULT_NUTS_SAMPLER`
Full run orchestration	`core/calib_run.py`	`run_calibration`
Domain-guard diagnostic before sampling	`core/calib_run.py`	`_domain_guard_report_for_tests`
Saving/loading a calibration project (.json+.csv)	`core/calib_project_io.py`	`export_calibration_project`, `import_calibration_project`
Resolving parameter draws for a prediction	`core/calib_diagnostics.py`	`resolve_predictive_param_draws`
Genuine posterior-predictive check	`core/calib_diagnostics.py`	`posterior_predictive_at`
Correlation matrix between parameters	`core/calib_diagnostics.py`	`posterior_correlation_matrix`
Convergence summary (R-hat/ESS, via arviz)	`core/calib_diagnostics.py`	`convergence_summary`
Hand-rebuilt forest plot / energy plot	`core/calib_diagnostics.py`	`forest_plot_rows`, `energy_diagnostic_arrays`
Shared Qt worker thread (off the UI thread)	`core/qt_worker.py`	`CalibWorker`

Topic	File	Function/method
Bayesian Calibration tab page (twelve cards)	`ui/pages/page_bayes_calib.py`	`BayesCalibPage`

10. Crack & wave analysis

This chapter documents the standalone "Crack & Wave Analysis" tool (`ui/pages/page_crack_wave_analysis.py`), which computes its own formal measurement-uncertainty estimates, like the DIC module of chapter 6. Unlike the other chapters, the uncertainty here is not produced by a Monte-Carlo draw (chapter 1): each method below relies on the asymptotic covariance of a least-squares fit (linear or non-linear), computed once at a single point, not by resampling — see §7 of chapter 1 for a comparison of the two paradigms.

The tool works on an already-exported DIC dataset (as chapter 9 recalls for the other standalone tools, no page shares its DIC results in memory once closed) and is organized into four steps, each usable independently of the others:

1. Data loading (`core/crack_wave_io.py`) — an already-exported DIC CSV (Digital Image Correlation page, "Export DIC results") plus the corresponding raw image sequence/video. An IDENTICAL "Data" card exists on both tabs ("Wave speed analysis" and "Crack analysis"): loading a file from either one updates the other identically (status, paths, video/sequence mode), since there is only ONE dataset loaded for the whole page — never two independent ones — the second card being there only to avoid having to switch tabs to reload a file.
2. Elastic wave speed (`core/elastic_wave_speed.py`) — "Wave speed analysis" tab.
3. Crack-tip detection/tracking (opening jump — position only, not K_I/K_{II}, §3) and stress intensity factors K_I/K_{II} via X-FEM-DIC (enriched direct correlation, §4) — "Crack analysis" tab. X-FEM-DIC is the K_I/K_{II} calculation method exposed in the interface; a standard ring-fit DIC method (with an optional T-stress extension) and an interaction integral (J-integral) remain available in the code for programmatic use — see §5.

The material properties needed to compute K_I/K_{II} (Young's modulus E, Poisson ratio ν, plane-stress/plane-strain state) are entered in a "Material & Calibration" card at the top of the "Crack analysis" tab (the only place on the page where they are used). The pixel size (mm/px) — used to convert all velocities on the "Wave speed analysis" tab (wave fronts, crack tip) to mm/s or m/s, and needed for the K_I/K_{II} calculation on the "Crack analysis" tab — is a SINGLE shared field, but built once per tab (the same pattern as the "Data" card above): it appears both on the "Material & Calibration" card of the "Wave speed analysis" tab and on that of the "Crack analysis" tab, the two instances remaining permanently synchronized (editing one immediately updates the other) — never two independent values, just more than one place to see/edit it without switching tabs. A "Sync from DIC tab" button, present on both instances, resynchronizes it from the Image Correlation page; it also auto-fills itself from the loaded CSV ("Data" card) when the latter was exported with a pixel size, using the same recovery method as the CSV import of chapter 6 (§4.8) — ratio u_mm/u and v_mm/v, aggregated by median over the whole file (`core.crack_wave_io._recover_pixel_size_mm`) — without ever overwriting a value that has already been entered manually or already synchronized: the status shown under the field explicitly states what happened (value detected and applied, detected but not applied because a value already exists, or nothing to detect if the CSV was exported without a pixel size).

10.1 Traced crack skeleton (core/crack_skeleton.py)

A crack is represented as a tree (not a simple polyline): `CrackPath`, a graph of nodes (x, y) connected by edges, with no privileged root — any node can be a branch point (degree > 2), a candidate endpoint/tip (degree 1), or an isolated point (degree 0). This choice reflects the fact that a real crack can branch; each branch has its own tip, tracked independently in step 3.

The "Crack Skeleton Tracing" card that holds this trace sits at the top of the "Crack analysis" tab (just below that tab's own "Data" card): the trace is used only by tip detection and X-FEM-DIC (step 3, both on this tab), never by wave-front picking on the "Wave speed analysis" tab (which works directly on the raw field, with no prior trace, §2 below).

Two ways to build a `CrackPath`:

- Manually, point by point, via the interactive widget `ui/crack_skeleton_picker.py`.
- Automatically (`detect_cracks`): thresholding a strain field (the same MISES/exx/eyy field already displayed for manual tracing) into a binary "localization band" mask, thinning it into a 1-pixel skeleton via `skimage.morphology.skeletonize` — preferred over a home-grown Zhang-Suen implementation: Zhang-Suen is known to erode thin diagonal bands (a staircase effect), a failure mode that would affect a diagonal crack band, a common case — then walking the skeleton as a graph, one connected component per detected crack, allowing several cracks to be proposed in a single call.

Polyline simplification (Ramer-Douglas-Peucker) is implemented directly in the module (a classic, easily-checked algorithm), in the same "literal and auditable code" spirit as `core/gaussian_process.py`.

The automatic detector is explicitly a starting suggestion, not a final answer: its result is an ordinary (editable) `CrackPath` list, subsequently modifiable exactly like a manual trace — achieving a perfect topology (never merging two neighboring fingers of the same crack, never splitting a crack into two components due to a noise pixel) is not the goal; giving a fast, reasonable starting point is. This module computes no uncertainty — it is a geometric tracing tool, not a physical measurement.

10.2 Elastic wave speed via front tracking (core/elastic_wave_speed.py)

10.2.1 Principle: time-of-flight along a front

For the measured direction (X or Y), the method reproduces how a propagation speed is read off a seismic "shot gather" or an ultrasonic B-scan:

1. The chosen strain field is averaged along the direction PERPENDICULAR to the measured one, at each position along the measured direction (e.g. averaging over y at each x, for a velocity along X) — this reinforces a roughly plane wave front, while noise/variation along Y cancels out instead of blurring the front.
2. The front's arrival time is detected at each position, by crossing a relative threshold (half-height by default) between that position's baseline and peak level — a standard "first break" convention in wave/impact testing, robust to an amplitude that decays with propagation distance (a fixed absolute threshold would not be).
3. Linear regression position $= v \cdot t_{arrive} + x_0$ over all positions with a detected arrival — a single regression over the whole set of field columns/rows (not just two points), with a directly checkable R^2 and an interpretable intercept (where/when the front would have

started).

This assumption (plane front, propagation mainly along the measured direction) is valid near the edge from which the wave is launched, early in its propagation — the typical situation this tool targets. A low R^2 signals a curved front, contamination by a later crack signal, or a threshold that captured noise rather than the real front — not something to hide behind a more sophisticated fit. `position_range` lets the FIT (not arrival detection, which keeps running over every position) be restricted to a single side of a point source, to treat two opposite fronts separately that would otherwise average their slopes into a near-zero, nearly meaningless slope.

10.2.2 Automatic detection of multiple branches (ridge + RANSAC)

For a field showing several reflections (a wave bouncing back and forth, each bounce reversing its propagation direction), a single manual separation by position range is no longer enough. `detect_wave_branches` automates the separation:

1. Unlike the single-arrival method above (suited to a front that, once arrived, stays high), this approach looks for a transient pulse: at each instant, it finds every contiguous band (along position) exceeding the arrival threshold — "where is the field high right now" — and reports one ridge point per band, at its amplitude-weighted centroid (`_band_centroids`), not its single strongest pixel — a band has a real physical width, and per-pixel measurement noise shifts, from one frame to the next, exactly which pixel is strongest; averaging over the whole band width cancels this noise (roughly as $1/\sqrt{N}$ over the band width) while still tracking a genuine sub-pixel shift of the center.
2. These ridge points are then separated into straight-line branches by a sequential RANSAC: repeated fitting of the line (position $= v \cdot t + x_0$) with the largest number of "inlier" points (within a position tolerance), removal of those inliers, then repetition on what remains until too few points are left or `max_branches` is reached — with no prior knowledge of the number of fronts or their propagation direction.

This ridge/RANSAC approach assumes a field showing a genuinely transient pulse (rises then falls) — true for an elastic wave with no significant residual plastic strain. If the field instead shows a permanent step (strain staying high after the front has passed, e.g. the field near a growing crack tip, or non-elastic residual strain), there is NO local maximum to track once the front has passed — use the single-front method (§2.1) instead in that case.

10.2.3 Manual picking via ROI polygon (extract_ridge_in_polygon)

The two automatic methods above share a structural limitation: they choose a threshold (fixed, or per-frame normalized) and trust it to separate "the wave" from "everything else" over the whole field at once. A real recording can show several fronts of markedly different intensities (one saturated, one barely visible), for which no single threshold works. `extract_ridge_in_polygon` entirely bypasses thresholding: the user draws a loose polygon around ONE visible band directly on the plot (just as one would manually point out a frequency ridge on a PDV/VISAR spectrogram), and every frame the polygon covers contributes one ridge point, at the local extremum (max or min, either sign) among the included samples — selected among competing candidates by proximity to the polygon's own extent rather than by intensity alone, then refined by a centroid tightened around its immediate neighborhood. Nothing here reads a threshold: the polygon itself isolates the front of interest, whatever its actual intensity. The method is also not constrained to a straight line (no RANSAC/line fit): a genuinely curved front (accelerating/decelerating, or non-elastic) is traced just as faithfully as a straight one; an indicative OLS fit (speed/R^2) is still reported for information, via the same `fit_ridge_points` function as the

automatic fit.

10.2.4 Absence of formal uncertainty

None of the methods at this stage produce a formal uncertainty on the wave speed itself: the R^2 of the (indicative, unweighted) OLS fit is the only quality indicator reported, to be interpreted qualitatively (a low R^2 signals an unreliable fit, without quantifying an error bar on the slope) — unlike the crack-tip treatment (§3 below), which has a formal uncertainty estimate.

10.3 Crack-tip detection and tracking (core/crack_tip_tracking.py)

10.3.1 Overview of available methods

Five methods for detecting the tip position along the traced skeleton exist in the code, tested and reusable; only one, "opening_jump" (§3.2), is wired by default in the current interface.

- "threshold" (`track_tip_threshold`): walks the (finely resampled) path from the origin and keeps the farthest point where the chosen field still exceeds a given absolute value (tolerating a few consecutive samples below the threshold), then linearly interpolates the exact crossing (`_interpolate_tip`) rather than reporting the raw resampled-grid point — without this interpolation, the tip could only sit on one of the fixed resampling steps, producing a non-physical staircase (several frames in a row at the exact same position, then a jump) even when the underlying field actually advances smoothly. Limitation: a single absolute threshold must separate "cracked" from "not yet cracked" at every frame, whereas the overall field intensity along the path can drift over the whole sequence (evolving damage, changing contrast/noise) — a threshold suited to the first frames can become too high or too low later in the sequence.
- "singular_fit" (`track_tip_singular_fit`): at each candidate point along the path, fits the field sampled in a forward-oriented cone/ring against the $1/\sqrt{r}$ radial decay of the first-order LEFM crack-tip field, and keeps the point with the best fit (the highest R^2, among those with a positive/increasing slope toward the tip) — more physically motivated, but requires sufficient field resolution to resolve a genuine $1/\sqrt{r}$ trend, and is more sensitive to noise and to the accuracy of the path's local tangent (used to orient the cone).
- "ridge" (recommended, not wired directly but reused via ROI): threshold-free. It treats the field concentration at the crack tip exactly as chapter 2 treats an elastic wave front: a moving, physically localized peak sweeping the path over time, whose map (arc position x time) is a 2D "image" in which a ridge can be traced (`build_tip_field_profile`). In the interface, this appears as "ridge (ROI)": the user draws a polygon around the visibly advancing band and `extract_tip_ridge_in_polygon` extracts one point per covered frame, at the strongest local extremum inside the polygon — no arrival threshold, so a weak or drifting signal is just as easy to pick as a strong one.
- "arrival (ROI)" (`extract_tip_arrival_time_in_polygon`): the reverse of "ridge" (which asks, frame by frame, "where is the strongest peak right now"), this one asks, position by position along the path, "the first time the field value at this point reaches a fixed absolute threshold, when was that" — then links these (arrival time, position) pairs in the order in which they occur. For a field such as the von Mises equivalent strain, where a genuinely cracked zone jumps to a level clearly separated from the undamaged background and stays there, this is quasi-monotone by construction. This is not the same algorithm as "threshold" above: "threshold" compares each position, at every frame, against a single

threshold, so it must keep working despite a global intensity drift over the whole sequence; "arrival" instead looks at each position's complete time history only once and asks only if/when it crossed the threshold, so a slower start or an intensity ramp elsewhere does not affect an already-crossed position at all.
- "opening_jump" (the default method, the only one wired in the current interface) — see §3.2.

A limitation common to "ridge (ROI)"/"arrival (ROI)" and the manual picking tool "manual (click)" (all three working exactly on the traced path): they track a derived field (strain, or any other chosen `field_key`), which can fade or shift slightly away from the exact line of the traced path before the crack has actually stopped opening at that spot. "opening_jump" (§3.2), which directly measures the displacement discontinuity rather than a derived field, is the method wired in the current interface; "ridge (ROI)" and "manual (click)" remain available in the code for reuse, and "arrival (ROI)" remains available in the interface as a complement when crossing of a derived field is indeed what is sought.

10.3.2 Method used: opening jump (crack_opening_jump_profile, track_tip_opening_jump)

Rather than thresholding/picking a derived field (strain) along the path, this method directly measures the raw DISPLACEMENT DISCONTINUITY u/v on either side of the path: a genuinely open crack shows a clean, unambiguous jump here, even where a derived field has already become too weak or shifted to be reliably thresholded/picked.

At every point of the traced path, the u/v fields are sampled at two points offset by `offset_px` on either side of the path, along its local NORMAL (perpendicular to the tangent — i.e. "on each side of the crack face", not forward/backward along the path), and the absolute difference is taken:

$$\text{jump}_u = |u(+\text{offset}) - u(-\text{offset})| \qquad \text{jump}_v = |v(+\text{offset}) - v(-\text{offset})| \qquad \text{jump}_{mag} = \sqrt{\text{jump}_u^2 + \text{jump}_v^2} \tag{10.1}$$

`jump_mag` combines the two components exactly as X-FEM-DIC's own Heaviside-jump detection combines its own fitted jump displacement (§4.6) — the default threshold, 0.15 px, is therefore shared and directly comparable between the two methods, not a coincidence. `offset_px` must be large enough that the two sampling points genuinely fall on either side of the discontinuity (potentially several pixels wide on a genuinely correlated image), but small enough to remain a "near-tip" measurement rather than capturing unrelated far-field motion.

`opening_extent_s`/`opening_extent_xy` answer "how far along the path does `jump_mag` stay continuously above `threshold_px`, walking from the path's origin (index 0)" — the same `_threshold_walk`/`_interpolate_tip` algorithm as the "threshold" method (§3.1), applied to `jump_mag` rather than to a raw exported field. This assumes that `path_positions[0]` is already genuinely open — true for a path traced from the crack's real initiation point. `max_gap_samples` tolerates a few consecutive samples below the threshold (correlation noise/an isolated point) without prematurely truncating the walk, including right at the origin itself.

Robustness to an elastic wave passing over the traced path (`smooth_window_samples`, `advance_confirm_frames`) — flagged in practice on real granite: an elastic wave passing through the DIC field of view produces very localized (a few samples wide along the path) and high-amplitude displacement disturbances, which shift from one frame to the next — exactly the profile that single- threshold filtering cannot distinguish from a genuine crack opening. Two complementary rejection mechanisms each tackle a different axis of the problem; neither assumes or bounds a

crack-propagation speed, a deliberate choice: crack speed can vary strongly in a heterogeneous material such as granite, so a fixed speed bound would wrongly reject a fast but real advance.

(1) Spatial filtering, within the profile of ONE frame — `smooth_window_samples` (1 by default = disabled) passes `jump_mag_px` (before the threshold walk) through a sliding median filter along the path (`_nanmedian_filter_1d`, centered window, ignoring non-finite samples rather than propagating them); `opening_extent_s/opening_extent_xy` are then computed on this smoothed signal (`jump_mag_smoothed_px`), not on raw `jump_mag_px`, which is nevertheless still reported as-is in the returned profile (CSV export, "Verify visually" plot) for direct comparison. A median, not a mean, cleanly rejects an isolated outlier sample rather than blending it into the result — exactly the "narrow spike vs. sustained plateau" distinction sought: a genuine opening remains wide by construction (every path point already open stays so at the following samples), a wave pulse is not. The window width should be chosen a bit wider than the pulse's visible extent (in the "Verify visually" plot), but noticeably narrower than the shortest real opening expected near initiation — too wide a window would eat into a genuine short opening.

(2) Temporal confirmation, across frames — the filtering above does not prevent a whole frame, while the wave crosses the path, from reporting an `opening_extent_s` that momentarily spikes well beyond the real trend (the pulse can occupy the path over a width wide enough to remain a "continuous opening" in the sense of the walk, even smoothed). The old behavior (before this parameter was added) of the "forward advance only" constraint (`enforce_monotonic`, §3.1) immediately accepted any forward advance and permanently raised the accepted running maximum to that value right away — a bug identified directly in the code: the only existing protection (`backslide_tolerance_px`) bounds the tolerated BACKWARD slide behind this maximum, never an abnormally large FORWARD jump beyond it. A single frame polluted by a wave could thus permanently make the running maximum "jump" far ahead, which then systematically rejected all subsequent, genuinely real points until the real trend caught up to it (if it ever did before the end of the frame range) — exactly the mechanism that made the jump curve "very, very erratic" when a wave passed through.

`advance_confirm_frames` (1 by default = unchanged behavior, an advance is accepted immediately as before) fixes this without introducing a speed bound: a candidate that advances by more than `backslide_tolerance_px` beyond the running maximum is accepted only if `advance_confirm_frames` consecutive raw points, starting from it inclusively, all stay at this advanced level (within `backslide_tolerance_px`) — the advance must therefore "hold" over several frames, not just appear on a single one, before being considered reliable. An advance that stays within the tolerance band around the running maximum continues to be accepted immediately, as before. With `advance_confirm_frames=1`, this confirmation window collapses to the point itself, which trivially confirms itself — exactly reproducing the old immediate-acceptance behavior, term for term, including for any already-polluted frame. At the very end of the frame range, there may not be enough subsequent points left to confirm a real, fast advance — these last points (at most `advance_confirm_frames - 1`) are then cautiously rejected rather than risk freezing an unconfirmed spike (see §6 for this documented limitation).

10.3.3 Position uncertainty (opening_extent_position_uncertainty_px)

Enabled via the "Estimate measurement uncertainty" checkbox of the "Crack Tip Detection" card — built entirely from data this method already computes, requiring no additional imaging assumption (a declared DIC noise floor, a calibration target tolerance, …), though supplying more does improve the result. The method, in one sentence: perturb EACH of the detector's three tunable parameters, one at a time (the other two held at their nominal value), and measure how much the detected position shifts; combine these contributions in quadrature with a discretization floor. When `smooth_window_samples` (§3.2) is active, this estimate stays consistent

with the nominal detection: `opening_extent_position_uncertainty_px` receives this same parameter and reruns each perturbation below on `jump_mag_smoothed_px`, never on raw `jump_mag_px` — otherwise the uncertainty envelope would silently end up measuring the sensitivity of a different signal from the one actually used to detect the front.

(1) Threshold, against the jump signal's own noise — `_local_noise_sigma`, a "blind" noise estimator (Immerkaer family, the 1D analogue of `core.dic_uncertainty.estimate_noise_sigma` in 2D): from the spread of consecutive first differences of `jump_mag` within a local window (15 samples on each side by default) around the detected front (a smooth underlying signal contributes little to a first difference, especially over such a narrow window, so the difference is mostly noise). For i.i.d. Gaussian noise with standard deviation σ, two independent samples differ by a Gaussian of standard deviation $\sigma\sqrt{2}$, whose mean absolute value is $\sigma\sqrt{2} \cdot \sqrt{2/\pi} = \sigma \cdot 2/\sqrt{\pi}$ — inverted to recover σ:

$$\hat{\sigma}_{jump} = |\Delta \,\mathrm{jump}_{mag}| \cdot \sqrt{\frac{\pi}{2}} \tag{10.2}$$

The nominal threshold is then perturbed to threshold $\pm\,\hat{\sigma}_{jump}$, the walk rerun for each of the two perturbed values, and the half-width of the resulting candidate-positions envelope (including the nominal one) kept as this contribution:

$$\delta_{threshold} = \frac{\max(\text{candidates}) - \min(\text{candidates})}{2} \tag{10.3}$$

(2) Gap tolerance (`max_gap_samples` $\pm\,1$) — the same walk rerun, on the same already-computed `jump_mag`, with a gap tolerance increased/decreased by one sample; the same envelope half-width formula, δ_{gap}.

(3) Sampling offset (`offset_px` $\pm\,\max(1\text{ px},\ 25\% \cdot$ `offset_px`$)$) — the only one of the three perturbations requiring genuine RE-sampling of the raw fields (not just a rerun of the walk on an already-computed `jump_mag`), since it changes where the two points on either side of the path are themselves placed; the same half-width formula, δ_{offset}.

(4) Path discretization floor, independent of the three previous ones — the traced path's own resampling step (read directly from the data, not hard-coded, though it defaults to 2 px on this page), divided by $\sqrt{12}$ — the standard variance of a rounding error uniformly distributed over one step:

$$\delta_{grid} = \frac{\text{resampling step}}{\sqrt{12}} \tag{10.4}$$

Final combination, in quadrature (independence-of-errors assumption — a simplification, since the threshold and offset perturbations actually both act through the same underlying noisy field, but a reasonable approximation for a practical estimate, always at least as large as any single isolated contribution, so never silently optimistic):

$$\sigma_{position} = \sqrt{\delta_{threshold}^2 + \delta_{gap}^2 + \delta_{offset}^2 + \delta_{grid}^2} \tag{10.5}$$

A perturbation that fails to detect any front at all (the walk never reaches the threshold) is simply dropped from that frame's sum — which then underestimates the uncertainty for that particular contribution; documented explicitly in the code rather than hidden, since a case where the perturbed run is genuinely undetectable usually signals that the nominal detection itself is already marginal for that frame.

10.3.4 Local speed and its uncertainty

Point-by-point local speed (`trace_velocity_px_per_s`): computed by centered finite differences (one-sided at both ends) of arc position with respect to time —

$$v_i = \frac{s_{i+1} - s_{i-1}}{t_{i+1} - t_{i-1}} \text{ (interior)} \qquad v_0 = \frac{s_1 - s_0}{t_1 - t_0} \qquad v_{n-1} = \frac{s_{n-1} - s_{n-2}}{t_{n-1} - t_{n-2}} \tag{10.6}$$

Deliberately not smoothed/fitted (see §3.5 for that) — this is the raw, noisy, point-by-point instantaneous speed that the velocity history of a genuinely propagating crack needs, not a single straight-line average.

Uncertainty of this local speed (`trace_velocity_uncertainty_px_per_s`): ordinary error propagation through this SAME finite-difference formula, treating `position_uncertainty_px[i+1]` and `[i-1]` as independent (each front position is detected separately, frame by frame):

$$\sigma_{v,i} = \frac{\sqrt{\sigma_{s,i+1}^2 + \sigma_{s,i-1}^2}}{t_{i+1} - t_{i-1}} \qquad \text{(interior, and the analogous rule at both ends)} \tag{10.7}$$

NaN wherever `trace_velocity_px_per_s` itself would be undefined (fewer than 2 points, zero time step), and wherever either of the two input position uncertainties is NaN (the NaN propagates through the square root — an unknown position uncertainty makes the resulting speed uncertainty unknown, not zero). It is this pair of arrays that feeds the shaded envelopes on the "arc position vs. time" plot of the "Verify visually" tab (position on the left, local speed on the right, dashed) and its CSV column `local_speed_uncertainty_px_per_s`/`_mm_per_s` (see §3.6).

10.3.5 Fitted speed (regression, weighted when an uncertainty is available)

`fit_tip_ridge_points` provides, in addition to the local speed above, an "indicative" speed via linear regression (time, arc position) over the whole trace:

- With no uncertainty supplied: an ordinary, unweighted OLS regression (`np.polyfit`), with R^2.
- With `uncertainty_px` supplied (a per-point position uncertainty, §3.3): becomes an inverse-variance-weighted least-squares regression ($w_i = 1/\sigma_{s,i}^2$), with a proper standard error on the fitted slope, from the weighted-regression covariance — a classic result for a weighted regression when the weights are genuine inverse variances:

$$\text{Var(slope)} = \frac{S_w}{S_w S_{wtt} - S_{wt}^2} \qquad \text{where } S_w = \sum w_i,\ S_{wt} = \sum w_i t_i,\ S_{wtt} = \sum w_i t_i^2 \tag{10.8}$$

More points, or a tighter per-point position uncertainty, both tighten this standard error as a real confidence measure should. This is where the recording frequency comes in: more, and more closely time-spaced, points (a tighter time spacing between accepted frames = the frame rate itself) directly tighten this standard error, with no need for a separate "frame-rate uncertainty" for this particular effect. An optional parameter `frame_rate_uncertainty_frac` (the UI field "Frame rate uncertainty (%)", 0 by default) additionally adds, in quadrature, a genuine camera time-base tolerance (trigger jitter/drift) — this one, by contrast, must be supplied by the user, since it is not observable from the trace's own data alone:

$$\sigma_{slope}^{final} = \sqrt{\sigma_{slope}^2 + (\text{slope} \times \texttt{frame_rate_uncertainty_frac})^2} \tag{10.9}$$

Falls back to the unweighted fit (last fields `None`) if `uncertainty_px` has fewer than two finite, positive entries matching valid times/arc_length, or if the weighted normal equations are degenerate.

Position and speed uncertainty are reported in px (`PolygonTipRidgeResult.position_uncertainty_px`, `speed_stderr_px_per_s`/`_mm_per_s` — converted to mm only at export/display time, via the page's pixel size); each is `None` when the "Estimate measurement uncertainty" checkbox is unchecked — the "None means not estimated, never zero uncertainty" convention shared with the rest of the tool.

10.3.6 CSV export

`export_tip_roi_csv` (`core/crack_tip_tracking.py`) writes, in long format (one row per path point per trace), the arc position, the local speed (§3.4), and — when uncertainty was estimated — the columns `position_uncertainty_px`/`_mm`, `local_speed_uncertainty_px_per_s`/`_mm_per_s` (per point, §3.4) and `trace_speed_stderr_px_per_s`/`_mm_per_s` (a single value for the whole trace, §3.5); these columns only appear in the header if at least one exported trace actually has an estimated uncertainty. The dedicated "Export this plot to CSV…" button on the "Verify visually" tab writes exactly the same content, restricted to the `opening_jump` traces of the current leaf — those the plot actually shows.

`export_opening_jump_profile_csv` writes, for a single frame, the complete `jump_u_px`/`jump_v_px`/`jump_mag_px` profile along the path (not just what the plot shows — the threshold line and detected-front marker — but the raw signal itself), with a per-point `is_open_at_this_point` indicator.

10.4 Stress intensity factors K_I/K_II: X-FEM-DIC (core/xfem_dic.py)

X-FEM-DIC is the K_I/K_{II} calculation method exposed on the "Crack analysis" tab (see §5 for the additional methods available in the code but not wired into the interface).

10.4.1 Principle and rationale: why the raw images

Subset-based DIC (`core/dic_processing.py`, chapter 6) tracks each point as a RIGID, purely translated template (normalized cross-correlation `cv2.matchTemplate` against a fixed reference template). A template whose image patch overlaps the crack face contains pixels from both sides of a real displacement discontinuity; no rigid translation can then match the corresponding template once the crack is open, even slightly — this is exactly why the ring fit available in the code (§5.1) must already exclude a region `r_min_px` and a band `flank_exclusion_deg` around the crack faces. Fitting a more elaborate model (even an enriched one) AFTER THE FACT on this already-exported field can never do more than re-fit the clean points, far from the crack: it cannot recover the near-field accuracy the enrichment is meant to provide, because the bias due to the discontinuity is already baked into the input before any post-processing even sees it.

X-FEM-DIC therefore re-correlates directly against the raw reference/deformed image pair itself, with a displacement model that ALREADY contains the crack discontinuity and the near-field singular field as first-order terms — an "Integrated DIC" formulation (Hild & Roux; "eXtended-DIC"/"X-DIC" papers by Réthoré et al.), solved directly against pixel gray levels rather than against an already-correlated point cloud.

10.4.2 Displacement model and deliberate simplifications

A textbook XFEM code enriches individual NODES near the crack (Heaviside for fully-cut elements, Williams branch functions for elements touching the tip), each carrying its own local degrees of freedom. Two deliberate simplifications keep this module tractable, while still being grounded in the literature (close to how Réthoré's X-DIC formulations actually proceed — a global rather than nodal singular enrichment is a published choice, not an ad hoc shortcut):

1. The tip singular term is GLOBAL, not nodal: exactly two extra scalar unknowns for the whole mesh, K_I and K_{II}, multiplying the same closed-form Williams/Irwin basis functions as the ring fit (`_williams_basis` — deliberately the same formulas/sign/κ convention, so that this method's K_I/K_{II} remain directly comparable to those of the additional methods in §5), smoothly windowed to tend to zero far from the tip (`_tip_window`) so it cannot leak into the background field and bias it. K_I/K_{II} therefore come directly out of the correlation as two of its fitted parameters — no separate post-fit ring step is needed.
2. The crack-face jump is nodal but NOT SHIFTED: $H(x, y)$ is evaluated at the query point itself (not the standard shifted form $H(x) - H(x_i)$) — simpler, at the cost of slightly less local mixing right at the boundary of a cut element with its non-enriched neighbors, with no consequence at the sampling density used here.

The rest — a structured Q4 mesh, bilinear shape functions, forward-additive Gauss-Newton against gray levels with an analytically exact Jacobian (the displacement model is LINEAR in each of its unknowns; only the image's own deformation is non-linear) — is standard.

10.4.3 Tip-centered Q4 mesh (uniform or graded)

The background mesh (`Q4Mesh`) is a grid of bilinear quadrilateral (Q4) elements, stored via the physical coordinates of each node column/row (`x_edges`/`y_edges`, strictly increasing) rather than by an origin plus a uniform element size — which lets `build_graded_tip_centered_mesh` grade the element size continuously, fine near the tip and coarse far from it, while the rest of the module (localization, shape functions, the fit itself) remains unchanged for a uniform mesh (`build_q4_mesh`, a special case where the steps are constant). Staying axis-aligned makes locating a physical point (which element contains it, and its local isoparametric coordinates) direct (a per-axis `searchsorted` lookup) — no per-point Newton solve needed, unlike a general unstructured FE mesh.

`build_tip_centered_mesh(tip_xy, roi_half_width_px, element_size_px)` builds the uniform version; `build_graded_tip_centered_mesh` the graded version, with a fine zone (`element_size_px`) over a radius `r_enrich_px` around the tip, growing up to `element_size_px * coarsen_factor` over the rest of `roi_half_width_px` — better resolution near the tip and fewer (larger) elements far from the crack for the same overall region, at no extra Gauss-Newton cost (the fit does not care about a given element's size, only about the (ξ, η) localization per sample).

The interface's "Suggest settings" button (`recommend_mesh_parameters`) pre-fills `element_size_px`, `r_enrich_px` and `roi_half_width_px` from the already-tracked crack distance (and, if the image is already loaded, caps `roi_half_width_px` to stay within the image) — heuristics explicitly acknowledged as such in the code (not derived from first-principles analysis), to be refined afterwards as needed with "Preview mesh". It also proposes a starting value for the sampling spacing (`sample_step_px`, "Sample spacing" in the interface, see §4.5 for exactly what this setting does): `element_size_px / 8`, capped between 1 and 10 px — roughly several samples across even the smallest element (the one near the tip on a graded mesh), without dropping arbitrarily below the correlation length of a real speckle pattern. This proposal is only a reasonable starting point, not a computed optimum: it remains editable, and §4.7 details why indiscriminately lowering it

below that has a cost on the reported formal uncertainty, not only on computation time.

10.4.4 Enrichment classification (classify_mesh_enrichment)

A node is Heaviside-enriched when an element carrying it is "cut" by the already-traced crack path, counting only the ALREADY OPEN portion (path arc $\leq$ `tip_s`). An element is classified "cut" when the signed distances of its 4 corners to the path change sign, the nearest crossing is genuinely close by (less than 1.5 times the element's local size — a LOCAL band, not a global one, to remain correct on a graded mesh where a coarse, far-away element can be much larger than the mesh's finest size), and it is not clearly beyond the current tip. Projection is always done onto the COMPLETE path (never truncated at $s \leq$ `tip_s`) and only filtered afterwards by projected arc length — truncating the polyline first would bias the projection of a node near the tip toward the artificial end of the truncation rather than the crack's real geometry.

10.4.5 Solving: damped Gauss-Newton, Jacobi scaling

The gray-level residuals compared at each iteration are not evaluated at every pixel or at the mesh nodes, but on a REGULAR grid of sampling points built independently of the Q4 mesh itself (`_sample_points(mesh, step_px)`, spacing `sample_step_px`, "Sample spacing" in the interface, 2 px by default) — this grid completely ignores element boundaries; the mesh is only used to interpolate the displacement at each point of this grid (shape functions) and to classify enrichment (§4.4). Too coarse a spacing starves the smallest elements in play (those near the tip on a graded mesh, §4.3) of enough samples within their bounds: their corner nodes can then end up under-constrained and flagged unreliable (`node_reliable=False`, left blank on the isocontours). Too fine a spacing, conversely, adds no real information once below the real speckle's correlation length (typically a few pixels): neighboring samples become increasingly redundant rather than more informative, for a cost that grows as $1/\text{step}^2$ per iteration — and, as detailed in §4.7, can make the formal uncertainty on K_I/K_{II} appear smaller than it really is.

Each frame is solved by forward-additive Gauss-Newton directly against the gray levels of the deformed image (interpolated by a bicubic spline, `RectBivariateSpline`), with an unknowns vector a organized as: $[a_x(n_{nodes}), a_y(n_{nodes}), b_x(n_{Heav}), b_y(n_{Heav}), K_I, K_{II}]$. At each iteration:

1. The current displacement (u, v) is evaluated at each sampling point (background mesh shape functions + Heaviside jump weighted by the local sign + windowed singular term), giving the warped deformed position $(x + u, y + v)$.
2. The gray-level residual $I_{ref} - I_{def}(x + u, y + v)$ is computed, along with the deformed image's gradient at the warped point (an exact analytical chain rule for all degrees of freedom, except the image's own gradient factor, re-evaluated every iteration).
3. A damped Gauss-Newton step is solved by sparse least squares (`scipy.sparse.linalg.lsqr`, with a damping parameter `damping`), AFTER Jacobi column scaling: the background/ Heaviside degrees of freedom are $O(1)$ in PIXELS, but K_I/K_{II} are $O(10^6)$ in $\text{Pa} \cdot \sqrt{\text{m}}$ — without this scaling, a single damping value cannot regularize both groups equitably (it would barely constrain the pixel-scale degrees of freedom, or would crush the K_I/K_{II} columns to near zero, leaving their estimate at the noise level).
4. Convergence when the largest step of any degree of freedom (other than K_I/K_{II}) between two iterations falls below `convergence_tol_px` (2×10^{-3} px by default), or after `max_iterations` (15 by default, adjustable from 5 to 500 in the interface).

This step criterion says nothing about the fit's actual improvement, only about its size — and this produces behavior observed in practice on real data: at a large tip displacement, the signal (gray-level gradient due to real motion) heavily dominates correlation noise and the numerical

noise of the spline derivative, so the Gauss-Newton corrections decay geometrically and cross `convergence_tol_px` within a few iterations; at a small tip displacement (a barely open crack), this same signal becomes comparable to that noise, each iteration's normal equations are then dominated by noise rather than by a genuine descent gradient, and since `damping` is FIXED (no adaptive Levenberg-Marquardt scheme), the step does not necessarily decay toward zero: it can jitter indefinitely around a noise floor above `convergence_tol_px`, regardless of `max_iterations` — this is then not slow convergence that a larger iteration budget would fix, but a step that structurally cannot reach THIS particular threshold on this type of frame.

Two mechanisms, orthogonal to the step criterion above, address this:

- Residual-plateau stopping (`stall_patience`, 5 iterations by default; `stall_rtol`, 0.1% by default): each iteration's gray-level RMS is compared to the best RMS observed so far for this frame; the "iterations without real improvement" counter only resets to zero if the improvement exceeds `stall_rtol` (relative) — a marginal or negative change (typical of jitter) lets it keep running. After `stall_patience` consecutive iterations with no clear improvement, the loop stops, reported as `stalled=True` — kept strictly distinct from `converged=True` (genuine convergence by step size), so that a caller that already prefers a converged candidate over a non-converged one (§4.6, automatic tip search) is never silently handed a plateaued fit as if it were a clean convergence.
- Best-observed-RMS state, not simply the last iterate: whatever criterion stops the loop — clean convergence, plateau, or simply `max_iterations` exhausted without either triggering — the degrees-of-freedom state actually reported is the one that produced the lowest RMS over the ENTIRE trajectory (including its very last iterate), never necessarily the loop's last iterate. For a frame that already converges cleanly by step size, this changes nothing: its best-RMS state coincides, in the vast majority of cases, with its final state, since the RMS decreases nearly monotonically in this well-conditioned regime. For a jittering frame, this guarantees that an early (plateau) or late (budget-exhaustion) stop never freezes the result on an arbitrarily worse point of the jitter than another already encountered.

In the "Crack analysis" tab's status text and in `export_xfem_k_csv` (column `xfem_stalled`, §4.9), a plateaued frame is distinguished from a simply non-converged one: unlike the latter, increasing "Max iterations" has structurally no reason to help a plateaued frame (it was not converging slowly, it stopped improving) — the displayed message says so explicitly rather than repeating the same advice ("try a larger Max iterations") that applies to the other case.

10.4.6 Automatic tip-position detection

The method always reuses the crack PATH already traced in step 1 — a "fixed path" is the boundary deliberately chosen for this feature (no re-detection of the crack geometry itself). What CAN be found independently, frame by frame, is the tip's POSITION along this already-traced path:

- Fixed trace (`auto_detect_tip=False`, default): the position at each frame is read directly from the tracking already performed in step 3 (`positions_xy`/`arc_length_s`) — unchanged, with no re-detection.
- Automatic detection (`auto_detect_tip=True`), two algorithms to choose from (`tip_detection_mode`):
  - "residual" (default): a grid search over candidate positions along the same path, within a radius `tip_search_radius_px` around THIS frame's nominal position (each frame is anchored independently to the step-3 estimate, NOT to the previous frame's auto-found position — to prevent a single search error from irreversibly drifting all subsequent frames), at a step of `tip_search_step_px`, keeping the CONVERGED candidate with the lowest gray-level correlation residual (falling back to the best non-

converged candidate only if none converged).
- "heaviside_jump": fits a deliberately over-covering trial mesh (starting at the nominal position, then pushed further out in bounded, growing steps as long as the crack still appears open up to the very edge of the trial), then reads the refined position directly off the Heaviside jump amplitude fitted by this fit (`heaviside_disp_x/y`), locating where the face opening falls below `heaviside_jump_threshold_px` — generally much cheaper (1-2 full fits per frame instead of 2 × radius/step + 1), at the cost of coarser spatial resolution (accurate to about one to two element widths, not pixel-accurate like the residual search).

Both modes enforce the same physical constraint: a crack cannot shrink — the ACCEPTED position at each frame can never move backward (in arc length, in the growth direction inferred from step 3's overall trend) relative to the last position this method has actually accepted.

10.4.7 Formal uncertainty (asymptotic Gauss-Newton covariance)

`estimate_uncertainty=True` (opt-in, checking "Estimate measurement uncertainty (K_I/K_II)") computes `K_I_stderr_pa_sqrt_m`/`K_II_stderr_pa_sqrt_m` from the asymptotic Gauss-Newton covariance that any non-linear least-squares fit admits at its converged point — exactly the same least-squares covariance (linear, at the local linearization point) as §5.1, applied here to the non-linear fit's local linearization rather than to an exactly linear model (the classic way of reporting a non-linear least-squares fit's parameter uncertainty):

1. The residual Jacobian J is reassembled ONCE MORE at the FINAL converged solution — not reused from the last Gauss-Newton loop iteration, whose own J was evaluated one step BEHIND the final state a.
2. The gray-level residual variance is $\sigma^2_{gris} = SSE/(n_{effective} - n_{dof})$, where $n_{effective}$ is the number of samples actually within the deformed image's bounds at this final iteration (not the raw $n_{samples}$) and n_{dof} the model's total number of degrees of freedom (background + Heaviside + K_I/K_{II}).
3. The parameter covariance is cov $= \sigma^2_{gris} \cdot \text{pinv}(J^T J)$ — with the same Jacobi column scaling as in the §4.5 solve: the K_I/K_{II} columns (Williams basis values weighted by the image gradient) differ from the background/Heaviside displacement columns by several orders of magnitude, and inverting the RAW normal matrix would silently truncate the tiny-but-real variance of K_I/K_{II} instead of honestly reporting it.

$$\sigma^2_{gris} = \frac{SSE}{n_{eff} - n_{dof}} \qquad \text{cov} = \sigma^2_{gris} \cdot \text{pinv}\left(\left(\frac{J}{\text{col_scale}}\right)^T \left(\frac{J}{\text{col_scale}}\right)\right) / (\text{col_scale}_i \cdot \text{col_scale}_j) \tag{10.10}$$

This estimate is opt-in (unlike the ring fit of §5.1, always active) because reassembling J and inverting the resulting $n_{dof} \times n_{dof}$ normal matrix costs a non-negligible real time on a fine mesh (n_{dof} grows with the number of mesh nodes), on top of a fit that is already, per the module's own documentation, "much heavier" than the ring fit's simple linear solve.

A source of uncertainty this formula does not capture: residuals correlated by too fine a `sample_step_px` (§4.5). $\sigma^2_{gris} = SSE/(n_{eff} - n_{dof})$ and the normal matrix $J^T J$ implicitly treat each grid sample as an INDEPENDENT observation — the standard assumption of any least-squares covariance (already flagged as not universally verified in point 2 of §6). A real speckle pattern, however, only has a finite correlation length (typically a few pixels): below it, two neighboring samples see essentially the same gray-level information rather than two genuinely independent measurements. Sampling finer than this length inflates n_{eff} without adding any

genuinely new constraint on K_I/K_{II} — σ^2_{gris} and, in turn, `K_I_stderr`/`K_II_stderr` can then come out smaller than the real uncertainty warrants, for the same reason that an average of repeated but correlated measurements underestimates its own standard error if treated as though the measurements were independent. The code does not correct for this effect (no Bretherton-style effective-degrees-of-freedom correction, nor reweighting by correlation length): the only protection is not to push `sample_step_px` arbitrarily low — §4.3/§4.5 recommend refining it below the value proposed by "Suggest settings" only as long as K_I/K_{II} themselves keep genuinely changing, not out of habit or for "more precision".

Case of automatic tip detection (`auto_detect_tip=True`): the search itself (by residual or by Heaviside jump) is NEVER asked to estimate an uncertainty for each of its (potentially numerous) tested candidates — that would multiply this search's already-high cost by the number of evaluated candidates, for an uncertainty immediately discarded on $N-1$ fits. Once the winning position is known, that position is refit a SECOND time with `estimate_uncertainty=True`, and this second fit (deterministic — same K_I/K_{II} as the first, only its uncertainty fields differ) becomes the value kept for that frame.

`K_I_stderr_pa_sqrt_m`/`K_II_stderr_pa_sqrt_m` remain `None` if the relevant diagonal entries of the scaled normal matrix come out non-finite or negative (a degenerate/rank-deficient fit), rather than reporting a misleading number.

10.4.8 "Critical K_I" convention and uncertainty envelope on the plot

$K_I \leq 0$ has no physical meaning (a closed/compressed crack tip has no mode-I stress intensity) — unlike K_{II}, whose SIGN is physically meaningful (the shear direction), so it is never filtered this way. The "critical" value reported for K_I — and the domain-bound markers on the plots — therefore reflects only the positive portion of a trace, over each continuous interval (a real, noisy trace frequently dips back below zero more than once, not only at its very start or very end):

$$K_I^{crit} = \text{mean}(\{K_I(s) : K_I(s) > 0\}) \tag{10.11}$$

implemented by `positive_value_mean_and_domain_bounds` — non-finite samples are discarded first, never treated as a domain bound; samples are sorted by distance before intervals are searched for, so that "contiguous" means contiguous IN DISTANCE.

On the "Crack analysis" tab's K_I/K_{II}(distance) plot, an "Uncertainty envelope" drop-down (Off/±1/±2/$\pm3\sigma$) controls a shaded envelope (`fill_between`) around the X-FEM-DIC curve when it has a formal uncertainty ("Estimate measurement uncertainty" checked for that calculation, §4.7) — with the curve itself always drawn on top of its own band (via zorder layering). The critical K_I/K_{II} mean line (in the status text and on the plot) is only shown if the corresponding "K_I"/"K_II" checkbox is checked.

The plot displays, per trace, a K_I curve (solid line, round marker) and a K_{II} curve (dashed, square marker), with the same color per trace and the legend label `"{trace} -- K_I"`/`"{trace} -- K_II"`.

10.4.9 CSV export (export_xfem_k_csv)

Writes, in long format (one row per frame sample per trace), the following columns: trace/frame identifiers, arc distance (px and mm), K_I/K_{II} (X-FEM-DIC) with their standard errors (§4.7, blank if not estimated), `xfem_converged` and `xfem_stalled` (§4.5 – never both 1 on the same

row), an `is_default_critical` indicator, and the critical mean (§4.8) of positive K_I, repeated identically on every row of the corresponding trace, for easy filtering/pivoting in a spreadsheet.

10.5 Additional methods available in the code (core/crack_k1c.py, core/crack_interaction_integral.py)

Two further K_I/K_{II} calculation methods are implemented and tested (`test_crack_k1c.py`, `test_crack_interaction_integral.py`), for programmatic use, but are not wired into the "Crack analysis" tab where X-FEM-DIC (§4) is the only exposed method. Both use as input the u/v displacement field already exported by standard DIC (chapter 6) rather than a new correlation on the raw images.

10.5.1 Standard ring-fit DIC (fit_k_single_frame, compute_k_time_series)

This method samples the displacement field in a ring around a given tip position, rotates it into the crack's local (tangent, normal) frame, and fits it against the first-order LEFM crack-tip displacement field, mode I + II (Williams/Irwin):

$$u' = \frac{K_I}{2\mu}\sqrt{\frac{r}{2\pi}}\cos\frac{\theta}{2}\left(\kappa - 1 + 2\sin^2\frac{\theta}{2}\right) + \frac{K_{II}}{2\mu}\sqrt{\frac{r}{2\pi}}\sin\frac{\theta}{2}\left(\kappa + 1 + 2\cos^2\frac{\theta}{2}\right) + T_x \qquad (10.12)$$

$$v' = \frac{K_I}{2\mu}\sqrt{\frac{r}{2\pi}}\sin\frac{\theta}{2}\left(\kappa + 1 - 2\cos^2\frac{\theta}{2}\right) - \frac{K_{II}}{2\mu}\sqrt{\frac{r}{2\pi}}\cos\frac{\theta}{2}\left(\kappa - 1 - 2\sin^2\frac{\theta}{2}\right) + T_y \qquad (10.13)$$

where (r, θ) are polar coordinates centered on the tip ($\theta = 0$ directly ahead of the tip, $\theta = \pm\pi$ along the two open crack faces), κ the Kolosov constant ($3 - 4\nu$ in plane strain, $(3-\nu)/(1+\nu)$ in plane stress), and $\mu = E/(2(1+\nu))$ the shear modulus. T_x, T_y are an UNKNOWN local rigid-body offset, included as two additional free fit parameters rather than assumed zero — a DIC displacement field is measured relative to the (arbitrary) reference frame, not relative to the crack tip itself; without them, the fit would silently absorb this offset into a biased K_I/K_{II}.

This model is LINEAR in its 4 unknowns (K_I, K_{II}, T_x, T_y), so a simple ordinary-least-squares (OLS) fit suffices at each frame, with a directly checkable per-frame R^2. Sampling covers a complete ring around the tip (the crack opening across its two faces, θ near $\pm\pi$, is where the fit is most sensitive to K_I), excluded only right at the faces themselves (`flank_exclusion_deg`), where standard DIC correlation commonly fails on the image's real discontinuity. `r_min_px`/`r_max_px` bound the ring radially, excluding respectively near-tip noise/DIC decorrelation and the departure from the first-order singular field too far from it.

`recommend_flank_exclusion_deg` (§5.3) provides a starting value for `flank_exclusion_deg` adapted to the analyzed trace rather than a fixed constant for every crack.

This method is "static first": a dynamic fit (Freund-type, a kernel depending on crack-propagation speed) is a possible, unimplemented extension.

Formal uncertainty (OLS covariance, Jacobi scaling): since the model is linear, classic OLS theory applies directly and exactly (no linearization/approximation needed, unlike X-FEM-DIC, §4.7): with a residual variance $\sigma^2 = SS_{res}/(n_{eq} - n_{param})$ and a design matrix X (2 rows per

valid point — one for u, one for v — 4 columns, one per unknown), the parameter covariance is $\sigma^2 \cdot (X^T X)^{-1}$, and each parameter's standard error is the square root of its own diagonal term. This is ALWAYS computed (no checkbox, unlike X-FEM-DIC, §4.7) since the cost is essentially nil compared to the `lstsq` solve already performed — inverting a 4×4 matrix, not redoing a fit.

The K_I/K_{II}-specific columns (near-field Williams basis values) are about 10^{-13} (in meters per Pa $\cdot \sqrt{\text{m}}$), while the T_x/T_y columns are exactly 1.0 — a gap of about 10^{13} that makes $X^T X$'s eigenvalues span 26 orders of magnitude. `pinv`'s default `rcond` threshold is relative to the LARGEST singular value, so it would silently treat the (tiny but real) K_I/K_{II} directions as numerically zero and eliminate them — which does not blow up their variance (as an ill-conditioned/near-singular fit normally would) but instead truncates it to almost nothing. Scaling each column to unit L^2 norm before `pinv` (the same Jacobi-preconditioning trick as the X-FEM-DIC Gauss-Newton solve, §4.5/§4.7) equalizes each column's order of magnitude before `pinv` sees the matrix, so its `rcond` threshold no longer discriminates between them; the true covariance is then recovered by "unscaling" (dividing by each column's scale squared):

$$\text{col_scale}_j = \sqrt{\sum_i X_{ij}^2} \qquad \text{cov} = \frac{\sigma^2 \cdot \text{pinv}\big((X/\text{col_scale})^T (X/\text{col_scale})\big)}{\text{col_scale}_i \cdot \text{col_scale}_j} \tag{10.14}$$

`K_I_stderr_pa_sqrt_m`/`K_II_stderr_pa_sqrt_m` are `None` as soon as fewer than 3 valid points are available (below the `n_valid < 6` floor already applied upstream) or the scaled normal matrix remains singular (non-finite or negative variance).

The "critical K_I = mean of positive values" convention (§4.8) and `positive_value_mean_and_domain_bounds` are shared with X-FEM-DIC.

10.5.2 Optional extension: T-stress term (extended Williams series)

The model of §5.1 keeps only the $n = 1$ (singular, $1/\sqrt{r}$) term of the full Williams series. The next term, $n = 2$, is a displacement induced by a UNIFORM stress parallel to the crack faces — the "T-stress" T, classically defined by $\sigma_{xx} = T$ (local frame, crack faces along x) in addition to the singular field. It is added as a FIFTH linear fit parameter, A_2, on the same u'/v' columns as §5.1:

$$u'_T = A_2 \cdot \frac{\kappa + 1}{2\mu}\, x' \qquad v'_T = A_2 \cdot \frac{\kappa - 3}{2\mu}\, y' \qquad T = 4A_2 \tag{10.15}$$

where (x', y') are the local Cartesian coordinates (tangent, normal) already used to rotate the field. The factor $\kappa - 3$ is negative for any physical Poisson ratio, which automatically reproduces, once $T = 4A_2$ is substituted, the classic result $v = -(\nu T/E)\, y$ of a uniform stress, with no extra sign needing to be added by hand.

The model remains LINEAR (5 unknowns instead of 4), so still a simple OLS fit; its standard error, $T_{stderr} = 4\sqrt{\text{Var}(A_2)}$, comes out of the SAME 5×5 covariance matrix as $K_I/K_{II}/T_x/T_y$ (§5.1), with the same Jacobi column scaling applied both to the `lstsq` solve itself and to `pinv`: the $K_I/K_{II}/T$-specific columns (near-field basis values, $\sim 10^{-13}$–10^{-14}) remain orders of magnitude smaller than the T_x/T_y columns (=1.0 exactly), and an `rcond` threshold uncompensated by this scaling would silently truncate the smallest-magnitude direction (the T-stress, the smallest of the three).

This check is not a mere precision refinement: it is a SECOND consistency check on the ring fit itself (beyond R^2, §5.1) — an abnormally large fitted T-stress, or one whose standard error is not small compared to its value, typically indicates that the sampled ring (`r_max_px` too large,

or a background field that is genuinely non-uniform over the domain) is picking up a far-field gradient that the first-order-only model would silently absorb into a biased K_I/K_{II} — a directly actionable diagnostic (reduce `r_max_px`) rather than just one more numerical value to ignore.

10.5.3 Interaction integral / J-integral (core/crack_interaction_integral.py)

A mathematically distinct approach from the ring fit: an energetic domain integral rather than a pointwise displacement fit — a classic, well-established fracture-mechanics technique (Yau, Wang & Corten, 1980; Moran & Shih, 1987), adapted to DIC by Réthoré, Gravouil, Morestin & Combescure (2005). The REAL field (the same u/v displacement already exported by standard DIC, the same input as §5.1) is combined, as an integral over an annular domain weighted by a function q, against a known AUXILIARY field — the Williams/Irwin field in pure mode I or pure mode II, at $K = 1$:

$$M^{(1,2)} = \iint \left[\sigma_{ij}^{(1)} u_{i,1}^{(2)} + \sigma_{ij}^{(2)} u_{i,1}^{(1)} - W^{(1,2)}\delta_{1j}\right] q_{,j}\, dA \qquad W^{(1,2)} = \sigma_{ij}^{(1)}\varepsilon_{ij}^{(2)} \tag{10.16}$$

where (1) denotes the real field and (2) the auxiliary field (or vice versa — the integral is symmetric). The standard relation linking this integral to the stress intensity factors is $M = (2/E^*)(K_I^{(1)}K_I^{(2)} + K_{II}^{(1)}K_{II}^{(2)})$, with $E^* = E$ in plane stress and $E^* = E/(1-\nu^2)$ in plane strain — choosing the auxiliary field as pure mode I at $K_I^{(2)} = 1$ (respectively pure mode II), the integral directly isolates $K_I^{(1)}$ (resp. $K_{II}^{(1)}$) of the real field.

The integration domain reuses the same ring as §5.1: the same `r_min_px`/`r_max_px`/`flank_exclusion_deg`, so both methods remain directly comparable on an identical data domain. The chosen weighting function q is deliberately simple — conical, $q = 1$ at r_{min}, $q = 0$ at r_{max}, linear in between — rather than the "plateau" shape (constant over most of the domain, decaying quickly only near the edge) more common in the literature, for architectural comparability with the §5.1 ring rather than for optimal numerical accuracy.

Difference from §5.1 on `r_min_px`: this method requires `r_min_px` STRICTLY positive, whereas the ring fit (and X-FEM-DIC) tolerate `r_min_px=0` ("disabled") without issue. The reason is physical: the $K = 1$ auxiliary field used to isolate K_I/K_{II} has a singular STRESS at $r = 0$ (the very point of the singularity K_I/K_{II} characterize), so a domain reaching $r = 0$ would integrate through this singularity rather than evaluate a finite (however large) basis function at that point — `fit_k_interaction_integral_single_frame` explicitly raises an error if `r_min_px<=0`.

The integral itself is evaluated by midpoint quadrature on a regular (r, θ) grid (12 radial points $\times$ 24 angular points by default), not by a higher-order Gauss rule — consistent with the rest of the application ("literal, auditable code" rather than optimized).

The real field's gradient (needed to compute $W^{(1,2)}$ and $\sigma_{ij}^{(1)}$) is obtained by finite differences (`np.gradient`) on the already-exported u/v grid, interpolated to the quadrature points and then projected into the local (tangent, normal) frame — no analytical differentiation, since the real field has by definition no known closed form. As elsewhere in the application, u/v and the grid coordinates xs/ys are BOTH in pixels: the discrete gradient is therefore already dimensionless and numerically identical to physical strain, with no further unit conversion.

Auto-normalization: each mode is computed as $K_{mode} = M(\text{real}, \text{aux}_{mode})/M(\text{aux}_{mode}, \text{aux}_{mode})$, rather than by the standard closed-form $K = (E^*/2)\cdot M$ — i.e. by dividing by the same auxiliary field's OWN self-interaction at $K = 1$, computed on the SAME (potentially truncated) domain. E^* algebraically cancels in this ratio. The standard closed-form $K = (E^*/2) \cdot M$ implicitly assumes a COMPLETE angular domain (θ from $-\pi$ to π) around the tip; excluding a band

near the crack faces (`flank_exclusion_deg`, otherwise necessary) breaks this assumption NON-UNIFORMLY between the two modes, mode II (antisymmetric) losing proportionally more signal near the excluded band than mode I (symmetric) — the self-interaction normalization removes this dependence on the truncated domain regardless of the exclusion angle, for a pure mode I, a pure mode II, a mixed mode, any crack tangent, and in both plane stress and plane strain.

Confidence diagnostic: sub-domain independence (inner/outer half) — unlike the ring fit (whose R^2, §5.1, directly measures how well the model fits the data), this method is not a fit and therefore has no residual to report. The confidence diagnostic used instead is DOMAIN INDEPENDENCE: K_I/K_{II} are recomputed separately on the inner half (r_{min} to r_{mid}) and the outer half (r_{mid} to r_{max}) of the same ring (`K_I_inner_half_pa_sqrt_m`/`K_I_outer_half_pa_sqrt_m` and the analogous fields for K_{II}) — for a first-order LEFM field genuinely well described by the model's assumptions, the two halves must agree closely with each other AND with the full-ring value; a marked disagreement signals that the real field departs from the ideal model over part of the domain (near-field viscoplasticity, or a far-field gradient farther out) — a role analogous to the ring fit's R^2 or to X-FEM-DIC's "converged" status (§4.6), specific to this method for lack of an alternative.

Tuning `flank_exclusion_deg`: although the normalization above cancels the systematic bias due to domain truncation, a smaller exclusion angle remains preferable in practice: it retains more of the signal close to the crack faces (where sensitivity to K_I/K_{II} is strongest) while staying far enough from a genuine DIC correlation discontinuity on the faces themselves. `recommend_flank_exclusion_deg` (`core/crack_k1c.py`) provides a starting value adapted to the analyzed trace rather than a fixed constant for every crack: it compares, at the tracked tip's arc position, the RAW local tangent (nearest path sample) to the SMOOTHED tangent (smoothing window $2 \times r_{max,px}$, the same one already used internally by `compute_k_time_series`) — a large disagreement between the two is the same signal, already documented for `path_tangents` (§3), that a path traced on a pixel grid zigzags locally more than a real crack; this disagreement (in degrees) is added to a base margin of 15°, clipped between 8° and 40°:

$$\text{flank_exclusion_deg} = \text{clip}(15^\circ + \text{tangent disagreement}(^\circ),\ 8^\circ,\ 40^\circ) \tag{10.17}$$

This method computes no formal covariance uncertainty (it is not a least-squares fit — there is no Jacobian or residual in the sense of §4.7/§5.1); the sub-domain-independence diagnostic above serves as a confidence signal in its place.

10.6 Limitations and remarks noted in the code

1. Two uncertainty paradigms coexist in the application without being directly, term-for-term comparable: chapter 1's Monte-Carlo draw (replays the whole physical pipeline N times) versus this chapter's asymptotic least-squares covariance (computed once, at the fitted point, by local linearization) — see chapter 1, §7 for a comparison.
2. The uncertainty of the ring fit (§5.1) and of X-FEM-DIC (§4.7) is that of the fitted MODEL (Williams/Irwin or FE+Heaviside+singular), not an end-to-end DIC measurement uncertainty: it does not cover, for example, a possible systematic error in the upstream u/v field (chapter 6) that would affect ALL points of the ring/image in a correlated rather than independent way — the independent- residuals assumption underlying any least-squares covariance is not explicitly verified.
3. No formal uncertainty on the elastic wave speed (chapter 2) — only an indicative R^2 is reported, unlike the more detailed treatment of the crack tip (§3).

4. Automatic tip-position detection for X-FEM-DIC (§4.6) is a grid/greedy search, not a global optimization: it can settle on a wrong local minimum, particularly with noisy images or a coarse search step — each frame is anchored independently to the step-3 estimate rather than chaining off the previous frame's auto-detection (§4.6), to limit the drift of an error without fully eliminating it for a single isolated frame.
5. The opening-jump position uncertainty (§3.3) silently underestimates its own contribution as soon as a perturbation fails to detect any front at all — explicitly documented in the code rather than hidden (see §3.3, end).
6. X-FEM-DIC and the ring fit (§5.1) are "static first": neither models a dependence on crack- propagation speed (a Freund-type dynamic kernel) — a possible, unimplemented extension.
7. X-FEM-DIC's crack-face jump is not shifted (§4.2, simplification 2) — slightly less local mixing at the boundary of a cut element, judged to have no consequence at the sampling density used.
8. Enrichment classification (§4.4) uses a local tolerance band (1.5× the element size), not an exact geometric intersection — a practical robustness choice, not an absolute topological guarantee on a very irregular mesh.
9. The ring fit's optional T-stress term (§5.2) is not a systematic improvement of K_I/K_{II} themselves — it is a model consistency check (an abnormally large T-stress signals a ring sampled too wide, or a non-uniform background field); enabling it or not generally changes K_I/K_{II} very little on a field already well described by the first-order model alone.
10. The standard closed-form relation between the interaction integral and K_I/K_{II} ($K = (E^*/2)M$) assumes a complete angular domain around the tip — an assumption broken by `flank_exclusion_deg`, non-uniformly between the modes (mode II, antisymmetric, is affected more than mode I); the self-interaction normalization (§5.3) removes this dependence on the truncated domain regardless of the exclusion angle, rather than using the standard closed-form relation.
11. The interaction integral has no formal covariance/standard error (it is not a least-squares fit): its own confidence diagnostic is the agreement between the K_I/K_{II} computed on the inner half and the outer half of the same ring (§5.3).
12. The pixel size auto-detected from the loaded CSV (`_recover_pixel_size_mm`) can only be recovered if the original export itself had a pixel size entered (non-empty u_mm/v_mm columns) — an export done in pure pixels on the Image Correlation page leaves nothing to detect, and the field stays at 0 (px/s only) until a manual entry or a "Sync from DIC tab"; the status shown under the field explicitly says so rather than implying a calibration that does not exist.
13. `r_min_px=0` ("disabled"), a normal setting shared with the ring fit (§5.1) and X-FEM-DIC, is an INVALID configuration for the interaction integral (§5.3): checking "Also compute via interaction integral" without raising `r_min_px` makes this cross-check fail on every frame. This configuration case is distinguished, in the per-frame status text, from the case where the data itself is insufficient for an otherwise-valid `r_min_px`.
14. `advance_confirm_frames` (§3.2) can, out of caution, reject up to `advance_confirm_frames - 1` genuine points at the very end of the frame range: a real, fast advance occurring too close to the last analyzed frame may not have enough subsequent points to confirm itself, and is then discarded as an uncorroborated wave spike would be — a deliberate trade-off (better to lose a few end-of-trace points than risk freezing an artifact), not a detection flaw at that particular spot. An equally deliberate design choice: neither this parameter nor `smooth_window_samples` (§3.2) bound or assume any crack-propagation speed — a speed bound was explicitly ruled out, since crack speed can vary strongly in a heterogeneous material such as granite.
15. `stall_patience`/`stall_rtol` (§4.5) are heuristics tuned to a practical iteration budget (5 it-

erations, 0.1% relative), not derived from a formal analysis of each frame's own correlation noise — a frame whose signal sits right at this threshold's edge can stop a bit early or a bit late depending on noise variations from frame to frame; this is not exposed as an interface setting (unlike `max_iterations`), on the same footing as `damping`/`convergence_tol_px`, themselves already not exposed. A `stalled=True` signals that the RMS has stopped improving significantly, not that the reported fit is necessarily bad — the state actually reported is always the best RMS observed over that frame's entire trajectory (see §4.5), so it is often already close to what a full convergence would have given; it nonetheless remains prudent to treat a plateaued K_I/K_{II} with the same reservation as a non-converged fit.

16. The ratio `SAMPLE_STEP_RATIO=8.0` and the 1.0 px floor used by "Suggest settings" to propose `sample_step_px` (§4.3/§4.5) are likewise a chosen heuristic, not a value derived from a measurement of the actually-used speckle's correlation length — two speckle patterns of different grain size do not have the same correlation length, which the code never explicitly measures (neither at DIC export time, chapter 6, nor here); the proposal therefore remains a reasonable starting point on a given mesh, not a guaranteed optimum. A direct corollary of §4.7: nothing technically prevents lowering `sample_step_px` well below this proposal (down to the setting's own 0.5 px floor) — the code neither refuses it nor warns about it at run time – only this chapter's text documents why that is generally undesirable.

Appendix — Summary of Caution Points Noted in the Code

This appendix gathers, chapter by chapter, the complete set of points flagged throughout the body of the document : apparent inconsistencies between two computation paths, undocumented implicit assumptions, ambiguous units, or interface features that do not pass along all the information expected by the underlying calculation. Each point is stated exactly as observed in the code, with no correction or reinterpretation ; it is meant to serve as a single checklist, complementing the detailed "Limitations" sections at the end of each chapter.

Chapter 1 — Monte Carlo uncertainties and covariance

- Input parameters are drawn independently of one another : no input correlation is modeled (for example between the diameter and density of a single machined bar).
- Only two sampling distributions are supported (Normal, Uniform) — no log-normal, triangular, etc.
- The dispersion parameter of an `MCValue` has a different meaning depending on the chosen distribution (standard deviation for the Normal distribution, half-range for the Uniform distribution).
- Two distinct aggregation methods coexist in the application : `ui/pages/page_process.py::_aggregate_quantity` truncates the draws to their common length and uses a biased standard deviation (`ddof=0`), whereas `core/covariance.py` interpolates each draw onto a common grid and uses `numpy.cov` (unbiased, `ddof=1`) — the two standard deviations can therefore differ slightly for the same quantity.
- In the covariance module, a draw whose curve does not cover the entire requested strain range is silently excluded (no extrapolation) ; the number of draws actually used can be significantly lower than the number of successful draws.
- No normalized correlation matrix is computed natively by the application : only the raw covariance is produced.
- A custom sensor's uncertainty band (`core/custom_sensor.py`) is a deterministic additive combination, not a Monte Carlo draw in itself ; it is treated as a Gaussian standard deviation only once it re-enters a downstream Monte Carlo calculation.
- The time-varying thermal output correction of a custom sensor (`core/custom_sensor.py::CustomSensor`, chapter 1 §4.4) deliberately applies no delay/thermal-dynamics model between the supplied temperature profile and the gauge : the gauge is assumed to follow this profile instantaneously, regardless of the actual speed of thermal diffusion through its bonding layer — a deliberate design choice (to remain valid both in fast dynamic and in slow quasi-static regimes), not an undocumented approximation. It is, moreover, not restricted to sensors with a strain-gauge role : the checkbox can be ticked for any sensor, with no warning if its signal is not a strain.

- A Monte Carlo draw that fails (an exception in the physics pipeline) is simply dropped from the final mean/standard deviation, with no retry.
- The physical quantity (`quantity_key`) associated with a calibration-library entry (chapter 1, §3.5) is a purely indicative label, never validated against the entry's actual content — only the dropdown-menu filtering on the Bayesian Calibration page (chapter 9, §7) relies on it.
- **New since the previous version of this document** : loading a calibration library from a `.json` file (`core/covariance_library_io.py::import_covariance_library`, chapter 1, §3.5.1) only validates the presence of the `"name"` field on each entry — neither the mutual consistency of the `grid`/`mean`/`covariance` lengths (an $N \times N$ covariance should correspond to a grid of N points) nor the plausibility of the numeric values themselves is checked. A corrupted or hand-edited file can therefore introduce an inconsistent entry into the project with no immediate error, the error only arising once that entry is actually consumed downstream (for example by `core/calib_library_adapter.py`, chapter 9, §7.4).

Chapter 2 — Quasi-Static (QS) and specimen geometries

- On the QS tab, the `comp_check`/`tens_check` flags passed to `process_qs()` depend only on the chosen kinematic mode (Compression/Tension/Notched/Shear), never on the specimen geometry actually configured (`project.geometry`) — unlike the SHPB and Direct Impact tabs, which do derive these flags from the geometry. As a result, the Square/Rectangular and Flat geometries appear to be practically unreachable from the QS tab (the circular cross-section $\pi(D/2)^2$ is used in every case).
- Still on the QS tab, `shear_check` is never passed through : only simple shear (factor 1) is reachable, with the Hat-shaped and Double shear geometries remaining inaccessible from this tab.
- `notch_angle` is declared with the unit "deg" in `core/models.py`, but used directly in `math.cos()` (which expects radians) with no visible conversion in the code, including on the Monte Carlo draw side.
- Several geometric fields declared in `Specimen` (`total_length`, `H3`, `R3`, `head_width`, etc.) are never read by `core/qs_processing.py`, even though they are indeed used by the SHPB and Direct Impact pipelines.
- None of the four QS kinematics recomputes the resisting cross-section as a function of the current strain (no necking-type correction) ; only the nominal-to-true conversion is applied.
- The adiabaticity assumption of the temperature model (Taylor-Quinney) is not explicitly justified in the code.

Chapter 3 — SHPB (two bars)

- Explicit warning from the `process_shpb.py` module : the engine has not been independently validated on real data for the full option matrix, only tested on synthetic signals in the simplest combination.

- The punching correction (`punch_check`) always applies the compression true-stress/true-strain formula, including for a tensile test — whereas the path without this correction correctly branches between compression and tension.
- For shear, the punching correction introduces a division of the strain by 2 that is absent from the path without this correction.
- The friction correction in practice applies a factor of m^2 (rather than m) to the final true stress, as a result of the order of operations, whereas the nominal stress is reduced only by a factor of m.
- The inertia correction for a notched specimen uses a fixed angle of 45°, independent of the notch angle actually configured.
- The viscoelastic branch of the calculation (`disp_visco`) implicitly assumes that the two bars (incident/transmitted) share the same rheology, with this assumption not flagged as deliberate in the code.
- The frequency-domain complex modulus of the viscoelastic branch is indexed directly sample by sample in time rather than via a full Fourier/inverse-Fourier transform — a non-standard treatment, to be validated.
- The overall test mode (`type_check`) is determined by the "Test mode" setting on the Test Setup page, not per tab ; a guard (`_require_notched_geometry`) warns of the risk of processing a test with the geometry of a different mode.
- The toe correction now re-trims each Monte Carlo draw individually (not only the aggregated curve), but the `toe_n_removed` counter used to realign a confinement/laser sensor left as a raw signal is a median over the set of draws, not an exact per-draw count — a reasonable approximation as long as the draw-to-draw variation in the number of points removed remains small.

Chapter 4 — Direct Impact (DI)

- Explicit warning from the `process_di.py` module : only the simplest combination of options has been tested with real/synthetic signals ; the full cross-product of heat × platen × punching × friction × inertia × viscoelastic bar × 4 kinematics has not been independently revalidated.
- When `plat_check=True` and `heat_check=False`, the modulus and wave speed used revert to the bar's own values while the density remains the platen's : a potentially inconsistent (E, ρ, C_0) triplet.
- For a notched specimen (`type_check==3`), the `displacement_mm` field, by a strict tracking of the code's units, appears to be expressed in meters despite its name and despite an adjacent comment asserting the opposite.
- The viscoelastic recalculation of the shear branch systematically uses the "hat-shaped" geometry, even for double or simple shear — with no practical consequence as long as this branch remains inaccessible from the interface (see the next point).
- The true stress recomputed after the punching correction always uses the "compression" formula, including in tension.

- The friction correction subtracts the friction contribution a second time in the notched branch only, equivalent to applying m^2 instead of m.
- The `disp_visco` option (viscoelastic branch) is currently never enabled from the interface's Direct Impact tab, although the feature exists and is operational on the code side.
- `SG_unc_d` and `f_inc_velocity`, although defined as per-draw Monte Carlo parameters on the `process_di()` side, systematically receive a constant value from the DI interface : their own uncertainty is therefore never sampled.

Chapter 5 — Confinement (quasi-oedometric)

- Neither the linear nor the non-linear cell model imposes a bound ("clamp") on the calibration range : beyond $[0, \varepsilon_{ref}]$, the blending between regimes becomes a linear extrapolation that is not flagged as such in the code.
- The non-linear calibration polynomial is forced through the origin (constant term eliminated), which improves physical consistency but degrades the fit quality relative to an unconstrained polynomial.
- The fixed-point iteration of `radial_strain()` systematically performs 50 iterations with no convergence test at all.
- The functions `radial_strain()`, `volumetric_strain()`, `equivalent_strain()` use an inverted sign convention (compression negative) relative to the rest of the application (compression positive) ; the caller must explicitly negotiate this sign change. This inversion does not apply to the stresses (σ_r, p, q).
- The equivalent strain combines a true (logarithmic) axial strain with an engineering radial strain — a combination carried over as-is from the original code, with no further documented justification.
- In the non-linear model, no Monte Carlo uncertainty equivalent to the 2 % applied to the linear model's cell modulus is modeled on the calibration curves themselves.
- All functions in `core/confinement.py` silently truncate their input arrays to the common minimum length, with no warning.
- On SHPB and Direct Impact, the actual computation of confinement quantities inside the Monte Carlo loop is enabled only if the cell sensor additionally has its "Include this cut in the Monte-Carlo loop" checkbox ticked ; otherwise the application silently falls back (with an explicit but optional-to-read message) to the synthetic fallback envelope.
- On the Quasi-Static tab, the confinement cell signal is never varied across Monte Carlo draws (a single, fixed signal shared by every draw) even when the axial stress/strain is ; the resulting uncertainty on confinement quantities there is therefore real but partial.
- Quantities derived from laser velocimetry (`axial_stress_laser`, `pressure_laser`, `deviator_laser`) are always computed by the synthetic fallback envelope, never inside the Monte Carlo loop, regardless of the availability of real data for the other confinement quantities of the same result.
- When the confinement/laser sensor is not re-trimmed per draw (raw signal), `_refresh_confinement()` now corrects the leading offset that a toe correction applied after the fact

would otherwise introduce, but only that offset : a sampling-rate mismatch between this sensor and the stress-strain curve is, for its part, neither detected nor corrected (position-to-position truncation to the minimum length, with no common time base).

Chapter 6 — Digital image correlation (DIC)

- Tracking is rigid-translation per subset (no deformable subset) : suitable for moderate, predominantly uniaxial strains, not for a strongly heterogeneous field.
- The local strain calculation (`_grid_strain`) is a small-strain first-order gradient, not a rigorous large-strain DIC calculation.
- The nominal-to-true strain conversion ($\varepsilon = \ln(1+e)$) is exact only in 1D uniaxial conditions and is never applied to the shear component.
- The dynamic force calculation on the ROI assumes a constant cross-section (no necking).
- No automatic camera/gauge synchronization is performed : time alignment is manual.
- The DIC uncertainty model is predictive (built from image noise and correlation quality), not empirically calibrated against a ”zero” test or a known synthetic displacement ; it does not cover optical distortion, out-of-plane motion, or the sub-pixel interpolation bias of the parabolic refinement.
- The three DIC uncertainty terms are combined in quadrature assuming their statistical independence, and the propagation to strain/velocity/force assumes independent errors between components that in reality share the same underlying displacement noise.
- The effective number of points used to correct for spatial averaging of noise over the ROI relies on a crude approximation of the spatial correlation length.

Chapter 7 — Elastic dispersion

- Explicit warning at the top of the module : the port could not be re-tested against a known reference case.
- No guard exists on the convergence of the root search (`sympy.nsolve`) in the high-frequency elastic regime, unlike the viscoelastic path, which has a try/except fallback.
- No time windowing (apodization) is applied before the FFT of the pulse, and the root search itself has no divergence guard of its own.
- An optional bound on the correction's own bandwidth (`_bandwidth_rolloff_weight`, enabled by default, adjustable threshold — 0.6 by default) now limits the amplification of high-frequency noise by capping the correction's gain beyond an adjustable normalized frequency ; it remains distinct from the optional low-pass filter applied to the raw signal (`Bar.lowpass_enabled`), and its default value is a pragmatic heuristic, not a demonstrated optimum.
- The exact physical origin of the amplitude correction factors M_1/M_2 is not documented in the code ; their form is consistent with a Bancroft/Davies-type factor, with no explicit confirmation.

- The time positioning of the reflected/transmitted windows uses a constant wave speed C_0 even in the corrected elastic regime — only the spectral correction itself is dispersive.
- The `n=n` parameter of `np.fft.irfft` is a critical, non-optional fix for odd-length windows (the common case) : omitting it would produce a numerically wrong reconstructed signal over its entire length, not merely a shifted one.

Chapter 8 — Viscoelastic dispersion, Bayesian calibration, damping

- Only the Standard Linear Solid (SLS) model is currently wired into the actual dispersion correction on the "Dispersion Corrections" page ; the Kelvin-Voigt, Generalized Maxwell (2-branch), and Burgers models are available only for calibration.
- The viscoelastic-bar Monte Carlo expansion (`core/models.py::Bar.expand_viscoelastic`) remains hard-wired to the SLS model, independent of the four rheological models available for calibration.
- The SLS model formula corresponds, after an algebraic check documented in the code, to the "spring in series with a Kelvin-Voigt cell" topology.
- The Burgers model has an exactly zero static modulus, which requires special handling of the zero frequency (evaluated at the first non-zero frequency, with only the real part kept) to avoid a complex-to-real conversion crash.
- The viscoelastic Pochhammer-Chree root search has a fallback formulation for when the primary one fails to converge ; in that case, the attenuation is not recomputed but carried over from the previous frequency bin, with no physical justification detailed in the code.
- `core/visco_calibration.py` carries an explicit warning of no independent validation against real experimental data or the original application's output.
- **Since the previous version of this document**, this root solver has been extracted into a new shared module, `core/pochhammer_solver.py` (chapter 8, §8.3.2), and rewritten from a symbolic `sympy.nsolve` search to a compiled complex-secant iteration (`scipy.special.jv`) — a claimed performance gain of two to three orders of magnitude per call, with no change to the physics modeled and no fix to any numerical bug identified in the previous code ; the only new failure mode (floating-point range overflow for a point very far outside the domain) is handled by the pre-existing fallback mechanism, with no observable difference in behavior for the user.
- The posterior correlation matrix between calibrated parameters indicates a non-identifiability of the measurement (two parameters of which only a combination is constrained by the data), not a genuine physical coupling — and no amount of additional search budget can resolve it.
- The final predicted pulse displayed is not recomputed at the posterior mean of the parameters (risk of very slow solver convergence in that region), but reused from the best point actually evaluated during the Bayesian search.
- In `core/decay.py`, the unit annotated for the full damping factor ("rad/m") does not dimensionally match the formula used ($\zeta \cdot 2\pi f_n$, which gives rad/s for f_n in Hz) — an inconsistency noted with no further conversion visible in the surrounding code.

- Still in `core/decay.py`, the half-power point search scans the entire spectrum from index zero (not just a neighborhood of the peak), with no guard against a noisy or multi-peak spectrum.
- The same optional bandwidth bound (chapter 7) applies to the viscoelastic correction, but with a reference speed specific to each branch (incident/reflected/transmitted), since a viscoelastic medium has no single constant wave speed — this choice is not exposed or separately adjustable from the elastic case in the interface.

Chapter 9 — Standalone tools

- The CPG calculator (`core/cpg_calc.py`) is a literal port, not independently revalidated ; its temperature uncertainty propagation directly uses the temperature's relative standard deviation (σ_T/T), which does not correspond to a rigorous propagation through the term $[1 + SG_T(T - T_{ref})]$ actually present in the formula — the uncertainty on the gauge temperature coefficient SG_T itself is, moreover, not taken into account.
- The CPG calculator's optional transverse-sensitivity correction (chapter 9, §1.7) is derived for a single axial gauge under uniaxial stress, and applied the same way regardless of the selected bridge type (Quarter/Half/Full) — a first-order approximation, since the reference technical note (Vishay TN-509) explicitly covers only the case of a single gauge or a two-gauge rosette, not that of a wired bridge with electrical Poisson compensation.
- The CPG calculator's optional thermal-output correction (chapter 9, §1.8) acts on the strain (an additive offset), not on the CPG coefficient itself (a multiplicative factor) : it is therefore not folded into the value synchronized to `SG_coeff`, nor applied automatically anywhere else in the application — the result (apparent strain and equivalent voltage) is displayed for information only, to be applied manually.
- The trigger-to-exposure-pulse correspondence, in the camera/video synchronization tool, is carried out in sample-index space rather than in real time, which implicitly assumes regular sampling of the clock file, never explicitly verified by the code.
- Still in this tool, peak detection can over-detect on a noisy signal when no expected period is supplied ; a detected-peak count that differs from the number of loaded frames never blocks the computation, only a warning displayed to the user.
- In the batch analysis (`core/batch_analysis.py`), the isothermal correction treats the last point of each curve asymmetrically relative to the others (reusing the second-to-last raw value), a behavior not explained in the source-code comments.
- Still in the batch analysis, resampling onto a common grid (the "target_x" method) uses a search for the last point below the target (no linear interpolation), with a special-case fallback to index 1 (not 0) when no X point lies below the target — not documented in the source code.
- The threshold for switching between the monotonic ("target_x") and non-monotonic ("index_fraction") resampling methods, set at 15 % relative backtracking, is an empirical choice made in the code, not derived from a physical criterion.
- The Bayesian calibration of constitutive laws (port of MAMBA, chapter 9, §7) has never been run against a real PyMC/PyTensor/arviz during its development (no network access and no library installed in the development sandbox) — a manual numerical-validation

test on the user's machine remains required before any use in real conditions (chapter 9, §7.5, §7.10). This finding remains **unchanged** since the previous version of this document.

- **Updated since the previous version of this document** : the wiring of the `nuts_sampler="nutpie"` setting (chosen specifically so that a standalone PyInstaller executable would not need an embedded C/C++ compiler) through to the actual `pm.sample(...)` call was previously incomplete — a correctly set `settings["nuts_sampler"]` was in fact never passed to `sample_model()`. This wiring is now fixed (chapter 9, §7.6, §7.10) ; the setting, however, remains unexposed on the interface's "Sampler settings" card (§7.9).
- **Updated since the previous version of this document** : the question of the PyInstaller packaging of this feature, explicitly left open in the previous version, has since been addressed (choice of `nutpie`, `PYTENSOR_FLAGS=cxx=`, a dedicated `hydra.spec` — see chapter 9, §7.10, and `BUILD_WINDOWS.md`) but remains unverified end to end : none of these changes could be tested with a real PyMC/PyTensor/numba/PyInstaller in the environment where they were prepared: an actual compilation test and a functional test on a Windows machine remain required before any distribution.
- In this same feature, PyMC offers no real cancellation while sampling is in progress (the Stop button only takes effect before `pm.sample` starts) and the progress bar is updated only before/after sampling, never during (chapter 9, §7.5, §7.10).
- **New since the previous version of this document** : domain guards (`core/calib_equation.py`, §7.2) now clip a real evaluation point that falls outside the equation's mathematical domain (e.g., the square root of a negative number) rather than letting sampling crash or silently produce a `NaN`. The associated diagnostic (`_domain_guard_report_for_tests`, §7.10) is explicitly documented as a "best-effort" check before sampling : it evaluates the equation once per test with one representative value per parameter, not every actual MCMC draw — the absence of a warning in Result summary therefore does not guarantee that no clipping occurred during actual sampling.
- Still in this feature, the "Validation" card falls back to the population hyper-mean for a hierarchical parameter on a validation test genuinely held out of the fit, ignoring that parameter's own cross-test spread — the band displayed there is therefore narrower than the true predictive uncertainty (chapter 9, §7.7, §7.9, §7.10).
- The prior kind (fixed / pooled / hierarchical) of a Bayesian-calibration free parameter applies globally to all tests of a given run, with no partial mixing possible within a single run (chapter 9, §7.10).

Chapter 10 — Crack analysis & wave speed (Crack & Wave Analysis)

- X-FEM-DIC (`core/xfem_dic.py`) is the only K_I/K_{II} computation method exposed in the "Crack analysis" tab's interface ; a standard DIC ring fit with optional T-stress extension (`core/crack_k1c.py`) and an interaction integral (`core/crack_interaction_integral.py`) remain available in the code, tested, for programmatic use — see chapter 10, §5.
- Two uncertainty paradigms coexist without being directly comparable term by term : the Monte Carlo draw (chapter 1) and the asymptotic least-squares covariance (X-FEM-DIC and the ring fit), computed once at the fitted point by local linearization rather than by resampling ; the interaction integral falls into neither, since it is not a least-squares fit — its confidence is assessed via a sub-domain independence diagnostic (inner/outer half).

- The formal uncertainty of X-FEM-DIC and of the ring fit is that of the fitted model, not an end-to-end DIC measurement uncertainty ; the independent-residuals assumption underlying any least-squares covariance is not explicitly verified (a correlated systematic error upstream in the u/v field is not covered).
- No formal uncertainty is computed on the elastic wave speed (`core/elastic_wave_speed.py`) — only an indicative R^2 is reported, unlike the treatment of the crack tip.
- The automatic tip-position detection for X-FEM-DIC (`auto_detect_tip=True`) is a grid/ greedy search, not a global optimization ; it can settle on an incorrect local minimum on an isolated frame, despite each frame being independently anchored to the tip-tracking estimate.
- The opening-extent position uncertainty (`opening_extent_position_uncertainty_px`) silently underestimates its own contribution as soon as a perturbation (threshold, gap tolerance, sampling offset) fails to detect a front at all — explicitly documented in the code rather than hidden.
- X-FEM-DIC and the ring fit are deliberately "static first" : neither models any dependence on the crack propagation speed (a Freund-type dynamic kernel).
- X-FEM-DIC's crack-face jump is unshifted (the $H(x, y)$ form evaluated at the query point, not the standard shifted form), and the node-enrichment classification uses a local tolerance band (1.5× the element size) rather than an exact geometric intersection.
- The ring fit's optional T-stress term (`core/crack_k1c.py`) is not a systematic improvement of K_I/K_{II} themselves : it is a model-consistency check (an abnormally large T-stress signals too large an outer ring radius or a non-uniform background field), not an automatic correction.
- The standard closed-form formula relating the interaction integral to K_I/K_{II} assumes a complete angular domain around the tip, an assumption broken non-uniformly between modes by the crack-flank exclusion angle (`flank_exclusion_deg`) — the interaction integral therefore normalizes each mode by the auxiliary field's self-interaction over the same (potentially truncated) domain rather than using the standard closed-form formula, which eliminates this dependence on the truncated domain.

End of document.

SUPPLEMENTS TO THE V1.0 EDITION

Note on the v1.0 edition

This edition brings the documentation to **version 1.0** of the application (the number displayed on the Welcome page, constant `APP_VERSION` of `ui/pages/page_welcome.py`). Chapters 1 to 10 and the appendix reproduce the October 2026 edition unchanged; the code changes made since are described in the four chapters that follow:

- chapter 11 describes the **Cameras Synchronisation** page, which pre-processes 1 to 8 cameras (dark/flat calibration with per-image dark scaling, alignment, pairwise photometric equalisation, ImageJ-style contrast stretch, lossless MKV/AVI/MP4 videos);
- chapter 12 supplements image correlation (affine IC-GN refinement, principal strains and maximum shear, spatial smoothing, continuous isocontours, enlarged window);
- chapter 13 supplements crack & wave analysis (ridge detection and branching, tip tracking by opening jump with wave rejection, local frame, direct K, corrected X-FEM-DIC, K_I versus speed, videos);
- chapter 14 groups the more specific changes (confined strain rates, "pre-toe" curves, embedded bar signals, laser uncertainty).

The table of contents at the beginning of the document has been regenerated from the bookmarks of the file: it gives the correct page numbers (those printed at the bottom of the pages) and lists these four chapters. Code references follow the convention of the rest of the document (`file::function`, paths relative to `shpb_app/`). As in the rest of the document, the independent-validation warnings carried by the modules should be read before any quantitative use of the results.

Contents

Supplements to the v1.0 edition (chapters 11 to 14) **193**

11 Synchronisation and pre-processing of N cameras **195**
11.1 General architecture . . . 195
11.2 Calibration . . . 196
11.2.1 Dark and flat field . . . 196
11.2.2 Per-image dark scaling . . . 196
11.2.3 Calibration of one image and negative pixels . . . 196
11.3 Alignment and crop . . . 196
11.4 Pairwise photometric equalisation . . . 197
11.5 Time stamping . . . 198
11.6 "ImageJ Auto" contrast stretch . . . 198
11.7 Lossless video exports . . . 198
11.8 File / function summary table . . . 199
11.9 Limitations, assumptions and points to check . . . 199

12 Image correlation, supplements **200**
12.1 Sub-pixel refinement by affine IC-GN . . . 200
12.1.1 Why: stripes in the strain maps . . . 200

12.1.2 Algorithm . . . 200
12.1.3 Rejections and fallback . . . 201
12.1.4 Setting . . . 201
12.2 Strain-tensor invariants and spatial smoothing . . . 201
12.3 Continuous isocontours . . . 201
12.4 Enlarged window for frame-by-frame reading (`ui/frame_popup.py`) . . . 202
12.5 File / function summary . . . 202
12.6 Limitations, assumptions and points to check . . . 202

13 Crack & wave analysis, supplements 203
13.1 Crack representation and automatic ridge detection . . . 203
13.1.1 Threshold detector (historical, `detect_cracks`) . . . 203
13.1.2 Ridge detector (default, `detect_cracks_ridge`) . . . 203
13.1.3 Branch points (`_relocate_branch_points`) . . . 204
13.1.4 Splitting into independent cracks and editing . . . 204
13.2 Tip tracking: methods and rejection of elastic waves . . . 204
13.2.1 Opening-jump profile (`crack_opening_jump_profile`) . . . 204
13.2.2 Position and speed uncertainty (option “Estimate measurement uncertainty”) . . . 205
13.2.3 Local speed . . . 205
13.3 Local crack frame and local opening (`core/crack_local_frame.py`) . . . 206
13.4 Smoothing of the tip position . . . 206
13.5 Direct K_I and K_{II} from the measured field . . . 206
13.6 X-FEM-DIC: sub-pixel sampling and K from the fitted field . . . 207
13.7 K_I versus tip speed . . . 207
13.8 Video exports of the tip analyses . . . 208
13.9 File / function summary . . . 208
13.10 Limitations, assumptions and points to check . . . 208

14 Processing and exports, supplements 210
14.1 Strain rates of the confined test . . . 210
14.2 Curves before toe correction (“pre-toe”) . . . 210
14.3 Bar signals in profiles and in the full-test export . . . 210
14.4 Axial stress from laser velocimetry: uncertainty propagation . . . 211
14.5 Setup profiles and session . . . 211
14.6 File / function summary . . . 211
14.7 Limitations, assumptions and points to check . . . 211

11. SYNCHRONISATION AND PRE-PROCESSING OF N CAMERAS ("CAMERAS SYNCHRONISATION" PAGE)

Source files: `core/camera_sync.py` (pipeline, no Qt), `ui/pages/page_camera_sync.py` (user interface). Navigation entry: "Cameras Synchronisation" (Image Processing section), built in `ui/main_window.py`.

This page replaces the stand-alone three-camera script `images_processing2.py`, and then integrates the behaviours of the script `N_cameras_images_processing.py` (pairwise photometric equalisation, levelling of the reference camera, per-camera videos, calibration exports, optional tone matching by look-up table). The number of cameras N is arbitrary from 1 to `MAX_CAMERAS` = 8; every list of the configuration (dark folders, flat-field folders, image folders, time offsets…) has exactly N items (`CameraSyncConfig.validate`). The case $N = 1$ is a case in its own right: the equalisation between cameras becomes a plain equalisation between successive images, and there is no relative alignment to compute.

11.1 General architecture

The pipeline runs in three phases, driven by three worker threads of the page (`_PrepareWorker`, `_ProcessWorker`, `_StretchWorker`):

1. **Preparation** (`prepare_calibration`): for each camera, construction of the master dark, the master flat and the gain map, with automatic rejection of outlier images (too many saturated pixels, or a mean that departs from the median by more than `dark_outlier_std_factor` standard deviations), followed by calibrated check images and diagnostics. At most `max_calibration_frames` (100 by default) dark or flat images are used, evenly sampled if the folder holds more.
2. **Interactive steps** (user-interface side, modal matplotlib windows `ImagePickDialog`, since the application ships `opencv-python-headless` and therefore has no `cv2.imshow`): a line drawn on each camera for the rotation, a registration point for the translation, and a crop rectangle per camera (the intersection of the rectangles is applied to all cameras).
3. **Processing** (`process_sequences`): for each image, in streaming mode (one image in memory at a time), calibration, a single affine resampling (rotation and translation composed), crop, then photometric equalisation, optional background homogenisation, optional denoising, 16-bit writing, time stamping, videos and contrast stretch.

8-bit inputs are brought to the 16-bit scale by a factor 257 upon reading, so that the calibration, the thresholds and the exports share one grey-level scale (MAX16 = 65535). Files are sorted in natural order. The time order of the merged sequence is derived from the time stamps (see §11.5).

Output folders (under the master folder, `master_output`):

`calibrated_cam`k	calibrated check images of camera k
`cropped_cam`k	aligned and cropped images, per camera
`all_croppedANDaligned`	merged sequence, before equalisation
`all_processed`	final merged sequence (16-bit, after equalisation); the time stamp is burned in only if requested
`all_processed_no_timestamp`	copy without burned-in text (if burning is on and the copy is requested)
`all_stretched`	contrast-stretched copy (§11.6), meant for viewing
`video`	lossless videos (§11.7)
`comparison`, `alignment_check`	check figures (before/after, alignment check)
`calibration`	master dark, gain map, negative-pixel map (§11.2)
`_work`	temporary files

Three reports accompany the images: `rotation_translation_report.json` (angles, translations, crop), `gains.json` (equalisation gains, keys `cam<c>_idx<i>` with c the camera index counted from 0) and

`timestamps.csv`. The module's `IMPROVEMENTS` list, repeated in the JSON report, enumerates the differences with the original script.

11.2 Calibration

11.2.1 Dark and flat field

The flat of each camera is turned into a gain map G once per camera (the original script recomputed it for every image). Two methods exist (`flat_method`):

- `robust` (default): hot and dead pixels repaired, low-frequency vignetting kept without a hard clip, pixel-level part (dust, response non-uniformity) limited then lightly smoothed. The bounds on the total gain are `flat_total_gain_limits` $= (0.6\,;\,1.6)$;
- `legacy`: gaussian then hard clip, as in the original script.

11.2.2 Per-image dark scaling

The dark current varies with sensor temperature: subtracting a single master dark from every image leaves uncompensated hot-pixel patterns, or over-compensates them ("black" pixels). With `dark_scaling = "per_frame"` (default), each raw image R is calibrated with a scaled dark $a\,D$, where D is the master dark and a a factor estimated image by image (`CameraCalibration.estimate_dark_scale`):

$$R \;\approx\; a\,D + S, \qquad a \;=\; \frac{\sum_{p\in\mathcal{H}} \mathrm{HP}(R)(p)\,\mathrm{HP}(D)(p)}{\sum_{p\in\mathcal{H}} \mathrm{HP}(D)(p)^2}, \qquad a \in [a_{\min}, a_{\max}] = [0.2\,;\,1.5], \tag{11.1}$$

where $\mathrm{HP}(X) = X - G_\sigma * X$ is a high-pass filter (gaussian blur, $\sigma = 3$ pixels), S the scene and $\mathcal{H}$ the set of "hot" dark pixels ($|\mathrm{HP}(D)| > \max(5\,\hat{\sigma},\,3)$, with $\hat{\sigma} = 1.4826\,\mathrm{MAD}(\mathrm{HP}(D))$, at least 50 pixels). The scene being uncorrelated with the dark-current pattern, it only adds variance to the estimate. The factor equals 1 if the dark has no usable structure; the bounds are `dark_scale_min/max`.

11.2.3 Calibration of one image and negative pixels

The calibrated image (`CameraCalibration.calibrate`) is

$$I_{\mathrm{cal}} \;=\; \mathrm{clip}_{[0,\,65535]}\!\left(\frac{R - a\,D}{G} + p \;-\; P_{\mathrm{stat}}\right), \tag{11.2}$$

where p is the "pedestal" (`calibration_pedestal`, in grey levels, 0 by default) and P_{stat} an optional static pattern. When $R - aD < 0$ (read noise of a dark pixel whose dark was slightly over-estimated), the original script replaced the value by the noise floor, which produces patches of black pixels. With a pedestal $p > 0$, the negative noise is kept (symmetric noise, no bias) instead of being clipped. The `calibration/` report contains a negative-pixel map, the fraction of negative pixels per image, the number of images beyond 0.1 %, and a message saying whether the pedestal should be set. The master dark is written as a 16-bit TIFF, the gain map as a visualisation TIFF and as an exact `.npy`.

11.3 Alignment and crop

Levelling. A line $p_1 \to p_2$ drawn by the user on the image of each camera defines a rotation. The function `leveling_angle` gives the angle, in degrees, that makes the line horizontal ($+x$); `rotation_to_apply` gives the angle that makes the line of a camera parallel to that of the reference, with $\theta = \mathrm{atan2}(\Delta y, \Delta x)$. With `align_level_reference = True` (default), the line drawn on the reference camera also serves as a baseline: the reference camera is itself levelled, and the others are aligned on it.

A single resampling. Rotation (about the image centre) and translation (d_x, d_y) are composed into one 2×3 affine matrix (`build_affine`) applied in a single `warpAffine`, cubic by default (`linear`, `cubic` or `lanczos` at choice), instead of two successive bilinear resamplings that smooth the image twice. A validity mask of the warp is tracked throughout the pipeline (pixels outside the source image are excluded from the gain estimates).

Crop. The rectangle chosen on each camera is converted to the common frame; the intersection is applied to all cameras, so that every output image has the same size and the same field of view.

11.4 Pairwise photometric equalisation

Successive images (and those of different cameras) do not have the same exposure: varying illumination, camera gain, residual vignetting. The default method (`intensity_method = "pairwise"`) is that of the script `N_cameras_images_processing.py`: one multiplicative gain per image, estimated from logarithmic offsets between pairs.

Offset between two images. For two images i, j (area-averaged thumbnails, longest side `equalize_thumb_side` $= 160$ pixels, function `frame_thumbnail`),

$$d_{ij} \;=\; \underset{p \in \Omega_{ij}}{\operatorname{median}} \Big[\ln\big(I_i(p)+1\big) - \ln\big(I_j(p)+1\big) \Big], \tag{11.3}$$

where Ω_{ij} is the set of common pixels: valid (warp mask), unsaturated (level below 0.98×65535), and above the `equalize_low_pct` percentile (1 %) in both images. A pair whose overlap is below `equalize_min_overlap_frac` (5 %) is ignored.

Linear system. The unknowns are the logarithmic offsets o_k of each image (in time order). Each pair provides an equation $o_j - o_i = d_{ij}$; a weak regularisation equation $\lambda\, o_k = 0$ ($\lambda =$ `equalize_reg` $= 10^{-2}$) fixes the global level. The sparse system is solved in the least-squares sense by `scipy.sparse.linalg.lsqr` (`solve_pairwise_gains`), and the gain of image k is

$$g_k \;=\; \mathrm{clip}\big(e^{o_k},\; g_{\min},\; g_{\max}\big), \qquad [g_{\min}, g_{\max}] = [0.6\,;\,1.6]. \tag{11.4}$$

The number of pairs grows as $M(M-1)/2$ with the number M of images: beyond `equalize_max_pairs` (40 000) a connected sparse graph replaces the complete graph (each image is linked to its W following images, plus a few anchor images; the mode used is recorded, for example `sparse_window_`W`_anchors_`K). The equalisation report records the root-mean-square of the offsets d_{ij} before and after correction, the number of images whose gain reached a bound, and a warning if so ("widen the limits if this is a real brightness change"). The gains are written to `gains.json`.

Other methods. `median_scale` (scaling on the median of the levels) and `none` (no equalisation) remain available. The reference level of a sequence can be the median, the mean or the first image (`INTENSITY_MODES`).

Tone matching by look-up table (optional). With `tone_matching = "lut"`, a grey-level look-up table is built between each image and the reference image (`match_tone_to_reference`, port of `match_with_overlap_and_slope_limit`, without the scikit-image shortcut): a coarse table built on the pixels above the first percentile in both images, gaussian-smoothed and slope-limited; if too many pixels clip, successive attempts are made with a soft-knee compression, a linear percentile rescale, then a plain median gain.

Background homogenisation (optional). `background_mode = "divide"` divides the image by a masked low-frequency surface (gaussian or robust polynomial, `background_model`), normalised to unit mean to preserve brightness (illumination is a multiplicative factor); one single global output gain, identical for all images and all cameras, replaces the former per-image min/max stretch (`output_scaling`). By

default, `background_mode = "none"` and `output_scaling = "none"`: these two operations are optional and change the image content that DIC will correlate afterwards. The `legacy` mode reproduces the original script.

Denoising (optional). `denoiser` ∈ {`none`, `bilateral`, `nlmeans`, `bm3d`, `tv`, `temporal_median`}; in the original script the denoiser's result was overwritten (`denoised = recombined`); here it is actually applied, in floating point, with a temporal median taken camera by camera. A despeckle filter is available (`median`, `morphology`, `hybrid`).

11.5 Time stamping

The instant of an image is $t = t_0 + k\,\Delta t + \delta_c$ (in nanoseconds; `timestamp_ns`, `timestamp_step` (530 by default), and `camera_offsets_ns`: offset δ_c of camera c). The order of the images in the merged sequence is that of these times; it is identical to the interleaving of the original script as long as the camera offsets remain smaller than the time step. The file `timestamps.csv` is always written. Burning the time into the pixels (bottom right, white on black, `timestamp_font_scale` = 0.6) is off by default, because it alters the image that is correlated; when it is turned on, `export_no_timestamp_copy` additionally produces the copy `all_processed_no_timestamp`.

11.6 "ImageJ Auto" contrast stretch

The final step writes into `all_stretched` a contrast-stretched copy, meant for *looking at* details (not for correlation: the windowing clips the pixels outside the window). The file `all_processed` is never modified.

Automatic window. `stretch_limits_imagej_auto` reproduces ImageJ's `ij.plugin.frame.ContrastAdjuster.autoAdjust`. The N pixels are rebinned into 256 bins between their minimum and maximum; with limit = $N/10$ and threshold = $N/5000$, the lower bound is the first bin, from the bottom, whose count exceeds *threshold*, and the upper bound the first bin, from the top, whose count exceeds *threshold*; any bin whose count exceeds *limit* is ignored, so that a dominant background peak does not pin the bound. The other modes (`STRETCH_MODES`) are `percentile` (the window saturates `stretch_saturated_pct` = 0.35 % of the pixels, half at each end, like ImageJ's "Saturated pixels" parameter), `manual` and `none`.

Application. With the window $[\ell, h]$ (`apply_stretch`):

$$x = \mathrm{clip}_{[0,1]}\!\left(\frac{I-\ell}{\max(h-\ell,\,10^{-6})}\right), \qquad I_{\mathrm{out}} = \mathrm{round}\big(65535\;x^{\gamma}\big), \tag{11.5}$$

with an exponent γ = `stretch_gamma` (1 by default) and, optionally, adaptive CLAHE equalisation (`stretch_clahe`). The scope (`stretch_scope`) is `global` (default: a single window for the whole sequence, which preserves the relative brightness between images and cameras and avoids any flicker), `per_camera` or `per_frame`. `restretch_existing` re-runs only this step on an existing `all_processed` folder, which allows tuning the window without reprocessing the sequence; the report records the window used.

11.7 Lossless video exports

The sequences are also written as video, as MKV, AVI and MP4 at choice (`encode_videos`), for the merged sequence and, if requested, for each camera (`export_video_per_camera`); the base name is `video_filename` (name without extension, `video_stem`). Images are sent to `ffmpeg` as a raw stream (`rawvideo`), without temporary files.

- **MKV and AVI** (`create_16bit_video`): FFV1 codec, `gray16le`: lossless and bit-exact, 16-bit.

- **MP4** (`create_mp4_lossless`): the MP4 container accepts neither FFV1 nor 16-bit video; the encoding is therefore lossless H.264 (`qp 0`, High 4:4:4 Predictive profile, full range) on images first rescaled to 8 bits (`mp4_bit_depth` = 8, default) or 10 bits: this rescaling is the only loss, the encoding of the rescaled images being exact. The `yuv444p` format preserves odd image sizes.

11.8 File / function summary table

Function / class	Role
`prepare_calibration`	darks, flats, gain maps, check images, diagnostics
`CameraCalibration.calibrate`	calibration of one image (scaled dark, gain, pedestal)
`CameraCalibration.estimate_dark_scale`	dark factor a, per image
`leveling_angle, rotation_to_apply, build_affine`	geometry: levelling, relative rotation, single warp
`frame_thumbnail, solve_pairwise_gains`	pairwise photometric equalisation
`match_tone_to_reference, apply_lut_uint16`	tone matching by look-up table
`process_sequences`	complete streaming processing
`stretch_limits_imagej_auto, apply_stretch, stretch_sequence, restretch_existing`	contrast stretch
`create_16bit_video, create_mp4_lossless, encode_videos`	video exports

11.9 Limitations, assumptions and points to check

- The pairwise equalisation assumes a purely multiplicative photometric transformation ($I_i \propto I_j$ on the common pixels); a non-linear change of contrast is not corrected (that is the partial role of tone matching by look-up table). The gains are bounded (0.6 to 1.6): a stronger change of illumination is clipped, not corrected.
- The look-up table (`tone_matching = "lut"`) changes the relation between grey level and luminance: use it only for visual rendering, not before a quantitative image correlation.
- The estimate of the dark factor a relies on the hot pixels of the master dark; in their absence (< 50 pixels), $a = 1$.
- The pedestal p is a constant additive offset: it must be subtracted if absolute levels matter (for example for a conversion to luminance).
- MP4 is "lossless" with respect to the images rescaled to 8 or 10 bits, not with respect to the 16-bit images; only MKV/AVI (FFV1) are exact at 16 bits.
- The contrast stretch clips the pixels outside the window: `all_stretched` must not be used as correlation data.
- The automated tests of this page (core and interface) run with a stand-in for PySide6: the Qt components themselves (matplotlib picking windows, layouts) remain to be validated on the real application.

12. IMAGE CORRELATION: SUPPLEMENTS OF THE V1.0 EDITION

This chapter supplements chapter 6 (§6.4, tracking algorithm, and §6.10, visual rendering). The following sections describe what has been added or changed since the October 2026 edition in `core/dic_processing.py`, `core/dic_icgn.py`, `core/dic_fields.py`, `core/dic_render.py` and `ui/pages/page_image_correlation.py`.

12.1 Sub-pixel refinement by affine IC-GN

12.1.1 Why: stripes in the strain maps

The basic tracker (chapter 6 §6.4.3) locates each subset as a rigid, translated window by normalised cross-correlation (`cv2.matchTemplate`) and a 3-point parabolic refinement of the peak. Two systematic errors result, which show up as **vertical or horizontal stripes** in the strain maps:

- *peak locking*: the parabola fitted on an integer-shift correlation map is biased towards integer (and half-integer) displacements, by up to about 0.1–0.2 pixel, with a period of 1 pixel of displacement. The displacement varies smoothly over the specimen, so the bias oscillates along it; since strain is the *derivative* of the displacement, the oscillation is amplified by the local displacement gradient, so that the stripes follow the iso-displacement lines and get closer together as the strain grows;
- the rigid window cannot follow the deformation of its own subset (a 5 % strain over 21 pixels is a 1-pixel stretch), which adds an error depending on the position and on the speckle pattern of each subset.

12.1.2 Algorithm

The refinement starts from the integer + parabolic estimate of the correlation peak, then adjusts the displacement *and its first gradient* inside each subset: it is the inverse-compositional Gauss-Newton (IC-GN) algorithm with a first-order (affine) shape function and a ZNSSD criterion (zero-normalised sum of squared differences) used by the common 2D DIC packages (Ncorr, VIC-2D). Let ξ, η be the local coordinates of the subset pixels relative to its centre, and

$$\mathbf{p} = (u,\ u_x,\ u_y,\ v,\ v_x,\ v_y)^{\top}, \qquad W(\xi,\eta;\mathbf{p}) = \begin{pmatrix} \xi + u + u_x\xi + u_y\eta \\ \eta + v + v_x\xi + v_y\eta \end{pmatrix}. \tag{12.1}$$

The reference subset is normalised, $\tilde{T} = (T - \bar{T})/\|T - \bar{T}\|$, and so is the current subset, $\tilde{F} = (F\circ W - \overline{F\circ W})/\|F\circ W - \overline{F\circ W}\|$. The criterion is

$$C(\mathbf{p}) = \sum_{\xi,\eta} \big(\tilde{F}(W(\xi,\eta;\mathbf{p})) - \tilde{T}(\xi,\eta)\big)^2, \qquad \mathrm{ZNCC} = 1 - \tfrac{1}{2}\,C. \tag{12.2}$$

Since the method is inverse-compositional, everything that depends only on the reference is computed *once* when the model is created (`prepare_template`): the steepest-descent images $\mathrm{SD} = \big[g_x,\ g_x\xi,\ g_x\eta,\ g_y,\ g_y\xi,\ g_y\eta\big]/\|T - \bar{T}\|$ (gradient g by central differences, with border replication) and the inverse of the approximate Hessian $H = \mathrm{SD}^{\top}\mathrm{SD}$. At each iteration (`refine_icgn`):

1. the current subset is resampled by **cubic** interpolation (`cv2.remap`, `INTER_CUBIC`, replicated border) through the current shape W;
2. the residual $r = \tilde{F} - \tilde{T}$ gives the increment $\Delta\mathbf{p} = H^{-1}\,\mathrm{SD}^{\top} r$;
3. the shape function is updated by inverse composition, $W \leftarrow W \circ \Delta W^{-1}$ (product of homogeneous 3×3 affine matrices).

The iteration stops when $|\Delta u|, |\Delta v| < \mathrm{tol}$ (10^{-3} pixel) and the gradient increments are below 0.1 tol, or after `max_iter` = 20 iterations.

12.1.3 Rejections and fallback

`refine_icgn` returns $(u, v, \text{ZNCC}, \text{ok})$; `ok = False` flags an implausible result, in which case the caller keeps the initial estimate (u_0, v_0):

- flat subset (no texture: zero standard deviation, tolerance `_FLAT_EPS`) or ill-conditioned Hessian ($\text{cond}(H) > 10^{12}$) at model creation (the model is then not created);
- flat current subset, non-invertible matrix, or non-finite value during the iterations;
- refinement that drifts by more than `max_shift_px` $= 2$ pixels from the starting estimate.

The batched version (`IcgnBatch`, `refine_batch`) stacks the models of m subsets of the same size and does the same computations with a single call to `cv2.remap` and a few `einsum` calls per iteration for the whole batch (an order of magnitude faster than a Python loop); the numpy/OpenCV calls release the GIL, which allows spreading batches over several threads.

12.1.4 Setting

The field `DICSettings.subpixel_method` (`core/project.py`) is `"icgn"` (application default) or `"parabolic"` (rigid tracking of chapter 6, faster). The template of each subset is always cut from the *reference* image (never re-cut from a later image), so that the per-step error does not accumulate as drift; `tracking_mode` (`fixed`/`moving`) only changes the centre of the search window (§6.4.4). IC-GN costs several times the run time of the parabolic mode.

12.2 Strain-tensor invariants and spatial smoothing

The components $\varepsilon_{xx}, \varepsilon_{yy}, \varepsilon_{xy}$ depend on the camera axes: a crack at 35° from the horizontal appears split between the three components. The invariants of the plane tensor are independent of orientation and highlight a localisation band whatever its direction. With the conventions of `_grid_strain` and `von_mises_strain` (`exy` is the *tensor* shear strain, half of the engineering shear γ_{xy}),

$$c = \frac{\varepsilon_{xx} + \varepsilon_{yy}}{2}, \qquad r = \sqrt{\left(\frac{\varepsilon_{xx} - \varepsilon_{yy}}{2}\right)^2 + \varepsilon_{xy}^2}, \qquad \varepsilon_1 = c + r, \quad \varepsilon_2 = c - r, \quad \gamma_{\max} = \frac{\varepsilon_1 - \varepsilon_2}{2} = r. \tag{12.3}$$

ε_1 and ε_2 are the maximum and minimum in-plane principal strains (`principal_strains`); $\gamma_{\max} = r$ is the radius of Mohr's circle (`max_shear_strain`, tensor convention, consistent with `exy`; the engineering maximum shear is $2r$). These fields are added to $\varepsilon_{xx}, \varepsilon_{yy}, \varepsilon_{xy}$ and ε_{eq} in the list of displayable fields of the Image Correlation page (keys `e1`, `e2`, `gmax`) and of the Crack & Wave Analysis page.

Spatial smoothing. The displayed field can be smoothed by a gaussian filter (`smooth_nan_gaussian`, σ in mesh points; 0 = no smoothing; "Spatial smoothing" field of the page). The filter uses *normalised convolution*: lost or excluded points do not contribute to the average of their neighbours (the weights are renormalised, which also handles the grid edges) and are not filled in. The strain components are smoothed *first*; the invariants $(\varepsilon_1, \varepsilon_2, \gamma_{\max}, \varepsilon_{eq})$ are then computed from the smoothed components (`smoothed_strain_fields`), because these invariants are non-linear (square roots): smoothing them directly would retain the positive bias that measurement noise gives them. Smoothing only affects the display: the CSV export of the results remains unsmoothed (all fields, all processed frames).

12.3 Continuous isocontours

The rendering of a field by `tricontourf` on a Delaunay triangulation of the *current* positions of the points (§6.10: no fixed-connectivity quadrilaterals that fold over when the specimen necks) is shared by the live display of the Image Correlation page, the annotated video export (`core/dic_video_export.py`), and the video exports of the Crack & Wave Analysis page (`core/tip_video_export.py`, `core/xfem_video_export.py`). A point with no reliable position (lost track, masked, rejected by `correlation_threshold` or `max_neighbor_deviation_px`) is simply left out of the triangulation, with no invented filler value.

12.4 Enlarged window for frame-by-frame reading (`ui/frame_popup.py`)

Several views (correlation results, the "Verify visually" view of tip detection, X-FEM/DIC isocontours, crack tracing) can be opened in a large window ("Pop out" button). The page's canvas is *moved* into it (and put back on closing), so that everything the page draws and each of its click handlers keep working. The window adds a transport bar driving the page's frame slider: Play/Pause, Reset, "Go to frame", speed (frames per second), loop; the space bar toggles playback, the left/right arrows step to the previous/next frame. The frame numbers displayed are counted from 1.

12.5 File / function summary

Function / class	Role
`dic_icgn.prepare_template`	reference model: $\tilde{T}$, SD, H^{-1}
`dic_icgn.refine_icgn, IcgnBatch, refine_batch`	IC-GN refinement of one subset / of a batch
`dic_processing.run_dic (subpixel_method)`	`icgn` / `parabolic` choice
`dic_fields.principal_strains, max_shear_strain`	$\varepsilon_1, \varepsilon_2, \gamma_{\max}$
`dic_fields.smooth_nan_gaussian, smoothed_strain_fields`	smoothing without filling, invariants of the smoothed components
`dic_render`	isocontours on a Delaunay triangulation
`frame_popup.make_popout_button`	enlarged window with transport bar

12.6 Limitations, assumptions and points to check

- The shape function is affine: a second-order strain gradient inside the subset (strongly curved field) is not represented. On a subset straddling an open crack, a displacement discontinuity is representable neither by the rigid nor by the affine shape; this is what motivates X-FEM-DIC (chapter 10 §10.4).
- The 2-pixel rejection from the initial estimate assumes that the integer + parabolic estimate is correct to better than 2 pixels; otherwise (`ok = False`) the subset is kept at the parabolic estimate.
- The smoothed invariants are derived from components smoothed on a regular mesh of points; strong smoothing ($\sigma \gtrsim 2$ points) widens localisation bands and lowers their peak.

13. CRACK & WAVE ANALYSIS: SUPPLEMENTS OF THE V1.0 EDITION

This chapter supplements chapter 10. It describes the modules and options added to the Crack & Wave Analysis page since the October 2026 edition: automatic ridge detection and handling of branches, crack-tip detection by opening jump and rejection of waves, local crack frame, stress-intensity factors computed directly from the measured field, the fix to X-FEM-DIC sub-pixel sampling and the computation of K from the fitted field, and K_I versus tip-speed curves. Source files: `core/crack_skeleton.py`, `ui/crack_skeleton_picker.py`, `core/crack_tip_tracking.py`, `core/crack_local_frame.py`, `core/direct_k.py`, `core/tip_speed.py`, `core/xfem_dic.py`, `core/tip_video_export.py`, `core/xfem_video_export.py`, `ui/pages/page_crack_wave_analysis.py`.

13.1 Crack representation and automatic ridge detection

A crack (`CrackPath`) is a *tree* of (x, y) nodes (pixels of the reference image) joined by edges, not a polyline: it can branch. The user can trace it by hand (left click: adds a node from the active node; right click on a node: makes it active, the next click opens a new branch), as many cracks as needed. The "Auto-detect path(s)" button proposes the cracks from the displayed field. Two detectors exist.

13.1.1 Threshold detector (historical, `detect_cracks`)

It thresholds the strain field, thins the binary mask, then walks the skeleton into a graph (Ramer-Douglas-Peucker simplification). On a real DIC field it fails in three ways: speckle noise is as strong as the weak branches, so a threshold low enough to keep a branch lets noise blobs in, while one high enough to reject noise cuts the branches; thinning a thick, ragged band produces dozens of fake branches (spurs); a band several cells wide at its trunk and one cell wide at its tips has no single "right" threshold.

13.1.2 Ridge detector (default, `detect_cracks_ridge`)

A crack is a *ridge*: a thin line brighter than both its sides. The detector searches for it as follows:

1. **Multi-scale ridge response.** For each scale s (in grid cells, derived from real widths of 7, 11 and 17 pixels, hence independent of the correlation mesh step), the smoothed field f_s has a Hessian whose smallest eigenvalue is $\lambda_{\min} = \frac{1}{2}(h_{xx} + h_{yy}) - \sqrt{\frac{1}{4}(h_{xx} - h_{yy})^2 + h_{xy}^2}$. The ridge strength is

$$R = \max_s \left[\max(0,\, -\lambda_{\min}(f_s))\; s^2 \right], \tag{13.1}$$

   scale-normalised: strong on a line of any width, weak on a noise blob or a smooth gradient. Points outside the correlated domain are filled with the median (no artificial edge) then forced to 0; the response next to a hole or the edge of the region of interest is set to zero.
2. **Automatic threshold.** The threshold on R is chosen by Otsu's method on the positive ridge strengths; `sensitivity` > 1 keeps weaker ridges (more branches, more noise). On a pure-noise field Otsu still finds a threshold: a minimum ridge contrast (`_MIN_RIDGE_CONTRAST` $= 1.6$ times the median ridge strength) is required for a detection to be retained. A percentile can also be imposed.
3. **Hysteresis.** Ridge pixels exceeding `low_ratio` $\times$ threshold (0.85) and touching a sure pixel are kept: a fading branch stays attached to its trunk.
4. **Thinning and graph clean-up.** Thinning (scikit-image), removal of staircase corner pixels, pruning of spurs shorter than `spur_length_px` (by default three cells, at least 21 pixels) repeated as long as a new spur appears, contraction of the small cycles that a thinned crossing leaves, merging of clusters of junction pixels into a single branch point, and breaking of the remaining cycles by removing the longest edge of each (minimum spanning forest: a crack is a tree).

5. **Dense geometry.** Each chain between two junctions or ends becomes a series of nodes spaced by `node_spacing_px` (one cell, at least 5 pixels), gaussian-smoothed along the arc length (ends pinned), then re-centred on the ridge (`_snap_to_ridge`): each interior node is moved along its normal towards the strongest ridge response within ±`reach_px`.
6. **Root.** Node 0 must be a leaf of the tree, chosen as the most likely origin of the crack (`_root_at_origin_leaf`): the tip-tracking step starts from this node.

13.1.3 Branch points (`_relocate_branch_points`)

Thinning places a fork where the two bands are far enough apart for the medial axis to split, i.e. downstream of the real branch point (by about half the parent band's width divided by tan of the fork angle, i.e. 10 to 20 pixels). Every three-arm branch point is therefore moved:

- for a **T** fork, the two most collinear arms form the through path; the branch is the line fitted (by PCA, on the part where it is clearly separate, at arc distances $d_1 = 12$ to $d_2 = 40$ pixels from the old node, where it is straight and no longer "hooked" towards the parent band), extended backwards; the new node is its intersection with the through path, searched up to $s_{\max} = 50$ pixels behind the old one;
- for a **Y** fork (two daughters of similar strength leaving the trunk at a modest angle), the daughters' straight runs lie too far out (they curve): the apex is read on the strain profile across the fork (`_split_apex`).

A manual move is possible (`move_branch_point`): the user clicks where the fork really is; the attached cracks stay attached (`_snap_attached_ends`).

13.1.4 Splitting into independent cracks and editing

`split_into_branches` turns *one* tree into several non-branching cracks: the main one from the root, plus a new crack *per branch*, starting at the branch point (hence always attached to its parent). With `master_first_crack`, the first crack of each tree is a "master" crack that ends at its first branching, where all the arms become independent cracks. The manual editing operations are: correction of the path of a crack by forcing it through a clicked point (`reroute_crack_through_point`, neighbourhood re-smoothed and evenly spaced, ends attached to another crack followed), choice of the crack a click is meant for (`choose_crack_for_click`), attaching a crack onto another (`set_crack_parent`), computation of the parentage (`crack_parents`: child index → (parent index, branch point)), search of the junction nearest to a click (`nearest_junction`, with a 30 % preference for the junctions of the selected crack), reordering of the list (`move_crack_in_list`) and renumbering of the automatic names (`renumber_cracks`).

13.2 Tip tracking: methods and rejection of elastic waves

The page now offers the **opening jump** (`track_tip_opening_jump`) as its default method; the "ridge (ROI)" and "manual (click)" methods have been removed from the interface (their core functions remain), "arrival (ROI)" remains available as an alternative. Multiple cracks are handled by a drop-down list with check boxes (`ui/checkable_combo.py`): the detection runs on every ticked crack. The tip export can be written as CSV or as an Excel workbook with one sheet per crack (`export_tip_roi_xlsx`).

13.2.1 Opening-jump profile (`crack_opening_jump_profile`)

At each point of the traced path, the displacement fields u, v of the standard DIC tracking are sampled at two points offset by ± offset (by default 4 pixels) along the *local normal* of the path (the tangent is smoothed over a window of 4 × offset by default, to get rid of the grid zigzag); the jump is the difference:

$$J_u = |u(+\text{offset}) - u(-\text{offset})|, \quad J_v = |v(+\text{offset}) - v(-\text{offset})|, \quad J = \sqrt{J_u^2 + J_v^2}. \tag{13.2}$$

An open crack shows a true jump; a smooth strain gradient, however intense, does not. The open extent is the arc distance, from the path origin, up to which J stays continuously above the threshold

`threshold_px` (0.15 pixel, the same default as the Heaviside-jump detection of X-FEM-DIC so that the two numbers are comparable), tolerating `max_gap_samples` (2) consecutive samples below the threshold, including at the origin (`_threshold_walk`), then by linear interpolation of the exact threshold crossing between the last retained sample and the next one (`_interpolate_tip`), so that the tip does not fall on the resampling grid (staircase). A frame where the jump never reaches the threshold near the origin yields *no* point (no filler NaN). The path is assumed to have its origin (index 0) already open.

Rejection of a passing wave. An elastic-wave pulse or a smooth strain around the crack makes $u(+d) - u(-d)$ non-zero even where the faces are not separated (adding about $\varepsilon \cdot 2\,\text{offset}$). Two devices act:

- `remove_continuous_part` (with `n_fit_samples` = 3): each side is extrapolated *to the crack face* by a straight line $a + b\,d$ least-squares fitted to the values at distances $\text{offset}, 2\,\text{offset}, \ldots, n\,\text{offset}$; a is the value at the face seen from that side (`_face_values`) and the jump is the *discontinuity* at the face, which rejects the continuous part; fewer than two valid samples gives NaN;
- `smooth_window_samples` (1 = off): a *running median* filter along the path (`_nanmedian_filter_1d`, which ignores non-finite values), which suppresses the narrow, very intense peaks of a wave without altering a sustained opening plateau; the raw signal J is kept next to the filtered one (plotted and exported).

The criterion (`jump_criterion`) is `magnitude` (default, $|J|$), `normal` (the *opening*, component normal to the crack in its local frame) or `tangential` (the sliding). Across successive frames, `enforce_monotonic` (default) keeps a point only if its position does not recede by more than `backslide_tolerance_px` (3 times the resampling step) relative to the furthest position already kept; and `advance_confirm_frames` (1 = off, immediate acceptance) holds back any candidate point that exceeds the high-water mark by more than `backslide_tolerance_px` until `advance_confirm_frames` consecutive raw points, starting at it, all stay at least at its level (within the tolerance): an advance has to hold over several frames, not just flash on one, to become the new high-water mark. A wave can indeed, on one frame, push the extent far ahead of the true tip, and the old behaviour then permanently raised the mark, corrupting every later backslide comparison. This rule assumes no bound on the crack speed (too variable in a heterogeneous material such as granite); at the very end of the sequence, the last `advance_confirm_frames`-1 points, which can no longer be confirmed, are conservatively dropped.

13.2.2 Position and speed uncertainty (option “Estimate measurement uncertainty”)

For each frame, the three settings of the detector are perturbed *one at a time* and the extent is read again: the threshold ± a local estimate of the noise of the signal J (a 1D Immerkaer-type estimator, on consecutive differences in a window around the front, `_local_noise_sigma`), the offset ±25 % (at least 1 pixel; the only perturbation that resamples u, v), and `max_gap_samples` ±1. Each half-extent (max−min)/2 is an independent contribution; a perturbation that no longer detects anything is dropped from that frame's total (which then under-states the uncertainty rather than hiding it). A path-discretisation floor is added: resampling step$/\sqrt{12}$ (quantisation noise of a uniform error). The contributions are combined in quadrature. The speed is fitted by *weighted* least squares (weights $1/\sigma_i^2$), with the standard deviation on the slope taken from the covariance of the weighted regression; a “Frame rate uncertainty (%)” field (0 by default) adds in quadrature a camera time-base tolerance (jitter, trigger drift), which is not observable in the data. Quantities that are not estimated are `None` (“not estimated”, never “zero uncertainty”).

13.2.3 Local speed

`trace_velocity_px_per_s` is the central-difference derivative of the arc-length position versus time (one-sided at both ends), and `trace_velocity_uncertainty_px_per_s` propagates the position uncertainty through exactly the same finite-difference formula. The local-fit speed in m/s (`core/tip_speed.py::local_tip_speed`) is the slope of a straight line fitted to the position over a window of `window` frames (odd, ≥ 3) centred on the current frame, which shrinks at the ends of the record; the sign is made positive in the direction of propagation.

13.3 Local crack frame and local opening (`core/crack_local_frame.py`)

A branching specimen has several cracks running in different directions: "the displacement perpendicular to the crack" is not an image axis but $\mathbf{u} \cdot \mathbf{n}$, with $\mathbf{n}$ the normal of the *closest* crack to each point. Each node of the DIC grid is assigned to the nearest point of a crack path, $\mathbf{t}$ is the tangent (growth direction, smoothed over `tangent_window_px`) and $\mathbf{n} = \mathbf{t}$ rotated by $+90°$ in image coordinates (x to the right, y downwards):

$$u_t = u\,t_x + v\,t_y, \qquad u_n = -u\,t_y + v\,t_x. \tag{13.3}$$

Across an open crack u_n changes sign (red/blue bands on either side, each in its own frame). But u_n itself is unreadable as soon as there is a bulk displacement (rigid translation, wave) that is the same on both faces and swamps the opening. `local_jump_fields` therefore computes the *local discontinuity*: for a node p beside the crack, with nearest point q and signed distance $d = (p - q) \cdot \mathbf{n}$, the *mirror* node is $p' = p - 2d\,\mathbf{n}$ (same position along the crack, other face, same distance). With $\mathbf{w} = (u, v)$ and the sign $\mathrm{sgn}(d)$ that always counts "$+\mathbf{n}$ side minus $-\mathbf{n}$ side":

$$\mathbf{J} = \mathrm{sgn}(d)\,\big(\mathbf{w}(p) - \mathbf{w}(p')\big), \qquad \text{opening} = \mathbf{J} \cdot \mathbf{n}\ (> 0 \text{ if the faces move apart}), \qquad \text{sliding} = \mathbf{J} \cdot \mathbf{t}. \tag{13.4}$$

The same $\mathbf{J}$ is given to p and p'; both quantities are independent of the direction in which the path is walked; nodes outside the band (`band_inner_px`, `band_outer_px`=10), beyond the end of a crack, or whose mirror falls outside the field or on a NaN, are NaN. The function also returns the index of the crack owning each node, which allows normalising per crack (`normalise_per_crack`, by the detection threshold or by each crack's own percentile: all cracks read from 0 to 1 whatever their absolute opening).

The page's displayable fields also include the displacement magnitude $|u| = \sqrt{u^2 + v^2}$ (key `umag`, in pixels, converted to mm if a pixel size is supplied) and the principal fields $\varepsilon_1, \varepsilon_2, \gamma_{\max}$ (chapter 12).

13.4 Smoothing of the tip position

The near-tip basis diverges as $1/\sqrt{r}$ towards the origin: a noisy tip position (a few pixels from one frame to the next on a fast crack) moves the polar origin of the fit and makes K_I/K_{II} vary far more than the tip trace itself. The positions are therefore smoothed first (`smooth_tip_arc_length`, window of `window` frames, odd, 7 by default):

`none`	raw positions (only monotonicity is applied, if requested)
`median`	running median: rejects an isolated bad detection
`linear`	local straight-line regression versus time: damps jitter and follows a speed ramp (no lag, unlike a moving average)
`quadratic`	local parabola: also follows an acceleration
`isotonic`	monotone (non-decreasing, pool-adjacent-violators) fit, then local regression to round off the staircase

`monotonic=True` finishes with the closest monotone sequence in the L^2 sense, in the direction of the overall trend: the tip never moves backwards (a growing crack does not heal). The smoothed positions are *re-projected* onto the traced path (`smooth_tip_trace`), so they stay exactly on the crack; they replace the raw positions in a *copy* of the trace, the raw trace is never modified.

13.5 Direct K_I and K_{II} from the measured field

`compute_direct_k_series` computes $K_I(t)$ and $K_{II}(t)$ at each tip position directly from the displacement field (u, v) of the exported DIC tracking, without a new X-FEM-DIC correlation. Four methods (`DIRECT_METHODS`):

- `ring`: Williams/Irwin ring fit (`core.crack_k1c.fit_k_single_frame`), with four unknowns (K_I, K_{II}, T_x, T_y) by linear ordinary least squares; $\kappa = 3 - 4\nu$ (plane strain) or $(3 - \nu)/(1 + \nu)$

(plane stress); ring of radii $r_{\min}$ to $r_{\max}$, flanks of $\pm$`flank_exclusion_deg` (15°) excluded; formal uncertainty from the regression covariance $\sigma^2(X^\top X)^{-1}$;
- `ring_t`: same with the T-stress term (fifth parameter);
- `ring_affine`: same with a uniform strain and a rigid-body rotation (extra parameters, `include_affine`);
- `interaction`: domain-form interaction integral (`core/crack_interaction_integral.py`).

The local crack direction comes from the traced path (smoothed over `tangent_window_px`, by default $2\,r_{\max}$) and is oriented along the tip's actual advance. `tip_offset_px` moves the origin of the fit along this direction (positive: ahead of the detected tip): the opening-jump tip lies *behind* the true tip (the opening drops below the DIC resolution before the tip), and a fit origin behind the true tip *overestimates* K, the more so the smaller $r_{\max}$ is. With `trim_after_arrest` (default), the frames after the crack has stopped (`growth_end_index`: first frame at which the tip reaches its final position, within max(`min_tol_px`, `rel_tol` × total travel)) are dropped: the field relaxes and K is no longer a propagation value there. The result (`DirectKSeries`) has the same shape as that of X-FEM-DIC, so the page's graphs, status text and CSV export treat it identically.

Sensitivity scans. Two tools make it possible to judge how robust the choice of parameters is, on at most `max_frames` evenly spaced frames: `scan_k_sensitivity` gives the median of K_I for each pair (tip offset, $r_{\max}$); `scan_k_window` for each ring ($r_{\min}, r_{\max}$) with $r_{\max} \geq r_{\min} + 8$ pixels. A K-dominated zone appears as a block of the table where K no longer changes; a K that still drifts with $r_{\min}$ signals that the ring is too close to the tip (unresolved jump zone) or already in the far field. The tables are formatted as text (`format_scan_table`, `format_window_table`, in MPa$\sqrt{\text{m}}$).

13.6 X-FEM-DIC: sub-pixel sampling and K from the fitted field

Sampling of the reference. The reference image must be sampled at the *exact* (sub-pixel) position of each sample point, not at the nearest integer pixel: otherwise a ± 0.5 pixel position error creates a residual floor and noisy or "blocky" strain maps. The code evaluates the reference by `RectBivariateSpline.ev` at the real coordinates (the deformed image is already sampled by a bicubic spline, $k_x = k_y = 3$).

K computed on the fitted field (`refit_k_from_fitted_field`). When K_I and K_{II} are unknowns of the grey-level fit (`fit_singular_term=True`, §10.4.2), they are almost collinear with the free nodal field and come out with an arbitrary sign and size. The alternative is to fit first the model *without* the singular term (`fit_singular_term=False`: bilinear background + Heaviside jump), then to apply to the *fitted* displacement field (evaluated by `evaluate_xfem_dic_displacement_field` / `evaluate_xfem_dic_field_grid`) the *same* ring fit that the direct method applies to the subset field (methods `ring`, `ring_t`, `ring_affine`; `K_FIELD_METHODS`). The Heaviside jump resolves the crack-face discontinuity that subset DIC smears out. On a synthetic test (true $K_I = 1.8$ MPa$\sqrt{\text{m}}$, uniform background strain, grey-level noise of 2), the joint fit gave -0.4 to $+3$ (biased even without noise) whereas the ring re-fit with `ring_affine` gives 1.7 to 1.9. The fitted fields can also be displayed (strains by central finite differences on a regular grid, like `_grid_strain` of chapter 6, for a point-by-point comparison with the subset field; equivalent strain by `von_mises_strain`).

13.7 K_I versus tip speed

In dynamic fracture, the toughness of a brittle material is not a constant: it rises steeply with crack speed, so a K_I measured on a fast, branching crack is not comparable with the quasi-static K_{Ic}. The "K_I versus tip speed" plot of the Results card plots K_I against the local tip speed; the Rayleigh wave speed, to which it is usually compared, is approximated by Viktorov's formula:

$$c_R = c_s \, \frac{0.862 + 1.14\,\nu}{1 + \nu}, \tag{13.5}$$

(c_s shear-wave speed, ν Poisson's ratio; `None` if an input is missing), and the Spearman rank correlation between K_I and speed (`spearman`: NaN pairs dropped; `None` below 4 pairs or if one variable is constant).

13.8 Video exports of the tip analyses

Two exporters produce an MP4 of the evolution of the analysis, in the same convention as `core/dic_video_export.py` (white background, fixed colour and axis scales supplied by the caller so that successive frames stay comparable, matplotlib Agg figure without Qt, hence runnable in a background thread):

- `core/tip_video_export.py` ("Crack Tip Detection" card): at the top, standard DIC isocontours with the trajectory of the detected tip (`opening_jump`); at the bottom, position and speed versus time (two ordinate axes), growing frame by frame;
- `core/xfem_video_export.py` ("Field isocontours" card, X-FEM-DIC tab): at the top, the standard DIC field and the fitted X-FEM-DIC field side by side, same frame, same colour scale; at the bottom, K_I (and K_{II}) on the left and tip speed on the right, *versus the distance travelled along the path* (not versus time: the physical question is "how does the toughness change as the crack advances").

In both cases the trajectory "so far" is drawn by walking the *real* resampled path (`path_polyline_up_to_arc_length`) from the origin to the last detected position, rather than by joining the temporally sparse detected points with straight segments; on a curved or branching crack the chord visibly cut corners.

13.9 File / function summary

Function / class	Role
`crack_skeleton.detect_cracks_ridge`	ridge-based crack detection, branches included
`crack_skeleton.split_into_branches`	one tree → non-branching cracks (optional master)
`crack_skeleton.move_branch_point, reroute_crack_through_point, set_crack_parent`	manual corrections
`crack_tip_tracking.crack_opening_jump_profile`	opening jump along the path, wave rejection
`crack_tip_tracking.track_tip_opening_jump`	frame-by-frame tip tracking
`crack_tip_tracking.opening_extent_position_uncertainty_px`	position uncertainty
`crack_local_frame.local_frame_displacement, local_jump_fields`	u_t, u_n; local opening and sliding
`direct_k.smooth_tip_trace`	smoothing of the tip position
`direct_k.compute_direct_k_series`	direct K_I, K_{II} (ring, ring_t, ring_affine, interaction)
`direct_k.scan_k_sensitivity, scan_k_window`	sensitivity scans
`xfem_dic.refit_k_from_fitted_field`	K on the fitted X-FEM-DIC field
`tip_speed.local_tip_speed, rayleigh_speed, spearman`	speed, Rayleigh reference, rank correlation
`tip_video_export, xfem_video_export`	MP4 videos of the analyses

13.10 Limitations, assumptions and points to check

- The opening-jump tip is a *lower* bound of the position of the true tip (the opening drops below the DIC resolution before it); the `tip_offset_px` offset and the sensitivity scans serve to assess its effect on K, they do not correct it automatically.
- The ridge detector assumes a crack brighter than both its sides in the displayed field (equivalent or principal strain); a closed crack or a saturated field can produce false negatives, and Otsu's threshold a trace of noise if the minimum contrast is crossed by chance.

- The position uncertainty combines the contributions in quadrature assuming they are independent, which is a simplification (the threshold and the offset both act through the same noisy field); it is never lower than the largest single contribution, hence never optimistic in that sense.
- The direct K values rely on a displacement field exported from a subset-based tracking that may have rejected the points close to the faces: the fitting ring already excludes $r < r_{\min}$ and the flanks of $\pm 15^{\circ}$; the choice of $(r_{\min}, r_{\max})$ and of the tip offset must be justified by the sensitivity scans.

14. PROCESSING AND EXPORTS: SUPPLEMENTS OF THE V1.0 EDITION

This chapter groups the smaller changes made since the October 2026 edition to chapters 2 to 5 (processing, confinement) and to the Results & Export, Test Setup and Covariance pages. Source files: `core/confinement.py`, `core/covariance.py`, `core/setup_profile.py`, `core/models.py`, `ui/pages/page_process.py`, `ui/pages/page_results_export.py`, `ui/pages/page_covariance.py`, `ui/pages/page_test_setup.py`, `ui/pages/page_bar.py`.

14.1 Strain rates of the confined test

To the radial, volumetric and equivalent strains of chapter 5 (§5.5) are added their *rates*: `strain_rate_from_strain(strain, time)` (`core/confinement.py`) returns the derivative $\mathrm{d}\varepsilon/\mathrm{d}t$ by central differences in the interior and one-sided differences at both ends (`numpy.gradient`), of the same length as the inputs truncated to their common length. It differentiates exactly the strain definitions returned by `radial_strain`, `volumetric_strain` and `equivalent_strain`: the volumetric rate is negative while the specimen compacts, and the equivalent rate can change sign on unloading. A time vector with duplicated or non-increasing stamps (for example a repeated row in the raw time file) would make a plain gradient divide by zero: in that case, the median of the strictly positive steps is used as a uniform step (same idea as `process_shpb._robust_dt`); a `ValueError` is raised if no usable step exists.

The corresponding keys are `confinement_radial_strain_rate`, `confinement_volumetric_strain_rate` and `confinement_equivalent_strain_rate` (output of the confined computation when a time vector is supplied), and `radial_strain_rate`, `volumetric_strain_rate`, `equivalent_strain_rate` in the synthetic draws (`triaxial_synthetic_draws`); the Covariance page offers them among its quantities, with the same Monte-Carlo treatment as stresses and strains (point-by-point mean and standard deviation over the N draws, §1.1.3), and they are exported by the Results & Export page. The rate is always computed from the curves *of each draw*, never from an average of curves.

14.2 Curves before toe correction ("pre-toe")

The toe correction (§3.5, `core/toe_correction.py`) removes the first points of the corrected curves: the corrected columns are therefore shorter than the original curves. To keep in the exports the information that this correction discards, the processing keeps the original set of curves (before correction) and exports it next to the corrected curves under names suffixed with `_pre_toe` (`PRE_TOE_SUFFIX`): `stress_pre_toe`, `stress_pre_toe_std`, etc., i.e. the mean and the standard deviation (`_std`) of the original draws (`_pre_toe_quantities`). The raw sensor signals (`sensor_<i>_signal…`) are excluded: they are inputs, not results.

So that the longer `_pre_toe` columns are not truncated to the shortest column as all exports used to do, `equalize_export_columns` makes the table rectangular in a different way: the corrected columns are truncated to the shortest *of their own group* (behaviour unchanged), each `_pre_toe` column keeps *all* its points (they may differ by one or two points depending on the quantity, for example for a differenced strain rate), and columns shorter than the table are padded with NaN. Without any `_pre_toe` column, the former behaviour is recovered exactly.

14.3 Bar signals in profiles and in the full-test export

The function `core.setup_profile.to_dict(project, include_signals=True)` can embed in the JSON profile the *raw signals* of the input-bar and output-bar gauges (`bar_signals_to_dict`, signal encoded by `encode_signal`, keys `input_bar` and `output_bar`), i.e. the signal before the bar's sign and offset are applied (`raw_bar_signal`: (signal + offset)/sign). This option is used by "Export full test"

of the Results & Export page, so that the test can be reloaded without the original signal files. On reading back (`bar_signals_from_dict`), the raw signal is restored, then the *current* sign and offset of each bar are re-applied (`signal = raw × sign − offset`), so that a change of sign or offset made since the export is respected. A corrupt blob is ignored. For test JSON files exported before the signals were embedded, `reload_bar_signal_from_file` re-reads the signal from the file remembered in `project.last_paths` (same reading and same optional downsampling as `ui/pages/page_bar.py`), then re-applies sign and offset; the operation fails (returns `False`) if the file has disappeared.

14.4 Axial stress from laser velocimetry: uncertainty propagation

The axial stress derived from the laser surface velocity of the output bar (§5.5.6) is

$$\sigma_{\text{laser}} = \frac{D_{\text{OB}}^2}{D_{\text{spe}}^2} \, \frac{\rho_{\text{OB}} \, C_{0,\text{OB}}}{2} \, v_{\text{laser}} \times 10^{-6} \quad [\text{MPa}] \tag{14.1}$$

(`confinement.laser_stress`). The uncertainty of the laser signal (`laser_signal_std`) is that of the *velocity* (declared tolerance of the sensor, plus Monte-Carlo alignment jitter), whereas `triaxial_synthetic_draws` expects the uncertainty of the *stress* (MPa). Since `laser_stress` is a simple proportionality, the same function converts the velocity standard deviation into a stress standard deviation; passing the velocity standard deviation as is (a few tenths of m/s against stresses of hundreds of MPa) made the laser band vanish. The conversion is applied in the Covariance page and in the Results & Export page.

14.5 Setup profiles and session

The named setup profiles (`core/setup_profile.py`), distinct from the automatic restoration of the last session (`core/session.py`), group what is reused from one test to the next on the same equipment: bar properties, template specimen, confinement cell, platens, custom sensors and, for quasi-static tests, the machine-compliance settings. They are saved and loaded from the Test Setup page.

14.6 File / function summary

Function	Role
`confinement.strain_rate_from_strain`	robust time derivative of a strain
`page_process._pre_toe_quantities`	original curves, `_pre_toe` suffix
`page_results_export.equalize_export_columns`	rectangular export table with complete `_pre_toe` columns
`setup_profile.bar_signals_to_dict, bar_signals_from_dict`	embedded raw bar signals
`setup_profile.reload_bar_signal_from_file`	fallback for old test JSON files
`confinement.laser_stress`	laser axial stress; conversion of the velocity standard deviation

14.7 Limitations, assumptions and points to check

- Strain rates are numerical derivatives: measurement noise is amplified in them, and the result depends on the time step (the median of the positive steps is used when the time axis is defective, which hides an acquisition anomaly).
- The `_pre_toe` columns do not all have exactly the same length; shorter columns are padded with NaN, not extrapolated.

- The sign and offset of a bar re-applied on reading back are the *current* ones of the project, not those in force at export time.